

Part I

Foundations



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

Introduction

When I got my first game console in 1979—a way-cool Intellivision system by Mattel—the term “game engine” did not exist. Back then, video and arcade games were considered by most adults to be nothing more than toys, and the software that made them tick was highly specialized to both the game in question and the hardware on which it ran. Today, games are a multi-billion-dollar mainstream industry rivaling Hollywood in size and popularity. And the software that drives these now-ubiquitous three-dimensional worlds—*game engines* like Epic Games’ Unreal Engine 4, Valve’s Source engine and, Crytek’s CRYENGINE® 3, Electronic Arts DICE’s Frostbite™ engine, and the Unity game engine—have become fully featured reusable software development kits that can be licensed and used to build almost any game imaginable.

While game engines vary widely in the details of their architecture and implementation, recognizable coarse-grained patterns have emerged across both publicly licensed game engines and their proprietary in-house counterparts. Virtually all game engines contain a familiar set of core components, including the rendering engine, the collision and physics engine, the animation system, the audio system, the game world object model, the artificial intelligence system and so on. Within each of these components, a relatively small number of semi-standard design alternatives are also beginning to emerge.

There are a great many books that cover individual game engine subsystems, such as three-dimensional graphics, in exhaustive detail. Other books cobble together valuable tips and tricks across a wide variety of game technology areas. However, I have been unable to find a book that provides its reader with a reasonably complete picture of the entire gamut of components that make up a modern game engine. The goal of this book, then, is to take the reader on a guided hands-on tour of the vast and complex landscape of game engine architecture.

In this book you will learn:

- how real industrial-strength production game engines are architected;
- how game development teams are organized and work in the real world;
- which major subsystems and design patterns appear again and again in virtually every game engine;
- the typical requirements for each major subsystem;
- which subsystems are genre- or game-agnostic, and which ones are typically designed explicitly for a specific genre or game; and
- where the engine normally ends and the game begins.

We'll also get a first-hand glimpse into the inner workings of some popular game engines, such as Quake, Unreal and Unity, and some well-known middleware packages, such as the Havok Physics library, the OGRE rendering engine and Rad Game Tools' Granny 3D animation and geometry management toolkit. And we'll explore a number of proprietary game engines that I've had the pleasure to work with, including the engine Naughty Dog developed for its *Uncharted* and *The Last of Us* game series.

Before we get started, we'll review some techniques and tools for large-scale software engineering in a game engine context, including:

- the difference between logical and physical software architecture;
- configuration management, revision control and build systems; and
- some tips and tricks for dealing with one of the common development environments for C and C++, Microsoft Visual Studio.

In this book I assume that you have a solid understanding of C++ (the language of choice among most modern game developers) and that you understand basic software engineering principles. I also assume you have some

exposure to linear algebra, three-dimensional vector and matrix math and trigonometry (although we'll review the core concepts in Chapter 5). Ideally, you should have some prior exposure to the basic concepts of real time and event-driven programming. But never fear—I will review these topics briefly, and I'll also point you in the right direction if you feel you need to hone your skills further before we embark.

1.1 Structure of a Typical Game Team

Before we delve into the structure of a typical game engine, let's first take a brief look at the structure of a typical game development team. Game studios are usually composed of five basic disciplines: engineers, artists, game designers, producers and other management and support staff (marketing, legal, information technology/technical support, administrative, etc.). Each discipline can be divided into various subdisciplines. We'll take a brief look at each below.

1.1.1 Engineers

The engineers design and implement the software that makes the game, and the tools, work. Engineers are often categorized into two basic groups: *runtime* programmers (who work on the engine and the game itself) and *tools* programmers (who work on the offline tools that allow the rest of the development team to work effectively). On both sides of the runtime/tools line, engineers have various specialties. Some engineers focus their careers on a single engine system, such as rendering, artificial intelligence, audio or collision and physics. Some focus on gameplay programming and scripting, while others prefer to work at the systems level and not get too involved in how the game actually plays. Some engineers are generalists—jacks of all trades who can jump around and tackle whatever problems might arise during development.

Senior engineers are sometimes asked to take on a technical leadership role. Lead engineers usually still design and write code, but they also help to manage the team's schedule, make decisions regarding the overall technical direction of the project, and sometimes also directly manage people from a human resources perspective.

Some companies also have one or more technical directors (TD), whose job it is to oversee one or more projects from a high level, ensuring that the teams are aware of potential technical challenges, upcoming industry developments, new technologies and so on. The highest engineering-related position at a game studio is the chief technical officer (CTO), if the studio has one. The

CTO's job is to serve as a sort of technical director for the entire studio, as well as serving a key executive role in the company.

1.1.2 Artists

As we say in the game industry, "Content is king." The artists produce all of the visual and audio content in the game, and the quality of their work can literally make or break a game. Artists come in all sorts of flavors:

- *Concept artists* produce sketches and paintings that provide the team with a vision of what the final game will look like. They start their work early in the concept phase of development, but usually continue to provide visual direction throughout a project's life cycle. It is common for screenshots taken from a shipping game to bear an uncanny resemblance to the concept art.
- *3D modelers* produce the three-dimensional geometry for everything in the virtual game world. This discipline is typically divided into two sub-disciplines: foreground modelers and background modelers. The former create objects, characters, vehicles, weapons and the other objects that populate the game world, while the latter build the world's static background geometry (terrain, buildings, bridges, etc.).
- *Texture artists* create the two-dimensional images known as textures, which are applied to the surfaces of 3D models in order to provide detail and realism.
- *Lighting artists* lay out all of the light sources in the game world, both static and dynamic, and work with color, intensity and light direction to maximize the artfulness and emotional impact of each scene.
- *Animators* imbue the characters and objects in the game with motion. The animators serve quite literally as actors in a game production, just as they do in a CG film production. However, a game animator must have a unique set of skills in order to produce animations that mesh seamlessly with the technological underpinnings of the game engine.
- *Motion capture actors* are often used to provide a rough set of motion data, which are then cleaned up and tweaked by the animators before being integrated into the game.
- *Sound designers* work closely with the engineers in order to produce and mix the sound effects and music in the game.

- *Voice actors* provide the voices of the characters in many games.
- Many games have one or more *composers*, who compose an original score for the game.

As with engineers, senior artists are often called upon to be team leaders. Some game teams have one or more *art directors*—very senior artists who manage the look of the entire game and ensure consistency across the work of all team members.

1.1.3 Game Designers

The game designers' job is to design the interactive portion of the player's experience, typically known as *gameplay*. Different kinds of designers work at different levels of detail. Some (usually senior) game designers work at the macro level, determining the story arc, the overall sequence of chapters or levels, and the high-level goals and objectives of the player. Other designers work on individual levels or geographical areas within the virtual game world, laying out the static background geometry, determining where and when enemies will emerge, placing supplies like weapons and health packs, designing puzzle elements and so on. Still other designers operate at a highly technical level, working closely with gameplay engineers and/or writing code (often in a high-level scripting language). Some game designers are ex-engineers, who decided they wanted to play a more active role in determining how the game will play.

Some game teams employ one or more *writers*. A game writer's job can range from collaborating with the senior game designers to construct the story arc of the entire game, to writing individual lines of dialogue.

As with other disciplines, some senior designers play management roles. Many game teams have a game director, whose job it is to oversee all aspects of a game's design, help manage schedules, and ensure that the work of individual designers is consistent across the entire product. Senior designers also sometimes evolve into producers.

1.1.4 Producers

The role of producer is defined differently by different studios. In some game companies, the producer's job is to manage the schedule and serve as a human resources manager. In other companies, producers serve in a senior game design capacity. Still other studios ask their producers to serve as liaisons between the development team and the business unit of the company (finance, legal, marketing, etc.). Some smaller studios don't have producers at all. For

example, at Naughty Dog, literally everyone in the company, including the two co-presidents, plays a direct role in constructing the game; team management and business duties are shared between the senior members of the studio.

1.1.5 Other Staff

The team of people who directly construct the game is typically supported by a crucial team of support staff. This includes the studio's executive management team, the marketing department (or a team that liaises with an external marketing group), administrative staff and the IT department, whose job is to purchase, install and configure hardware and software for the team and to provide technical support.

1.1.6 Publishers and Studios

The marketing, manufacture and distribution of a game title are usually handled by a *publisher*, not by the game studio itself. A publisher is typically a large corporation, like Electronic Arts, THQ, Vivendi, Sony, Nintendo, etc. Many game studios are not affiliated with a particular publisher. They sell each game that they produce to whichever publisher strikes the best deal with them. Other studios work exclusively with a single publisher, either via a long-term publishing contract or as a fully owned subsidiary of the publishing company. For example, THQ's game studios are independently managed, but they are owned and ultimately controlled by THQ. Electronic Arts takes this relationship one step further, by directly managing its studios. *First-party developers* are game studios owned directly by the console manufacturers (Sony, Nintendo and Microsoft). For example, Naughty Dog is a first-party Sony developer. These studios produce games exclusively for the gaming hardware manufactured by their parent company.

1.2 What Is a Game?

We probably all have a pretty good intuitive notion of what a game is. The general term "game" encompasses board games like chess and *Monopoly*, card games like poker and blackjack, casino games like roulette and slot machines, military war games, computer games, various kinds of play among children, and the list goes on. In academia we sometimes speak of *game theory*, in which multiple agents select strategies and tactics in order to maximize their gains within the framework of a well-defined set of game rules. When used in the context of console or computer-based entertainment, the word "game"

usually conjures images of a three-dimensional virtual world featuring a humanoid, animal or vehicle as the main character under player control. (Or for the old geezers among us, perhaps it brings to mind images of two-dimensional classics like *Pong*, *Pac-Man*, or *Donkey Kong*.) In his excellent book, *A Theory of Fun for Game Design*, Raph Koster defines a game to be an interactive experience that provides the player with an increasingly challenging sequence of patterns which he or she learns and eventually masters [30]. Koster's assertion is that the activities of learning and mastering are at the heart of what we call "fun," just as a joke becomes funny at the moment we "get it" by recognizing the pattern.

For the purposes of this book, we'll focus on the subset of games that comprise two- and three-dimensional virtual worlds with a small number of players (between one and 16 or thereabouts). Much of what we'll learn can also be applied to HTML5/JavaScript games on the Internet, pure puzzle games like *Tetris*, or massively multiplayer online games (MMOG). But our primary focus will be on game engines capable of producing first-person shooters, third-person action/platform games, racing games, fighting games and the like.

1.2.1 Video Games as Soft Real-Time Simulations

Most two- and three-dimensional video games are examples of what computer scientists would call *soft real-time interactive agent-based computer simulations*. Let's break this phrase down in order to better understand what it means.

In most video games, some subset of the real world—or an imaginary world—is *modeled* mathematically so that it can be manipulated by a computer. The model is an approximation to and a simplification of reality (even if it's an *imaginary* reality), because it is clearly impractical to include every detail down to the level of atoms or quarks. Hence, the mathematical model is a *simulation* of the real or imagined game world. Approximation and simplification are two of the game developer's most powerful tools. When used skillfully, even a greatly simplified model can sometimes be almost indistinguishable from reality—and a lot more fun.

An *agent-based* simulation is one in which a number of distinct entities known as "agents" interact. This fits the description of most three-dimensional computer games very well, where the agents are vehicles, characters, fireballs, power dots and so on. Given the agent-based nature of most games, it should come as no surprise that most games nowadays are implemented in an object-oriented, or at least loosely object-based, programming language.

All interactive video games are *temporal simulations*, meaning that the virtual game world model is *dynamic*—the state of the game world changes over time as the game’s events and story unfold. A video game must also respond to unpredictable inputs from its human player(s)—thus *interactive temporal simulations*. Finally, most video games present their stories and respond to player input in real time, making them *interactive real-time simulations*. One notable exception is in the category of turn-based games like computerized chess or turn-based strategy games. But even these types of games usually provide the user with some form of real-time graphical user interface. So for the purposes of this book, we’ll assume that all video games have at least *some* real-time constraints.

At the core of every real-time system is the concept of a *deadline*. An obvious example in video games is the requirement that the screen be updated at least 24 times per second in order to provide the illusion of motion. (Most games render the screen at 30 or 60 frames per second because these are multiples of an NTSC monitor’s refresh rate.) Of course, there are many other kinds of deadlines in video games as well. A physics simulation may need to be updated 120 times per second in order to remain stable. A character’s artificial intelligence system may need to “think” at least once every second to prevent the appearance of stupidity. The audio library may need to be called at least once every 1/60 second in order to keep the audio buffers filled and prevent audible glitches.

A “soft” real-time system is one in which missed deadlines are not catastrophic. Hence, all video games are *soft real-time systems*—if the frame rate dies, the human player generally doesn’t! Contrast this with a *hard real-time system*, in which a missed deadline could mean severe injury to or even the death of a human operator. The avionics system in a helicopter or the control-rod system in a nuclear power plant are examples of hard real-time systems.

Mathematical models can be *analytic* or *numerical*. For example, the analytic (closed-form) mathematical model of a rigid body falling under the influence of constant acceleration due to gravity is typically written as follows:

$$y(t) = \frac{1}{2}gt^2 + v_0t + y_0. \quad (1.1)$$

An analytic model can be evaluated for any value of its independent variables, such as the time t in the above equation, given only the initial conditions v_0 and y_0 and the constant g . Such models are very convenient when they can be found. However, many problems in mathematics have no closed-form solution. And in video games, where the user’s input is unpredictable, we cannot hope to model the entire game analytically.

A numerical model of the same rigid body under gravity can be expressed

as follows:

$$y(t + \Delta t) = F(y(t), \dot{y}(t), \ddot{y}(t), \dots). \quad (1.2)$$

That is, the height of the rigid body at some future time $(t + \Delta t)$ can be found as a function of the height and its first, second, and possibly higher-order time derivatives at the current time t . Numerical simulations are typically implemented by running calculations repeatedly, in order to determine the state of the system at each discrete time step. Games work in the same way. A main “game loop” runs repeatedly, and during each iteration of the loop, various game systems such as artificial intelligence, game logic, physics simulations and so on are given a chance to calculate or update their state for the next discrete time step. The results are then “rendered” by displaying graphics, emitting sound and possibly producing other outputs such as force-feedback on the joystick.

1.3 What Is a Game Engine?

The term “game engine” arose in the mid-1990s in reference to first-person shooter (FPS) games like the insanely popular *Doom* by id Software. *Doom* was architected with a reasonably well-defined separation between its core software components (such as the three-dimensional graphics rendering system, the collision detection system or the audio system) and the art assets, game worlds and rules of play that comprised the player’s gaming experience. The value of this separation became evident as developers began licensing games and retooling them into new products by creating new art, world layouts, weapons, characters, vehicles and game rules with only minimal changes to the “engine” software. This marked the birth of the “mod community”—a group of individual gamers and small independent studios that built new games by modifying existing games, using free toolkits provided by the original developers.

Towards the end of the 1990s, some games like *Quake III Arena* and *Unreal* were designed with reuse and “modding” in mind. Engines were made highly customizable via scripting languages like id’s Quake C, and engine licensing began to be a viable secondary revenue stream for the developers who created them. Today, game developers can license a game engine and reuse significant portions of its key software components in order to build games. While this practice still involves considerable investment in custom software engineering, it can be much more economical than developing all of the core engine components in-house.

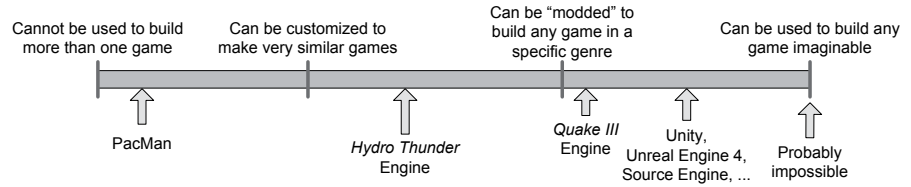


Figure 1.1. Game engine reusability gamut.

The line between a game and its engine is often blurry. Some engines make a reasonably clear distinction, while others make almost no attempt to separate the two. In one game, the rendering code might “know” specifically how to draw an orc. In another game, the rendering engine might provide general-purpose material and shading facilities, and “orc-ness” might be defined entirely in data. No studio makes a perfectly clear separation between the game and the engine, which is understandable considering that the definitions of these two components often shift as the game’s design solidifies.

Arguably a *data-driven architecture* is what differentiates a game engine from a piece of software that is a game but not an engine. When a game contains hard-coded logic or game rules, or employs special-case code to render specific types of game objects, it becomes difficult or impossible to reuse that software to make a different game. We should probably reserve the term “game engine” for software that is extensible and can be used as the foundation for many different games without major modification.

Clearly this is not a black-and-white distinction. We can think of a gamut of reusability onto which every engine falls. Figure 1.1 takes a stab at the locations of some well-known games/engines along this gamut.

One would think that a game engine could be something akin to Apple QuickTime or Microsoft Windows Media Player—a general-purpose piece of software capable of playing virtually *any* game content imaginable. However, this ideal has not yet been achieved (and may never be). Most game engines are carefully crafted and fine-tuned to run a particular game on a particular hardware platform. And even the most general-purpose multiplatform engines are really only suitable for building games in one particular genre, such as first-person shooters or racing games. It’s safe to say that the more general-purpose a game engine or middleware component is, the less optimal it is for running a particular game on a particular platform.

This phenomenon occurs because designing any efficient piece of software invariably entails making trade-offs, and those trade-offs are based on assumptions about how the software will be used and/or about the target

hardware on which it will run. For example, a rendering engine that was designed to handle intimate indoor environments probably won't be very good at rendering vast outdoor environments. The indoor engine might use a binary space partitioning (BSP) tree or portal system to ensure that no geometry is drawn that is being occluded by walls or objects that are closer to the camera. The outdoor engine, on the other hand, might use a less-exact occlusion mechanism, or none at all, but it probably makes aggressive use of level-of-detail (LOD) techniques to ensure that distant objects are rendered with a minimum number of triangles, while using high-resolution triangle meshes for geometry that is close to the camera.

The advent of ever-faster computer hardware and specialized graphics cards, along with ever-more-efficient rendering algorithms and data structures, is beginning to soften the differences between the graphics engines of different genres. It is now possible to use a first-person shooter engine to build a strategy game, for example. However, the trade-off between generality and optimality still exists. A game can always be made more impressive by fine-tuning the engine to the specific requirements and constraints of a particular game and/or hardware platform.

1.4 Engine Differences across Genres

Game engines are typically somewhat genre specific. An engine designed for a two-person fighting game in a boxing ring will be very different from a massively multiplayer online game (MMOG) engine or a first-person shooter (FPS) engine or a real-time strategy (RTS) engine. However, there is also a great deal of overlap—all 3D games, regardless of genre, require some form of low-level user input from the joypad, keyboard and/or mouse, some form of 3D mesh rendering, some form of heads-up display (HUD) including text rendering in a variety of fonts, a powerful audio system, and the list goes on. So while the Unreal Engine, for example, was designed for first-person shooter games, it has been used successfully to construct games in a number of other genres as well, including the wildly popular third-person shooter franchise *Gears of War* by Epic Games, the hit action-adventure games in the *Batman: Arkham* series by Rocksteady Studios, the well-known fighting game *Tekken 7* by Bandai Namco Studios, and the first three role-playing third-person shooter games in the *Mass Effect* series by BioWare.

Let's take a look at some of the most common game genres and explore some examples of the technology requirements particular to each.



Figure 1.2. *Overwatch* by Blizzard Entertainment (Xbox One, PlayStation 4, Windows). (See Color Plate I.)

1.4.1 First-Person Shooters (FPS)

The first-person shooter (FPS) genre is typified by games like *Quake*, *Unreal Tournament*, *Half-Life*, *Battlefield*, *Destiny*, *Titanfall* and *Overwatch* (see Figure 1.2). These games have historically involved relatively slow on-foot roaming of a potentially large but primarily corridor-based world. However, modern first-person shooters can take place in a wide variety of virtual environments including vast open outdoor areas and confined indoor areas. Modern FPS traversal mechanics can include on-foot locomotion, rail-confined or free-roaming ground vehicles, hovercraft, boats and aircraft. For an overview of this genre, see http://en.wikipedia.org/wiki/First-person_shooter.

First-person games are typically some of the most technologically challenging to build, probably rivaled in complexity only by third-person shooters, action-platformer games, and massively multiplayer games. This is because first-person shooters aim to provide their players with the illusion of being immersed in a detailed, hyperrealistic world. It is not surprising that many of the game industry's big technological innovations arose out of the games in this genre.

First-person shooters typically focus on technologies such as:

- efficient rendering of large 3D virtual worlds;

- a responsive camera control/aiming mechanic;
- high-fidelity animations of the player’s virtual arms and weapons;
- a wide range of powerful handheld weaponry;
- a forgiving player character motion and collision model, which often gives these games a “floaty” feel;
- high-fidelity animations and artificial intelligence for the non-player characters (NPCs)—the player’s enemies and allies; and
- small-scale online multiplayer capabilities (typically supporting between 10 and 100 simultaneous players), and the ubiquitous “death match” gameplay mode.

The rendering technology employed by first-person shooters is almost always highly optimized and carefully tuned to the particular type of environment being rendered. For example, indoor “dungeon crawl” games often employ binary space partitioning trees or portal-based rendering systems. Outdoor FPS games use other kinds of rendering optimizations such as occlusion culling, or an offline sectorization of the game world with manual or automated specification of which target sectors are visible from each source sector.

Of course, immersing a player in a hyperrealistic game world requires much more than just optimized high-quality graphics technology. The character animations, audio and music, rigid body physics, in-game cinematics and myriad other technologies must all be cutting-edge in a first-person shooter. So this genre has some of the most stringent and broad technology requirements in the industry.

1.4.2 Platformers and Other Third-Person Games

“Platformer” is the term applied to third-person character-based action games where jumping from platform to platform is the primary gameplay mechanic. Typical games from the 2D era include *Space Panic*, *Donkey Kong*, *Pitfall!* and *Super Mario Brothers*. The 3D era includes platformers like *Super Mario 64*, *Crash Bandicoot*, *Rayman 2*, *Sonic the Hedgehog*, the *Jak and Daxter* series (Figure 1.3), the *Ratchet & Clank* series and *Super Mario Galaxy*. See <http://en.wikipedia.org/wiki/Platformer> for an in-depth discussion of this genre.

In terms of their technological requirements, platformers can usually be lumped together with third-person shooters and third-person action/adventure games like *Just Cause 2*, *Gears of War 4* (Figure 1.4), the *Uncharted* series, the *Resident Evil* series, the *The Last of Us* series, *Red Dead Redemption 2*, and the list goes on.

Third-person character-based games have a lot in common with first-person shooters, but a great deal more emphasis is placed on the main character's abilities and locomotion modes. In addition, high-fidelity full-body character animations are required for the player's avatar, as opposed to the somewhat less-taxing animation requirements of the "floating arms" in a typical FPS game. It's important to note here that almost all first-person shooters have an online multiplayer component, so a full-body player avatar must be rendered in addition to the first-person arms. However, the fidelity of these FPS player avatars is usually not comparable to the fidelity of the non-player characters in these same games; nor can it be compared to the fidelity of the player avatar in a third-person game.

In a platformer, the main character is often cartoon-like and not particularly realistic or high-resolution. However, third-person shooters often feature a highly realistic humanoid player character. In both cases, the player character typically has a very rich set of actions and animations.

Some of the technologies specifically focused on by games in this genre include:



Figure 1.3. *Jak II* by Naughty Dog (Jak, Daxter, Jak and Daxter, and Jak II © 2003, 2013/™ SIE. Created and developed by Naughty Dog, PlayStation 2.) (See Color Plate II.)



Figure 1.4. *Gears of War 4* by The Coalition (Xbox One). (See Color Plate III.)

- moving platforms, ladders, ropes, trellises and other interesting locomotion modes;
- puzzle-like environmental elements;
- a third-person “follow camera” which stays focused on the player character and whose rotation is typically controlled by the human player via the right joypad stick (on a console) or the mouse (on a PC—note that while there are a number of popular third-person shooters on a PC, the platformer genre exists almost exclusively on consoles); and
- a complex camera collision system for ensuring that the view point never “clips” through background geometry or dynamic foreground objects.

1.4.3 Fighting Games

Fighting games are typically two-player games involving humanoid characters pummeling each other in a ring of some sort. The genre is typified by games like *Soul Calibur* and *Tekken 3* (see Figure 1.5). The Wikipedia page http://en.wikipedia.org/wiki/Fighting_game provides an overview of this genre.

Traditionally games in the fighting genre have focused their technology efforts on:

- a rich set of fighting animations;
- accurate hit detection;
- a user input system capable of detecting complex button and joystick combinations; and
- crowds, but otherwise relatively static backgrounds.

Since the 3D world in these games is small and the camera is centered on the action at all times, historically these games have had little or no need for world subdivision or occlusion culling. They would likewise not be expected to employ advanced three-dimensional audio propagation models, for example.

Modern fighting games like EA's *Fight Night Round 4* and NetherRealm Studios' *Injustice 2* (Figure 1.6) have upped the technological ante with features like:

- high-definition character graphics;
- realistic skin shaders with subsurface scattering and sweat effects;
- photo-realistic lighting and particle effects;
- high-fidelity character animations; and



Figure 1.5. *Tekken 3* by Namco (PlayStation). (See Color Plate IV.)

- physics-based cloth and hair simulations for the characters.

It's important to note that some fighting games like Ninja Theory's *Heavenly Sword* and *For Honor* by Ubisoft Montreal take place in a large-scale virtual world, not a confined arena. In fact, many people consider this to be a separate genre, sometimes called a *brawler*. This kind of fighting game can have technical requirements more akin to those of a third-person shooter or a strategy game.

1.4.4 Racing Games

The racing genre encompasses all games whose primary task is driving a car or other vehicle on some kind of track. The genre has many subcategories. Simulation-focused racing games ("sims") aim to provide a driving experience that is as realistic as possible (e.g., *Gran Turismo*). Arcade racers favor over-the-top fun over realism (e.g., *San Francisco Rush*, *Cruis'n USA*, *Hydro Thunder*). One subgenre explores the subculture of street racing with tricked out consumer vehicles (e.g., *Need for Speed*, *Juiced*). Kart racing is a subcategory in which popular characters from platformer games or cartoon characters from TV are re-cast as the drivers of whacky vehicles (e.g., *Mario Kart*, *Jak X*, *Freaky Flyers*). Racing games need not always involve time-based competition. Some kart racing games, for example, offer



Figure 1.6. *Injustice 2* by NetherRealm Studios (PlayStation 4, Xbox One, Android, iOS, Microsoft Windows). (See Color Plate V.)



Figure 1.7. *Gran Turismo Sport* by Polyphony Digital (PlayStation 4). (See Color Plate VI.)

modes in which players shoot at one another, collect loot or engage in a variety of other timed and untimed tasks. For a discussion of this genre, see http://en.wikipedia.org/wiki/Racing_game.

A racing game is often very linear, much like older FPS games. However, travel speed is generally much faster than in an FPS. Therefore, more focus is placed on very long corridor-based tracks, or looped tracks, sometimes with various alternate routes and secret short-cuts. Racing games usually focus all their graphic detail on the vehicles, track and immediate surroundings. As an example of this, Figure 1.7 shows a screenshot from the latest installment in the well-known *Gran Turismo* racing game series, *Gran Turismo Sport*, developed by Polyphony Digital and published by Sony Interactive Entertainment. However, kart racers also devote significant rendering and animation bandwidth to the characters driving the vehicles.

Some of the technological properties of a typical racing game include the following techniques:

- Various “tricks” are used when rendering distant background elements, such as employing two-dimensional cards for trees, hills and mountains.
- The track is often broken down into relatively simple two-dimensional regions called “sectors.” These data structures are used to optimize rendering and visibility determination, to aid in artificial intelligence and path finding for non-human-controlled vehicles, and to solve many other technical problems.
- The camera typically follows behind the vehicle for a third-person per-



Figure 1.8. *Age of Empires* by Ensemble Studios (Windows). (See Color Plate VII.)

spective, or is sometimes situated inside the cockpit first-person style.

- When the track involves tunnels and other “tight” spaces, a good deal of effort is often put into ensuring that the camera does not collide with background geometry.

1.4.5 Strategy Games

The modern strategy game genre was arguably defined by *Dune II: The Building of a Dynasty* (1992). Other games in this genre include *Warcraft*, *Command & Conquer*, *Age of Empires* and *Starcraft*. In this genre, the player deploys the battle units in his or her arsenal strategically across a large playing field in an attempt to overwhelm his or her opponent. The game world is typically displayed at an oblique top-down viewing angle. A distinction is often made between turn-based strategy games and real-time strategy (RTS). For a discussion of this genre, see https://en.wikipedia.org/wiki/Strategy_video_game.

The strategy game player is usually prevented from significantly changing the viewing angle in order to see across large distances. This restriction permits developers to employ various optimizations in the rendering engine of a strategy game.



Figure 1.9. *Total War: Warhammer 2* by Creative Assembly (Windows). (See Color Plate VIII.)

Older games in the genre employed a grid-based (cell-based) world construction, and an orthographic projection was used to greatly simplify the renderer. For example, Figure 1.8 shows a screenshot from the classic strategy game *Age of Empires*.

Modern strategy games sometimes use perspective projection and a true 3D world, but they may still employ a grid layout system to ensure that units and background elements, such as buildings, align with one another properly. A popular example, *Total War: Warhammer 2*, is shown in Figure 1.9.

Some other common practices in strategy games include the following techniques:

- Each unit is relatively low-res, so that the game can support large numbers of them on-screen at once.
- Height-field terrain is usually the canvas upon which the game is designed and played.
- The player is often allowed to build new structures on the terrain in addition to deploying his or her forces.
- User interaction is typically via single-click and area-based selection of units, plus menus or toolbars containing commands, equipment, unit types, building types, etc.



Figure 1.10. *World of Warcraft* by Blizzard Entertainment (Windows, MacOS). (See Color Plate IX.)

1.4.6 Massively Multiplayer Online Games (MMOG)

The massively multiplayer online game (MMOG or just MMO) genre is typified by games like *Guild Wars 2* (AreaNet/NCsoft), *EverQuest* (989 Studios/SOE), *World of Warcraft* (Blizzard) and *Star Wars Galaxies* (SOE/Lucas Arts), to name a few. An MMO is defined as any game that supports huge numbers of simultaneous players (from thousands to hundreds of thousands), usually all playing in one very large, *persistent* virtual world (i.e., a world whose internal state persists for very long periods of time, far beyond that of any one player's gameplay session). Otherwise, the gameplay experience of an MMO is often similar to that of their small-scale multiplayer counterparts. Subcategories of this genre include MMO role-playing games (MMORPG), MMO real-time strategy games (MMORTS) and MMO first-person shooters (MMOFPS). For a discussion of this genre, see <http://en.wikipedia.org/wiki/MMOG>. Figure 1.10 shows a screenshot from the hugely popular MMORPG *World of Warcraft*.

At the heart of all MMOGs is a very powerful battery of servers. These servers maintain the authoritative state of the game world, manage users signing in and out of the game, provide inter-user chat or voice-over-IP (VoIP) services and more. Almost all MMOGs require users to pay some kind of regular

subscription fee in order to play, and they may offer micro-transactions within the game world or out-of-game as well. Hence, perhaps the most important role of the central server is to handle the billing and micro-transactions which serve as the game developer's primary source of revenue.

Graphics fidelity in an MMO is almost always lower than its non-massively multiplayer counterparts, as a result of the huge world sizes and extremely large numbers of users supported by these kinds of games.

Figure 1.11 shows a screen from Bungie's latest FPS game, *Destiny 2*. This game has been called an MMOFPS because it incorporates some aspects of the MMO genre. However, Bungie prefers to call it a "shared world" game because unlike a traditional MMO, in which a player can see and interact with literally any other player on a particular server, *Destiny* provides "on-the-fly match-making." This permits the player to interact only with the other players with whom they have been matched by the server; this matchmaking system has been significantly improved for *Destiny 2*. Also unlike a traditional MMO, the graphics fidelity in *Destiny 2* is on par with first- and third-person shooters.

We should note here that the game *Player Unknown's Battlegrounds* (PUBG) has recently popularized a subgenre known as *battle royale*. This type of game blurs the line between regular multiplayer shooters and massively multiplayer online games, because they typically pit on the order of 100 players against each other in an online world, employing a survival-based "last man standing" gameplay style.



Figure 1.11. *Destiny 2* by Bungie, © 2018 Bungie Inc. (Xbox One, PlayStation 4, PC) (See Color Plate X.)

1.4.7 Player-Authored Content

As social media takes off, games are becoming more and more collaborative in nature. A recent trend in game design is toward *player-authored content*. For example, Media Molecule's *LittleBigPlanet*,TM *LittleBigPlanet*TM 2 (Figure 1.12) and *LittleBigPlanet*TM 3: *The Journey Home* are technically *puzzle platformers*, but their most notable and unique feature is that they encourage players to create, publish and share their own game worlds. Media Molecule's latest installment in this engaging genre is *Dreams* for the PlayStation 4 (Figure 1.13).

Perhaps the most popular game today in the player-created content genre is *Minecraft* (Figure 1.14). The brilliance of this game lies in its simplicity: *Minecraft* game worlds are constructed from simple cubic voxel-like elements mapped with low-resolution textures to mimic various materials. Blocks can be solid, or they can contain items such as torches, anvils, signs, fences and panes of glass. The game world is populated with one or more player characters, animals such as chickens and pigs, and various “mobs”—good guys like villagers and bad guys like zombies and the ubiquitous *creepers* who sneak up on unsuspecting players and explode (only scant moments after warning the player with the “hiss” of a burning fuse).

Players can create a randomized world in *Minecraft* and then dig into the generated terrain to create tunnels and caverns. They can also construct their own structures, ranging from simple terrain and foliage to vast and complex



Figure 1.12. *LittleBigPlanet*TM 2 by Media Molecule, © 2014 Sony Interactive Entertainment (PlayStation 3). (See Color Plate XI.)



Figure 1.13. *Dreams* by Media Molecule, © 2017 Sony Computer Computer Europe (PlayStation 4). (See Color Plate XII.)

buildings and machinery. Perhaps the biggest stroke of genius in *Minecraft* is *redstone*. This material serves as “wiring,” allowing players to lay down circuitry that controls pistons, hoppers, mine carts and other dynamic elements in the game. As a result, players can create virtually anything they can imagine, and then share their worlds with their friends by hosting a server and inviting them to play online.



Figure 1.14. *Minecraft* by Markus “Notch” Persson / Mojang AB (Windows, MacOS, Xbox 360, PlayStation 3, PlayStation Vita, iOS). (See Color Plate XIII.)

1.4.8 Virtual, Augmented and Mixed Reality

Virtual, augmented and mixed reality are exciting new technologies that aim to immerse the viewer in a 3D world that is either entirely generated by a computer, or is augmented by computer-generated imagery. These technologies have many applications outside the game industry, but they have also become viable platforms for a wide range of gaming content.

1.4.8.1 Virtual Reality

Virtual reality (VR) can be defined as an immersive multimedia or computer-simulated reality that simulates the user's presence in an environment that is either a place in the real world or in an imaginary world. Computer-generated VR (CG VR) is a subset of this technology in which the virtual world is exclusively generated via computer graphics. The user views this virtual environment by donning a headset such as HTC Vive, Oculus Rift, Sony PlayStation VR, Samsung Gear VR or Google Daydream View. The headset displays the content directly in front of the user's eyes; the system also tracks the movement of the headset in the real world, so that the virtual camera's movements can be perfectly matched to those of the person wearing the headset. The user typically holds devices in his or her hands which allow the system to track the movements of each hand. This allows the user to interact in the virtual world: Objects can be pushed, picked up or thrown, for example.

1.4.8.2 Augmented and Mixed Reality

The terms *augmented reality* (AR) and *mixed reality* (MR) are often confused or used interchangeably. Both technologies present the user with a view of the real world, but with computer graphics used to enhance the experience. In both technologies, a viewing device like a smart phone, tablet or tech-enhanced pair of glasses displays a real-time or static view of a real-world scene, and computer graphics are overlaid on top of this image. In real-time AR and MR systems, accelerometers in the viewing device permit the virtual camera's movements to track the movements of the device, producing the illusion that the device is simply a window through which we are viewing the actual world, and hence giving the overlaid computer graphics a strong sense of realism.

Some people make a distinction between these two technologies by using the term "augmented reality" to describe technologies in which computer graphics are overlaid on a live, direct or indirect view of the real world, but are not anchored to it. The term "mixed reality," on the other hand, is more often

applied to the use of computer graphics to render imaginary objects which are anchored to the real world and appear to exist within it. However, this distinction is by no means universally accepted.

Here are a few examples of AR technology in action:

- The U.S. Army provides its soldiers with improved tactical awareness using a system dubbed “tactical augmented reality” (TAR)—it overlays a video-game-like heads-up display (HUD) complete with a mini-map and object markers onto the soldier’s view of the real world (<https://youtu.be/x8p19j8C6VI>).
- In 2015, Disney demonstrated some cool AR technology that renders a 3D cartoon character on top of a sheet of real-world paper on which a 2D version of the character is colored with a crayon (<https://youtu.be/SWzurBQ81CM>).
- PepsiCo also pranked commuting Londoners with an AR-enabled bus stop. People sitting in the bus stop enclosure were treated to AR images of a prowling tiger, a meteor crashing, and an alien tentacle grabbing unwitting passers by off the street (<https://youtu.be/Go9rf9GmYpM>).

And here are a few examples of MR:

- Starting with Android 8.1, the camera app on the Pixel 1 and Pixel 2 supports *AR Stickers*, a fun feature that allows users to place animated 3D objects and characters into videos and photos.
- Microsoft’s HoloLens is another example of mixed reality. It overlays world-anchored graphics onto a live video image, and can be used for a wide range of applications including education and training, engineering, health care, and entertainment.

1.4.8.3 VR/AR/MR Games

The game industry is currently experimenting with VR and AR/MR technologies, and is trying to find its footing within these new media. Some traditional 3D games have been “ported” to VR, yielding very interesting, if not particularly innovative, experiences. But perhaps more exciting, entirely new game genres are starting to emerge, offering gameplay experiences that could not be achieved without VR or AR/MR.

For example, *Job Simulator* by Owlchemy Labs plunges the user into a virtual job museum run by robots, and asks them to perform tongue-in-cheek approximations of various real-world jobs, making use of game mechanics

that simply wouldn't work on a non-VR platform. Owlchemy's next installment, *Vacation Simulator*, applies the same whimsical sense of humour and art style to a world in which the robots of *Job Simulator* invite the player to relax and perform various tasks. Figure 1.15 shows a screenshot from another innovative (and somewhat disturbing!) game for HTC Vive called *Accounting*, from the creators of "Rick & Morty" and *The Stanley Parable*.

1.4.8.4 VR Game Engines

VR game engines are technologically similar in many respects to first-person shooter engines, and in fact many FPS-capable engines such as Unity and Unreal Engine support VR "out of the box." However, VR games differ from FPS games in a number of significant ways:

- *Stereoscopic rendering.* A VR game needs to render the scene twice, once for each eye. This doubles the number of graphics primitives that must be rendered, although other aspects of the graphics pipeline such as visibility culling can be performed only once per frame, since the eyes are reasonably close together. As such, a VR game isn't quite as expensive to render as the same game would be to render in split-screen multiplayer mode, but the principle of rendering each frame twice from two (slightly) different virtual cameras is the same.
- *Very high frame rate.* Studies have shown that VR running at below 90



Figure 1.15. *Accounting* by Squanchtendo and Crows Crows Crows (HTC Vive). (See Color Plate XIV.)

frames per second is likely to induce disorientation, nausea, and other negative user effects. This means that not only do VR systems need to render the scene twice per frame, they need to do so at 90+ FPS. This is why VR games and applications are generally required to run on high-powered CPU and GPU hardware.

- *Navigation issues.* In an FPS game, the player can simply walk around the game world with the joypad or the WASD keys. In a VR game, a small amount of movement can be realized by the user physically walking around in the real world, but the safe physical play area is typically quite small (the size of a small bathroom or closet). Travelling by “flying” tends to induce nausea as well, so most games opt for a point-and-click teleportation mechanism to move the virtual player/camera across larger distances. Various real-world devices have also been conceived that allow a VR user to “walk” in place with their feet in order to move around in a VR world.

Of course, VR makes up for these limitations somewhat by enabling new user interaction paradigms that aren’t possible in traditional video games. For example,

- users can reach in the real world to touch, pick up and throw objects in the virtual world;
- a player can dodge an attack in the virtual world by dodging physically in the real world;
- new user interface opportunities are possible, such as having floating menus attached to one’s virtual hands, or seeing a game’s credits written on a whiteboard in the virtual world;
- a player can even pick up a pair of virtual VR goggles and place them onto his or her head, thereby transporting them into a “nested” VR world—an effect that might best be called “VR-ception.”

1.4.8.5 Location-Based Entertainment

Games like *Pokémon Go* neither overlay graphics onto an image of the real world, nor do they generate a completely immersive virtual world. However, the user’s view of the computer-generated world of *Pokémon Go* does react to movements of the user’s phone or tablet, much like a 360-degree video. And the game is aware of your actual location in the real world, prompting you to go searching for Pokémon in nearby parks, malls and restaurants. This kind of game can’t really be called AR/MR, but neither does it fall into the VR category. Such a game might be better described as a form of *location-based*

entertainment, although some people do use the AR moniker for these kinds of games.

1.4.9 Other Genres

There are of course many other game genres which we won't cover in depth here. Some examples include:

- sports, with subgenres for each major sport (football, baseball, soccer, golf, etc.);
- role-playing games (RPG);
- God games, like *Populous* and *Black & White*;
- environmental/social simulation games, like *SimCity* or *The Sims*;
- puzzle games like *Tetris*;
- conversions of non-electronic games, like chess, card games, go, etc.;
- web-based games, such as those offered at Electronic Arts' Pogo site;

and the list goes on.

We have seen that each game genre has its own particular technological requirements. This explains why game engines have traditionally differed quite a bit from genre to genre. However, there is also a great deal of technological overlap between genres, especially within the context of a single hardware platform. With the advent of more and more powerful hardware, differences between genres that arose because of optimization concerns are beginning to evaporate. It is therefore becoming increasingly possible to reuse the same engine technology across disparate genres, and even across disparate hardware platforms.

1.5 Game Engine Survey

1.5.1 The Quake Family of Engines

The first 3D first-person shooter (FPS) game is generally accepted to be *Castle Wolfenstein 3D* (1992). Written by id Software of Texas for the PC platform, this game led the game industry in a new and exciting direction. id Software went on to create *Doom*, *Quake*, *Quake II* and *Quake III*. All of these engines are very similar in architecture, and I will refer to them as the Quake family of engines. Quake technology has been used to create many other games and even other engines. For example, the lineage of *Medal of Honor* for the PC platform goes something like this:

- *Quake III* (id Software);
- *Sin* (Ritual);
- *F.A.K.K. 2* (Ritual);
- *Medal of Honor: Allied Assault* (2015 & Dreamworks Interactive); and
- *Medal of Honor: Pacific Assault* (Electronic Arts, Los Angeles).

Many other games based on Quake technology follow equally circuitous paths through many different games and studios. In fact, Valve's Source engine (used to create the *Half-Life* games) also has distant roots in Quake technology.

The *Quake* and *Quake II* source code is freely available, and the original Quake engines are reasonably well architected and "clean" (although they are of course a bit outdated and written entirely in C). These code bases serve as great examples of how industrial-strength game engines are built. The full source code to *Quake* and *Quake II* is available at <https://github.com/id-Software/Quake-2>.

If you own the Quake and/or Quake II games, you can actually build the code using Microsoft Visual Studio and run the game under the debugger using the real game assets from the disk. This can be incredibly instructive. You can set breakpoints, run the game and then analyze how the engine actually works by stepping through the code. I highly recommend downloading one or both of these engines and analyzing the source code in this manner.

1.5.2 Unreal Engine

Epic Games, Inc. burst onto the FPS scene in 1998 with its legendary game *Unreal*. Since then, the Unreal Engine has become a major competitor to Quake technology in the FPS space. Unreal Engine 2 (UE2) is the basis for *Unreal Tournament 2004* (UT2004) and has been used for countless "mods," university projects and commercial games. Unreal Engine 4 (UE4) is the latest evolutionary step, boasting some of the best tools and richest engine feature sets in the industry, including a convenient and powerful graphical user interface for creating shaders and a graphical user interface for game logic programming called *Blueprints* (previously known as Kismet).

The Unreal Engine has become known for its extensive feature set and cohesive, easy-to-use tools. The Unreal Engine is not perfect, and most developers modify it in various ways to run their game optimally on a particular hardware platform. However, Unreal is an incredibly powerful prototyping tool and commercial game development platform, and it can be used to build virtually any 3D first-person or third-person game (not to mention games in

other genres as well). Many exciting games in all sorts of genres have been developed with UE4, including *Rime* by Tequila Works, *Genesis: Alpha One* by Radiation Blue, *A Way Out* by Hazelight Studios, and *Crackdown 3* by Microsoft Studios.

The Unreal Developer Network (UDN) provides a rich set of documentation and other information about all released versions of the Unreal Engine (see <http://udn.epicgames.com/Main/WebHome.html>). Some documentation is freely available. However, access to the full documentation for the latest version of the Unreal Engine is generally restricted to licensees of the engine. There are plenty of other useful websites and wikis that cover the Unreal Engine. One popular one is <http://www.beyondunreal.com>.

Thankfully, Epic now offers full access to Unreal Engine 4, source code and all, for a low monthly subscription fee plus a cut of your game's profits if it ships. This makes UE4 a viable choice for small independent game studios.

1.5.3 The Half-Life Source Engine

Source is the game engine that drives the well-known *Half-Life 2* and its sequels *HL2: Episode One* and *HL2: Episode Two*, *Team Fortress 2* and *Portal* (shipped together under the title *The Orange Box*). Source is a high-quality engine, rivaling Unreal Engine 4 in terms of graphics capabilities and tool set.

1.5.4 DICE's Frostbite

The Frostbite engine grew out of DICE's efforts to create a game engine for *Battlefield Bad Company* in 2006. Since then, the Frostbite engine has become the most widely adopted engine within Electronic Arts (EA); it is used by many of EA's key franchises including *Mass Effect*, *Battlefield*, *Need for Speed*, *Dragon Age*, and *Star Wars Battlefront II*. Frostbite boasts a powerful unified asset creation tool called FrostEd, a powerful tools pipeline known as Backend Services, and a powerful runtime game engine. It is a proprietary engine, so it's unfortunately unavailable for use by developers outside EA.

1.5.5 Rockstar Advanced Game Engine (RAGE)

RAGE is the engine that drives the insanely popular *Grand Theft Auto V*. Developed by RAGE Technology Group, a division of Rockstar Games' Rockstar San Diego studio, RAGE has been used by Rockstar Games' internal studios to develop games for PlayStation 4, Xbox One, PlayStation 3, Xbox 360, Wii, Windows, and MacOS. Other games developed on this proprietary engine include *Grand Theft Auto IV*, *Red Dead Redemption* and *Max Payne 3*.

1.5.6 CRYENGINE

Crytek originally developed their powerful game engine known as CRYENGINE as a tech demo for NVIDIA. When the potential of the technology was recognized, Crytek turned the demo into a complete game and *Far Cry* was born. Since then, many games have been made with CRYENGINE including *Crysis*, *Codename Kingdoms*, *Ryse: Son of Rome*, and *Everyone's Gone to the Rapture*. Over the years the engine has evolved into what is now Crytek's latest offering, CRYENGINE V. This powerful game development platform offers a powerful suite of asset-creation tools and a feature-rich runtime engine featuring high-quality real-time graphics. CRYENGINE can be used to make games targeting a wide range of platforms including Xbox One, Xbox 360, PlayStation 4, PlayStation 3, Wii U, Linux, iOS and Android.

1.5.7 Sony's PhyreEngine

In an effort to make developing games for Sony's PlayStation 3 platform more accessible, Sony introduced PhyreEngine at the Game Developer's Conference (GDC) in 2008. As of 2013, PhyreEngine has evolved into a powerful and full-featured game engine, supporting an impressive array of features including advanced lighting and deferred rendering. It has been used by many studios to build over 90 published titles, including thatgamecompany's hits *fIOW*, *Flower* and *Journey*, and Coldwood Interactive's *Unravel*. PhyreEngine now supports Sony's PlayStation 4, PlayStation 3, PlayStation 2, PlayStation Vita and PSP platforms. PhyreEngine gives developers access to the power of the highly parallel Cell architecture on PS3 and the advanced compute capabilities of the PS4, along with a streamlined new world editor and other powerful game development tools. It is available free of charge to any licensed Sony developer as part of the PlayStation SDK.

1.5.8 Microsoft's XNA Game Studio

Microsoft's XNA Game Studio is an easy-to-use and highly accessible game development platform based on the C# language and the Common Language Runtime (CLR), and aimed at encouraging players to create their own games and share them with the online gaming community, much as YouTube encourages the creation and sharing of home-made videos.

For better or worse, Microsoft officially retired XNA in 2014. However, developers can port their XNA games to iOS, Android, Mac OS X, Linux and Windows 8 Metro via an open-source implementation of XNA called MonoGame. For more details, see <https://www.windowscentral.com/xna-dead-long-live-xna>.

1.5.9 Unity

Unity is a powerful cross-platform game development environment and run-time engine supporting a wide range of platforms. Using Unity, developers can deploy their games on mobile platforms (e.g., Apple iOS, Google Android), consoles (Microsoft Xbox 360 and Xbox One, Sony PlayStation 3 and PlayStation 4, and Nintendo Wii, Wii U), handheld gaming platforms (e.g., Playstation Vita, Nintendo Switch), desktop computers (Microsoft Windows, Apple Macintosh and Linux), TV boxes (e.g., Android TV and tvOS) and virtual reality (VR) systems (e.g., Oculus Rift, Steam VR, Gear VR).

Unity's primary design goals are ease of development and cross-platform game deployment. As such, Unity provides an easy-to-use integrated editor environment, in which you can create and manipulate the assets and entities that make up your game world and quickly preview your game in action right there in the editor, or directly on your target hardware. Unity also provides a powerful suite of tools for analyzing and optimizing your game on each target platform, a comprehensive asset conditioning pipeline, and the ability to manage the performance-quality trade-off uniquely on each deployment platform. Unity supports scripting in JavaScript, C# or Boo; a powerful animation system supporting animation retargeting (the ability to play an animation authored for one character on a totally different character); and support for networked multiplayer games.

Unity has been used to create a wide variety of published games, including *Deus Ex: The Fall* by N-Fusion/Eidos Montreal, *Hollow Knight* by Team Cherry, and the subversive retro-style *Cuphead* by StudioMDHR. The Webby Award winning short film *Adam* was rendered in real time using Unity.

1.5.10 Other Commercial Game Engines

There are lots of other commercial game engines out there. Although indie developers may not have the budget to purchase an engine, many of these products have great online documentation and/or wikis that can serve as a great source of information about game engines and game programming in general. For example, check out the Tombstone engine (<http://tombstoneengine.com/>) by Terathon Software, the LeadWerks engine (<https://www.leadwerks.com/>), and HeroEngine by Idea Fabrik, PLC (<http://www.heroengine.com/>).

1.5.11 Proprietary In-House Engines

Many companies build and maintain proprietary in-house game engines. Electronic Arts built many of its RTS games on a proprietary engine called Sage, developed at Westwood Studios. Naughty Dog's *Crash Bandicoot* and

Jak and Daxter franchises were built on a proprietary engine custom tailored to the PlayStation and PlayStation 2. For the *Uncharted* series, Naughty Dog developed a brand new engine custom tailored to the PlayStation 3 hardware. This engine evolved and was ultimately used to create Naughty Dog's *The Last of Us* series on the PlayStation 3 and PlayStation 4, as well as its most recent releases, *Uncharted 4: A Thief's End* and *Uncharted: The Lost Legacy*. And of course, most commercially licensed game engines like Quake, Source, Unreal Engine 4 and CRYENGINE all started out as proprietary in-house engines.

1.5.12 Open Source Engines

Open source 3D game engines are engines built by amateur and professional game developers and provided online for free. The term "open source" typically implies that source code is freely available and that a somewhat open development model is employed, meaning almost anyone can contribute code. Licensing, if it exists at all, is often provided under the Gnu Public License (GPL) or Lesser Gnu Public License (LGPL). The former permits code to be freely used by anyone, as long as their code is also freely available; the latter allows the code to be used even in proprietary for-profit applications. Lots of other free and semi-free licensing schemes are also available for open source projects.

There are a staggering number of open source engines available on the web. Some are quite good, some are mediocre and some are just plain awful! The list of game engines provided online at http://en.wikipedia.org/wiki/List_of_game_engines will give you a feel for the sheer number of engines that are out there. (The list at http://www.worldofleveldesign.com/categories/level_design_tutorials/recommended-game-engines.php is a bit more digestible.) Both of these lists include both open-source and commercial game engines.

OGRE is a well-architected, easy-to-learn and easy-to-use 3D rendering engine. It boasts a fully featured 3D renderer including advanced lighting and shadows, a good skeletal character animation system, a two-dimensional overlay system for heads-up displays and graphical user interfaces, and a post-processing system for full-screen effects like bloom. OGRE is, by its authors' own admission, not a full game engine, but it does provide many of the foundational components required by pretty much any game engine.

Some other well-known open source engines are listed here:

- Panda3D is a script-based engine. The engine's primary interface is the Python custom scripting language. It is designed to make prototyping

3D games and virtual worlds convenient and fast.

- Yake is a game engine built on top of OGRE.
- Crystal Space is a game engine with an extensible modular architecture.
- Torque and Irrlicht are also well-known open-source game engines.
- While not technically open-source, the Lumberyard engine does provide source code to its developers. It is a free cross-platform engine developed by Amazon, and based on the CRYENGINE architecture.

1.5.13 2D Game Engines for Non-programmers

Two-dimensional games have become incredibly popular with the recent explosion of casual web gaming and mobile gaming on platforms like Apple iPhone/iPad and Google Android. A number of popular game/multimedia authoring toolkits have become available, enabling small game studios and independent developers to create 2D games for these platforms. These toolkits emphasize ease of use and allow users to employ a graphical user interface to create a game rather than requiring the use of a programming language. Check out this YouTube video to get a feel for the kinds of games you can create with these toolkits: <https://www.youtube.com/watch?v=3Zq1yo0lxOU>

- *Multimedia Fusion 2* (<http://www.clickteam.com/website/world>) is a 2D game/multimedia authoring toolkit developed by Clickteam. Fusion is used by industry professionals to create games, screen savers and other multimedia applications. Fusion and its simpler counterpart, The Games Factory 2, are also used by educational camps like PlanetBravo (<http://www.planetbravo.com>) to teach kids about game development and programming/logic concepts. Fusion supports the iOS, Android, Flash, and Java platforms.
- *Game Salad Creator* (<http://gamesalad.com/creator>) is another graphical game/multimedia authoring toolkit aimed at non-programmers, similar in many respects to Fusion.
- *Scratch* (<http://scratch.mit.edu>) is an authoring toolkit and graphical programming language that can be used to create interactive demos and simple games. It is a great way for young people to learn about programming concepts such as conditionals, loops and event-driven programming. Scratch was developed in 2003 by the Lifelong Kindergarten group, led by Mitchel Resnick at the MIT Media Lab.

1.6 Runtime Engine Architecture

A game engine generally consists of a tool suite and a runtime component. We'll explore the architecture of the runtime piece first and then get into tool architecture in the following section.

Figure 1.16 shows all of the major runtime components that make up a typical 3D game engine. Yeah, it's *big*! And this diagram doesn't even account for all the tools. Game engines are definitely large software systems.

Like all software systems, game engines are built in *layers*. Normally upper layers depend on lower layers, but not vice versa. When a lower layer depends upon a higher layer, we call this a *circular dependency*. Dependency cycles are to be avoided in any software system, because they lead to undesirable coupling between systems, make the software untestable and inhibit code reuse. This is especially true for a large-scale system like a game engine.

What follows is a brief overview of the components shown in the diagram in Figure 1.16. The rest of this book will be spent investigating each of these components in a great deal more depth and learning how these components are usually integrated into a functional whole.

1.6.1 Target Hardware

The target hardware layer represents the computer system or console on which the game will run. Typical platforms include Microsoft Windows, Linux and MacOS-based PCs; mobile platforms like the Apple iPhone and iPad, Android smart phones and tablets, Sony's PlayStation Vita and Amazon's Kindle Fire (among others); and game consoles like Microsoft's Xbox, Xbox 360 and Xbox One, Sony's PlayStation, PlayStation 2, PlayStation 3 and PlayStation 4, and Nintendo's DS, GameCube, Wii, Wii U and Switch. Most of the topics in this book are platform-agnostic, but we'll also touch on some of the design considerations peculiar to PC or console development, where the distinctions are relevant.

1.6.2 Device Drivers

Device drivers are low-level software components provided by the operating system or hardware vendor. Drivers manage hardware resources and shield the operating system and upper engine layers from the details of communicating with the myriad variants of hardware devices available.

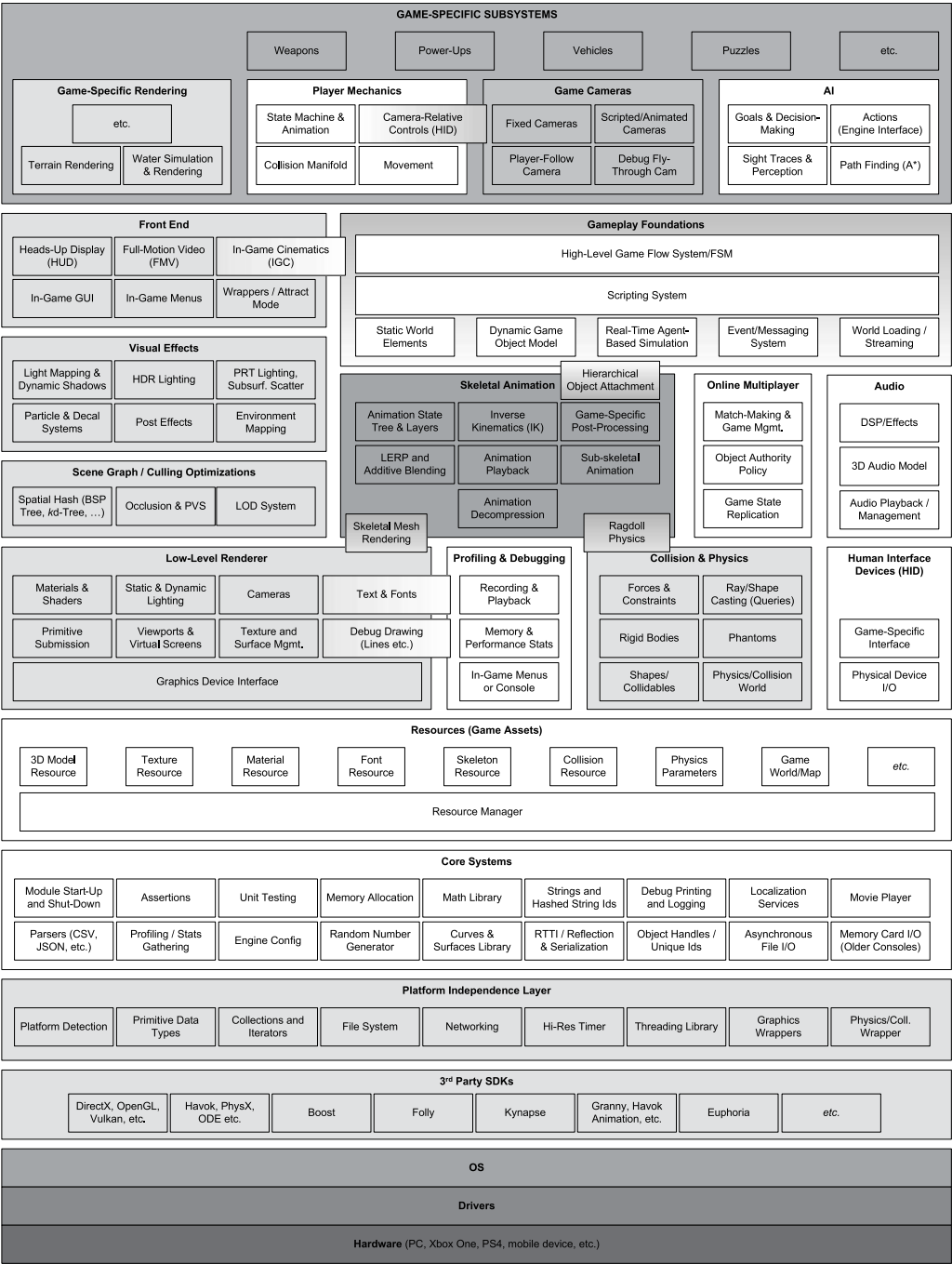


Figure 1.16. Runtime game engine architecture.

1.6.3 Operating System

On a PC, the operating system (OS) is running all the time. It orchestrates the execution of multiple programs on a single computer, one of which is your game. Operating systems like Microsoft Windows employ a time-sliced approach to sharing the hardware with multiple running programs, known as preemptive multitasking. This means that a PC game can never assume it has full control of the hardware—it must “play nice” with other programs in the system.

On early consoles, the operating system, if one existed at all, was just a thin library layer that was compiled directly into your game executable. On those early systems, the game “owned” the entire machine while it was running. However, on modern consoles this is no longer the case. The operating system on the Xbox 360, PlayStation 3, Xbox One and PlayStation 4 can interrupt the execution of your game, or take over certain system resources, in order to display online messages, or to allow the player to pause the game and bring up the PS4’s “XMB” user interface or the Xbox One’s dashboard, for example. On the PS4 and Xbox One, the OS is continually running background tasks, such as recording video of your playthrough in case you decide to share it via the PS4’s Share button, or downloading games, patches and DLC, so you can have fun playing a game while you wait. So the gap between console and PC development is gradually closing (for better or for worse).

1.6.4 Third-Party SDKs and Middleware

Most game engines leverage a number of third-party software development kits (SDKs) and middleware, as shown in Figure 1.17. The functional or class-based interface provided by an SDK is often called an application programming interface (API). We will look at a few examples.



Figure 1.17. Third-party SDK layer.

1.6.4.1 Data Structures and Algorithms

Like any software system, games depend heavily on *container* data structures and algorithms to manipulate them. Here are a few examples of third-party libraries that provide these kinds of services:

- *Boost*. Boost is a powerful data structures and algorithms library, designed in the style of the standard C++ library and its predecessor, the standard template library (STL). (The online documentation for Boost is also a great place to learn about computer science in general!)
- *Folly*. Folly is a library used at Facebook whose goal is to extend the standard C++ library and Boost with all sorts of useful facilities, with an emphasis on maximizing code performance.
- *Loki*. Loki is a powerful generic programming template library which is exceedingly good at making your brain hurt!

The C++ Standard Library and STL

The C++ standard library also provides many of the same kinds of facilities found in third-party libraries like Boost. The subset of the standard library that implements generic container classes such as `std::vector` and `std::list` is often referred to as the *standard template library* (STL), although this is technically a bit of a misnomer: The standard template library was written by Alexander Stepanov and David Musser in the days before the C++ language was standardized. Much of this library's functionality was absorbed into what is now the C++ standard library. When we use the term STL in this book, it's usually in the context of the subset of the C++ standard library that provides generic container classes, not the original STL.

1.6.4.2 Graphics

Most game rendering engines are built on top of a hardware interface library, such as the following:

- *Glide* is the 3D graphics SDK for the old Voodoo graphics cards. This SDK was popular prior to the era of hardware transform and lighting (hardware T&L) which began with DirectX 7.
- *OpenGL* is a widely used portable 3D graphics SDK.
- *DirectX* is Microsoft's 3D graphics SDK and primary rival to OpenGL.
- *libgcm* is a low-level direct interface to the PlayStation 3's RSX graphics hardware, which was provided by Sony as a more efficient alternative to OpenGL.
- *Edge* is a powerful and highly efficient rendering and animation engine produced by Naughty Dog and Sony for the PlayStation 3 and used by a number of first- and third-party game studios.

- *Vulkan* is a low-level library created by the Khronos™ Group which enables game programmers to submit rendering batches and GPGPU compute jobs directly to the GPU as command lists, and provides them with fine-grained control over memory and other resources that are shared between the CPU and GPU. (See Section 4.11 for more on GPGPU programming.)

1.6.4.3 Collision and Physics

Collision detection and rigid body dynamics (known simply as “physics” in the game development community) are provided by the following well-known SDKs:

- *Havok* is a popular industrial-strength physics and collision engine.
- *PhysX* is another popular industrial-strength physics and collision engine, available for free download from NVIDIA.
- *Open Dynamics Engine (ODE)* is a well-known open source physics/collision package.

1.6.4.4 Character Animation

A number of commercial animation packages exist, including but certainly not limited to the following:

- *Granny*. Rad Game Tools’ popular Granny toolkit includes robust 3D model and animation exporters for all the major 3D modeling and animation packages like Maya, 3D Studio MAX, etc., a runtime library for reading and manipulating the exported model and animation data, and a powerful runtime animation system. In my opinion, the Granny SDK has the best-designed and most logical animation API of any I’ve seen, commercial or proprietary, especially its excellent handling of time.
- *Havok Animation*. The line between physics and animation is becoming increasingly blurred as characters become more and more realistic. The company that makes the popular Havok physics SDK decided to create a complimentary animation SDK, which makes bridging the physics-animation gap much easier than it ever has been.
- *OrbisAnim*. The OrbisAnim library produced for the PS4 by SN Systems in conjunction with the ICE and game teams at Naughty Dog, the Tools and Technology group of Sony Interactive Entertainment, and Sony’s Advanced Technology Group in Europe includes a powerful and efficient animation engine and an efficient geometry-processing engine for rendering.

1.6.4.5 Biomechanical Character Models

- *Endorphin and Euphoria*. These are animation packages that produce character motion using advanced biomechanical models of realistic human movement.

As we mentioned previously, the line between character animation and physics is beginning to blur. Packages like Havok Animation try to marry physics and animation in a traditional manner, with a human animator providing the majority of the motion through a tool like Maya and with physics augmenting that motion at runtime. But a firm called Natural Motion Ltd. has produced a product that attempts to redefine how character motion is handled in games and other forms of digital media.

Its first product, *Endorphin*, is a Maya plug-in that permits animators to run full biomechanical simulations on characters and export the resulting animations as if they had been hand animated. The biomechanical model accounts for center of gravity, the character's weight distribution, and detailed knowledge of how a real human balances and moves under the influence of gravity and other forces.

Its second product, *Euphoria*, is a real-time version of *Endorphin* intended to produce physically and biomechanically accurate character motion at runtime under the influence of unpredictable forces.

1.6.5 Platform Independence Layer

Most game engines are required to be capable of running on more than one hardware platform. Companies like Electronic Arts and ActivisionBlizzard Inc., for example, always target their games at a wide variety of platforms because it exposes their games to the largest possible market. Typically, the only game studios that do not target at least two different platforms per game are first-party studios, like Sony's Naughty Dog and Insomniac studios. Therefore, most game engines are architected with a platform independence layer, like the one shown in Figure 1.18. This layer sits atop the hardware, drivers, operating system and other third-party software and shields the rest of the engine from the majority of knowledge of the underlying platform by "wrapping" certain interface functions in custom functions over which you, the game developer, will have control on every target platform.

There are two primary reasons to "wrap" functions as part of your game engine's platform independence layer like this: First, some application programming interfaces (APIs), like those provided by the operating system, or even some functions in older "standard" libraries like the C standard library,

differ significantly from platform to platform; wrapping these functions provides the rest of your engine with a consistent API across all of your targeted platforms. Second, even when using a fully cross-platform library such as Havok, you might want to insulate yourself from future changes, such as transitioning your engine to a different collision/physics library in the future.

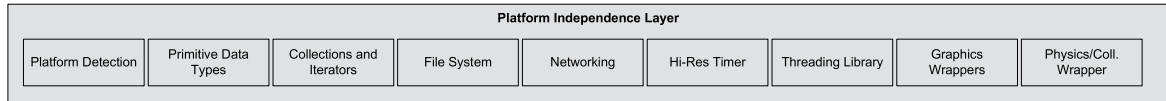


Figure 1.18. Platform independence layer.

1.6.6 Core Systems

Every game engine, and really every large, complex C++ software application, requires a grab bag of useful software utilities. We'll categorize these under the label "core systems." A typical core systems layer is shown in Figure 1.19. Here are a few examples of the facilities the core layer usually provides:

- *Assertions* are lines of error-checking code that are inserted to catch logical mistakes and violations of the programmer's original assumptions. Assertion checks are usually stripped out of the final production build of the game. (Assertions are covered in Section 3.2.3.3.)
- *Memory management*. Virtually every game engine implements its own custom memory allocation system(s) to ensure high-speed allocations and deallocations and to limit the negative effects of memory fragmentation (see Section 6.2.1).
- *Math library*. Games are by their nature highly mathematics-intensive. As such, every game engine has at least one, if not many, math libraries. These libraries provide facilities for vector and matrix math, quaternion rotations, trigonometry, geometric operations with lines, rays, spheres, frusta, etc., spline manipulation, numerical integration, solving systems of equations and whatever other facilities the game programmers require.
- *Custom data structures and algorithms*. Unless an engine's designers decided to rely entirely on third-party packages such as Boost and Folly, a suite of tools for managing fundamental data structures (linked lists, dynamic arrays, binary trees, hash maps, etc.) and algorithms (search, sort, etc.) is usually required. These are often hand coded to minimize or eliminate dynamic memory allocation and to ensure optimal runtime performance on the target platform(s).

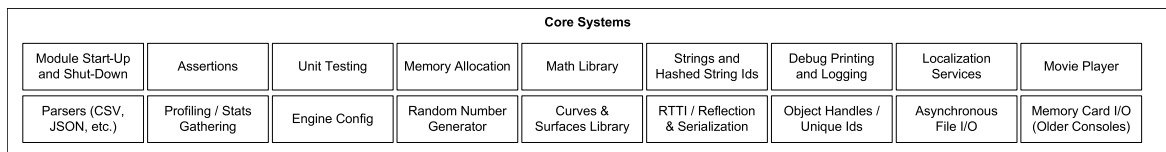


Figure 1.19. Core engine systems.

A detailed discussion of the most common core engine systems can be found in Part II.

1.6.7 Resource Manager

Present in every game engine in some form, the resource manager provides a unified interface (or suite of interfaces) for accessing any and all types of game assets and other engine input data. Some engines do this in a highly centralized and consistent manner (e.g., Unreal's packages, OGRE's Resource-Manager class). Other engines take an ad hoc approach, often leaving it up to the game programmer to directly access raw files on disk or within compressed archives such as Quake's PAK files. A typical resource manager layer is depicted in Figure 1.20.

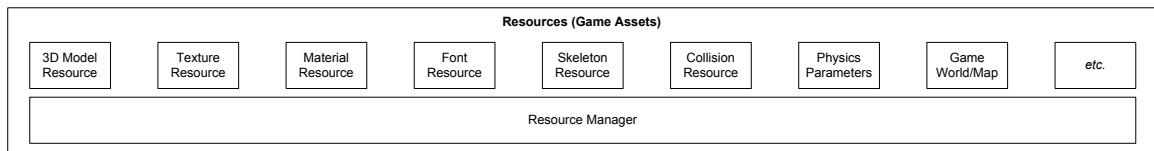


Figure 1.20. Resource manager.

1.6.8 Rendering Engine

The rendering engine is one of the largest and most complex components of any game engine. Renderers can be architected in many different ways. There is no one accepted way to do it, although as we'll see, most modern rendering engines share some fundamental design philosophies, driven in large part by the design of the 3D graphics hardware upon which they depend.

One common and effective approach to rendering engine design is to employ a layered architecture as follows.

1.6.8.1 Low-Level Renderer

The *low-level renderer*, shown in Figure 1.21, encompasses all of the raw rendering facilities of the engine. At this level, the design is focused on rendering a collection of geometric primitives as quickly and richly as possible, without much regard for which portions of a scene may be visible. This component is broken into various subcomponents, which are discussed below.

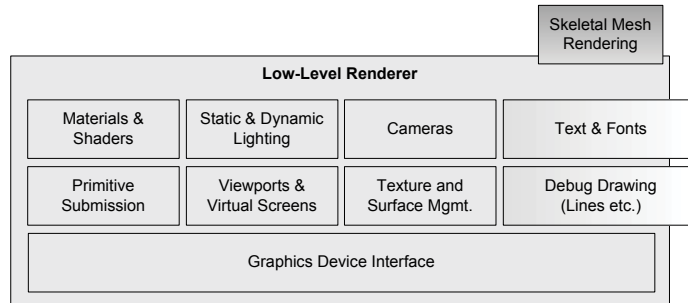


Figure 1.21. Low-level rendering engine.

Graphics Device Interface

Graphics SDKs, such as DirectX, OpenGL or Vulkan, require a reasonable amount of code to be written just to enumerate the available graphics devices, initialize them, set up render surfaces (back-buffer, stencil buffer, etc.) and so on. This is typically handled by a component that I'll call the *graphics device interface* (although every engine uses its own terminology).

For a PC game engine, you also need code to integrate your renderer with the Windows message loop. You typically write a "message pump" that services Windows messages when they are pending and otherwise runs your render loop over and over as fast as it can. This ties the game's keyboard polling loop to the renderer's screen update loop. This coupling is undesirable, but with some effort it is possible to minimize the dependencies. We'll explore this topic in more depth later.

Other Renderer Components

The other components in the low-level renderer cooperate in order to collect submissions of *geometric primitives* (sometimes called *render packets*), such as meshes, line lists, point lists, particles, terrain patches, text strings and whatever else you want to draw, and render them as quickly as possible.

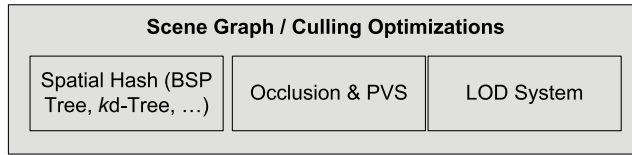


Figure 1.22. A typical scene graph/spatial subdivision layer, for culling optimization.

The low-level renderer usually provides a viewport abstraction with an associated camera-to-world matrix and 3D projection parameters, such as field of view and the location of the near and far clip planes. The low-level renderer also manages the state of the graphics hardware and the game's shaders via its *material system* and its *dynamic lighting system*. Each submitted primitive is associated with a material and is affected by n dynamic lights. The material describes the texture(s) used by the primitive, what device state settings need to be in force, and which vertex and pixel shader to use when rendering the primitive. The lights determine how dynamic lighting calculations will be applied to the primitive. Lighting and shading is a complex topic. We'll discuss the fundamentals in Chapter 11, but these topics are covered in depth in many excellent books on computer graphics, including [16], [49] and [2].

1.6.8.2 Scene Graph/Culling Optimizations

The low-level renderer draws all of the geometry submitted to it, without much regard for whether or not that geometry is actually visible (other than back-face culling and clipping triangles to the camera frustum). A higher-level component is usually needed in order to limit the number of primitives submitted for rendering, based on some form of visibility determination. This layer is shown in Figure 1.22.

For very small game worlds, a simple *frustum cull* (i.e., removing objects that the camera cannot "see") is probably all that is required. For larger game worlds, a more advanced *spatial subdivision* data structure might be used to improve rendering efficiency by allowing the potentially visible set (PVS) of objects to be determined very quickly. Spatial subdivisions can take many forms, including a binary space partitioning tree, a quadtree, an octree, a *kd-tree* or a sphere hierarchy. A spatial subdivision is sometimes called a scene graph, although technically the latter is a particular kind of data structure and does not subsume the former. Portals or occlusion culling methods might also be applied in this layer of the rendering engine.

Ideally, the low-level renderer should be completely agnostic to the type of spatial subdivision or scene graph being used. This permits different game

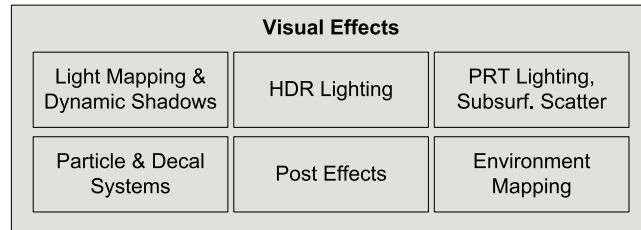


Figure 1.23. Visual effects.

teams to reuse the primitive submission code but to craft a PVS determination system that is specific to the needs of each team's game. The design of the OGRE open source rendering engine (<http://www.ogre3d.org>) is a great example of this principle in action. OGRE provides a plug-and-play scene graph architecture. Game developers can either select from a number of preimplemented scene graph designs, or they can provide a custom scene graph implementation.

1.6.8.3 Visual Effects

Modern game engines support a wide range of visual effects, as shown in Figure 1.23, including:

- particle systems (for smoke, fire, water splashes, etc.);
- decal systems (for bullet holes, foot prints, etc.);
- light mapping and environment mapping;
- dynamic shadows; and
- full-screen post effects, applied after the 3D scene has been rendered to an off-screen buffer.

Some examples of full-screen post effects include:

- high dynamic range (HDR) tone mapping and bloom;
- full-screen anti-aliasing (FSAA); and
- color correction and color-shift effects, including bleach bypass, saturation and desaturation effects, etc.

It is common for a game engine to have an *effects system* component that manages the specialized rendering needs of particles, decals and other visual effects. The particle and decal systems are usually distinct components of the rendering engine and act as inputs to the low-level renderer. On the other

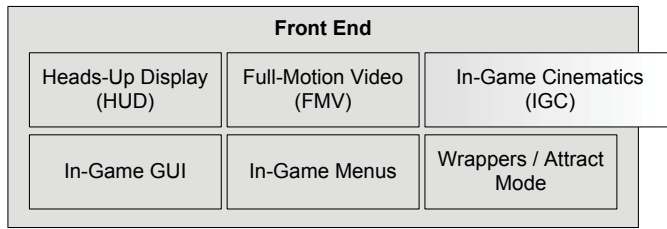


Figure 1.24. Front end graphics.

hand, light mapping, environment mapping and shadows are usually handled internally within the rendering engine proper. Full-screen post effects are either implemented as an integral part of the renderer or as a separate component that operates on the renderer's output buffers.

1.6.8.4 Front End

Most games employ some kind of 2D graphics overlaid on the 3D scene for various purposes. These include:

- the game's *heads-up display* (HUD);
- in-game menus, a console and /or other *development tools*, which may or may not be shipped with the final product; and
- possibly an in-game *graphical user interface* (GUI), allowing the player to manipulate his or her character's inventory, configure units for battle or perform other complex in-game tasks.

This layer is shown in Figure 1.24. Two-dimensional graphics like these are usually implemented by drawing textured quads (pairs of triangles) with an orthographic projection. Or they may be rendered in full 3D, with the quads bill-boarded so they always face the camera.

We've also included the *full-motion video* (FMV) system in this layer. This system is responsible for playing full-screen movies that have been recorded earlier (either rendered with the game's rendering engine or using another rendering package).

A related system is the *in-game cinematics* (IGC) system. This component typically allows cinematic sequences to be choreographed within the game itself, in full 3D. For example, as the player walks through a city, a conversation between two key characters might be implemented as an in-game cinematic. IGCs may or may not include the player character(s). They may be done as a deliberate cut-away during which the player has no control, or they may be subtly integrated into the game without the human player even realizing that

an IGC is taking place. Some games, such as Naughty Dog's *Uncharted 4: A Thief's End*, have moved away from pre-rendered movies entirely, and display *all* cinematic moments in the game as real-time IGCs.

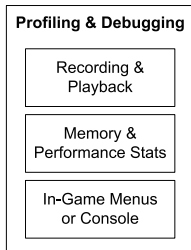


Figure 1.25. Profiling and debugging tools.

1.6.9 Profiling and Debugging Tools

Games are real-time systems and, as such, game engineers often need to profile the performance of their games in order to optimize performance. In addition, memory resources are usually scarce, so developers make heavy use of memory analysis tools as well. The profiling and debugging layer, shown in Figure 1.25, encompasses these tools and also includes in-game debugging facilities, such as debug drawing, an in-game menu system or console and the ability to record and play back gameplay for testing and debugging purposes.

There are plenty of good general-purpose software profiling tools available, including:

- Intel's *VTune*,
- IBM's *Quantify* and *Purify* (part of the *PurifyPlus* tool suite),
- *Insure++* by Parasoft, and
- *Valgrind* by Julian Seward and the Valgrind development team.

However, most game engines also incorporate a suite of custom profiling and debugging tools. For example, they might include one or more of the following:

- a mechanism for manually instrumenting the code, so that specific sections of code can be timed;
- a facility for displaying the profiling statistics on-screen while the game is running;
- a facility for dumping performance stats to a text file or to an Excel spreadsheet;
- a facility for determining how much memory is being used by the engine, and by each subsystem, including various on-screen displays;
- the ability to dump memory usage, high water mark and leakage stats when the game terminates and/or during gameplay;
- tools that allow debug print statements to be peppered throughout the code, along with an ability to turn on or off different categories of debug output and control the level of verbosity of the output; and

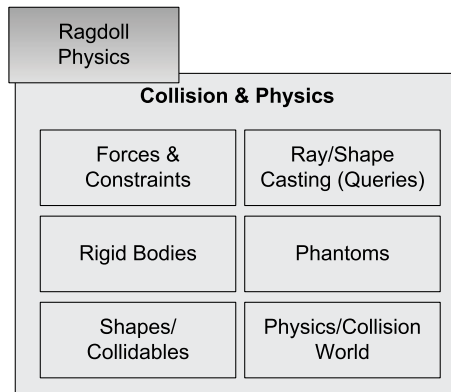


Figure 1.26. Collision and physics subsystem.

- the ability to record game events and then play them back. This is tough to get right, but when done properly it can be a very valuable tool for tracking down bugs.

The PlayStation 4 provides a powerful core dump facility to aid programmers in debugging crashes. The PlayStation 4 is always recording the last 15 seconds of gameplay video, to allow players to share their experiences via the Share button on the controller. Because of this, the PS4's core dump facility automatically provides programmers not only with a complete call stack of what the program was doing when it crashed, but also with a screenshot of the moment of the crash and 15 seconds of video footage showing what was happening just prior to the crash. Core dumps can be automatically uploaded to the game developer's servers whenever the game crashes, even after the game has shipped. These facilities revolutionize the tasks of crash analysis and repair.

1.6.10 Collision and Physics

Collision detection is important for every game. Without it, objects would interpenetrate, and it would be impossible to interact with the virtual world in any reasonable way. Some games also include a realistic or semi-realistic dynamics simulation. We call this the "physics system" in the game industry, although the term *rigid body dynamics* is really more appropriate, because we are usually only concerned with the motion (kinematics) of rigid bodies and the forces and torques (dynamics) that cause this motion to occur. This layer is depicted in Figure 1.26.

Collision and physics are usually quite tightly coupled. This is because when collisions are detected, they are almost always resolved as part of the physics integration and constraint satisfaction logic. Nowadays, very few game companies write their own collision/physics engine. Instead, a third-party SDK is typically integrated into the engine.

- *Havok* is the gold standard in the industry today. It is feature-rich and performs well across the boards.
- *PhysX* by NVIDIA is another excellent collision and dynamics engine. It was integrated into Unreal Engine 4 and is also available for free as a stand-alone product for PC game development. PhysX was originally designed as the interface to Ageia's physics accelerator chip. The SDK is now owned and distributed by NVIDIA, and the company has adapted PhysX to run on its latest GPUs.

Open source physics and collision engines are also available. Perhaps the best-known of these is the Open Dynamics Engine (ODE). For more information, see <http://www.ode.org>. I-Collide, V-Collide and RAPID are other popular non-commercial collision detection engines. All three were developed at the University of North Carolina (UNC). For more information, see http://www.cs.unc.edu/~geom/I_COLLIDE/index.html and http://www.cs.unc.edu/~geom/V_COLLIDE/index.html.

1.6.11 Animation

Any game that has organic or semi-organic characters (humans, animals, cartoon characters or even robots) needs an animation system. There are five basic types of animation used in games:

- sprite/texture animation,
- rigid body hierarchy animation,
- skeletal animation,
- vertex animation, and
- morph targets.

Skeletal animation permits a detailed 3D character mesh to be posed by an animator using a relatively simple system of bones. As the bones move, the vertices of the 3D mesh move with them. Although morph targets and vertex animation are used in some engines, skeletal animation is the most prevalent animation method in games today; as such, it will be our primary focus in this book. A typical skeletal animation system is shown in Figure 1.27.

You'll notice in Figure 1.16 that the skeletal mesh rendering component bridges the gap between the renderer and the animation system. There is a tight cooperation happening here, but the interface is very well defined. The animation system produces a pose for every bone in the skeleton, and then these poses are passed to the rendering engine as a palette of matrices. The renderer transforms each vertex by the matrix or matrices in the palette, in order to generate a final blended vertex position. This process is known as *skinning*.

There is also a tight coupling between the animation and physics systems when *rag dolls* are employed. A rag doll is a limp (often dead) animated character, whose bodily motion is simulated by the physics system. The physics system determines the positions and orientations of the various parts of the body by treating them as a constrained system of rigid bodies. The animation system calculates the palette of matrices required by the rendering engine in order to draw the character on-screen.

1.6.12 Human Interface Devices (HID)

Every game needs to process input from the player, obtained from various *human interface devices* (HIDs) including:

- the keyboard and mouse,
- a joystick, or
- other specialized game controllers, like steering wheels, fishing rods, dance pads, the Wiimote, etc.

We sometimes call this component the *player I/O* component, because

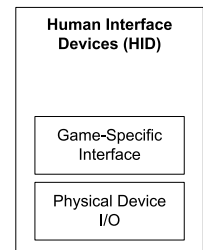


Figure 1.28. The player input/output system, also known as the human interface device (HID) layer.

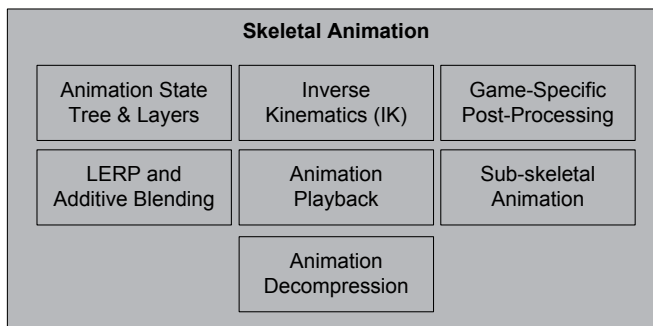


Figure 1.27. Skeletal animation subsystem.

we may also provide *output* to the player through the HID, such as force-feedback/ rumble on a joypad or the audio produced by the Wiimote. A typical HID layer is shown in Figure 1.28.

The HID engine component is sometimes architected to divorce the low-level details of the game controller(s) on a particular hardware platform from the high-level game controls. It massages the raw data coming from the hardware, introducing a dead zone around the center point of each joypad stick, debouncing button-press inputs, detecting button-down and button-up events, interpreting and smoothing accelerometer inputs (e.g., from the PlayStation Dualshock controller) and more. It often provides a mechanism allowing the player to customize the mapping between physical controls and logical game functions. It sometimes also includes a system for detecting chords (multiple buttons pressed together), sequences (buttons pressed in sequence within a certain time limit) and gestures (sequences of inputs from the buttons, sticks, accelerometers, etc.).

1.6.13 Audio

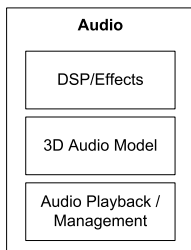


Figure 1.29. Audio subsystem.

Audio is just as important as graphics in any game engine. Unfortunately, audio often gets less attention than rendering, physics, animation, AI and game-play. Case in point: Programmers often develop their code with their speakers turned off! (In fact, I've known quite a few game programmers who didn't even *have* speakers or headphones.) Nonetheless, no great game is complete without a stunning audio engine. The audio layer is depicted in Figure 1.29.

Audio engines vary greatly in sophistication. Quake's audio engine is pretty basic, and game teams usually augment it with custom functionality or replace it with an in-house solution. Unreal Engine 4 provides a reasonably robust 3D audio rendering engine (discussed in detail in [45]), although its feature set is limited and many game teams will probably want to augment and customize it to provide advanced game-specific features. For DirectX platforms (PC, Xbox 360, Xbox One), Microsoft provides an excellent runtime audio engine called XAudio2. Electronic Arts has developed an advanced, high-powered audio engine internally called SoundR!OT. In conjunction with first-party studios like Naughty Dog, Sony Interactive Entertainment (SIE) provides a powerful 3D audio engine called Scream, which has been used on a number of PS3 and PS4 titles including Naughty Dog's *Uncharted 4: A Thief's End* and *The Last of Us: Remastered*. However, even if a game team uses a preexisting audio engine, every game requires a great deal of custom software development, integration work, fine-tuning and attention to detail in order to produce high-quality audio in the final product.

1.6.14 Online Multiplayer/Networking

Many games permit multiple human players to play within a single virtual world. Multiplayer games come in at least four basic flavors:

- *Single-screen multiplayer.* Two or more human interface devices (joypads, keyboards, mice, etc.) are connected to a single arcade machine, PC or console. Multiple player characters inhabit a single virtual world, and a single camera keeps all player characters in frame simultaneously. Examples of this style of multiplayer gaming include *Smash Brothers*, *Lego Star Wars* and *Gauntlet*.
- *Split-screen multiplayer.* Multiple player characters inhabit a single virtual world, with multiple HIDs attached to a single game machine, but each with its own camera, and the screen is divided into sections so that each player can view his or her character.
- *Networked multiplayer.* Multiple computers or consoles are networked together, with each machine hosting one of the players.
- *Massively multiplayer online games (MMOG).* Literally hundreds of thousands of users can be playing simultaneously within a giant, persistent, online virtual world hosted by a powerful battery of central servers.

The multiplayer networking layer is shown in Figure 1.30.

Multiplayer games are quite similar in many ways to their single-player counterparts. However, support for multiple players can have a profound impact on the design of certain game engine components. The game world object model, renderer, human input device system, player control system and animation systems are all affected. Retrofitting multiplayer features into a pre-existing single-player engine is certainly not impossible, although it can be a daunting task. Still, many game teams have done it successfully. That said, it is usually better to design multiplayer features from day one, if you have that luxury.

It is interesting to note that going the other way—converting a multiplayer game into a single-player game—is typically trivial. In fact, many game engines treat single-player mode as a special case of a multiplayer game, in which there happens to be only one player. The Quake engine is well known for its *client-on-top-of-server* mode, in which a single executable, running on a single PC, acts both as the client and the server in single-player campaigns.

1.6.15 Gameplay Foundation Systems

The term *gameplay* refers to the action that takes place in the game, the rules that govern the virtual world in which the game takes place, the abilities of the

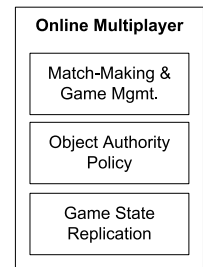


Figure 1.30. Online multiplayer networking subsystem.

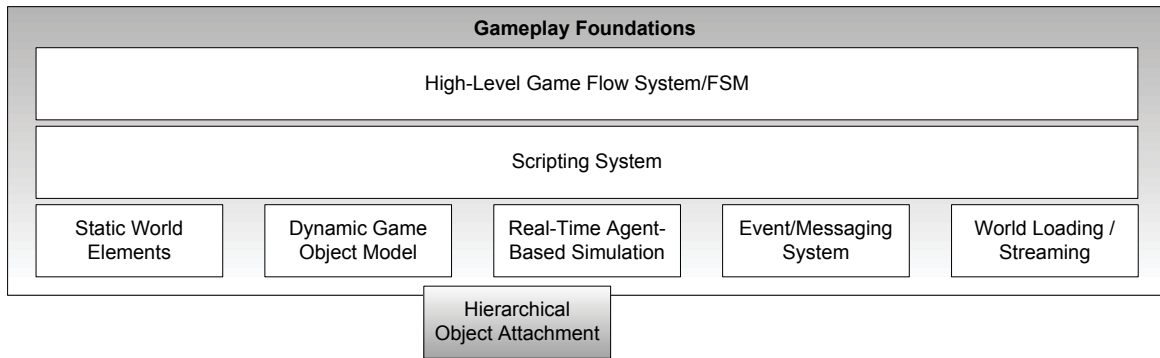


Figure 1.31. Gameplay Foundation systems.

player character(s) (known as *player mechanics*) and of the other characters and objects in the world, and the goals and objectives of the player(s). Gameplay is typically implemented either in the native language in which the rest of the engine is written or in a high-level scripting language—or sometimes both. To bridge the gap between the gameplay code and the low-level engine systems that we’ve discussed thus far, most game engines introduce a layer that I’ll call the *gameplay foundations* layer (for lack of a standardized name). Shown in Figure 1.31, this layer provides a suite of core facilities, upon which game-specific logic can be implemented conveniently.

1.6.15.1 Game Worlds and Object Models

The gameplay foundations layer introduces the notion of a game world, containing both static and dynamic elements. The contents of the world are usually modeled in an object-oriented manner (often, but not always, using an object-oriented programming language). In this book, the collection of object types that make up a game is called the *game object model*. The game object model provides a real-time simulation of a heterogeneous collection of objects in the virtual game world.

Typical types of game objects include:

- static background geometry, like buildings, roads, terrain (often a special case), etc.;
- dynamic rigid bodies, such as rocks, soda cans, chairs, etc.;
- player characters (PC);
- non-player characters (NPC);

- weapons;
- projectiles;
- vehicles;
- lights (which may be present in the dynamic scene at runtime, or only used for static lighting offline);
- cameras;

and the list goes on.

The game world model is intimately tied to a *software object model*, and this model can end up pervading the entire engine. The term software object model refers to the set of language features, policies and conventions used to implement a piece of object-oriented software. In the context of game engines, the software object model answers questions, such as:

- Is your game engine designed in an object-oriented manner?
- What language will you use? C? C++? Java? OCaml?
- How will the static class hierarchy be organized? One giant monolithic hierarchy? Lots of loosely coupled components?
- Will you use templates and policy-based design, or traditional polymorphism?
- How are objects referenced? Straight old pointers? Smart pointers? Handles?
- How will objects be uniquely identified? By address in memory only? By name? By a global unique identifier (GUID)?
- How are the lifetimes of game objects managed?
- How are the states of the game objects simulated over time?

We'll explore software object models and game object models in considerable depth in Section 16.2.

1.6.15.2 Event System

Game objects invariably need to communicate with one another. This can be accomplished in all sorts of ways. For example, the object sending the message might simply call a member function of the receiver object. An event-driven architecture, much like what one would find in a typical graphical user interface, is also a common approach to inter-object communication. In an event-driven system, the sender creates a little data structure called an *event* or *message*, containing the message's type and any argument data that are to be sent. The event is passed to the receiver object by calling its *event handler function*. Events can also be stored in a queue for handling at some future time.

1.6.15.3 Scripting System

Many game engines employ a scripting language in order to make development of game-specific gameplay rules and content easier and more rapid. Without a scripting language, you must recompile and relink your game executable every time a change is made to the logic or data structures used in the engine. But when a scripting language is integrated into your engine, changes to game logic and data can be made by modifying and reloading the script code. Some engines allow script to be reloaded while the game continues to run. Other engines require the game to be shut down prior to script recompilation. But either way, the turnaround time is still much faster than it would be if you had to recompile and relink the game's executable.

1.6.15.4 Artificial Intelligence Foundations

Traditionally, artificial intelligence has fallen squarely into the realm of game-specific software—it was usually not considered part of the game engine per se. More recently, however, game companies have recognized patterns that arise in almost every AI system, and these foundations are slowly starting to fall under the purview of the engine proper.

For example, a company called Kynogon developed a middleware SDK named Kynapse, which provides much of the low-level technology required to build commercially viable game AI. This technology was purchased by Autodesk and has been superseded by a totally redesigned AI middleware package called Gameware Navigation, designed by the same engineering team that invented Kynapse. This SDK provides low-level AI building blocks such as nav mesh generation, path finding, static and dynamic object avoidance, identification of vulnerabilities within a play space (e.g., an open window from which an ambush could come) and a well-defined interface between AI and animation.

1.6.16 Game-Specific Subsystems

On top of the gameplay foundation layer and the other low-level engine components, gameplay programmers and designers cooperate to implement the features of the game itself. Gameplay systems are usually numerous, highly varied and specific to the game being developed. As shown in Figure 1.32, these systems include, but are certainly not limited to the mechanics of the player character, various in-game camera systems, artificial intelligence for the control of non-player characters, weapon systems, vehicles and the list goes on. If a clear line could be drawn between the engine and the game,

it would lie between the game-specific subsystems and the gameplay foundations layer. Practically speaking, this line is never perfectly distinct. At least some game-specific knowledge invariably seeps down through the gameplay foundations layer and sometimes even extends into the core of the engine itself.

1.7 Tools and the Asset Pipeline

Any game engine must be fed a great deal of data, in the form of game assets, configuration files, scripts and so on. Figure 1.33 depicts some of the types of game assets typically found in modern game engines. The thicker dark-grey arrows show how data flows from the tools used to create the original source assets all the way through to the game engine itself. The thinner light-grey arrows show how the various types of assets refer to or use other assets.

1.7.1 Digital Content Creation Tools

Games are multimedia applications by nature. A game engine's input data comes in a wide variety of forms, from 3D mesh data to texture bitmaps to animation data to audio files. All of this source data must be created and manipulated by artists. The tools that the artists use are called *digital content creation* (DCC) applications.

A DCC application is usually targeted at the creation of one particular type of data—although some tools can produce multiple data types. For example, Autodesk's Maya and 3ds Max and Pixologic's ZBrush are prevalent in the creation of both 3D meshes and animation data. Adobe's Photoshop and its ilk are aimed at creating and editing bitmaps (textures). SoundForge is a popular tool for creating audio clips. Some types of game data cannot be created using an off-the-shelf DCC app. For example, most game engines provide a custom editor for laying out game worlds. Still, some engines do make use of

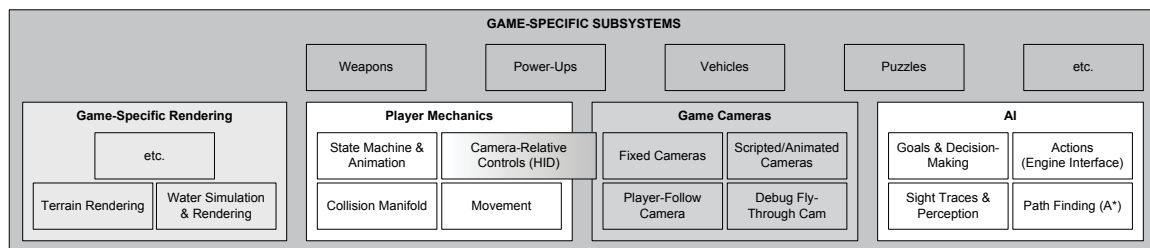


Figure 1.32. Game-specific subsystems.

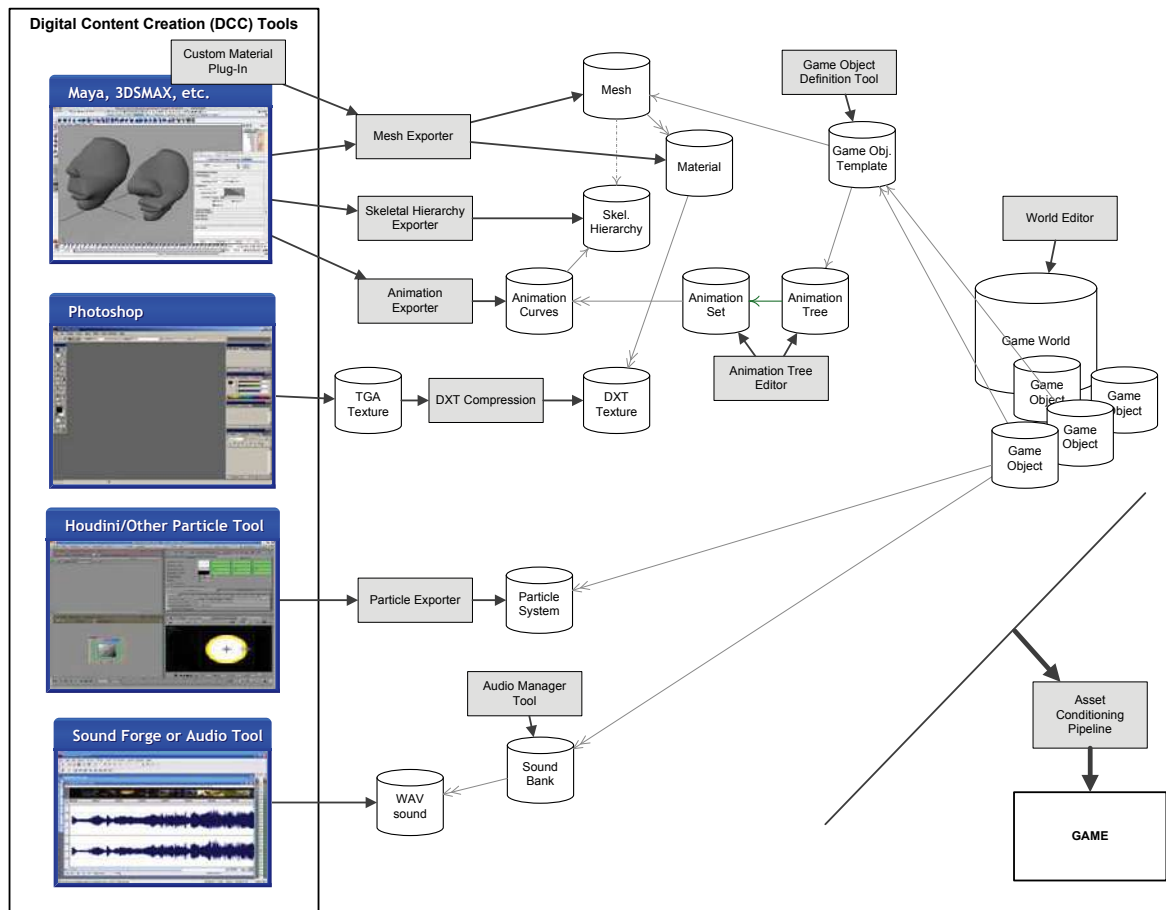


Figure 1.33. Tools and the asset pipeline.

preexisting tools for game world layout. I've seen game teams use 3ds Max or Maya as a world layout tool, with or without custom plug-ins to aid the user. Ask most game developers, and they'll tell you they can remember a time when they laid out terrain height fields using a simple bitmap editor, or typed world layouts directly into a text file by hand. Tools don't have to be pretty—game teams will use whatever tools are available and get the job done. That said, tools must be relatively *easy to use*, and they absolutely must be *reliable*, if a game team is going to be able to develop a highly polished product in a timely manner.

1.7.2 The Asset Conditioning Pipeline

The data formats used by digital content creation (DCC) applications are rarely suitable for direct use in-game. There are two primary reasons for this.

1. The DCC app's in-memory model of the data is usually much more complex than what the game engine requires. For example, Maya stores a directed acyclic graph (DAG) of scene nodes, with a complex web of interconnections. It stores a history of all the edits that have been performed on the file. It represents the position, orientation and scale of every object in the scene as a full hierarchy of 3D transformations, decomposed into translation, rotation, scale and shear components. A game engine typically only needs a tiny fraction of this information in order to render the model in-game.
2. The DCC application's file format is often too slow to read at runtime, and in some cases it is a closed proprietary format.

Therefore, the data produced by a DCC app is usually exported to a more accessible standardized format, or a custom file format, for use in-game.

Once data has been exported from the DCC app, it often must be further processed before being sent to the game engine. And if a game studio is shipping its game on more than one platform, the intermediate files might be processed differently for each target platform. For example, 3D mesh data might be exported to an intermediate format, such as XML, JSON or a simple binary format. Then it might be processed to combine meshes that use the same material, or split up meshes that are too large for the engine to digest. The mesh data might then be organized and packed into a memory image suitable for loading on a specific hardware platform.

The pipeline from DCC app to game engine is sometimes called the *asset conditioning pipeline* (ACP). Every game engine has this in some form.

1.7.2.1 3D Model/Mesh Data

The visible geometry you see in a game is typically constructed from triangle meshes. Some older games also make use of volumetric geometry known as *brushes*. We'll discuss each type of geometric data briefly below. For an in-depth discussion of the techniques used to describe and render 3D geometry, see Chapter 11.

3D Models (Meshes)

A mesh is a complex shape composed of triangles and vertices. Renderable geometry can also be constructed from quads or higher-order subdivision

surfaces. But on today's graphics hardware, which is almost exclusively geared toward rendering rasterized triangles, all shapes must eventually be translated into triangles prior to rendering.

A mesh typically has one or more *materials* applied to it in order to define visual surface properties (color, reflectivity, bumpiness, diffuse texture, etc.). In this book, I will use the term “mesh” to refer to a single renderable shape, and “model” to refer to a composite object that may contain multiple meshes, plus animation data and other metadata for use by the game.

Meshes are typically created in a 3D modeling package such as 3ds Max, Maya or SoftImage. A powerful and popular tool by Pixologic called ZBrush allows ultra high-resolution meshes to be built in a very intuitive way and then down-converted into a lower-resolution model with normal maps to approximate the high-frequency detail.

Exporters must be written to extract the data from the digital content creation (DCC) tool (Maya, Max, etc.) and store it on disk in a form that is digestible by the engine. The DCC apps provide a host of standard or semi-standard export formats, although none are perfectly suited for game development (with the possible exception of COLLADA). Therefore, game teams often create custom file formats and custom exporters to go with them.

Brush Geometry

Brush geometry is defined as a collection of convex hulls, each of which is defined by multiple planes. Brushes are typically created and edited directly in the game world editor. This is essentially an “old school” approach to creating renderable geometry, but it is still used in some engines.

Pros:

- fast and easy to create;
- accessible to game designers—often used to “block out” a game level for prototyping purposes;
- can serve both as collision volumes and as renderable geometry.

Cons:

- low-resolution;
- difficult to create complex shapes;
- cannot support articulated objects or animated characters.

1.7.2.2 Skeletal Animation Data

A *skeletal mesh* is a special kind of mesh that is bound to a skeletal hierarchy for the purposes of articulated animation. Such a mesh is sometimes called a

skin because it forms the skin that surrounds the invisible underlying skeleton. Each vertex of a skeletal mesh contains a list of indices indicating to which joint(s) in the skeleton it is bound. A vertex usually also includes a set of joint weights, specifying the amount of influence each joint has on the vertex.

In order to render a skeletal mesh, the game engine requires three distinct kinds of data:

1. the mesh itself,
2. the skeletal hierarchy (joint names, parent-child relationships and the base pose the skeleton was in when it was originally bound to the mesh), and
3. one or more animation clips, which specify how the joints should move over time.

The mesh and skeleton are often exported from the DCC application as a single data file. However, if multiple meshes are bound to a single skeleton, then it is better to export the skeleton as a distinct file. The animations are usually exported individually, allowing only those animations which are in use to be loaded into memory at any given time. However, some game engines allow a bank of animations to be exported as a single file, and some even lump the mesh, skeleton and animations into one monolithic file.

An unoptimized skeletal animation is defined by a stream of 4×3 matrix samples, taken at a frequency of at least 30 frames per second, for each of the joints in a skeleton (of which there can be 500 or more for a realistic humanoid character). Thus, animation data is inherently memory-intensive. For this reason, animation data is almost always stored in a highly compressed format. Compression schemes vary from engine to engine, and some are proprietary. There is no one standardized format for game-ready animation data.

1.7.2.3 Audio Data

Audio clips are usually exported from Sound Forge or some other audio production tool in a variety of formats and at a number of different data sampling rates. Audio files may be in mono, stereo, 5.1, 7.1 or other multi-channel configurations. Wave files (.wav) are common, but other file formats such as PlayStation ADPCM files (.vag) are also commonplace. Audio clips are often organized into banks for the purposes of organization, easy loading into the engine, and streaming.

1.7.2.4 Particle Systems Data

Modern games make use of complex particle effects. These are authored by artists who specialize in the creation of visual effects. Third-party tools, such as Houdini, permit film-quality effects to be authored; however, most game engines are not capable of rendering the full gamut of effects that can be created with Houdini. For this reason, many game companies create a custom particle effect editing tool, which exposes only the effects that the engine actually supports. A custom tool might also let the artist see the effect exactly as it will appear in-game.

1.7.3 The World Editor

The game world is where everything in a game engine comes together. To my knowledge, there are no commercially available game world editors (i.e., the game world equivalent of Maya or Max). However, a number of commercially available game engines provide good world editors:

- Some variant of the *Radiant* game editor is used by most game engines based on Quake technology.
- The *Half-Life 2* Source engine provides a world editor called *Hammer*.
- *UnrealEd* is the Unreal Engine's world editor. This powerful tool also serves as the asset manager for all data types that the engine can consume.
- *Sandbox* is the world editor in CRYENGINE.

Writing a good world editor is difficult, but it is an extremely important part of any good game engine.

1.7.4 The Resource Database

Game engines deal with a wide range of asset types, from renderable geometry to materials and textures to animation data to audio. These assets are defined in part by the raw data produced by the artists when they use a tool like Maya, Photoshop or SoundForge. However, every asset also carries with it a great deal of *metadata*. For example, when an animator authors an animation clip in Maya, the metadata provides the asset conditioning pipeline, and ultimately the game engine, with the following information:

- A unique id that identifies the animation clip at runtime.
- The name and directory path of the source Maya (.ma or .mb) file.
- The *frame range*—on which frame the animation begins and ends.
- Whether or not the animation is intended to loop.

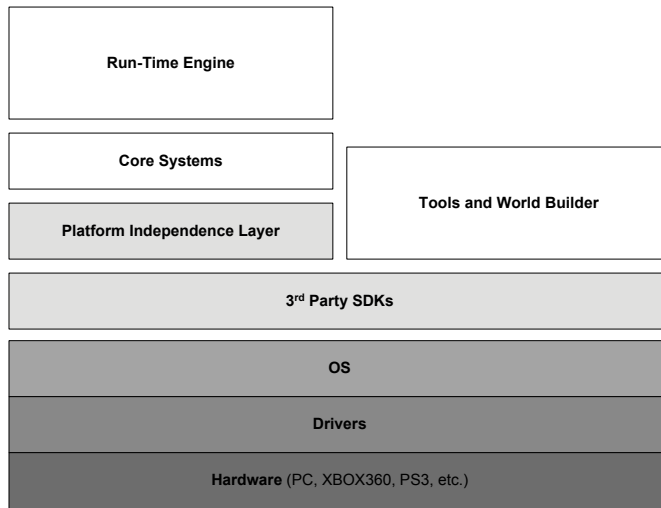


Figure 1.34. Stand-alone tools architecture.

- The animator’s choice of compression technique and level. (Some assets can be highly compressed without noticeably degrading their quality, while others require less or no compression in order to look right in-game.)

Every game engine requires some kind of database to manage all of the metadata associated with the game’s assets. This database might be implemented using an honest-to-goodness relational database such as MySQL or Oracle, or it might be implemented as a collection of text files, managed by a revision control system such as Subversion, Perforce or Git. We’ll call this metadata the *resource database* in this book.

No matter in what format the resource database is stored and managed, some kind of user interface must be provided to allow users to author and edit the data. At Naughty Dog, we wrote a custom GUI in C# called Builder for this purpose. For more information on Builder and a few other resource database user interfaces, see Section 7.2.1.3.

1.7.5 Some Approaches to Tool Architecture

A game engine’s tool suite may be architected in any number of ways. Some tools might be stand-alone pieces of software, as shown in Figure 1.34. Some tools may be built on top of some of the lower layers used by the runtime engine, as Figure 1.35 illustrates. Some tools might be built into the game itself.

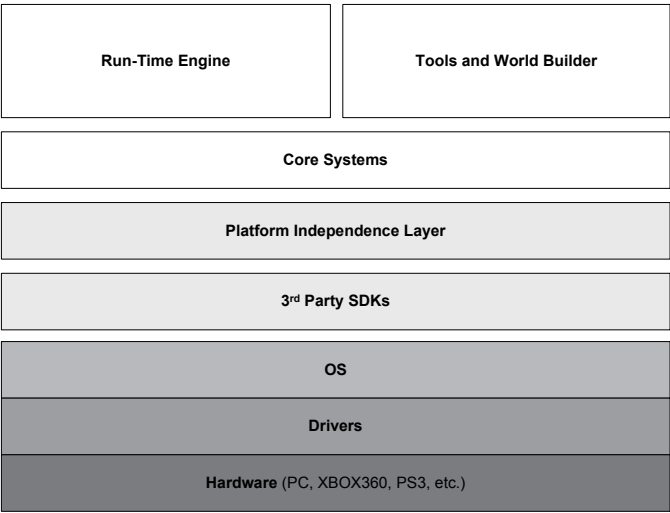


Figure 1.35. Tools built on a framework shared with the game.

For example, Quake- and Unreal-based games both boast an in-game console that permits developers and “modders” to type debugging and configuration commands while running the game. Finally, web-based user interfaces are becoming more and more popular for certain kinds of tools.

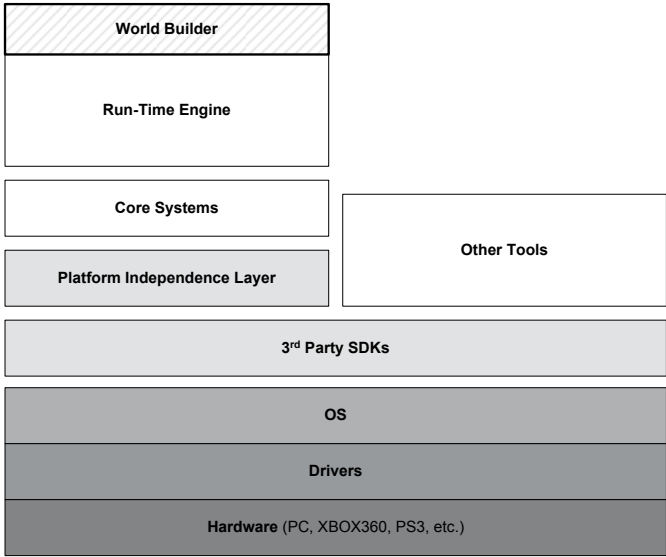


Figure 1.36. Unreal Engine's tool architecture.

As an interesting and unique example, Unreal's world editor and asset manager, UnrealEd, is built right into the runtime game engine. To run the editor, you run your game with a command-line argument of "editor." This unique architectural style is depicted in Figure 1.36. It permits the tools to have total access to the full range of data structures used by the engine and avoids a common problem of having to have two representations of every data structure—one for the runtime engine and one for the tools. It also means that running the game from within the editor is very fast (because the game is actually already running). Live in-game editing, a feature that is normally very tricky to implement, can be developed relatively easily when the editor is a part of the game. However, an in-engine editor design like this does have its share of problems. For example, when the engine is crashing, the tools become unusable as well. Hence a tight coupling between engine and asset creation tools can tend to slow down production.

1.7.5.1 Web-Based User Interfaces

Web-based user interfaces are quickly becoming the norm for certain kinds of game development tools. At Naughty Dog, we use a number of web-based UIs. Naughty Dog's localization tool serves as the front-end portal into our localization database. *Tasker* is the web-based interface used by all Naughty Dog employees to create, manage, schedule, communicate and collaborate on game development tasks during production. A web-based interface known as *Connector* also serves as our window into the various streams of debugging information that are emitted by the game engine at runtime. The game spits out its debug text into various named channels, each associated with a different engine system (animation, rendering, AI, sound, etc.) These data streams are collected by a lightweight Redis database. The browser-based Connector interface allows users to view and filter this information in a convenient way.

Web-based UIs offer a number of advantages over stand-alone GUI applications. For one thing, web apps are typically easier and faster to develop and maintain than a stand-alone app written in a language like Java, C# or C++. Web apps require no special installation—all the user needs is a compatible web browser. Updates to a web-based interface can be pushed out to the users without the need for an installation step—they need only refresh or restart their browser to receive the update. Web interfaces also force us to design our tools using a client-server architecture. This opens up the possibility of distributing our tools to a wider audience. For example, Naughty Dog's localization tool is available directly to outsourcing partners around the globe

who provide language translation services to us. Stand-alone tools still have their place of course, especially when specialized GUIs such as 3D visualization are required. But if your tool only needs to present the user with editable forms and tabular data, a web-based tool may be your best bet.

2

Tools of the Trade

Before we embark on our journey across the fascinating landscape of game engine architecture, it is important that we equip ourselves with some basic tools and provisions. In the next two chapters, we will review the software engineering concepts and practices that we will need during our voyage. In Chapter 2, we'll explore the tools used by the majority of professional game engineers. Then in Chapter 3, we'll round out our preparations by reviewing some key topics in the realms of object-oriented programming, design patterns and large-scale C++ programming.

Game development is one of the most demanding and broad areas of software engineering, so believe me, we'll want to be well equipped if we are to safely navigate the sometimes-treacherous terrain we'll be covering. For some readers, the contents of this chapter and the next will be very familiar. However, I encourage you not to skip these chapters entirely. I hope that they will serve as a pleasant refresher; and who knows, you might even pick up a new trick or two.

2.1 Version Control

A *version control* system is a tool that permits multiple users to work on a group of files collectively. It maintains a history of each file so that changes

can be tracked and reverted if necessary. It permits multiple users to modify files—even the same file—simultaneously, without everyone stomping on each other’s work. Version control gets its name from its ability to track the version history of files. It is sometimes called *source control*, because it is primarily used by computer programmers to manage their source code. However, version control can be used for other kinds of files as well. Version control systems are usually best at managing text files, for reasons we will discover below. However, many game studios use a single version control system to manage both source code files (which are text) and game assets like textures, 3D meshes, animations and audio files (which are usually binary).

2.1.1 Why Use Version Control?

Version control is crucial whenever software is developed by a team of multiple engineers. Version control

- provides a central repository from which engineers can share source code;
- keeps a history of the changes made to each source file;
- provides mechanisms allowing specific versions of the code base to be tagged and later retrieved; and
- permits versions of the code to be branched off from the main development line, a feature often used to produce demos or make patches to older versions of the software.

A source control system can be useful even on a single-engineer project. Although its multiuser capabilities won’t be relevant, its other abilities, such as maintaining a history of changes, tagging versions, creating branches for demos and patches, tracking bugs, etc., are still invaluable.

2.1.2 Common Version Control Systems

Here are the most common source control systems you’ll probably encounter during your career as a game engineer.

- *SCCS and RCS*. The Source Code Control System (SCCS) and the Revision Control System (RCS) are two of the oldest version control systems. Both employ a command-line interface. They are prevalent primarily on UNIX platforms.
- *CVS*. The Concurrent Version System (CVS) is a heavy-duty professional-grade command-line-based source control system, originally built on top

of RCS (but now implemented as a stand-alone tool). CVS is prevalent on UNIX systems but is also available on other development platforms such as Microsoft Windows. It is open source and licensed under the Gnu General Public License (GPL). CVSNT (also known as WinCVS) is a native Windows implementation that is based on, and compatible with, CVS.

- *Subversion*. Subversion is an open source version control system aimed at replacing and improving upon CVS. Because it is open source and hence free, it is a great choice for individual projects, student projects and small studios.
- *Git*. This is an open source revision control system that has been used for many venerable projects, including the Linux kernel. In the git development model, the programmer makes changes to files and commits the changes to a branch. The programmer can then merge his changes into any other code branch quickly and easily, because git “knows” how to rewind a sequence of diffs and reapply them onto a new base revision—a process git calls *rebasing*. The net result is a revision control system that is highly efficient and fast when dealing with multiple code branches. Git is a distributed version control system; individual programmers can work locally much of the time, yet they can merge their changes into a shared codebase easily. It’s also very easy to use on a one-person software project, because there’s zero server set-up to worry about. More information on git can be found at <http://git-scm.com/>.
- *Perforce*. Perforce is a professional-grade source control system, with both text-based and GUI interfaces. One of Perforce’s claims to fame is its concept of *change lists*. A change list is a collection of source files that have been modified as a logical unit. Change lists are checked into the repository atomically—either the entire change list is submitted, or none of it is. Perforce is used by many game companies, including *Naughty Dog* and *Electronic Arts*.
- *NxN Alienbrain*. Alienbrain is a powerful and feature-rich source control system designed explicitly for the game industry. Its biggest claim to fame is its support for very large databases containing both text source code files and binary game art assets, with a customizable user interface that can be targeted at specific disciplines such as artists, producers or programmers.
- *ClearCase*. Rational ClearCase is a professional-grade source control system aimed at very large-scale software projects. It is powerful and employs a unique user interface that extends the functionality of Windows

Explorer. I haven't seen ClearCase used much in the game industry, perhaps because it is one of the more expensive version control systems.

- *Microsoft Visual SourceSafe*. SourceSafe is a lightweight source control package that has been used successfully on some game projects.

2.1.3 Overview of Subversion and TortoiseSVN

I have chosen to highlight Subversion in this book for a few reasons. First off, it's free, which is always nice. It works well and is reliable, in my experience. A Subversion central repository is quite easy to set up, and as we'll see, there are already a number of free repository servers out there if you don't want to go to the trouble of setting one up yourself. There are also a number of good Windows and Mac Subversion clients, such as the freely available TortoiseSVN for Windows. So while Subversion may not be the best choice for a large commercial project (I personally prefer Perforce or git for that purpose), I find it perfectly suited to small personal and educational projects. Let's take a look at how to set up and use Subversion on a Microsoft Windows PC development platform. As we do so, we'll review core concepts that apply to virtually any version control system.

Subversion, like most other version control systems, employs a client-server architecture. The server manages a central repository, in which a version-controlled directory hierarchy is stored. Clients connect to the server and request operations, such as checking out the latest version of the directory tree, committing new changes to one or more files, tagging revisions, branching the repository and so on. We won't discuss setting up a server here; we'll assume you have a server, and instead we will focus on setting up and using the client. You can learn how to set up a Subversion server by reading Chapter 6 of [43]. However, you probably will never need to do so, because you can always find free Subversion servers. For example, HelixTeamHub provides Subversion code hosting at <http://info.perforce.com/try-perforce-helix-teamhub-free.html> that's free for projects of 5 users or fewer and up to 1 GB of storage. Beanstalk is another good hosting service, but they do charge a nominal monthly fee.

2.1.4 Setting up a Code Repository

The easiest way to get started with Subversion is to visit the website of HelixTeamHub or a similar SVN hosting service, and set up a Subversion repository. Create an account and you're off to the races. Most hosting sites offer easy-to-follow instructions.

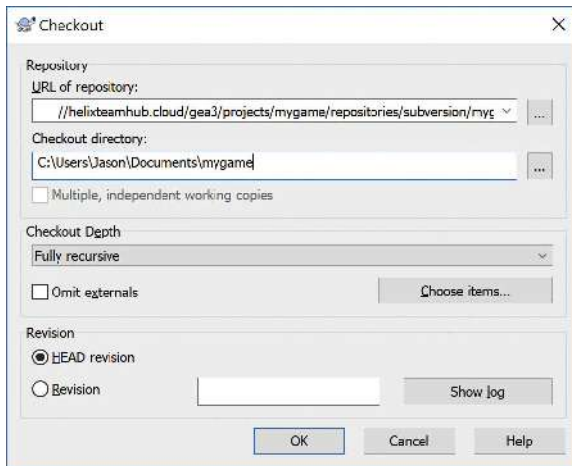


Figure 2.1. TortoiseSVN initial check-out dialog.

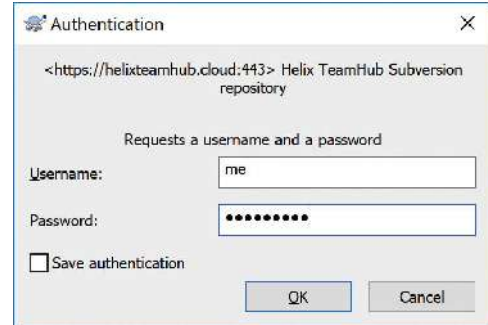


Figure 2.2. TortoiseSVN user authentication dialog.

Once you’ve created your repository, you can typically administer it on the hosting service’s website. You can add and remove users, control options and perform a wealth of advanced tasks. But all you really need to do next is set up a Subversion client and start using your repository.

2.1.5 Installing TortoiseSVN

TortoiseSVN is a popular front end for Subversion. It extends the functionality of the Microsoft Windows Explorer via a convenient right-click menu and overlay icons to show you the status of your version-controlled files and folders.

To get TortoiseSVN, visit <http://tortoisesvn.tigris.org/>. Download the latest version from the download page. Install it by double-clicking the .msi file that you’ve downloaded and following the installation wizard’s instructions.

Once TortoiseSVN is installed, you can go to any folder in Windows Explorer and right-click—TortoiseSVN’s menu extensions should now be visible. To connect to an existing code repository (such as one you created on Helix TeamHub), create a folder on your local hard disk and then right-click and select “SVN Checkout...” The dialog shown in Figure 2.1 will appear. In the “URL of repository” field, enter your repository’s URL. If you are using Helix TeamHub, it would be `https://helixteamhub.cloud/mr3/projects/myprojectname/repositories/subversion/myrepository`, where *myprojectname* is whatever you named your project when you first created it, and *myrepository* is the name of your SVN code repository.

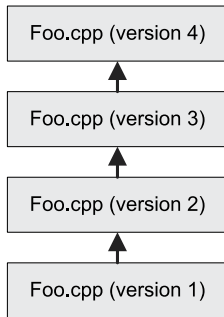


Figure 2.3. File version histories.

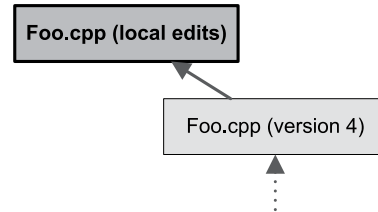
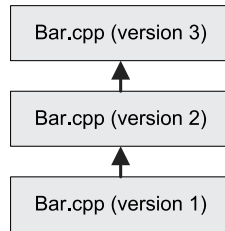


Figure 2.4. Editing the local copy of a version-controlled file.

You should now see the dialog shown in Figure 2.2. Enter your user name and password; checking the “Save authentication” option on this dialog allows you to use your repository without ever having to log in again. Only select this option if you are working on your own personal machine—never on a machine that is shared by many users.

Once you’ve authenticated your user name, TortoiseSVN will download (“check out”) the entire contents of your repository to your local disk. If you have just set up your repository, this will be ...nothing! The folder you created will still be empty. But now it is connected to your Subversion repository on HelixTeamHub (or wherever your server is located). If you refresh your Windows Explorer window (hit F5), you should now see a little green and white checkmark on your folder. This icon indicates that the folder is connected to a Subversion repository via TortoiseSVN and that the local copy of the repository is up to date.

2.1.6 File Versions, Updating and Committing

As we’ve seen, one of the key purposes of any source control system like Subversion is to allow multiple programmers to work on a single software code base by maintaining a central repository or “master” version of all the source code on a server. The server maintains a version history for each file, as shown in Figure 2.3. This feature is crucial to large-scale multiprogrammer software development. For example, if someone makes a mistake and checks in code that “breaks the build,” you can easily go back in time to undo those changes (and check the log to see who the culprit was!). You can also grab a snapshot of the code as it existed at any point in time, allowing you to work with, demonstrate or patch previous versions of the software.

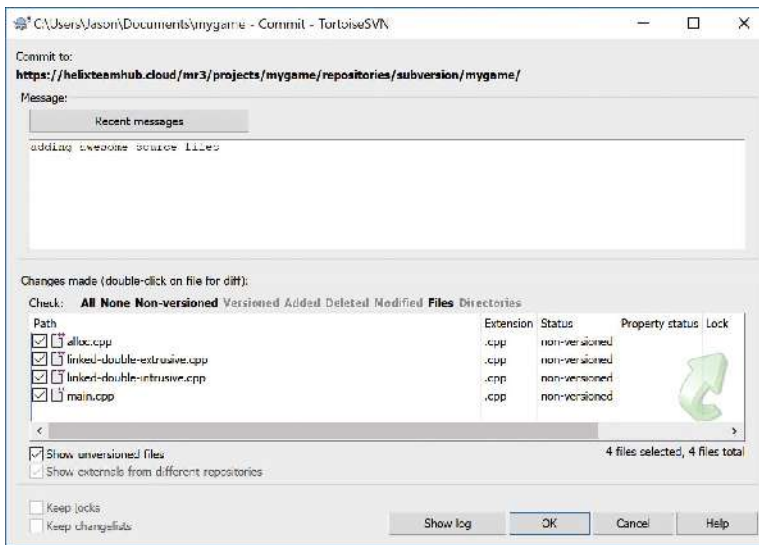


Figure 2.5. TortoiseSVN Commit dialog.

Each programmer gets a local copy of the code on his or her machine. In the case of TortoiseSVN, you obtain your initial working copy by “checking out” the repository, as described above. Periodically you should update your local copy to reflect any changes that may have been made by other programmers. You do this by right-clicking on a folder and selecting “SVN Update” from the pop-up menu.

You can work on your local copy of the code base without affecting the other programmers on the team (Figure 2.4). When you are ready to share your changes with everyone else, you *commit* your changes to the repository (also known as *submitting* or *checking in*). You do this by right-clicking on the folder you want to commit and selecting “SVN Commit...” from the pop-up menu. You will get a dialog like the one shown in Figure 2.5, asking you to confirm the changes.

During a commit operation, Subversion generates a *diff* between your local version of each file and the latest version of that same file in the repository. The term “diff” means difference, and it is typically produced by performing a line-by-line comparison of the two versions of the file. You can double-click on any file in the TortoiseSVN Commit dialog (Figure 2.5) to see the diffs between your version and the latest version on the server (i.e., the changes you made). Files that have changed (i.e., any files that “have diffs”) are committed. This replaces the latest version in the repository with your local version, adding a

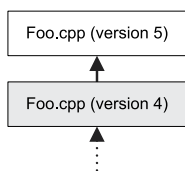


Figure 2.6. Committing local edits to the repository.

new entry to the file’s version history. Any files that have not changed (i.e., your local copy is identical to the latest version in the repository) are ignored by default during a commit. An example commit operation is shown in Figure 2.6.

If you created any new files prior to the commit, they will be listed as “non-versioned” in the Commit dialog. You can check the little check boxes beside them in order to add them to the repository. Any files that you deleted locally will likewise show up as “missing”—if you check their check boxes, they will be deleted from the repository. You can also type a comment in the Commit dialog. This comment is added to the repository’s history log, so that you and others on your team will know why these files were checked in.

2.1.7 Multiple Check-Out, Branching and Merging

Some version control systems require *exclusive check-out*. This means that you must first indicate your intentions to modify a file by *checking it out* and *locking* it. The file(s) that are checked out to you are writable on your local disk and cannot be checked out by anyone else. All other files in the repository are read-only on your local disk. Once you’re done editing the file, you can check it in, which releases the lock and commits the changes to the repository for everyone else to see. The process of exclusively locking files for editing ensures that no two people can edit the same file simultaneously.

Subversion, CVS, Perforce and many other high-quality version control systems also permit *multiple check-out*, i.e., you can edit a file while someone else is editing that same file. Whichever user’s changes are committed first become the latest version of the file in the repository. Any subsequent commits by other users require that programmer to merge his or her changes with the changes made by the programmer(s) who committed previously.

Because more than one set of changes (diffs) have been made to the same file, the version control system must *merge* the changes in order to produce a final version of the file. This is often not a big deal, and in fact many conflicts can be resolved automatically by the version control system. For example, if you changed function `f()` and another programmer changed function `g()`, then your edits would have been to a different range of lines in the file than those of the other programmer. In this case, the merge between your changes and his or her changes will usually resolve automatically without any conflicts. However, if you were both making changes to the same function `f()`, then the second programmer to commit his or her changes will need to do a *three-way merge* (see Figure 2.7).

For three-way merges to work, the version control server has to be smart

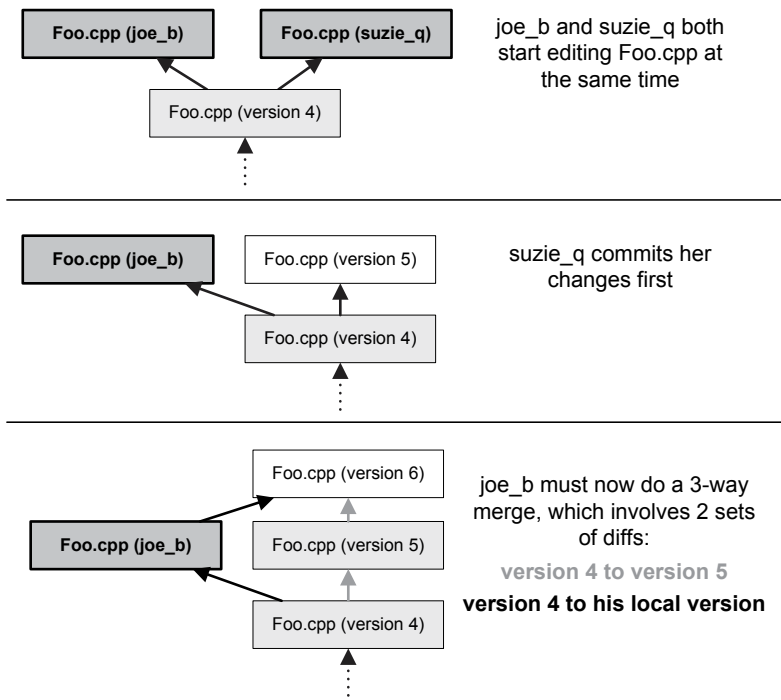


Figure 2.7. Three-way merge due to local edits by two different users.

enough to keep track of which version of each file you currently have on your local disk. That way, when you merge the files, the system will know which version is the base version (the common ancestor, such as version 4 in Figure 2.7).

Subversion permits multiple check-out, and in fact it doesn't require you to check out files explicitly at all. You simply start editing the files locally—all files are writable on your local disk at all times. (By the way, this is one reason that Subversion doesn't scale well to large projects, in my opinion. To determine which files you have changed, Subversion must search the entire tree of source files, which can be slow. Version control systems like Perforce, which explicitly keep track of which files you have modified, are usually easier to work with when dealing with large amounts of code. But for small projects, Subversion's approach works just fine.)

When you perform a commit operation by right-clicking on any folder and selecting "SVN Commit..." from the pop-up menu, you may be prompted to *merge* your changes with changes made by someone else. But if no one has changed the file since you last updated your local copy, then your changes

will be committed without any further action on your part. This is a very convenient feature, but it can also be dangerous. It's a good idea to always check your commits carefully to be sure you aren't committing any files that you didn't intend to modify. When TortoiseSVN displays its Commit Files dialog, you can double-click on an individual file in order to see the diffs you made prior to hitting the "OK" button.

2.1.8 Deleting Files

When a file is deleted from the repository, it's not really gone. The file still exists in the repository, but its latest version is simply marked "deleted" so that users will no longer see the file in their local directory trees. You can still see and access previous versions of a deleted file by right-clicking on the folder in which the file was contained and selecting "Show log" from the TortoiseSVN menu.

You can undelete a deleted file by updating your local directory to the version immediately before the version in which the file was marked deleted. Then simply commit the file again. This replaces the latest deleted version of the file with the version just prior to the deletion, effectively undeleting the file.

2.2 Compilers, Linkers and IDEs

Compiled languages, such as C++, require a *compiler* and *linker* in order to transform source code into an executable program. There are many compilers/linkers available for C++, but for the Microsoft Windows platform, the most commonly used package is probably Microsoft Visual Studio. The fully-featured Professional and Enterprise Editions of the product can be purchased online from the Microsoft store, and its lighter-weight cousin Visual Studio Community Edition (previously known as Visual Studio Express) is available for free download at <https://www.visualstudio.com/downloads>. Documentation on Visual Studio and the standard C and C++ libraries is available online at the Microsoft Developer Network (MSDN) site (<https://msdn.microsoft.com/en-us>).

Visual Studio is more than just a compiler and linker. It is an *integrated development environment* (IDE), including a slick and fully featured *text editor* for source code and a powerful source-level and machine-level *debugger*. In this book, our primary focus is the Windows platform, so we'll investigate Visual Studio in some depth. Much of what you learn below will be applicable to other compilers, linkers and debuggers such as LLVM/Clang, gcc/gdb, and

the Intel C/C++ compiler. So even if you're not planning on ever using Visual Studio, I suggest you read this section carefully. You'll find all sorts of useful tips on using compilers, linkers and debuggers in general.

2.2.1 Source Files, Headers and Translation Units

A program written in C++ is comprised of *source files*. These typically have a .c, .cc, .cxx or .cpp extension, and they contain the bulk of your program's source code. Source files are technically known as *translation units*, because the compiler *translates* one source file at a time from C++ into machine code.

A special kind of source file, known as a *header file*, is often used in order to share information, such as type declarations and function prototypes, between translation units. Header files are not seen by the compiler. Instead, the C++ *preprocessor* replaces each `#include` statement with the contents of the corresponding header file prior to sending the translation unit to the compiler. This is a subtle but very important distinction to make. Header files exist as distinct files from the point of view of the programmer—but thanks to the preprocessor's header file expansion, all the compiler ever sees are translation units.

2.2.2 Libraries, Executables and Dynamic Link Libraries

When a translation unit is compiled, the resulting machine code is placed in an object file (files with a .obj extension under Windows or .o under UNIX-based operating systems). The machine code in an object file is:

- *relocatable*, meaning that the memory addresses at which the code resides have not yet been determined, and
- *unlinked*, meaning that any external references to functions and global data that are defined outside the translation unit have not yet been resolved.

Object files can be collected into groups called *libraries*. A library is simply an archive, much like a ZIP or tar file, containing zero or more object files. Libraries exist merely as a convenience, permitting a large number of object files to be collected into a single easy-to-use file.

Object files and libraries are *linked* into an executable by the linker. The executable file contains fully resolved machine code that can be loaded and run by the operating system. The linker's jobs are:

- to calculate the final relative addresses of all the machine code, as it will appear in memory when the program is run, and
- to ensure that all external references to functions and global data made by each translation unit (object file) are properly resolved.

It's important to remember that the machine code in an executable file is still relocatable, meaning that the addresses of all instructions and data in the file are still *relative* to an arbitrary base address, not absolute. The final absolute base address of the program is not known until the program is actually loaded into memory, just prior to running it.

A *dynamic link library* (DLL) is a special kind of library that acts like a hybrid between a regular static library and an executable. The DLL acts like a library, because it contains functions that can be called by any number of different executables. However, a DLL also acts like an executable, because it can be loaded by the operating system independently, and it contains some start-up and shut-down code that runs much the way the `main()` function in a C++ executable does.

The executables that use a DLL contain *partially linked* machine code. Most of the function and data references are fully resolved within the final executable, but any references to external functions or data that exist in a DLL remain unlinked. When the executable is run, the operating system resolves the addresses of all unlinked functions by locating the appropriate DLLs, loading them into memory if they are not already loaded, and patching in the necessary memory addresses. Dynamically linked libraries are a very useful operating system feature, because individual DLLs can be updated without changing the executable(s) that use them.

2.2.3 Projects and Solutions

Now that we understand the difference between libraries, executables and dynamic link libraries (DLLs), let's see how to create them. In Visual Studio, a *project* is a collection of *source files* which, when compiled, produce a library, an executable or a DLL. In all versions of Visual Studio since VS 2013, projects are stored in project files with a `.vcxproj` extension. These files are in XML format, so they are reasonably easy for a human to read and even edit by hand if necessary.

All versions of Visual Studio since version 7 (Visual Studio 2003) employ *solution files* (files with a `.sln` extension) as a means of containing and managing collections of projects. A solution is a collection of dependent and/or independent projects intended to build one or more libraries, executables and/or DLLs. In the Visual Studio graphical user interface, the *Solution Explorer* is

usually displayed along the right or left side of the main window, as shown in Figure 2.8.

The Solution Explorer is a tree view. The solution itself is at the root, with the projects as its immediate children. Source files and headers are shown as children of each project. A project can contain any number of user-defined folders, nested to any depth. Folders are for organizational purposes only and have nothing to do with the folder structure in which the files may reside on-disk. However, it is common practice to mimic the on-disk folder structure when setting up a project's folders.

2.2.4 Build Configurations

The C/C++ preprocessor, compiler and linker offer a wide variety of options to control how your code will be built. These options are normally specified on the command line when the compiler is run. For example, a typical command to build a single translation unit with the Microsoft compiler might look like this:

```
> cl /c foo.cpp /Fo foo.obj /Wall /Od /Zi
```

This tells the compiler/linker to compile but not link (/c) the translation unit named foo.cpp, output the result to an object file named foo.obj (/Fo foo.obj), turn on all warnings (/Wall), turn off all optimizations (/Od) and generate debugging information (/Zi). A roughly equivalent command line for LLVM/Clang would look something like this:

```
> clang --std=c++14 foo.cpp -o foo.o --Wall -O0 -g
```

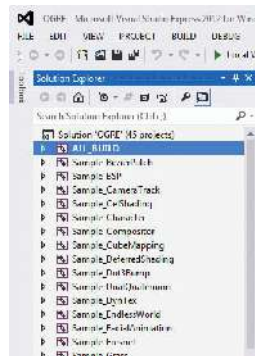


Figure 2.8. The VisualStudio Solution Explorer window.

Modern compilers provide so many options that it would be impractical and error prone to specify all of them every time you build your code. That's where *build configurations* come in. A build configuration is really just a collection of preprocessor, compiler and linker options associated with a particular project in your solution. You can define any number of build configurations, name them whatever you want, and configure the preprocessor, compiler and linker options differently in each configuration. By default, the same options are applied to every translation unit in the project, although you can override the global project settings on an individual translation unit basis. (I recommend avoiding this if at all possible, because it becomes difficult to tell which .cpp files have custom settings and which do not.)

When you create a new project/solution in Visual Studio, by default it creates two build configurations named "Debug" and "Release." The release build is intended for the final version of your software that you'll ship (release) to customers, while the debug build is for development purposes. A debug build runs more slowly than a non-debug build, but it provides the programmer with invaluable information for developing and debugging the program.

Professional software developers often set up more than two build configurations for their software. To understand why, we'll need to understand how local (compile-time) and global (link-time) optimizations work—we'll discuss these kinds of optimizations in Section 2.2.4.2. For now, let's drop the somewhat confusing term "release build" and stick with the terms "debug build" (meaning local and global optimizations are disabled) and "non-debug build" (meaning local and/or global optimizations are enabled).

2.2.4.1 Common Build Options

This section lists some of the most common options you'll want to control via build configurations for a game engine project.

Preprocessor Settings

The C++ preprocessor handles the expansion of `#included` files and the definition and substitution of `#defined` macros. One extremely powerful feature of all modern C++ preprocessors is the ability to define preprocessor macros via command-line options (and hence via build configurations). Macros defined in this way act as though they had been written into your source code with a `#define` statement. For most compilers, the command line option for this is `-D` or `/D`, and any number of these directives can be used.

This feature allows you to communicate various build options to your code,

without having to modify the source code itself. As a ubiquitous example, the symbol `_DEBUG` is always defined for a debug build, while in non-debug builds, the symbol `NDEBUG` is defined instead. The source code can check these flags and in effect “know” whether it is being built in debug or non-debug mode. This is known as *conditional compilation*. For example,

```
void f()
{
#ifdef _DEBUG
    printf("Calling function f()\n");
#endif
    // ...
}
```

The compiler is also free to introduce “magic” preprocessor macros into your code, based on its knowledge of the compilation environment and target platform. For example, the macro `__cplusplus` is defined by most C/C++ compilers when compiling a C++ file. This allows code to be written that automatically adapts to being compiled for C or C++.

As another example, every compiler identifies itself to the source code via a “magic” preprocessor macro. When compiling code under the Microsoft compiler, the macro `_MSC_VER` is defined; when compiling under the GNU compiler (gcc), the macro `__GNUC__` is defined instead and so on for the other compilers. The target platform on which the code will be run is likewise identified via macros. For example, when building for a 32-bit Windows machine, the symbol `_WIN32` is always defined. These key features permit cross-platform code to be written, because they allow your code to “know” what compiler is compiling it and on which target platform it is destined to be run.

Compiler Settings

One of the most common compiler options controls whether or not the compiler should include *debugging information* with the object files it produces. This information is used by debuggers to step through your code, display the values of variables and so on. Debugging information makes your executables larger on disk and also opens the door for hackers to reverse-engineer your code, so it is always stripped from the final shipping version of your executable. However, during development, debugging information is invaluable and should always be included in your builds.

The compiler can also be told whether or not to expand *inline functions*. When inline function expansion is turned off, every inline function appears only once in memory, at a distinct address. This makes the task of tracing

through the code in the debugger much simpler, but obviously comes at the expense of the execution speed improvements normally achieved by inlining.

Inline function expansion is but one example of generalized code transformations known as *optimizations*. The aggressiveness with which the compiler attempts to optimize your code, and the kinds of optimizations it uses, can be controlled via compiler options. Optimizations have a tendency to reorder the statements in your code, and they also cause variables to be stripped out of the code altogether, or moved around, and can cause CPU registers to be reused for new purposes later in the same function. Optimized code usually confuses most debuggers, causing them to “lie” to you in various ways, and making it difficult or impossible to see what’s really going on. As a result, all optimizations are usually turned off in a debug build. This permits every variable and every line of code to be scrutinized as it was originally coded. But, of course, such code will run much more slowly than its fully optimized counterpart.

Linker Settings

The linker also exposes a number of options. You can control what type of output file to produce—an executable or a DLL. You can also specify which external libraries should be linked into your executable, and which directory paths to search in order to find them. A common practice is to link with debug libraries when building a debug executable and with optimized libraries when building in non-debug mode.

Linker options also control things like stack size, the preferred base address of your program in memory, what type of machine the code will run on (for machine-specific optimizations), whether or not global (link-time) optimizations are enabled, and some other minutia with which we will not concern ourselves here.

2.2.4.2 Local and Global Optimizations

An optimizing compiler is one that can automatically optimize the machine code it generates. All of the C/C++ compilers commonly used today are optimizing compilers. These include Microsoft Visual Studio, gcc, LLVM/Clang and the Intel C/C++ compiler.

Optimizations can come in two basic forms:

- local optimizations and
- global optimizations,

although other kinds of optimizations are possible (e.g., *peep-hole optimizations*, which enable the optimizer to make platform- or CPU-specific optimizations).

Local optimizations operate only on small chunks of code known as *basic blocks*. Roughly speaking, a basic block is a sequence of assembly language instructions that doesn't involve a branch. Local optimizations include things like:

- *algebraic simplification*,
- *operator strength reduction* (e.g., converting $x / 2$ into $x \gg 1$ because the shift operator is “lower strength” and therefore less expensive than the integer division operator),
- *code inlining*,
- *constant folding* (recognizing expressions that are constant at compile time and replacing such expressions with their known value),
- *constant propagation* (replacing all instances of a variable whose value turns out to be constant with the literal constant itself),
- *loop unrolling* (e.g., converting a loop that always iterates exactly four times with four copies of the code within the loop, in order to eliminate conditional branching),
- *dead code elimination* (removal of code that has no effect, such as removing the assignment expression $x = 5$; if it is immediately followed by another assignment to x like $x = y + 1$), and
- *instruction reordering* (in order to minimize CPU pipeline stalls).

Global optimizations operate beyond the scope of basic code blocks—they take the whole control flow graph of a program into account. An example of this kind of optimization would be *common sub-expression elimination*. Global optimizations ideally operate across translation unit boundaries, and hence are performed by the linker rather than the compiler. Aptly, optimizations performed by the linker are known as *link-time optimizations* (LTO).

Some modern compilers like LLVM/Clang support *profile-guided optimizations* (PGO). As the name implies, these optimizations use profiling information obtained from previous runs of your software to iteratively identify and optimize its most performance-critical code paths. PGO and LTO optimizations can produce an impressive performance boost, but they come with a cost. LTO optimizations greatly increase the amount of time required to link an executable. And PGO optimizations, being iterative in nature, require the software to be run (via the QA team or an automated test suite) in order to generate the profiling information that drives further optimizations.

Most compilers offer various options controlling how aggressive its optimization efforts will be. Optimizations can be disabled entirely (useful for debug builds, where debuggability is more important than performance), or increasing “levels” of optimizations can be applied up to some predetermined maximum. For example, optimizations for gcc, Clang and Visual C++ all range from -O0 (meaning no optimizations are performed) to -O3 (all optimizations enabled). Individual optimizations can be turned on or off as well, via other command line options.

2.2.4.3 Typical Build Configurations

Game projects often have more than just two build configurations. Here are a few of the common configurations I’ve seen used in game development.

- *Debug.* A debug build is a very slow version of your program, with all optimizations turned off, all function inlining disabled, and full debugging information included. This build is used when testing brand new code and also to debug all but the most trivial problems that arise during development.
- *Development.* A development build (or “dev build”) is a faster version of your program, with most or all local optimizations enabled, but with debugging information and assertions still turned on. (See Section 3.2.3.3 for a discussion of assertions.) This allows you to see your game running at a speed representative of the final product, but it still gives you some opportunity to debug problems.
- *Ship.* A ship configuration is intended for building the final game that you will ship to your customers. It is sometimes called a “final” build or “disk” build. Unlike a development build, all debugging information is stripped out of a ship build, most or all assertions are compiled out, and optimizations are cranked all the way up, including global optimizations (LTO and PGO). A ship build is very tricky to debug, but it is the fastest and leanest of all build types.

Hybrid Builds

A hybrid build is a build configuration in which the majority of the translation units are built in development mode, but a small subset of them is built in debug mode. This permits the segment of code that is currently under scrutiny to be easily debugged, while the rest of the code continues to run at close to full speed.

With a text-based build system like make, it is quite easy to set up a hybrid build that permits users to specify the use of debug mode on a per-

translation-unit basis. In a nutshell, we define a make variable called something like `$HYBRID_SOURCES`, which lists the names of all translation units (.cpp files) that should be compiled in debug mode for our hybrid build. We set up build rules for compiling both debug and development versions of every translation unit, and arrange for the resulting object files (.obj/.o) to be placed into two different folders, one for debug and one for development. The final link rule is set up to link with the debug versions of the object files listed in `$HYBRID_SOURCES` and with the non-debug versions of all other object files. If we've set it up properly, make's dependency rules will take care of the rest.

Unfortunately, this is not so easy to do in Visual Studio, because its build configurations are designed to be applied on a per-project basis, not per-translation unit. The crux of the problem is that we cannot easily define a list of the translation units that we want to build in debug mode. One solution to this problem is to write a script (in Python or another suitable language) that automatically generates Visual Studio .vcxproj files, given a list of the .cpp files you want to be built in debug mode in your hybrid configuration. Another alternative that works if your source code is already organized into libraries is to set up a "Hybrid" build configuration at the solution level, which picks and chooses between debug and development builds on a per-project (and hence per-library) basis. This isn't as flexible as having control on a per-translation-unit basis, but it does work reasonably well if your libraries are sufficiently granular.

Build Configurations and Testability

The more build configurations your project supports, the more difficult testing becomes. Although the differences between the various configurations may be slight, there's a finite probability that a critical bug may exist in one of them but not in the others. Therefore, each build configuration must be tested equally thoroughly. Most game studios do not formally test their debug builds, because the debug configuration is primarily intended for internal use during initial development of a feature and for the debugging of problems found in one of the other configurations. However, if your testers spend most of their time testing your development build configuration, then you cannot simply make a ship build of your game the night before Gold Master and expect it to have an identical bug profile to that of the development build. Practically speaking, the test team must test both your development and ship builds equally throughout alpha and beta to ensure that there aren't any nasty surprises lurking in your ship build. In terms of testability, there is a clear advantage to keeping your build configurations to a minimum, and in

fact some studios don't have a separate ship build for this reason—they simply ship their *development* build once it has been thoroughly tested (but with the debugging information stripped out).

2.2.4.4 Project Configuration Tutorial

Right-clicking on any project in the Solution Explorer and selecting “Properties...” from the menu brings up the project's “Property Pages” dialog. The tree view on the left shows various categories of settings. Of these, the four we will use most are:

- Configuration Properties/General,
- Configuration Properties/Debugging,
- Configuration Properties/C++, and
- Configuration Properties/Linker.

Configurations Drop-Down Combo Box

Notice the drop-down combo box labeled “Configuration:” at the top-left corner of the window. All of the properties displayed on these property pages apply separately to each build configuration. If you set a property for the debug configuration, this does not necessarily mean that the same setting exists for the other configuration(s).

If you click on the combo box to drop down the list, you'll find that you can select a single configuration or multiple configurations, including “All configurations.” As a rule of thumb, try to do most of your build configuration editing with “All configurations” selected. That way, you won't have to make the same edits multiple times, once for each configuration—and you don't risk setting things up incorrectly in one of the configurations by accident. However, be aware that some settings *need* to be different between the debug and development configurations. For example, function inlining and code optimization settings should, of course, be different between debug and development builds.

General Property Page

On the *General* property page, shown in Figure 2.9, the most useful fields are the following:

- *Output directory*. This defines where the final product(s) of the build will go—namely, the executable, library or DLL that the compiler/linker ultimately outputs.

- `$(IntDir)`. The value of the “Intermediate Directory” field in this dialog.
- `$(VCInstallDir)`. The directory in which Visual Studio’s C++ standard library is currently installed.

The benefit of using macros instead of hard-wiring your configuration settings is that a simple change of the global macro’s value will automatically affect all configuration settings in which the macro is used. Also, some macros like `$(ConfigurationName)` automatically change their values depending on the build configuration, so using them can permit you to use identical settings across all your configurations.

To see a complete list of all available macros, click in either the “Output Directory” field or the “Intermediate Directory” field on the “General” property page, click the little arrow to the right of the text field, select “Edit...” and then click the “Macros” button in the dialog that comes up.

Debugging Property Page

The “Debugging” property page is where the name and location of the executable to debug is specified. On this page, you can also specify the command-line argument(s) that should be passed to the program when it runs. We’ll discuss debugging your program in more depth below.

C/C++ Property Page

The C/C++ property page controls compile-time language settings—things that affect how your source files will be compiled into object files (.obj extension). The settings on this page do *not* affect how your object files are linked into a final executable or DLL.

You are encouraged to explore the various subpages of the C/C++ page to see what kinds of settings are available. Some of the most commonly used settings include the following:

- *General Property Page/Additional Include Directories*. This field lists the on-disk directories that will be searched when looking for `#included` header files.

Important: It is always best to specify these directories using *relative paths* and/or with Visual Studio macros like `$(OutDir)` or `$(IntDir)`. That way, if you move your build tree to a different location on disk or to another computer with a different root folder, everything will continue to work properly.

- *General Property Page/Debug Information Format.* This field controls whether or not debug information is generated and in what format. Typically both debug and development configurations include debugging information so that you can track down problems during the development of your game. The ship build will have all the debug info stripped out to prevent hacking.
- *Preprocessor Property Page/Preprocessor Definitions.* This very handy field lists any number of C/C++ preprocessor symbols that should be defined in the code when it is compiled. See *Preprocessor Settings* in Section 2.2.4.1 for a discussion of preprocessor-defined symbols.

Linker Property Page

The “Linker” property page lists properties that affect how your object code files will be linked into an executable or DLL. Again, you are encouraged to explore the various subpages. Some commonly used settings follow:

- *General Property Page/Output File.* This setting lists the name and location of the final product of the build, usually an executable or DLL.
- *General Property Page/Additional Library Directories.* Much like the C/C++ Additional Include Directories field, this field lists zero or more directories that will be searched when looking for libraries and object files to link into the final executable.
- *Input Property Page/Additional Dependencies.* This field lists external libraries that you want linked into your executable or DLL. For example, the OGRE libraries would be listed here if you are building an OGRE-enabled application.

Note that Visual Studio employs various “magic spells” to specify libraries that should be linked into an executable. For example, a special `#pragma` instruction in your source code can be used to instruct the linker to automatically link with a particular library. For this reason, you may not see all of the libraries you’re actually linking to in the “Additional Dependencies” field. (In fact, that’s why they are called *additional* dependencies.) You may have noticed, for example, that Direct X applications do not list all of the DirectX libraries manually in their “Additional Dependencies” field. Now you know why.

2.2.5 Debugging Your Code

One of the most important skills any programmer can learn is how to effectively debug code. This section provides some useful debugging tips and

tricks. Some are applicable to any debugger and some are specific to Microsoft Visual Studio. However, you can usually find an equivalent to Visual Studio's debugging features in other debuggers, so this section should prove useful even if you don't use Visual Studio to debug your code.

2.2.5.1 The Start-Up Project

A Visual Studio solution can contain more than one project. Some of these projects build executables, while others build libraries or DLLs. It's possible to have more than one project that builds an executable in a single solution. Visual Studio provides a setting known as the "Start-Up Project." This is the project that is considered "current" for the purposes of the debugger. Typically a programmer will debug one project at a time by setting a single start-up project. However, it is possible to debug multiple projects simultaneously (see [http://msdn.microsoft.com/en-us/library/0s590bew\(v=vs.100\).aspx](http://msdn.microsoft.com/en-us/library/0s590bew(v=vs.100).aspx) for details).

The start-up project is highlighted in bold in the Solution Explorer. By default, hitting F5 will run the .exe built by the start-up project, if the start-up project builds an executable. (Technically speaking, F5 runs whatever command you type into the Command field in the Debugging property page, so it's not limited to running the .exe built by your project.)

2.2.5.2 Breakpoints

Breakpoints are the bread and butter of code debugging. A breakpoint instructs the program to stop at a particular line in your source code so that you can inspect what's going on.

In Visual Studio, select a line and hit F9 to toggle a breakpoint. When you run your program and the line of code containing the breakpoint is about to be executed, the debugger will stop the program. We say that the breakpoint has been "hit." A little arrow will show you which line of code the CPU's program counter is currently on. This is shown in Figure 2.10.

2.2.5.3 Stepping through Your Code

Once a breakpoint has been hit, you can single-step your code by hitting the F10 key. The yellow program-counter arrow moves to show you the lines as they execute. Hitting F11 steps *into* a function call (i.e., the next line of code you'll see is the first line of the called function), while F10 steps *over* that function call (i.e., the debugger calls the function at full speed and then breaks again on the line right after the call).

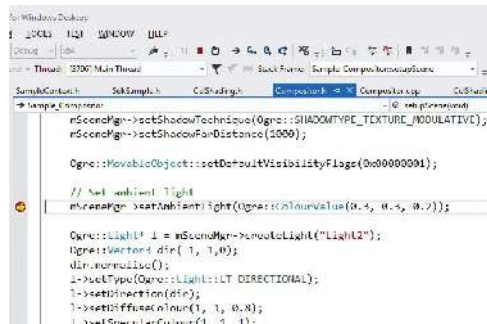


Figure 2.10. Setting a breakpoint in Visual Studio.

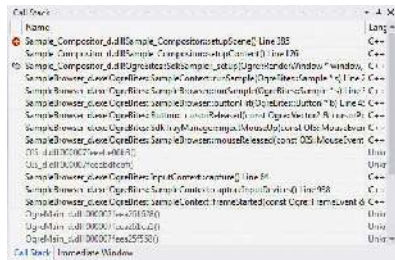


Figure 2.11. The call stack window.

2.2.5.4 The Call Stack

The *call stack* window, shown in Figure 2.11, shows you the stack of functions that have been called at any given moment during the execution of your code. (See Section 3.3.5.2 for more details on a program's stack.) To display the call stack (if it is not already visible), go to the “Debug” menu on the main menu bar, select “Windows” and then “Call Stack.”

Once a breakpoint has been hit (or the program is manually paused), you can move up and down the call stack by double-clicking on entries in the “Call Stack” window. This is very useful for inspecting the chain of function calls that were made between `main()` and the current line of code. For example, you might trace back to the root cause of a bug in a parent function that has manifested itself in a deeply nested child function.

2.2.5.5 The Watch Window

As you step through your code and move up and down the call stack, you will want to be able to inspect the values of the variables in your program. This is what *watch windows* are for. To open a watch window, go to the “Debug” menu, select “Windows...,” then select “Watch...,” and finally select one of

inspect the rotation angle of any quaternion within the debugger.

You can also use various suffixes on the expressions in the watch window in order to change the way Visual Studio displays the data, as shown in Figure 2.13.

- The “,d” suffix forces values to be displayed in decimal notation.
- The “,x” suffix forces values to be displayed in hexadecimal notation.
- The “,n” suffix (where *n* is any positive integer) forces Visual Studio to treat the value as an array with *n* elements. This allows you to expand array data that is referenced through a pointer.
- You can also write simple expressions in square brackets that *calculate* the value of *n* in a “,n” suffix. For example, you can type something like this:

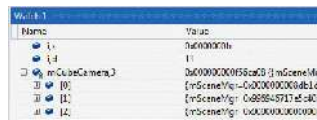
```
my_array, [my_array_count]
```

to ask the debugger to show `my_array_count` elements of the array named `my_array`.

Be careful when expanding very large data structures in the watch window, because it can sometimes slow the debugger down to the point of being unusable.

2.2.5.6 Data Breakpoints

Regular breakpoints trip when the CPU’s program counter hits a particular machine instruction or line of code. However, another incredibly useful feature of modern debuggers is the ability to set a breakpoint that trips whenever a specific memory address is *written to* (i.e., changed). These are called *data breakpoints*, because they are triggered by changes to data, or sometimes *hardware breakpoints*, because they are implemented via a special feature of the CPU’s hardware—namely, the ability to raise an interrupt when a predefined memory address is written to.



Name	Value
i	0x00000000
d	1
mCubeCamera.D	0x0000000000000000 [mSceneMgr]
[0]	{mSceneMgr: 0x0000000000000000}
[1]	{mSceneMgr: 0x0000000000000000}
[2]	{mSceneMgr: 0x0000000000000000}

Figure 2.13. Comma suffixes in the Visual Studio watch window.

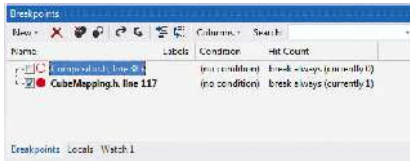


Figure 2.14. The Visual Studio breakpoints window.

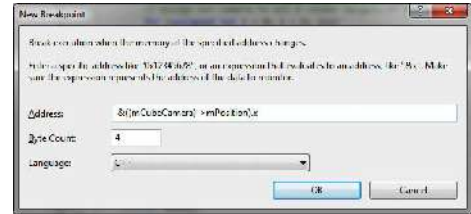


Figure 2.15. Defining a data breakpoint.

Here's how data breakpoints are typically used. Let's say you are tracking down a bug that manifests itself as a zero (0.0f) value mysteriously appearing inside a member variable of a particular object called `m_angle` that *should* always contain a nonzero angle. You have no idea which function might be writing that zero into your variable. However, you do know the address of the variable. (You can just type "`&object.m_angle`" into the watch window to find its address.) To track down the culprit, you can set a data breakpoint on the address of `object.m_angle`, and then simply let the program run. When the value changes, the debugger will stop automatically. You can then inspect the call stack to catch the offending function red-handed.

To set a data breakpoint in Visual Studio, take the following steps.

- Bring up the "Breakpoints" window found on the "Debug" menu under "Windows" and then "Breakpoints" (Figure 2.14).
- Select the "New" drop-down button in the upper-left corner of the window.
- Select "New Data Breakpoint."
- Type in the raw address or an address-valued expression, such as "`&myVariable`" (Figure 2.15).

2.2.5.7 Conditional Breakpoints

You'll also notice in the "Breakpoints" window that you can set conditions and hit counts on any type breakpoint—data breakpoints or regular line-of-code breakpoints.

A *conditional breakpoint* causes the debugger to evaluate the C/C++ expression you provide every time the breakpoint is hit. If the expression is true, the debugger stops your program and gives you a chance to see what's going on. If the expression is false, the breakpoint is ignored and the program continues. This is very useful for setting breakpoints that only trip when a function is called on a particular instance of a class. For example,

let's say you have a game level with 20 tanks on-screen, and you want to stop your program when the third tank, whose memory address you know to be 0x12345678, is running. By setting the breakpoint's condition expression to something like `"(uintptr_t)this == 0x12345678"`, you can restrict the breakpoint only to the class instance whose memory address (`this` pointer) is 0x12345678.

Specifying a *hit count* for a breakpoint causes the debugger to decrement a counter every time the breakpoint is hit, and only actually stop the program when that counter reaches zero. This is really useful for situations where your breakpoint is inside a loop, and you need to inspect what's happening during the 376th iteration of the loop (e.g., the 376th element in an array). You can't very well sit there and hit the F5 key 375 times! But you *can* let the hit count feature of Visual Studio do it for you.

One note of caution: conditional breakpoints cause the debugger to evaluate the conditional expression every time the breakpoint is hit, so they can bog down the performance of the debugger and your game.

2.2.5.8 Debugging Optimized Builds

I mentioned above that it can be very tricky to debug problems using a development or ship build, due primarily to the way the compiler optimizes the code. Ideally, every programmer would prefer to do all of his or her debugging in a debug build. However, this is often not possible. Sometimes a bug occurs so rarely that you'll jump at any chance to debug the problem, even if it occurs in a non-debug build on someone else's machine. Other bugs only occur in your non-debug builds, but they magically disappear whenever you run the debug build. These dreaded *non-debug-only bugs* are sometimes caused by uninitialized variables, because variables and dynamically allocated memory blocks are often set to zero in debug mode but are left containing garbage in a non-debug build. Other common causes of non-debug-only bugs include code that has been accidentally omitted from the non-debug build(s) (e.g., when important code is erroneously placed inside an assertion statement), data structures whose size or data member packing changes between debug and development/ship builds, bugs that are only triggered by inlining or compiler-introduced optimizations, and (in rare cases) bugs in the compiler's optimizer itself, causing it to emit incorrect code in a fully optimized build.

Clearly, it behooves every programmer to be capable of debugging problems in a non-debug build, unpleasant as it may seem. The best ways to reduce the pain of debugging optimized code is to practice doing it and to expand your skill set in this area whenever you have the opportunity. Here are a few tips.

- *Learn to read and step through disassembly in the debugger.* In a non-debug build, the debugger often has trouble keeping track of which line of source code is currently being executed. Thanks to instruction reordering, you'll often see the program counter jump around erratically within the function when viewed in source code mode. However, things become sane again when you work with the code in disassembly mode (i.e., step through the assembly language instructions individually). Every C/C++ programmer should be at least a little bit familiar with the architecture and assembly language of their target CPU(s). That way, even if the debugger is confused, you won't be. (For an introduction to assembly language, see Section 3.4.7.3.)
- *Use registers to deduce variables' values or addresses.* The debugger will sometimes be unable to display the value of a variable or the contents of an object in a non-debug build. However, if the program counter is not too far away from the initial use of the variable, there's a good chance its address or value is still stored in one of the CPU's registers. If you can trace back through the disassembly to where the variable is first loaded into a register, you can often discover its value or its address by inspecting that register. Use the register window, or type the name of the register into a watch window, to see its contents.
- *Inspect variables and object contents by address.* Given the address of a variable or data structure, you can usually see its contents by casting the address to the appropriate type in a watch window. For example, if we know that an instance of the `Foo` class resides at address `0x1378A0C0`, we can type "`(Foo*) 0x1378A0C0`" in a watch window, and the debugger will interpret that memory address as if it were a pointer to a `Foo` object.
- *Leverage static and global variables.* Even in an optimized build, the debugger can usually inspect global and static variables. If you cannot deduce the address of a variable or object, keep your eye open for a static or global that might contain its address, either directly or indirectly. For example, if we want to find the address of an internal object within the physics system, we might discover that it is in fact stored in a member variable of the global `PhysicsWorld` object.
- *Modify the code.* If you can reproduce a non-debug-only bug relatively easily, consider modifying the source code to help you debug the problem. Add print statements so you can see what's going on. Introduce a global variable to make it easier to inspect a problematic variable or

object in the debugger. Add code to detect a problem condition or to isolate a particular instance of a class.

2.3 Profiling Tools

Games are typically high-performance real-time programs. As such, game engine programmers are always looking for ways to speed up their code. In this section, we'll investigate some of the tools we can use to measure the performance of our software. We'll have more to say about how to use this profiling data in order to optimize our software in Chapter 4.

There is a well-known, albeit rather unscientific, rule of thumb known as the *Pareto principle* (see http://en.wikipedia.org/wiki/Pareto_principle). It is also known as the *80/20 rule*, because it states that in many situations, 80% of the effects of some event come from only 20% of the possible causes. In computer science, this principle has been applied both to bug-fixing (80% of the perceived bugs in a piece of software can be eliminated by fixing bugs in only 20% of the code), and to software optimization, where as a rule of thumb 80% (or more) of the wall clock time spent running any piece of software is accounted for by only 20% (or less) of the code. In other words, if you optimize 20% of your code, you can potentially realize 80% of all the gains in execution speed you'll ever realize.

So, how do you know *which* 20% of your code to optimize? For that, you need a *profiler*. A profiler is a tool that measures the execution time of your code. It can tell you how much time is spent in each function. You can then direct your optimizations toward only those functions that account for the lion's share of the execution time.

Some profilers also tell you how many *times* each function is called. This is an important dimension to understand. A function can eat up time for two reasons: (a) it takes a long time to execute on its own, or (b) it is called frequently. For example, a function that runs an A* algorithm to compute the optimal paths through the game world might only be called a few times each frame

Even more information can be obtained if you use the right profiler. Some profilers report the call graph, meaning that for any given function, you can see which functions called it (these are known as *parent functions*) and which functions it called (these are known as *child functions* or *descendants*). You can even see what *percentage* of the function's time was spent calling each of its descendants and the percentage of the overall running time accounted for by each individual function.

Profilers fall into two broad categories.

1. *Statistical profilers.* This kind of profiler is designed to be unobtrusive, meaning that the target code runs at almost the same speed, whether or not profiling is enabled. These profilers work by sampling the CPU's program counter register periodically and noting which function is currently running. The number of samples taken within each function yields an approximate percentage of the total running time that is eaten up by that function. Intel's *VTune* is the gold standard in statistical profilers for Windows machines employing Intel Pentium processors, and it is now also available for Linux. See <https://software.intel.com/en-us/intel-vtune-amplifier-xe> for details.
2. *Instrumenting profilers.* This kind of profiler is aimed at providing the most accurate and comprehensive timing data possible, but at the expense of real-time execution of the target program—when profiling is turned on, the target program usually slows to a crawl. These profilers work by preprocessing your executable and inserting special prologue and epilogue code into every function. The prologue and epilogue code calls into a profiling library, which in turn inspects the program's call stack and records all sorts of details, including which parent function called the function in question and how many times that parent has called the child. This kind of profiler can even be set up to monitor every line of code in your source program, allowing it to report how long each line is taking to execute. The results are stunningly accurate and comprehensive, but turning on profiling can make a game virtually unplayable. IBM's Rational Quantify, available as part of the Rational Purify Plus tool suite, is an excellent instrumenting profiler. See <http://www.ibm.com/developerworks/rational/library/957.html> for an introduction to profiling with Quantify.

Microsoft has also published a profiler that is a hybrid between the two approaches. It is called LOP, which stands for low-overhead profiler. LOP uses a statistical approach, sampling the state of the processor periodically, which means it has a low impact on the speed of the program's execution. However, with each sample, it analyzes the call stack, thereby determining the chain of parent functions that resulted in each sample. This allows LOP to provide information normally not available with a statistical profiler, such as the distribution of calls across parent functions.

On the PlayStation 4, SN Systems' *Razor CPU* is the profiler of choice for measuring game software running on the PS4's CPU. It supports both statis-

tical and instrumenting methods of profiling. (See <https://www.snsystems.com/tech-blog/2014/02/14/function-level-profiling/> for some more details.) Its counterpart, *Razor GPU*, provides profiling and debugging facilities for shaders and compute jobs running on the PS4's GPU.

2.3.1 List of Profilers

There are a great many profiling tools available. See http://en.wikipedia.org/wiki/List_of_performance_analysis_tool for a reasonably comprehensive list.

2.4 Memory Leak and Corruption Detection

Two other problems that plague C and C++ programmers are memory leaks and memory corruption. A memory leak occurs when memory is allocated but never freed. This wastes memory and eventually leads to a potentially fatal out-of-memory condition. Memory corruption occurs when the program inadvertently writes data to the wrong memory location, overwriting the important data that was there—while simultaneously failing to update the memory location where that data *should* have been written. Blame for both of these problems falls squarely on the language feature known as the *pointer*.

A pointer is a powerful tool. It can be an agent of good when used properly—but it can also be all-too-easily transformed into an agent of evil. If a pointer points to memory that has been freed, or if it is accidentally assigned a nonzero integer or floating-point value, it becomes a dangerous tool for corrupting memory, because data written through it can quite literally end up anywhere. Likewise, when pointers are used to keep track of allocated memory, it is all too easy to forget to free the memory when it is no longer needed. This leads to memory leaks.

Clearly, good coding practices are one approach to avoiding pointer-related memory problems. And it is certainly possible to write solid code that essentially never corrupts or leaks memory. Nonetheless, having a tool to help you detect potential memory corruption and leak problems certainly can't hurt. Thankfully, many such tools exist.

My personal favorite is IBM's Rational Purify, which comes as part of the Purify Plus toolkit. Purify instruments your code prior to running it, in order to hook into all pointer dereferences and all memory allocations and deallocations made by your code. When you run your code under Purify, you get a live report of the problems—real and potential—encountered by your code. And when the program exits, you get a detailed memory leak report. Each problem is linked directly to the source code that caused the problem, mak-

ing tracking down and fixing these kinds of problems relatively easy. You can find more information on Purify at <http://www-306.ibm.com/software/awdtools/purify>.

Two other popular tools are Insure++ by Parasoft, and Valgrind by Julian Seward and the Valgrind development team. These tools provide both memory debugging and profiling facilities.

2.5 Other Tools

There are a number of other commonly used tools in a game programmer's toolkit. We won't cover them in any depth here, but the following list will make you aware of their existence and point you in the right direction if you want to learn more.

- *Difference tools.* A difference tool, or *diff tool*, is a program that compares two versions of a text file and determines what has changed between them. (See <http://en.wikipedia.org/wiki/Diff> for a discussion of diff tools.) Diffs are usually calculated on a line-by-line basis, although modern diff tools can also show you a range of characters on a changed line that have been modified. Most version control systems come with a diff tool. Some programmers like a particular diff tool and configure their version control software to use the tool of their choice. Popular tools include ExamDiff (http://www.prestosoft.com/edp_examdiff.asp), AraxisMerge (<http://www.araxis.com>), WinDiff (available in the Options Packs for most Windows versions and available from many independent websites as well), and the GNU diff tools package (<http://www.gnu.org/software/diffutils/diffutils.html>).
- *Three-way merge tools.* When two people edit the same file, two independent sets of diffs are generated. A tool that can merge two sets of diffs into a final version of the file that contains both person's changes is called a three-way merge tool. The name "three-way" refers to the fact that three versions of the file are involved: the original, user A's version and user B's version. (See http://en.wikipedia.org/wiki/3-way_merge#Three-way_merge for a discussion of two-way and three-way merge technologies.) Many merge tools come with an associated diff tool. Some popular merge tools include AraxisMerge (<http://www.araxis.com>) and WinMerge (<http://winmerge.org>). Perforce also comes with an excellent three-way merge tool (<http://www.perforce.com/perforce/products/merge.html>).

- *Hex editors.* A hex editor is a program used for inspecting and modifying the contents of binary files. The data are usually displayed as integers in hexadecimal format, hence the name. Most good hex editors can display data as integers from one byte to 16 bytes each, in 32- and 64-bit floating-point format and as ASCII text. Hex editors are particularly useful when tracking down problems with binary file formats or when reverse-engineering an unknown binary format—both of which are relatively common endeavors in game engine development circles. There are quite literally a million different hex editors out there; I've had good luck with HexEdit by Expert Commercial Software (<http://www.expertcomsoft.com/index.html>), but your mileage may vary.

As a game engine programmer you will undoubtedly come across other tools that make your life easier, but I hope this chapter has covered the main tools you'll use on a day-to-day basis.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

3

Fundamentals of Software Engineering for Games

In this chapter, we'll discuss the foundational knowledge needed by any professional game programmer. We'll explore numeric bases and representations, the components and architecture of a typical computer and its CPU, machine and assembly language, and the C++ programming language. We'll review some key concepts in object-oriented programming (OOP), and then delve into some advanced topics that should prove invaluable in any software engineering endeavor (and especially when creating games). As with Chapter 2, some of this material may already be familiar to some readers. However, I highly recommend that all readers at least skim this chapter, so that we all embark on our journey with the same set of tools and supplies.

3.1 C++ Review and Best Practices

Because C++ is arguably the most commonly used language in the game industry, we will focus primarily on C++ in this book. However, most of the concepts we'll cover apply equally well to *any* object-oriented programming language. Certainly a great many other languages are used in the game industry—imperative languages like C; object-oriented languages like C# and

Java; scripting languages like Python, Lua and Perl; functional languages like Lisp, Scheme and F#, and the list goes on. I highly recommend that every programmer learn at least two high-level languages (the more the merrier), as well as learning at least some *assembly language* programming (see Section 3.4.7.3). Every new language that you learn further expands your horizons and allows you to think in a more profound and proficient way about programming overall. That being said, let's turn our attention now to object-oriented programming concepts in general, and C++ in particular.

3.1.1 Brief Review of Object-Oriented Programming

Much of what we'll discuss in this book assumes you have a solid understanding of the principles of object-oriented design. If you're a bit rusty, the following section should serve as a pleasant and quick review. If you have no idea what I'm talking about in this section, I recommend you pick up a book or two on object-oriented programming (e.g., [7]) and C++ in particular (e.g., [46] and [36]) before continuing.

3.1.1.1 Classes and Objects

A *class* is a collection of attributes (data) and behaviors (code) that together form a useful, meaningful whole. A class is a *specification* describing how individual *instances* of the class, known as *objects*, should be constructed. For example, your pet Rover is an instance of the class "dog." Thus, there is a one-to-many relationship between a class and its instances.

3.1.1.2 Encapsulation

Encapsulation means that an object presents only a limited interface to the outside world; the object's internal state and implementation details are kept hidden. Encapsulation simplifies life for the user of the class, because he or she need only understand the class' limited interface, not the potentially intricate details of its implementation. It also allows the programmer who wrote the class to ensure that its instances are always in a logically consistent state.

3.1.1.3 Inheritance

Inheritance allows new classes to be defined as *extensions* to preexisting classes. The new class modifies or extends the data, interface and/or behavior of the existing class. If class Child extends class Parent, we say that Child *inherits from* or is *derived from* Parent. In this relationship, the class Parent is known as the *base class* or *superclass*, and the class Child is the *derived class*.

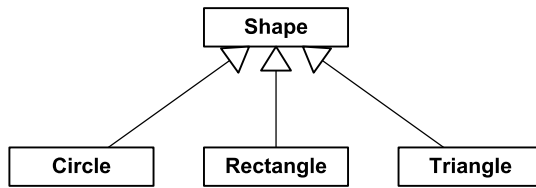


Figure 3.1. UML static class diagram depicting a simple class hierarchy.

or *subclass*. Clearly, inheritance leads to hierarchical (tree-structured) relationships between classes.

Inheritance creates an “is-a” relationship between classes. For example, a circle *is a* type of shape. So, if we were writing a 2D drawing application, it would probably make sense to derive our `Circle` class from a base class called `Shape`.

We can draw diagrams of class hierarchies using the conventions defined by the Unified Modeling Language (UML). In this notation, a rectangle represents a class, and an arrow with a hollow triangular head represents inheritance. The inheritance arrow points from child class to parent. See Figure 3.1 for an example of a simple class hierarchy represented as a UML *static class diagram*.

Multiple Inheritance

Some languages support *multiple inheritance* (MI), meaning that a class can have more than one parent class. In theory MI can be quite elegant, but in practice this kind of design usually gives rise to a lot of confusion and technical difficulties (see http://en.wikipedia.org/wiki/Multiple_inheritance). This is because multiple inheritance transforms a simple *tree* of classes into a potentially complex *graph*. A class graph can have all sorts of problems that never plague a simple tree—for example, the *deadly diamond* (http://en.wikipedia.org/wiki/Diamond_problem), in which a derived class ends up containing *two copies* of a grandparent base class (see Figure 3.2). (In C++, *virtual inheritance* allows one to avoid this doubling of the grandparent’s data.) Multiple inheritance also complicates casting, because the actual address of a pointer may change depending on which base class it is cast to. This happens because of the presence of multiple vtable pointers within the object.

Most C++ software developers avoid multiple inheritance completely or only permit it in a limited form. A common rule of thumb is to allow only simple, parentless classes to be multiply inherited into an otherwise strictly single-inheritance hierarchy. Such classes are sometimes called *mix-in classes*

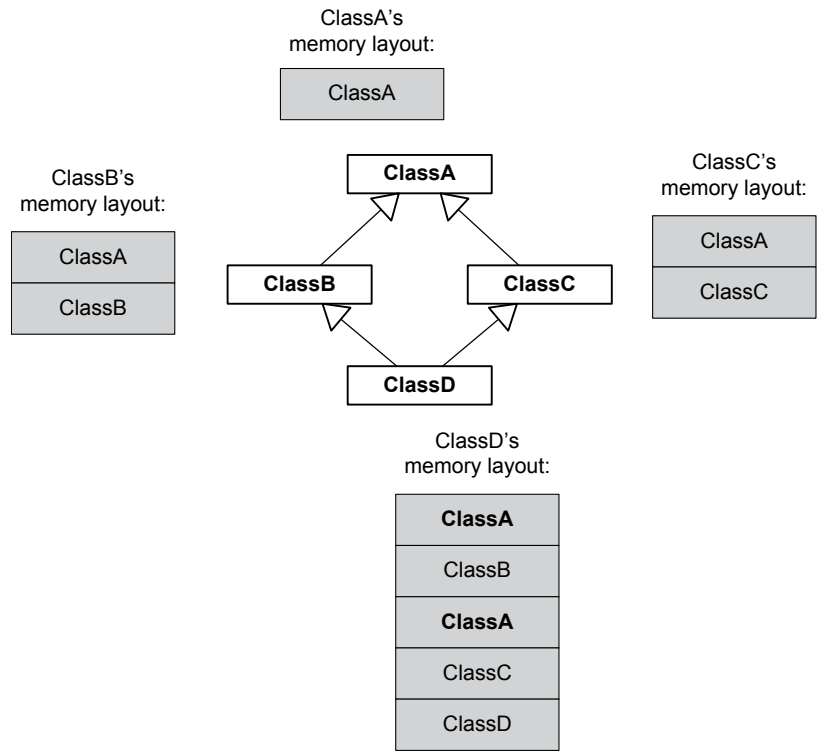


Figure 3.2. "Deadly diamond" in a multiple inheritance hierarchy.

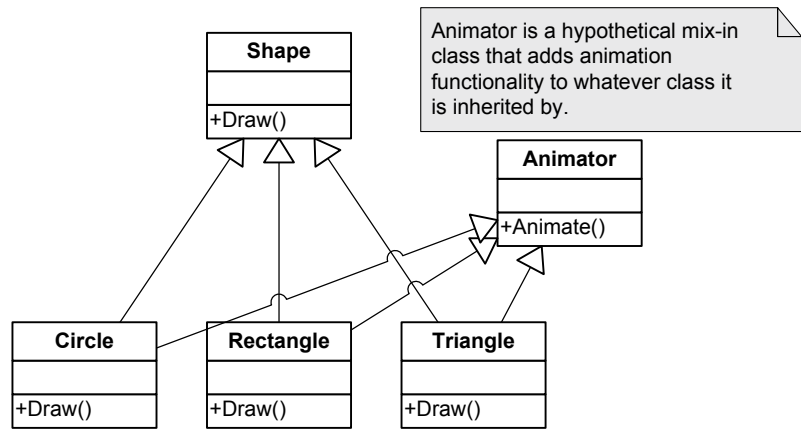


Figure 3.3. Example of a mix-in class.

because they can be used to introduce new functionality at arbitrary points in a class tree. See Figure 3.3 for a somewhat contrived example of a mix-in class.

3.1.1.4 Polymorphism

Polymorphism is a language feature that allows a collection of objects of different types to be manipulated through a single *common interface*. The common interface makes a heterogeneous collection of objects *appear* to be homogeneous, from the point of view of the code using the interface.

For example, a 2D painting program might be given a list of various shapes to draw on-screen. One way to draw this heterogeneous collection of shapes is to use a switch statement to perform different drawing commands for each distinct type of shape.

```
void drawShapes(std::list<Shape*>& shapes)
{
    std::list<Shape*>::iterator pShape = shapes.begin();
    std::list<Shape*>::iterator pEnd = shapes.end();

    for ( ; pShape != pEnd; pShape++)
    {
        switch (pShape->mType)
        {
            case CIRCLE:
                // draw shape as a circle
                break;

            case RECTANGLE:
                // draw shape as a rectangle
                break;

            case TRIANGLE:
                // draw shape as a triangle
                break;

            //...
        }
    }
}
```

The problem with this approach is that the `drawShapes()` function needs to “know” about all of the kinds of shapes that can be drawn. This is fine in a simple example, but as our code grows in size and complexity, it can become difficult to add new types of shapes to the system. Whenever a new shape

type is added, one must find every place in the code base where knowledge of the set of shape types is embedded—like this switch statement—and add a case to handle the new type.

The solution is to insulate the majority of our code from any knowledge of the types of objects with which it might be dealing. To accomplish this, we can define classes for each of the types of shapes we wish to support. All of these classes would inherit from the common base class *Shape*. A *virtual function*—the C++ language’s primary polymorphism mechanism—would be defined called *Draw()*, and each distinct shape class would implement this function in a different way. Without “knowing” what specific types of shapes it has been given, the drawing function can now simply call each shape’s *Draw()* function in turn.

```
struct Shape
{
    virtual void Draw() = 0; // pure virtual function
    virtual ~Shape() { } // ensure derived dtors are virtual
};

struct Circle : public Shape
{
    virtual void Draw()
    {
        // draw shape as a circle
    }
};

struct Rectangle : public Shape
{
    virtual void Draw()
    {
        // draw shape as a rectangle
    }
};

struct Triangle : public Shape
{
    virtual void Draw()
    {
        // draw shape as a triangle
    }
};
```

```
void drawShapes(std::list<Shape*>& shapes)
{
    std::list<Shape*>::iterator pShape = shapes.begin();
    std::list<Shape*>::iterator pEnd = shapes.end();

    for ( ; pShape != pEnd; pShape++)
    {
        pShape->Draw(); // call virtual function
    }
}
```

3.1.1.5 Composition and Aggregation

Composition is the practice of using a *group* of *interacting* objects to accomplish a high-level task. Composition creates a “has-a” or “uses-a” relationship between classes. (Technically speaking, the “has-a” relationship is called *composition*, while the “uses-a” relationship is called *aggregation*.) For example, a spaceship *has an* engine, which in turn *has a* fuel tank. Composition/aggregation usually results in the individual classes being simpler and more focused. Inexperienced object-oriented programmers often rely too heavily on inheritance and tend to underutilize aggregation and composition.

As an example, imagine that we are designing a graphical user interface for our game’s front end. We have a class `Window` that represents any rectangular GUI element. We also have a class called `Rectangle` that encapsulates the mathematical concept of a rectangle. A naïve programmer might derive the `Window` class from the `Rectangle` class (using an “is-a” relationship). But in a more flexible and well-encapsulated design, the `Window` class would *refer to* or *contain* a `Rectangle` (employing a “has-a” or “uses-a” relationship). This makes both classes simpler and more focused and allows the classes to be more easily tested, debugged and reused.

3.1.1.6 Design Patterns

When the same type of problem arises over and over, and many different programmers employ a very similar solution to that problem, we say that a *design pattern* has arisen. In object-oriented programming, a number of common design patterns have been identified and described by various authors. The most well-known book on this topic is probably the “Gang of Four” book [19].

Here are a few examples of common general-purpose design patterns.

- *Singleton*. This pattern ensures that a particular class has only one instance (the *singleton instance*) and provides a global point of access to it.
- *Iterator*. An iterator provides an efficient means of accessing the individual elements of a collection, without exposing the collection’s underlying

implementation. The iterator “knows” the implementation details of the collection so that its users don’t have to.

- *Abstract factory.* An abstract factory provides an interface for creating families of related or dependent classes without specifying their concrete classes.

The game industry has its own set of design patterns for addressing problems in every realm from rendering to collision to animation to audio. In a sense, this book is all about the high-level design patterns prevalent in modern 3D game engine design.

Janitors and RAII

As one very useful example of a design pattern, let’s have a brief look at the “resource acquisition is initialization” pattern (RAII). In this pattern, the acquisition and release of a resource (such as a file, a block of dynamically allocated memory, or a mutex lock) are bound to the constructor and destructor of a class, respectively. This prevents programmers from accidentally forgetting to release the resource—you simply construct a local instance of the class to acquire the resource, and let it fall out of scope to release it automatically. At Naughty Dog, we call such classes *janitors* because they “clean up” after you.

For example, whenever we need to allocate memory from a particular type of allocator, we *push* that allocator onto a global *allocator stack*, and when we’re done allocating we must always remember to *pop* the allocator off the stack. To make this more convenient and less error-prone, we use an *allocation janitor*. This tiny class’s constructor pushes the allocator, and its destructor pops the allocator:

```
class AllocJanitor
{
public:
    explicit AllocJanitor(mem::Context context)
    {
        mem::PushAllocator(context);
    }
    ~AllocJanitor()
    {
        mem::PopAllocator();
    }
};
```

To use the janitor class, we simply construct a local instance of it. When this instance drops out of scope, the allocator will be popped automatically:

```

void f()
{
    // do some work...

    // allocate temp buffers from single-frame allocator
    {
        AllocJanitor janitor(mem::Context::kSingleFrame);

        U8* pByteBuffer = new U8[SIZE];
        float* pFloatBuffer = new float[SIZE];

        // use buffers...

        // (NOTE: no need to free the memory because we
        // used a single-frame allocator)
    } // janitor pops allocator when it drops out of scope

    // do more work...
}

```

See <http://en.cppreference.com/w/cpp/language/raii> for more information on the highly useful RAII pattern.

3.1.2 C++ Language Standardization

Since its inception in 1979, the C++ language has been continually evolving. Bjarne Stroustrup, its inventor, originally named the language “C with Classes”, but it was renamed “C++” in 1983. The International Organization for Standardization (ISO, www.iso.org) first standardized the language in 1998—this version is known today as C++98. Since then, the ISO has been periodically publishing updated standards for the C++ language, with the goals of making the language more powerful, easier to use, and less ambiguous. These goals are achieved by refining the semantics and rules of the language, by adding new, more-powerful language features, and by deprecating, or removing entirely, those aspects of the language which have proven problematic or unpopular.

The most-recent variant of the C++ programming language standard is called C++17, which was published on July 31, 2017. The next iteration of the standard, C++2a, was in development at the time of this publication. The various versions of the C++ standard are summarized in chronological order below.

- C++98 was the first official C++ standard, established by the ISO in 1998.
- C++03 was introduced in 2003, to address various problems that had been identified in the C++98 standard.

- C++11 (also known as C++0x during much of its development) was approved by the ISO on August 12, 2011. C++11 added a large number of powerful new features to the language, including:
 - a type-safe `nullptr` literal, to replace the bug-prone `NULL` macro that had been inherited from the C language;
 - the `auto` and `decltype` keywords for type inference;
 - a “trailing return type” syntax, which allows a `decltype` of a function’s input arguments to be used to describe the return type of that function;
 - the `override` and `final` keywords for improved expressiveness when defining and overriding virtual functions;
 - defaulted and deleted functions (allowing the programmer to explicitly request that a compiler-generated default implementation be used, or that a function’s implementation should be undefined);
 - delegating constructors—the ability of one constructor to invoke another within the same class;
 - strongly-typed enumerations;
 - the `constexpr` keyword for defining compile-time constant values by evaluating expressions at compile time;
 - a uniform initialization syntax that extends the original braces-based POD initializers to cover non-POD types as well;
 - support for lambda functions and variable capture (closures);
 - the introduction of rvalue references and move semantics for more efficient handling of temporary objects; and
 - standardized attribute specifiers, to replace compiler-specific specifiers such as `__attribute__((...))` and `__declspec()`.
- C++11 also introduced an improved and expanded standard library, including support for threading (concurrent programming), improved smart pointer facilities and an expanded set of generic algorithms.
- C++14 was approved by the ISO on August 18, 2014 and released on December 15, 2014. Its additions and improvements to C++11 include:
 - return type deduction, which in many situations allows function return types to be declared using a simple `auto` keyword, without the need for the verbose trailing `decltype` expression required by C++11;

- generic lambdas, allowing a lambda to act like a templated function by using `auto` to declare its input arguments;
 - the ability to initialize “captured” variables in lambdas;
 - binary literals prefaced with `0b` (e.g., `0b10110110`);
 - support for digit separators in numeric literals, for improved readability (e.g., `1'000'000` instead of `1000000`);
 - variable templates, which allow template syntax to be used when declaring variables; and
 - relaxation of some restrictions on `constexpr`, including the ability to use `if`, `switch` and loops within constant expressions.
- C++17 was published by the ISO on July 31, 2017. It extends and improves C++14 in many ways, including but not limited to the following:
 - removed a number of out-dated and/or dangerous language features, including trigraphs, the `register` keyword, and the already-deprecated `auto_ptr` smart pointer class;
 - guarantees copy *elision*, the omission of unnecessary object copying;
 - exception specifications are now part of the type system, meaning that `void f() noexcept(true);` and `void f() noexcept(false);` are now distinct types;
 - adds two new literals to the language: UTF-8 character literals (e.g., `u8'x'`), and floating-point literals with a hexadecimal base and decimal exponent (e.g., `0xC.68p+2`);
 - introduces *structured bindings* to C++, allowing the values in a collection data type to be “unpacked” into individual variables (e.g., `auto [a, b] = func_that_returns_a_pair();`)—a syntax that is strikingly similar to that of returning multiple values from a function via a tuple in Python;
 - adds some useful standardized attributes, including `[[fallthrough]]` which allows you to explicitly document the fact that a missing `break` statement in a `switch` is intentional, thereby suppressing the warning that would otherwise be generated.

3.1.2.1 Further Reading

There are plenty of great online resources and books that describe the features of C++11, C++14 and C++17 in detail, so we won't attempt to cover them here. Here are a few useful references:

- The site <http://en.cppreference.com/w/cpp> provides an excellent C++ reference manual, including call-outs such as “since C++11” or “until C++17” to indicate when certain language features were added to or removed from the standard, respectively.
- Information about the ISO’s standardization efforts can be found at <https://isocpp.org/std>.
- See the following sites for good summaries of C++11’s major new features:
 - <https://www.codeproject.com/Articles/570638/Ten-Cplusplus-Features-Every-Cplusplus-Developer>, and
 - <https://blog.smartbear.com/development/the-biggest-changes-in-c11-and-why-you-should-care>.
- See http://thbecker.net/articles/auto_and_decltype/section_01.html for a great treatment of `auto` and `decltype`.
- See <http://www.drdobbs.com/cpp/the-c14-standard-what-you-need-to-know/240169034> for a good treatment of C++14’s changes to the C++11 standard.
- See <https://isocpp.org/files/papers/p0636r0.html> for a complete list of how the C++17 standard differs from C++14.

3.1.2.2 Which Language Features to Use?

As you read about all the cool new features being added to C++, it’s tempting to think that you need to use *all* of these features in your engine or game. However, just because a feature exists doesn’t mean your team needs to immediately start making use of it.

At Naughty Dog, we tend to take a conservative approach to adopting new language features into our codebase. As part of our studio’s coding standards, we have a list of those C++ language features that are approved for use in our runtime code, and another somewhat more liberal list of language features that are allowed in our offline tools code. There are a number of reasons for this cautious approach, which I’ll outline in the following sections.

Lack of Full Feature Support

The “bleeding edge” features may not be fully supported by your compiler. For example, LLVM/Clang, the C++ compiler used on the Sony Playstation

4, currently supports the entire C++11 standard in versions 3.3 and later, and all of C++14 in versions 3.4 and later. But its support for C++17 is spread across Clang versions 3.5 through 4.0, and it currently has no support for the draft C++2a standard. Also, Clang compiles code in C++98 mode by default—support for some of the more-advanced standards are accepted as extensions, but in order to enable full support one must pass specific command-line arguments to the compiler. See https://clang.llvm.org/cxx_status.html for details.

Cost of Switching between Standards

There's a non-zero cost to switching your codebase from one standard to another. Because of this, it's important for a game studio to decide on the most-advanced C++ standard to support, and then stick with it for a reasonable length of time (e.g., for the duration of one project). At Naughty Dog, we adopted the C++11 standard only relatively recently, and we only allowed its use in the code branch in which *The Last of Us Part II* is being developed. The code in the branch used for *Uncharted 4: A Thief's End* and *Uncharted: The Lost Legacy* had been written originally using the C++98 standard, and we decided that the relatively minor benefits of adopting C++11 features in that codebase did not outweigh the cost and risks of doing so.

Risk versus Reward

Not every C++ language feature is created equal. Some features are useful and pretty much universally acceptable, like `nullptr`. Others may have benefits, but also associated negatives. Still other language features may be deemed inappropriate for use in runtime engine code altogether.

As an example of a language feature with both benefits and downsides, consider the new C++11 interpretation of the `auto` keyword. This keyword certainly makes variables and functions more convenient to write. But the Naughty Dog programmers recognized that over-use of `auto` can lead to obfuscated code: As an extreme example, imagine trying to read a `.cpp` file written by somebody else, in which virtually every variable, function argument and return value is declared `auto`. It would be like reading a typeless language such as Python or Lisp. One of the benefits of a strongly-typed language like C++ is the programmer's ability to quickly and easily determine the types of all variables. As such, we decided to adopt a simple rule: `auto` may only be used when declaring iterators, in situations where no other approach works (such as within template definitions), or in special cases when code clarity, readability and maintainability is significantly improved by its use. In all other cases, we require the use of explicit type declarations.

As an example of a language feature that could be deemed inappropriate for use in a commercial product like a game, consider *template metaprogramming*. Andrei Alexandrescu's Loki library [3] makes heavy use of template metaprogramming to do some pretty interesting and amazing things. However, the resulting code is tough to read, is sometimes non-portable, and presents programmers with an extremely high barrier to understanding. The programming leads at Naughty Dog believe that any programmer should be able to jump in and debug a problem on short notice, even in code with which he or she may not be very familiar. As such, Naughty Dog prohibits complex template metaprogramming in runtime engine code, with exceptions made only on a case-by-case basis where the benefits are deemed to outweigh the costs.

In summary, remember that when you have a hammer, everything can tend to look like a nail. Don't be tempted to use features of your language just because they're there (or because they're new). A judicious and carefully considered approach will result in a stable codebase that's as easy as possible to understand, reason about, debug and maintain.

3.1.3 Coding Standards: Why and How Much?

Discussions of coding conventions among engineers can often lead to heated "religious" debates. I do not wish to spark any such debate here, but I will go so far as to suggest that following at least a minimal set of coding standards is a good idea. Coding standards exist for two primary reasons.

1. Some standards make the code more readable, understandable and maintainable.
2. Other conventions help to prevent programmers from shooting themselves in the foot. For example, a coding standard might encourage the programmer to use only a smaller, more testable and less error-prone subset of the whole language. The C++ language is rife with possibilities for abuse, so this kind of coding standard is particularly important when using C++.

In my opinion, the most important things to achieve in your coding conventions are the following.

- *Interfaces are king.* Keep your interfaces (.h files) clean, simple, minimal, easy to understand and well-commented.
- *Good names encourage understanding and avoid confusion.* Stick to intuitive names that map directly to the purpose of the class, function or variable in question. Spend time up-front identifying a good name. Avoid

a naming scheme that requires programmers to use a look-up table in order to decipher the meaning of your code. Remember that high-level programming languages like C++ are intended for *humans* to read. (If you disagree, just ask yourself why you don't write all your software directly in machine language.)

- *Don't clutter the global namespace.* Use C++ namespaces or a common naming prefix to ensure that your symbols don't collide with symbols in other libraries. (But be careful not to overuse namespaces, or nest them too deeply.) Name `#defined` symbols with extra care; remember that C++ preprocessor macros are really just text substitutions, so they cut across all C/C++ scope and namespace boundaries.
- *Follow C++ best practices.* Books like the *Effective C++* series by Scott Meyers [36,37], *Meyers' Effective STL* [38] and *Large-Scale C++ Software Design* by John Lakos [31] provide excellent guidelines that will help keep you out of trouble.
- *Be consistent.* The rule I try to use is as follows: If you're writing a body of code from scratch, feel free to invent any convention you like—then stick to it. When editing preexisting code, try to follow whatever conventions have already been established.
- *Make errors stick out.* Joel Spolsky wrote an excellent article on coding conventions, which can be found at <http://www.joelonsoftware.com/articles/Wrong.html>. Joel suggests that the “cleanest” code is not necessarily code that looks neat and tidy on a superficial level, but rather the code that is written in a way that makes common programming errors *easier to see*. Joel's articles are always fun and educational, and I highly recommend this one.

3.2 Catching and Handling Errors

There are a number of ways to catch and handle error conditions in a game engine. As a game programmer, it's important to understand these different mechanisms, their pros and cons and when to use each one.

3.2.1 Types of Errors

In any software project there are two basic kinds of error conditions: *user errors* and *programmer errors*. A user error occurs when the user of the program does something incorrect, such as typing an invalid input, attempting to open a file that does not exist, etc. A programmer error is the result of a *bug* in the code itself. Although it may be triggered by something the user has done, the

essence of a programmer error is that the problem could have been avoided if the programmer had not made a mistake, and the user has a reasonable expectation that the program *should* have handled the situation gracefully.

Of course, the definition of “user” changes depending on context. In the context of a game project, user errors can be roughly divided into two categories: errors caused by the person playing the game and errors caused by the people who are making the game during development. It is important to keep track of which type of user is affected by a particular error and handle the error appropriately.

There’s actually a third kind of user—the other programmers on your team. (And if you are writing a piece of game middleware software, like Havok or OpenGL, this third category extends to other programmers all over the world who are using your library.) This is where the line between *user errors* and *programmer errors* gets blurry. Let’s imagine that programmer A writes a function $f()$, and programmer B tries to call it. If B calls $f()$ with invalid arguments (e.g., a null pointer, or an out-of-range array index), then this could be seen as a user error by programmer A, but it would be a programmer error from B’s point of view. (Of course, one can also argue that programmer A should have anticipated the passing of invalid arguments and should have handled them gracefully, so the problem really is a programmer error, on A’s part.) The key thing to remember here is that the line between user and programmer can shift depending on context—it is rarely a black-and-white distinction.

3.2.2 Handling Errors

When handling errors, the requirements differ significantly between the two types. It is best to handle *user errors* as gracefully as possible, displaying some helpful information to the user and then allowing him or her to continue working—or in the case of a game, to continue playing. Programmer errors, on the other hand, should *not* be handled with a graceful “inform and continue” policy. Instead, it is usually best to halt the program and provide detailed low-level debugging information, so that a programmer can quickly identify and fix the problem. In an ideal world, *all* programmer errors would be caught and fixed before the software ships to the public.

3.2.2.1 Handling Player Errors

When the “user” is the person playing your game, errors should obviously be handled within the context of gameplay. For example, if the player attempts to reload a weapon when no ammo is available, an audio cue and/or an animation can indicate this problem to the player without taking him or her “out of the game.”

3.2.2.2 Handling Developer Errors

When the “user” is someone who is making the game, such as an artist, animator or game designer, errors may be caused by an invalid asset of some sort. For example, an animation might be associated with the wrong skeleton, or a texture might be the wrong size, or an audio file might have been sampled at an unsupported sample rate. For these kinds of *developer errors*, there are two competing camps of thought.

On the one hand, it seems important to prevent bad game assets from persisting for too long. A game typically contains literally thousands of assets, and a problem asset might get “lost,” in which case one risks the possibility of the bad asset surviving all the way into the final shipping game. If one takes this point of view to an extreme, then the best way to handle bad game assets is to prevent the entire game from running whenever even a single problematic asset is encountered. This is certainly a strong incentive for the developer who created the invalid asset to remove or fix it immediately.

On the other hand, game development is a messy and iterative process, and generating “perfect” assets the first time around is rare indeed. By this line of thought, a game engine should be robust to almost any kind of problem imaginable, so that work can continue even in the face of totally invalid game asset data. But this too is not ideal, because the game engine would become bloated with error-catching and error-handling code that won’t be needed once the development pace settles down and the game ships. And the probability of shipping the product with “bad” assets becomes too high.

In my experience, the best approach is to find a middle ground between these two extremes. When a developer error occurs, I like to *make the error obvious* and then allow the team to continue to work in the presence of the problem. It is extremely costly to prevent all the other developers on the team from working, just because one developer tried to add an invalid asset to the game. A game studio pays its employees well, and when multiple team members experience downtime, the costs are multiplied by the number of people who are prevented from working. Of course, we should only handle errors in this way when it is practical to do so, without spending inordinate amounts of engineering time, or bloating the code.

As an example, let’s suppose that a particular mesh cannot be loaded. In my view, it’s best to draw a big red box in the game world at the places that mesh would have been located, perhaps with a text string hovering over each one that reads, “Mesh *blah-dee-blah* failed to load.” This is superior to printing an easy-to-miss message to an error log. And it’s *far* better than just crashing the game, because then no one will be able to work until that one mesh

reference has been repaired. Of course, for particularly egregious problems it's fine to just spew an error message and crash. There's no silver bullet for all kinds of problems, and your judgment about what type of error handling approach to apply to a given situation will improve with experience.

3.2.2.3 Handling Programmer Errors

The best way to detect and handle programmer errors (a.k.a. bugs) is often to embed error-checking code into your source code and arrange for failed error checks to halt the program. Such a mechanism is known as an *assertion system*; we'll investigate assertions in detail in Section 3.2.3.3. Of course, as we said above, one programmer's user error is another programmer's bug; hence, assertions are not always the right way to handle every programmer error. Making a judicious choice between an assertion and a more graceful error-handling technique is a skill that one develops over time.

3.2.3 Implementation of Error Detection and Handling

We've looked at some philosophical approaches to handling errors. Now let's turn our attention to the choices we have as programmers when it comes to implementing error detection and handling code.

3.2.3.1 Error Return Codes

A common approach to handling errors is to return some kind of failure code from the function in which the problem is first detected. This could be a Boolean value indicating success or failure, or it could be an "impossible" value, one that is outside the range of normally returned results. For example, a function that returns a positive integer or floating-point value could return a negative value to indicate that an error occurred. Even better than a Boolean or an "impossible" return value, the function could be designed to return an enumerated value to indicate success or failure. This clearly separates the error code from the output(s) of the function, and the exact nature of the problem can be indicated on failure (e.g., `enum Error { kSuccess, kAssetNotFound, kInvalidRange, ... }`).

The calling function should intercept error return codes and act appropriately. It might handle the error immediately. Or, it might work around the problem, complete its own execution and then pass the error code on to whatever function called *it*.

3.2.3.2 Exceptions

Error return codes are a simple and reliable way to communicate and respond to error conditions. However, error return codes have their drawbacks. Perhaps the biggest problem with error return codes is that the function that detects an error may be totally unrelated to the function that is capable of handling the problem. In the worst-case scenario, a function that is 40 calls deep in the call stack might detect a problem that can only be handled by the top-level game loop, or by `main()`. In this scenario, every one of the 40 functions on the call stack would need to be written so that it can pass an appropriate error code all the way back up to the top-level error-handling function.

One way to solve this problem is to throw an exception. *Exception handling* is a very powerful feature of C++. It allows the function that detects a problem to communicate the error to the rest of the code without knowing anything about which function might handle the error. When an exception is thrown, relevant information about the error is placed into a data object of the programmer's choice known as an *exception object*. The call stack is then automatically unwound, in search of a calling function that has wrapped its call in a `try-catch` block. If a `try-catch` block is found, the exception object is matched against all possible `catch` clauses, and if a match is found, the corresponding `catch`'s code block is executed. The destructors of any automatic variables are called as needed during the stack unwinding process.

The ability to separate error detection from error handling in such a clean way is certainly attractive, and exception handling is an excellent choice for some software projects. However, exception handling does add some overhead to the program. The stack frame of any function that contains a `try-catch` block must be augmented to contain additional information required by the stack unwinding process. Also, if even one function in your program (or a library that your program links with) uses exception handling, your entire program must use exception handling—the compiler can't know which functions might be above you on the call stack when you throw an exception.

That said, it is possible to “sandbox” a library or libraries that make use of exception handling, in order to avoid your entire game engine having to be written with exceptions enabled. To do this, you would wrap all the API calls into the libraries in question in functions that are implemented in a translation unit that has exception handling enabled. Each of these functions would catch all possible exceptions in a `try/catch` block and convert them into error return codes. Any code that links with your wrapper library can therefore safely disable exception handling.

Arguably more important than the overhead issue is the fact that excep-

tions are in some ways no better than `goto` statements. Joel Spolsky of Microsoft and Fog Creek Software fame argues that exceptions are in fact *worse* than `gotos` because they aren't easily seen in the source code. A function that neither throws nor catches exceptions may nevertheless become involved in the stack-unwinding process, if it finds itself sandwiched between such functions in the call stack. And the unwinding process is itself imperfect: Your software can easily be left in an invalid state unless the programmer considers every possible way that an exception can be thrown, and handles it appropriately. This can make writing robust software difficult. When the possibility for exception throwing exists, pretty much every function in your codebase needs to be robust to the carpet being pulled out from under it and all its local objects destroyed whenever it makes a function call.

Another issue with exception handling is its cost. Although in theory modern exception handling frameworks don't introduce additional runtime overhead in the error-free case, this is not necessarily true in practice. For example, the code that the compiler adds to your functions for unwinding the call stack when an exception occurs tends to produce an overall increase in code size. This might degrade I-cache performance, or cause the compiler to decide not to inline a function that it otherwise would have.

Clearly there are some pretty strong arguments for turning *off* exception handling in your game engine altogether. This is the approach employed at Naughty Dog and also on most of the projects I've worked on at Electronic Arts and Midway. In his capacity as Engine Director at Insomniac Games, Mike Acton has clearly stated his objection to the use of exception handling in runtime game code on numerous occasions. JPL and NASA also disallow exception handling in their mission-critical embedded software, presumably for the same reasons we tend to avoid it in the game industry. That said, your mileage certainly may vary. There is no perfect tool and no one right way to do anything. When used judiciously, exceptions *can* make your code easier to write and work with; just be careful out there!

There are many interesting articles on this topic on the web. Here's one good thread that covers most of the key issues on both sides of the debate:

- <http://www.joelonsoftware.com/items/2003/10/13.html>
- <http://www.nedbatchelder.com/text/exceptions-vs-status.html>
- <http://www.joelonsoftware.com/items/2003/10/15.html>

Exceptions and RAI

The “resource acquisition is initialization” pattern (RAII, see Section 3.1.1.6) is often used in conjunction with *exception handling*: The constructor attempts to

acquire the desired resource, and throws an exception if it fails to do so. This is done to avoid the need for an if check to test the status of the object after it has been created—if the constructor returns without throwing an exception, we know for certain that the resource was successfully acquired.

However, the RAII pattern can be used even without exceptions. All it requires is a little discipline to check the status of each new resource object when it is first created. After that, all of the other benefits of RAII can be reaped. (Exceptions can also be replaced by assertion failures to signal the failure of some kinds of resource acquisitions.)

3.2.3.3 Assertions

An *assertion* is a line of code that checks an expression. If the expression evaluates to true, nothing happens. But if the expression evaluates to false, the program is stopped, a message is printed and the debugger is invoked if possible.

Assertions check a programmer's assumptions. They act like *land mines* for bugs. They check the code when it is first written to ensure that it is functioning properly. They also ensure that the original assumptions continue to hold for long periods of time, even when the code around them is constantly changing and evolving. For example, if a programmer changes code that used to work, but accidentally violates its original assumptions, they'll hit the land mine. This immediately informs the programmer of the problem and permits him or her to rectify the situation with minimum fuss. Without assertions, bugs have a tendency to "hide out" and manifest themselves later in ways that are difficult and time-consuming to track down. But with assertions embedded in the code, bugs announce themselves the moment they are introduced—which is usually the best moment to fix the problem, while the code changes that caused the problem are fresh in the programmer's mind. Steve Maguire provides an in-depth discussion of assertions in his must-read book, *Writing Solid Code* [35].

The cost of the assertion checks can usually be tolerated during development, but stripping out the assertions prior to shipping the game can buy back that little bit of crucial performance if necessary. For this reason assertions are generally implemented in such a way as to allow the checks to be stripped out of the executable in non-debug build configurations. In C, an `assert()` macro is provided by the standard library header file `<assert.h>`; in C++, it's provided by the `<cassert>` header.

The standard library's definition of `assert()` causes it to be defined in debug builds (builds with the `DEBUG` preprocessor symbol defined) and

stripped in non-debug builds (builds with the `NDEBUG` preprocessor symbol defined). In a game engine, you may want finer-grained control over which build configurations retain assertions, and which configurations strip them out. For example, your game might support more than just a debug and development build configuration—you might also have a shipping build with global optimizations enabled, and perhaps even a PGO build for use by profile-guided optimization tools (see Section 2.2.4). Or you might also want to define different “flavors” of assertions—some that are always retained even in the shipping version of your game, and others that are stripped out of the non-shipping build. For these reasons, let’s take a look at how you can implement your own `ASSERT()` macro using the C/C++ preprocessor.

Assertion Implementation

Assertions are usually implemented via a combination of a `#defined` macro that evaluates to an `if/else` clause, a function that is called when the assertion fails (the expression evaluates to `false`), and a bit of assembly code that halts the program and breaks into the debugger when one is attached. Here’s a typical implementation:

```
#if ASSERTIONS_ENABLED

// define some inline assembly that causes a break
// into the debugger -- this will be different on each
// target CPU
#define debugBreak() asm { int 3 }

// check the expression and fail if it is false
#define ASSERT(expr) \
    if (expr) { } \
    else \
    { \
        reportAssertionFailure(#expr, \
                                __FILE__, __LINE__); \
        debugBreak(); \
    }

#else

#define ASSERT(expr) // evaluates to nothing

#endif
```

Let’s break down this definition so we can see how it works:

- The outer `#if/#else/#endif` is used to strip assertions from the code base. When `ASSERTIONS_ENABLED` is nonzero, the `ASSERT()` macro is defined in its full glory, and all assertion checks in the code will be included in the program. But when assertions are turned off, `ASSERT(expr)` evaluates to nothing, and all instances of it in the code are effectively removed.
- The `debugBreak()` macro evaluates to whatever assembly-language instructions are required in order to cause the program to halt and the debugger to take charge (if one is connected). This differs from CPU to CPU, but it is usually a single assembly instruction.
- The `ASSERT()` macro itself is defined using a full `if/else` statement (as opposed to a lone `if`). This is done so that the macro can be used in any context, even within *other* unbracketed `if/else` statements.

Here's an example of what would happen if `ASSERT()` were defined using a solitary `if`:

```
// WARNING: NOT A GOOD IDEA!
#define ASSERT(expr)  if (!(expr)) debugBreak()

void f()
{
    if (a < 5)
        ASSERT(a >= 0);
    else
        doSomething(a);
}
```

This expands to the following *incorrect* code:

```
void f()
{
    if (a < 5)
        if (!(a >= 0))
            debugBreak();
    else // oops! bound to the wrong if()!
        doSomething(a);
}
```

- The `else` clause of an `ASSERT()` macro does two things. It displays some kind of message to the programmer indicating what went wrong, and then it breaks into the debugger. Notice the use of `#expr` as the first argument to the message display function. The pound (`#`) preprocessor operator causes the expression `expr` to be turned into a string, thereby allowing it to be printed out as part of the assertion failure message.

- Notice also the use of `__FILE__` and `__LINE__`. These compiler-defined macros magically contain the .cpp file name and line number of the line of code on which they appear. By passing them into our message display function, we can print the exact location of the problem.

I highly recommend the use of assertions in your code. However, it's important to be aware of their performance cost. You may want to consider defining two kinds of assertion macros. The regular `ASSERT()` macro can be left active in *all* builds, so that errors are easily caught even when not running in debug mode. A second assertion macro, perhaps called `SLOW_ASSERT()`, could be activated only in debug builds. This macro could then be used in places where the cost of assertion checking is too high to permit inclusion in release builds. Obviously `SLOW_ASSERT()` is of lower utility, because it is stripped out of the version of the game that your testers play every day. But at least these assertions become active when programmers are debugging their code.

It's also extremely important to use assertions properly. They should be used to catch bugs in the program itself—*never* to catch user errors. Also, assertions should always cause the entire game to halt when they fail. It's usually a bad idea to allow assertions to be skipped by testers, artists, designers and other non-engineers. (This is a bit like the boy who cried wolf: if assertions can be skipped, then they cease to have any significance, rendering them ineffective.) In other words, assertions should only be used to catch fatal errors. If it's OK to continue past an assertion, then it's probably better to notify the user of the error in some other way, such as with an on-screen message, or some ugly bright-orange 3D graphics.

Compile-Time Assertions

One weakness of assertions, as we've discussed them thus far, is that the conditions encoded within them are only checked at runtime. We have to run the program, *and* the code path in question must actually *execute*, in order for an assertion's condition to be checked.

Sometimes the condition we're checking within an assertion involves information that is entirely known at compile time. For example, let's say we're defining a struct that for some reason needs to be exactly 128 bytes in size. We want to add an assertion so that if another programmer (or a future version of yourself) decides to change the size of the struct, the compiler will give us an error message. In other words, we'd like to write something like this:

```

struct NeedsToBe128Bytes
{
    U32    m_a;
    F32    m_b;
    // etc.
};

// sadly this doesn't work...
ASSERT(sizeof(NeedsToBe128Bytes) == 128);

```

The problem of course is that the `ASSERT()` (or `assert()`) macro needs to be executable at runtime, and one can't even put executable code at global scope in a .cpp file outside of a function definition. The solution to this problem is a *compile-time assertion*, also known as a *static assertion*.

Starting with C++11, the standard library defines a macro named `static_assert()` for us. So we can re-write the example above as follows:

```

struct NeedsToBe128Bytes
{
    U32    m_a;
    F32    m_b;
    // etc.
};

static_assert(sizeof(NeedsToBe128Bytes) == 128,
    "wrong size");

```

If you're not using C++11, you can always roll your own `STATIC_ASSERT()` macro. It can be implemented in a number of different ways, but the basic idea is always the same: The macro places a declaration into your code that (a) is legal at file scope, (b) evaluates the desired expression at compile time rather than runtime, and (c) produces a compile error if and only if the expression is false. Some methods of defining `STATIC_ASSERT()` rely on compiler-specific details, but here's one reasonably portable way to define it:

```

#define _ASSERT_GLUE(a, b)  a ## b
#define ASSERT_GLUE(a, b)  _ASSERT_GLUE(a, b)

#define STATIC_ASSERT(expr) \
    enum \
    { \
        ASSERT_GLUE(g_assert_fail_, __LINE__) \
            = 1 / (int)(!!(expr)) \
    }

STATIC_ASSERT(sizeof(int) == 4);    // should pass
STATIC_ASSERT(sizeof(float) == 1); // should fail

```

This works by defining an anonymous enumeration containing a single enumerator. The name of the enumerator is made unique (within the translation unit) by “gluing” a fixed prefix such as `g_assert_fail_` to a unique suffix—in this case, the line number on which the `STATIC_ASSERT()` macro is invoked. The value of the enumerator is set to `1 / (!! (expr))`. The double negation `!!` ensures that `expr` has a Boolean value. This value is then cast to an `int`, yielding either 1 or 0 depending on whether the expression is `true` or `false`, respectively. If the expression is `true`, the enumerator will be set to the value `1/1` which is one. But if the expression is `false`, we’ll be asking the compiler to set the enumerator to the value `1/0` which is illegal, and will trigger a compile error.

When our `STATIC_ASSERT()` macro as defined above fails, Visual Studio 2015 produces a compile-time error message like this:

```
1>test.cpp(48): error C2131: expression did not evaluate to
                    a constant
1> test.cpp(48): note: failure was caused by an undefined
                    arithmetic operation
```

Here’s another way to define `STATIC_ASSERT()` using template specialization. In this example, we first check to see if we’re using C++11 or beyond. If so, we use the standard library’s implementation of `static_assert()` for maximum portability. Otherwise we fall back to our custom implementation.

```
#ifndef __cplusplus
    #if __cplusplus >= 201103L
        #define STATIC_ASSERT(expr) \
            static_assert(expr, \
                "static assert failed:" \
                #expr)
    #else
        // declare a template but only define the
        // true case (via specialization)
        template<bool> class TStaticAssert;
        template<> class TStaticAssert<true> {};

        #define STATIC_ASSERT(expr) \
            enum \
            { \
                ASSERT_GLUE(g_assert_fail_, __LINE__) \
                = sizeof(TStaticAssert<!!(expr)>) \
            }
    #endif
#endif
```

This implementation, using template specialization, may be preferable to the previous one using division by zero, because it produces a slightly better error message in Visual Studio 2015:

```
1>test.cpp(48): error C2027: use of undefined type
           'TStaticAssert<false>'
1>test.cpp(48): note: see declaration of
           'TStaticAssert<false>'
```

However, each compiler handles error reporting differently, so your mileage may vary. For more implementation ideas for compile-time assertions, one good reference is http://www.pixelbeat.org/programming/gcc/static_assert.html.

3.3 Data, Code and Memory Layout

3.3.1 Numeric Representations

Numbers are at the heart of everything that we do in game engine development (and software development in general). Every software engineer should understand how numbers are represented and stored by a computer. This section will provide you with the basics you'll need throughout the rest of the book.

3.3.1.1 Numeric Bases

People think most naturally in *base ten*, also known as *decimal notation*. In this notation, ten distinct digits are used (0 through 9), and each digit from right to left represents the next highest power of 10. For example, the number 7803 = $(7 \times 10^3) + (8 \times 10^2) + (0 \times 10^1) + (3 \times 10^0) = 7000 + 800 + 0 + 3$.

In computer science, mathematical quantities such as integers and real-valued numbers need to be stored in the computer's memory. And as we know, computers store numbers in *binary* format, meaning that only the two digits 0 and 1 are available. We call this a *base-two* representation, because each digit from right to left represents the next highest power of 2. Computer scientists sometimes use a prefix of "0b" to represent binary numbers. For example, the binary number 0b1101 is equivalent to decimal 13, because $0b1101 = (1 \times 2^3) + (1 \times 2^2) + (0 \times 2^1) + (1 \times 2^0) = 8 + 4 + 0 + 1 = 13$.

Another common notation popular in computing circles is *hexadecimal*, or *base 16*. In this notation, the 10 digits 0 through 9 and the six letters A through F are used; the letters A through F replace the decimal values 10 through 15, respectively. A prefix of "0x" is used to denote hex numbers in the C and C++

programming languages. This notation is popular because computers generally store data in groups of 8 bits known as *bytes*, and since a single hexadecimal digit represents 4 bits exactly, a *pair* of hex digits represents a byte. For example, the value $0xFF = 0b11111111 = 255$ is the largest number that can be stored in 8 bits (1 byte). Each digit in a hexadecimal number, from right to left, represents the next power of 16. So, for example, $0xB052 = (11 \times 16^3) + (0 \times 16^2) + (5 \times 16^1) + (2 \times 16^0) = (11 \times 4096) + (0 \times 256) + (5 \times 16) + (2 \times 1) = 45,138$.

3.3.1.2 Signed and Unsigned Integers

In computer science, we use both signed and unsigned integers. Of course, the term “unsigned integer” is actually a bit of a misnomer—in mathematics, the *whole numbers* or *natural numbers* range from 0 (or 1) up to positive infinity, while the *integers* range from negative infinity to positive infinity. Nevertheless, we’ll use computer science lingo in this book and stick with the terms “signed integer” and “unsigned integer.”

Most modern personal computers and game consoles work most easily with integers that are 32 bits or 64 bits wide (although 8- and 16-bit integers are also used a great deal in game programming as well). To represent a 32-bit unsigned integer, we simply encode the value using binary notation (see above). The range of possible values for a 32-bit unsigned integer is $0x00000000$ (0) to $0xFFFFFFFF$ (4,294,967,295).

To represent a *signed* integer in 32 bits, we need a way to differentiate between positive and negative values. One simple approach called the *sign and magnitude* encoding reserves the most significant bit as a *sign bit*. When this bit is zero, the value is positive, and when it is one, the value is negative. This leaves us 31 bits to represent the magnitude of the value, effectively cutting the range of possible magnitudes in half (but allowing both positive and negative forms of every distinct magnitude, including zero).

Most microprocessors use a slightly more efficient technique for encoding negative integers, called *two’s complement* notation. This notation has only one representation for the value zero, as opposed to the two representations possible with simple sign bit (positive zero and negative zero). In 32-bit two’s complement notation, the value $0xFFFFFFFF$ is interpreted to mean -1 , and negative values count down from there. Any value with the most significant bit set is considered negative. So values from $0x00000000$ (0) to $0x7FFFFFFF$ (2,147,483,647) represent positive integers, and $0x80000000$ ($-2,147,483,648$) to $0xFFFFFFFF$ (-1) represent negative integers.

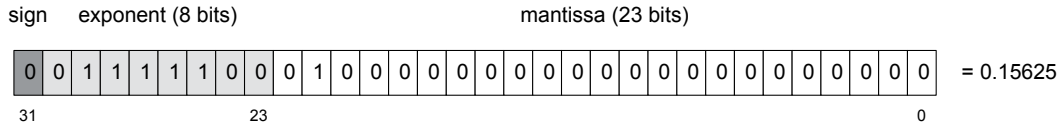


Figure 3.5. IEEE-754 32-bit floating-point format.

to lay out these three components in memory, but the most common standard is IEEE-754. It states that a 32-bit floating-point number will be represented with the sign in the most significant bit, followed by 8 bits of exponent and finally 23 bits of mantissa.

The value v represented by a sign bit s , an exponent e and a mantissa m is $v = s \times 2^{(e-127)} \times (1 + m)$.

The sign bit s has the value $+1$ or -1 . The exponent e is biased by 127 so that negative exponents can be easily represented. The mantissa begins with an implicit 1 that is not actually stored in memory, and the rest of the bits are interpreted as inverse powers of two. Hence the value represented is really $1 + m$, where m is the fractional value stored in the mantissa.

For example, the bit pattern shown in Figure 3.5 represents the value 0.15625, because $s = 0$ (indicating a positive number), $e = 0b01111100 = 124$ and $m = 0b0100... = 0 \times 2^{-1} + 1 \times 2^{-2} = \frac{1}{4}$. Therefore,

$$\begin{aligned}
 v &= s \times 2^{(e-127)} \times (1 + m) \\
 &= (+1) \times 2^{(124-127)} \times (1 + \frac{1}{4}) \\
 &= 2^{-3} \times \frac{5}{4} \\
 &= \frac{1}{8} \times \frac{5}{4} \\
 &= 0.125 \times 1.25 = 0.15625.
 \end{aligned}$$

The Trade-Off between Magnitude and Precision

The *precision* of a floating-point number increases as the *magnitude* decreases, and vice versa. This is because there are a fixed number of bits in the mantissa, and these bits must be shared between the whole part and the fractional part of the number. If a large percentage of the bits are spent representing a large magnitude, then a small percentage of bits are available to provide fractional precision. In physics the term *significant digits* is typically used to describe this concept (http://en.wikipedia.org/wiki/Significant_digits).

To understand the trade-off between magnitude and precision, let's look at the largest possible floating-point value, $FLT_MAX \approx 3.403 \times 10^{38}$, whose

representation in 32-bit IEEE floating-point format is 0x7F7FFFFF. Let's break this down:

- The largest absolute value that we can represent with a 23-bit mantissa is 0x00FFFFFF in hexadecimal, or 24 consecutive binary ones—that's 23 ones in the mantissa, plus the implicit leading one.
- An exponent of 255 has a special meaning in the IEEE-754 format—it is used for values like not-a-number (NaN) and infinity—so it cannot be used for regular numbers. Hence the maximum eight-bit exponent is actually 254, which translates into 127 after subtracting the implicit bias of 127.

So `FLT_MAX` is $0x00FFFFFF \times 2^{127} = 0xFFFFFFFF000000000000000000000000$. In other words, our 24 binary ones were shifted up by 127 bit positions, leaving $127 - 23 = 104$ binary zeros (or $104/4 = 26$ hexadecimal zeros) after the least significant digit of the mantissa. Those trailing zeros don't correspond to any actual bits in our 32-bit floating-point value—they just appear out of thin air because of the exponent. If we were to subtract a small number (where "small" means any number composed of fewer than 26 hexadecimal digits) from `FLT_MAX`, the result would still be `FLT_MAX`, because those 26 least significant hexadecimal digits don't really exist!

The opposite effect occurs for floating-point values whose magnitudes are much less than one. In this case, the exponent is large but negative, and the significant digits are shifted in the opposite direction. We trade the ability to represent large magnitudes for high precision. In summary, we always have the same number of *significant digits* (or really *significant bits*) in our floating-point numbers, and the exponent can be used to shift those significant bits into higher or lower ranges of magnitude.

Subnormal Values

Another subtlety to notice is that there is a finite gap between zero and the smallest nonzero value we can represent with floating-point notation (as it has been described thus far). The smallest nonzero magnitude we can represent is $\text{FLT_MIN} = 2^{-126} \approx 1.175 \times 10^{-38}$, which has a binary representation of 0x00800000 (i.e., the exponent is 0x01, or -126 after subtracting the bias, and the mantissa is all zeros except for the implicit leading one). The next smallest valid value is zero, so there is a finite gap between the values $-\text{FLT_MIN}$ and $+\text{FLT_MIN}$. This underscores the fact that the real number line is *quantized* when using a floating-point representation. (Note that the C++ standard library exposes `FLT_MIN` as the rather more verbose `std::numeric_limits`

`<float>::min()`. We'll stick to `FLT_MIN` in this book for brevity.)

The gap around zero can be filled by employing an extension to the floating-point representation known as *denormalized values*, also known as *subnormal values*. With this extension, any floating-point value with a biased exponent of 0 is interpreted as a subnormal number. The exponent is treated as if it had been a 1 instead of a 0, and the implicit leading 1 that normally sits in front of the bits of the mantissa is changed to a 0. This has the effect of filling the gap between `-FLT_MIN` and `+FLT_MIN` with a linear sequence of evenly-spaced subnormal values. The positive subnormal `float` that is closest to zero is represented by the constant `FLT_TRUE_MIN`.

The benefit of using subnormal values is that it provides greater precision near zero. For example, it ensures that the following two expressions are equivalent, even for values of `a` and `b` that are very close to `FLT_MIN`:

```
if (a == b) { ... }
if (a - b == 0.0f) { ... }
```

Without subnormal values, the expression `a - b` could evaluate to zero even when `a != b`.

Machine Epsilon

For a particular floating-point representation, the *machine epsilon* is defined to be the smallest floating-point value ϵ that satisfies the equation, $1 + \epsilon \neq 1$. For an IEEE-754 floating-point number, with its 23 bits of precision, the value of ϵ is 2^{-23} , which is approximately 1.192×10^{-7} . The most significant digit of ϵ falls *just inside* the range of significant digits in the value 1.0, so adding any value smaller than ϵ to 1.0 has no effect. In other words, any new bits contributed adding a value smaller than ϵ will get “chopped off” when we try to fit the sum into a mantissa with only 23 bits.

Units in the Last Place (ULP)

Consider two floating-point numbers which are identical in all respects except for the value of the least-significant bit in their mantissas. These two values are said to differ by one *unit in the last place* (1 ULP). The actual value of 1 ULP changes depending on the exponent. For example, the floating-point value `1.0f` has an unbiased exponent of zero, and a mantissa in which all bits are zero (except for the implicit leading 1). At this exponent, 1 ULP is equal to the machine epsilon (2^{-23}). If we change the exponent to a 1, yielding the value `2.0f`, the value of 1 ULP becomes equal to two times the machine epsilon. And if the exponent is 2, yielding the value `4.0f`, the value of 1 ULP is four

times the machine epsilon. In general, if a floating-point value's unbiased exponent is x , then $1 \text{ ULP} = 2^x \cdot \epsilon$.

The concept of units in the last place illustrates the idea that the precision of a floating-point number depends on its exponent, and is useful for quantifying the error inherent in any floating-point calculation. It can also be useful for finding the floating-point value that is the next largest representable value relative to a known value, or conversely the next smallest representable value relative to that value. This in turn can be useful for converting a greater-than-or-equal comparison into a greater-than comparison. Mathematically, the condition $a \geq b$ is equivalent to the condition $a + 1 \text{ ULP} > b$. We use this little “trick” in the Naughty Dog engine to simplify some logic in our character dialog system. In this system, simple comparisons can be used to select different lines of dialog for the characters to say. Rather than supporting all possible comparison operators, we only support greater-than and less-than checks, and we handle greater-than-or-equal-to and less-than-or-equal-to by adding or subtracting 1 ULP to or from the value being compared.

Impact of Floating-Point Precision on Software

The concepts of limited precision and the machine epsilon have real impacts on game software. For example, let's say we use a floating-point variable to track absolute game time in seconds. How long can we run our game before the magnitude of our clock variable gets so large that adding $1/30^{th}$ of a second to it no longer changes its value? The answer is 12.14 days or 2^{20} seconds. That's longer than most games will be left running, so we can probably get away with using a 32-bit floating-point clock measured in seconds in a game. But clearly it's important to understand the limitations of the floating-point format so that we can predict potential problems and take steps to avoid them when necessary.

IEEE Floating-Point Bit Tricks

See [9, Section 2.1] for a few really useful IEEE floating-point “bit tricks” that can make certain floating-point calculations lightning fast.

3.3.2 Primitive Data Types

C and C++ provide a number of primitive data types. The C and C++ standards provide guidelines on the relative sizes and signedness of these data types, but each compiler is free to define the types slightly differently in order to provide maximum performance on the target hardware.

- `char`. A `char` is usually 8 bits and is generally large enough to hold an ASCII or UTF-8 character (see Section 6.4.4.1). Some compilers define `char` to be signed, while others use unsigned `chars` by default.
- `int`, `short`, `long`. An `int` is supposed to hold a signed integer value that is the most efficient size for the target platform; it is usually defined to be 32 bits wide on a 32-bit CPU architecture, such as Pentium 4 or Xeon, and 64 bits wide on a 64-bit architecture, such as Intel Core i7, although the size of an `int` is also dependent upon other factors such as compiler options and the target operating system. A `short` is intended to be smaller than an `int` and is 16 bits on many machines. A `long` is as large as or larger than an `int` and may be 32 or 64 bits wide, or even wider, again depending on CPU architecture, compiler options and the target OS.
- `float`. On most modern compilers, a `float` is a 32-bit IEEE-754 floating-point value.
- `double`. A `double` is a double-precision (i.e., 64-bit) IEEE-754 floating-point value.
- `bool`. A `bool` is a true/false value. The size of a `bool` varies widely across different compilers and hardware architectures. It is never implemented as a single bit, but some compilers define it to be 8 bits while others use a full 32 bits.

Portable Sized Types

The built-in primitive data types in C and C++ were designed to be portable and therefore nonspecific. However, in many software engineering endeavors, including game engine programming, it is often important to know exactly how wide a particular variable is.

Before C++11, programmers had to rely on non-portable sized types provided by their compiler. For example, the Visual Studio C/C++ compiler defined the following extended keywords for declaring variables that are an explicit number of bits wide: `__int8`, `__int16`, `__int32` and `__int64`. Most other compilers have their own “sized” data types, with similar semantics but slightly different syntax.

Because of these differences between compilers, most game engines achieved source code portability by defining their own custom sized types. For example, at Naughty Dog we use the following sized types:

- `F32` is a 32-bit IEEE-754 floating-point value.
- `U8`, `I8`, `U16`, `I16`, `U32`, `I32`, `U64` and `I64` are unsigned and signed 8-, 16-, 32- and 64-bit integers, respectively.

- U32F and I32F are “fast” unsigned and signed 32-bit values, respectively. Each of these data types contains a value that is at least 32 bits wide, but may be wider if that would result in faster code on the target CPU.

`<stdint>`

The C++11 standard library introduces a set of standardized sized integer types. They are declared in the `<stdint>` header, and they include the signed types `std::int8_t`, `std::int16_t`, `std::int32_t` and `std::int64_t` and the unsigned types `std::uint8_t`, `std::uint16_t`, `std::uint32_t` and `std::uint64_t`, along with “fast” variants (like the I32F and U32F types we defined at Naughty Dog). These types free the programmer from having to “wrap” compiler-specific types in order to achieve portability. For a complete list of these sized types, see <http://en.cppreference.com/w/cpp/types/integer>.

OGRE's Primitive Data Types

OGRE defines a number of sized types of its own. `Ogre::uint8`, `Ogre::uint16` and `Ogre::uint32` are the basic unsigned sized integral types.

`Ogre::Real` defines a real floating-point value. It is usually defined to be 32 bits wide (equivalent to a `float`), but it can be redefined globally to be 64 bits wide (like a `double`) by defining the preprocessor macro `OGRE_DOUBLE_PRECISION` to 1. This ability to change the meaning of `Ogre::Real` is generally only used if one's game has a particular requirement for double-precision math, which is rare. Graphics chips (GPUs) always perform their math with 32-bit or 16-bit floats, the CPU/FPU is also usually faster when working in single-precision, and SIMD vector instructions operate on 128-bit registers that contain four 32-bit floats each. Hence, most games tend to stick to single-precision floating-point math.

The data types `Ogre::uchar`, `Ogre::ushort`, `Ogre::uint` and `Ogre::ulong` are just shorthand notations for C/C++'s unsigned `char`, unsigned short and unsigned long, respectively. As such, they are no more or less useful than their native C/C++ counterparts.

The types `Ogre::Radian` and `Ogre::Degree` are particularly interesting. These classes are wrappers around a simple `Ogre::Real` value. The primary role of these types is to permit the angular units of hard-coded literal constants to be documented and to provide automatic conversion between the two unit systems. In addition, the type `Ogre::Angle` represents an angle in the current “default” angle unit. The programmer can define whether the default will be radians or degrees when the OGRE application first starts up.

Perhaps surprisingly, OGRE does not provide a number of sized primitive data types that are commonplace in other game engines. For example, it defines no signed 8-, 16- or 64-bit integral types. If you are writing a game engine on top of OGRE, you will probably find yourself defining these types manually at some point.

3.3.2.1 Multibyte Values and Endianness

Values that are larger than eight bits (one byte) wide are called *multibyte quantities*. They’re commonplace on any software project that makes use of integers and floating-point values that are 16 bits or wider. For example, the integer value 4660 = 0x1234 is represented by the two bytes 0x12 and 0x34. We call 0x12 the most significant byte and 0x34 the least significant byte. In a 32-bit value, such as 0xABCD1234, the most-significant byte is 0xAB and the least-significant is 0x34. The same concepts apply to 64-bit integers and to 32- and 64-bit floating-point values as well.

Multibyte integers can be stored into memory in one of two ways, and different microprocessors may differ in their choice of storage method (see Figure 3.6).

- *Little-endian*. If a microprocessor stores the least significant byte of a multibyte value at a lower memory address than the most significant byte, we say that the processor is *little-endian*. On a little-endian machine, the number 0xABCD1234 would be stored in memory using the consecutive bytes 0x34, 0x12, 0xCD, 0xAB.
- *Big-endian*. If a microprocessor stores the most significant byte of a multibyte value at a lower memory address than the least significant byte,

```
U32 value = 0xABCD1234;  
U8* pBytes = (U8*)&value;
```

Big-endian		Little-endian	
pBytes + 0x0	0xAB	pBytes + 0x0	0x34
pBytes + 0x1	0xCD	pBytes + 0x1	0x12
pBytes + 0x2	0x12	pBytes + 0x2	0xCD
pBytes + 0x3	0x34	pBytes + 0x3	0xAB

Figure 3.6. Big- and little-endian representations of the value 0xABCD1234.

we say that the processor is *big-endian*. On a big-endian machine, the number 0xABCD1234 would be stored in memory using the bytes 0xAB, 0xCD, 0x12, 0x34.

Most programmers don't need to think much about endianness. However, when you're a game programmer, endianness can become a bit of a thorn in your side. This is because games are usually *developed* on a Windows or Linux machine running an Intel Pentium processor (which is little-endian), but *run* on a console such as the Wii, Xbox 360 or PlayStation 3—all three of which utilize a variant of the PowerPC processor (which can be configured to use either endianness, but is big-endian by default). Now imagine what happens when you generate a data file for consumption by your game engine on an Intel processor and then try to load that data file into your engine running on a PowerPC processor. Any multibyte value that you wrote out into that data file will be stored in little-endian format. But when the game engine reads the file, it expects all of its data to be in big-endian format. The result? You'll write 0xABCD1234, but you'll read 0x3412CDAB, and that's clearly not what you intended!

There are at least two solutions to this problem.

1. You could write all your data files as text and store all multibyte numbers as sequences of decimal or hexadecimal digits, one character (one byte) per digit. This would be an inefficient use of disk space, but it would work.
2. You can have your tools endian-swap the data prior to writing it into a binary data file. In effect, you make sure that the data file uses the endianness of the target microprocessor (the game console), even if the tools are running on a machine that uses the opposite endianness.

Integer Endian-Swapping

Endian-swapping an integer is not conceptually difficult. You simply start at the most significant byte of the value and swap it with the least significant byte; you continue this process until you reach the halfway point in the value. For example, 0xA7891023 would become 0x231089A7.

The only tricky part is knowing *which bytes* to swap. Let's say you're writing the contents of a C struct or C++ class from memory out to a file. To properly endian-swap this data, you need to keep track of the locations and sizes of each data member in the struct and swap each one appropriately based on its size. For example, the structure

```

struct Example
{
    U32    m_a;
    U16    m_b;
    U32    m_c;
};

```

might be written out to a data file as follows:

```

void writeExampleStruct(Example& ex, Stream& stream)
{
    stream.writeU32(swapU32(ex.m_a));
    stream.writeU16(swapU16(ex.m_b));
    stream.writeU32(swapU32(ex.m_c));
}

```

and the swap functions might be defined like this:

```

inline U16 swapU16(U16 value)
{
    return ((value & 0x00FF) << 8)
        | ((value & 0xFF00) >> 8);
}

inline U32 swapU32(U32 value)
{
    return ((value & 0x000000FF) << 24)
        | ((value & 0x0000FF00) << 8)
        | ((value & 0x00FF0000) >> 8)
        | ((value & 0xFF000000) >> 24);
}

```

You cannot simply cast the `Example` object into an array of bytes and blindly swap the bytes using a single general-purpose function. We need to know both *which data members* to swap and *how wide* each member is, and each data member must be swapped individually.

Some compilers provide built-in endian-swapping macros, freeing you from having to write your own. For example, gcc offers a family of macros named `__builtin_bswapXX()` for performing 16-, 32- and 64-bit endian swaps. However, such compiler-specific facilities are of course non-portable.

Floating-Point Endian-Swapping

As we've seen, an IEEE-754 floating-point value has a detailed internal structure involving some bits for the mantissa, some bits for the exponent and a sign bit. However, you can endian-swap it just as if it were an integer, because bytes are bytes. You merely reinterpret the bit pattern of your `float`

as if it were a `std::int32_t`, perform the endian swapping operation, and then reinterpret the result as a `float` again.

You can reinterpret floats as integers by using C++’s `reinterpret_cast` operator on a pointer to the float, and then dereferencing the type-cast pointer; this is known as *type punning*. But punning can lead to optimization bugs when strict aliasing is enabled. (See <http://www.cocoawithlove.com/2008/04/using-pointers-to-recast-in-c-is-bad.html> for an excellent description of this problem.) One alternative that’s guaranteed to be portable is to use a union, as follows:

```
union U32F32
{
    U32    m_asU32;
    F32    m_asF32;
};

inline F32 swapF32(F32 value)
{
    U32F32 u;
    u.m_asF32 = value;

    // endian-swap as integer
    u.m_asU32 = swapU32(u.m_asU32);

    return u.m_asF32;
}
```

3.3.3 Kilobytes versus Kibibytes

You’ve probably used metric (SI) units like kilobytes (kB) and megabytes (MB) to describe quantities of memory. However, the use of these units to describe quantities of memory that are measured in powers of two isn’t strictly correct. When a computer programmer speaks of a “kilobyte,” she or he usually means 1024 bytes. But SI units define the prefix “kilo” to mean 10^3 or 1000, not 1024.

To resolve this ambiguity, the International Electrotechnical Commission (IEC) in 1998 established a new set of SI-like prefixes for use in computer science. These prefixes are defined in terms of powers of two rather than powers of ten, so that computer engineers can precisely and conveniently specify quantities that are powers of two. In this new system, instead of kilobyte (1000 bytes), we say kibibyte (1024 bytes, abbreviated KiB). And instead of megabyte (1,000,000 bytes), we say mebibyte ($1024 \times 1024 = 1,048,576$ bytes, abbreviated MiB). Table 3.1 summarizes the sizes, prefixes and names of the

Metric (SI)			IEC		
Value	Unit	Name	Value	Unit	Name
1000	kB	kilobyte	1024	KiB	kibibyte
1000 ²	MB	megabyte	1024 ²	MiB	mebibyte
1000 ³	GB	gigabyte	1024 ³	GiB	gibibyte
1000 ⁴	TB	terabyte	1024 ⁴	TiB	tebibyte
1000 ⁵	PB	petabyte	1024 ⁵	PiB	pebibyte
1000 ⁶	EB	exabyte	1024 ⁶	EiB	exbibyte
1000 ⁷	ZB	zettabyte	1024 ⁷	ZiB	zebibyte
1000 ⁸	YB	yottabyte	1024 ⁸	YiB	yobibyte

Table 3.1. Comparison of metric (SI) units and IEC units for describing quantities of bytes.

most commonly used byte quantity units in both the SI and IEC systems. We'll use IEC units throughout this book.

3.3.4 Declarations, Definitions and Linkage

3.3.4.1 Translation Units Revisited

As we saw in Chapter 2, a C or C++ program is comprised of *translation units*. The compiler translates one .cpp file at a time, and for each one it generates an output file called an object file (.o or .obj). A .cpp file is the smallest unit of translation operated on by the compiler; hence, the name “translation unit.” An object file contains not only the compiled machine code for all of the functions defined in the .cpp file, but also all of its global and static variables. In addition, an object file may contain *unresolved references* to functions and global variables defined in *other* .cpp files.

The compiler only operates on one translation unit at a time, so whenever it encounters a reference to an external global variable or function, it must “go on faith” and assume that the entity in question really exists, as shown in Figure 3.7. It is the linker’s job to combine all of the object files into a final executable image. In doing so, the linker reads all of the object files and attempts to resolve all of the unresolved cross-references between them. If it is successful, an executable image is generated containing all of the functions, global variables and static variables, with all cross-translation-unit references properly resolved. This is depicted in Figure 3.8.

The linker’s primary job is to resolve external references, and in this

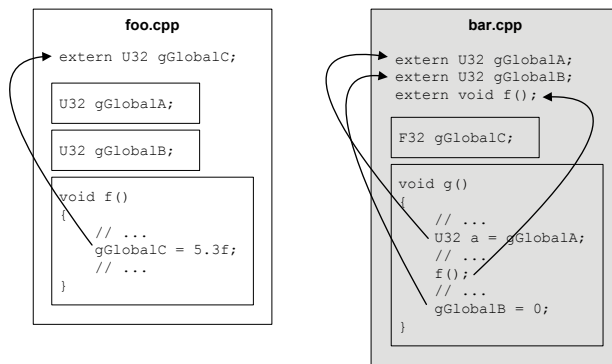


Figure 3.7. Unresolved external references in two translation units.

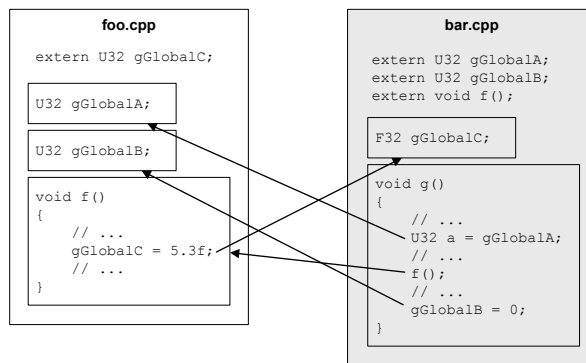


Figure 3.8. Fully resolved external references after successful linking.

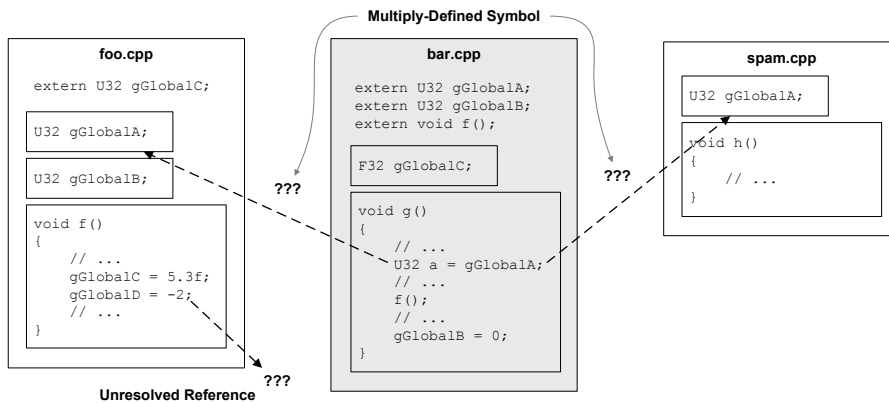


Figure 3.9. The two most common linker errors.

capacity it can generate only two kinds of errors:

1. The target of an `extern` reference might not be found, in which case the linker generates an “unresolved symbol” error.
2. The linker might find more than one variable or function with the same name, in which case it generates a “multiply defined symbol” error.

These two situations are shown in Figure 3.9.

3.3.4.2 Declaration versus Definition

In the C and C++ languages, variables and functions must be *declared* and *defined* before they can be used. It is important to understand the difference between a declaration and a definition in C and C++.

- A *declaration* is a description of a data object or function. It provides the compiler with the *name* of the entity and its *data type* or *function signature* (i.e., return type and argument type(s)).
- A *definition*, on the other hand, describes a unique region of memory in the program. This memory might contain a variable, an instance of a struct or class or the machine code of a function.

In other words, a declaration is a *reference to an entity*, while a definition is the *entity itself*. A definition is always a declaration, but the reverse is not always the case—it is possible to write a pure declaration in C and C++ that is not a definition.

Functions are *defined* by writing the body of the function immediately after the signature, enclosed in curly braces:

foo.cpp

```
// definition of the max() function
int max(int a, int b)
{
    return (a > b) ? a : b;
}

// definition of the min() function
int min(int a, int b)
{
    return (a <= b) ? a : b;
}
```

A pure declaration can be provided for a function so that it can be used in other translation units (or later in the same translation unit). This is done by writing a function signature followed by a semicolon, with an optional prefix of `extern`:

foo.h

```
extern int max(int a, int b); // a function declaration

int min(int a, int b); // also a declaration (the extern
                       // is optional/assumed)
```

Variables and instances of classes and `structs` are defined by writing the data type followed by the name of the variable or instance and an optional array specifier in square brackets:

foo.cpp

```
// All of these are variable definitions:
U32 gGlobalInteger = 5;
F32 gGlobalFloatArray[16];
MyClass gGlobalInstance;
```

A global variable defined in one translation unit can optionally be *declared* for use in other translation units by using the `extern` keyword:

foo.h

```
// These are all pure declarations:
extern U32 gGlobalInteger;
extern F32 gGlobalFloatArray[16];
extern MyClass gGlobalInstance;
```

Multiplicity of Declarations and Definitions

Not surprisingly, any particular data object or function in a C/C++ program can have multiple identical *declarations*, but each can have only one *definition*. If two or more identical definitions exist in a single translation unit, the compiler will notice that multiple entities have the same name and flag an error. If two or more identical definitions exist in *different* translation units, the compiler will not be able to identify the problem, because it operates on one translation unit at a time. But in this case, the linker will give us a “multiply defined symbol” error when it tries to resolve the cross-references.

Definitions in Header Files and Inlining

It is usually dangerous to place *definitions* in header files. The reason for this should be pretty obvious: if a header file containing a *definition* is `#included` into more than one `.cpp` file, it's a sure-fire way of generating a "multiply defined symbol" linker error.

Inline function definitions are an exception to this rule, because each invocation of an inline function gives rise to a brand new copy of that function's machine code, embedded directly into the calling function. In fact, inline function definitions *must* be placed in header files if they are to be used in more than one translation unit. Note that it is *not* sufficient to tag a function *declaration* with the `inline` keyword in a `.h` file and then place the body of that function in a `.cpp` file. The compiler must be able to "see" the body of the function in order to inline it. For example:

foo.h

```
// This function definition will be inlined properly.
inline int max(int a, int b)
{
    return (a > b) ? a : b;
}

// This declaration cannot be inlined because the
// compiler cannot "see" the body of the function.
inline int min(int a, int b);
```

foo.cpp

```
// The body of min() is effectively "hidden" from the
// compiler, so it can ONLY be inlined within foo.cpp.
int min(int a, int b)
{
    return (a <= b) ? a : b;
}
```

The `inline` keyword is really just a hint to the compiler. It does a cost/benefit analysis of each inline function, weighing the size of the function's code versus the potential performance benefits of inlining it, and the compiler gets the final say as to whether the function will really be inlined or not. Some compilers provide syntax like `__forceinline`, allowing the programmer to bypass the compiler's cost/benefit analysis and control function inlining directly.

Templates and Header Files

The definition of a templated class or function must be visible to the compiler across all translation units in which it is used. As such, if you want a template to be usable in more than one translation unit, the template must be placed into a header file (just as inline function definitions must be). The declaration and definition of a template are therefore inseparable: You cannot declare templated functions or classes in a header but “hide” their definitions inside a .cpp file, because doing so would render those definitions invisible within any other .cpp file that includes that header.

3.3.4.3 Linkage

Every definition in C and C++ has a property known as *linkage*. A definition with *external linkage* is visible to and can be referenced by translation units other than the one in which it appears. A definition with *internal linkage* can only be “seen” inside the translation unit in which it appears and thus cannot be referenced by other translation units. We call this property *linkage* because it dictates whether or not the linker is permitted to cross-reference the entity in question. So, in a sense, linkage is the translation unit’s equivalent of the `public:` and `private:` keywords in C++ class definitions.

By default, definitions have external linkage. The `static` keyword is used to change a definition’s linkage to internal. Note that two or more identical static definitions in two or more different .cpp files are considered to be *distinct entities* by the linker (just as if they had been given different names), so they will *not* generate a “multiply defined symbol” error. Here are some examples:

foo.cpp

```
// This variable can be used by other .cpp files
// (external linkage).
U32 gExternalVariable;

// This variable is only usable within foo.cpp (internal
// linkage).
static U32 gInternalVariable;

// This function can be called from other .cpp files
// (external linkage).
void externalFunction()
{
    // ...
}
```

```

// This function can only be called from within foo.cpp
// (internal linkage).
static void internalFunction()
{
    // ...
}

```

bar.cpp

```

// This declaration grants access to foo.cpp's variable.
extern U32 gExternalVariable;

// This 'gInternalVariable' is distinct from the one
// defined in foo.cpp -- no error. We could just as
// well have named it gInternalVariableForBarCpp -- the
// net effect is the same.
static U32 gInternalVariable;

// This function is distinct from foo.cpp's
// version -- no error. It acts as if we had named it
// internalFunctionForBarCpp().
static void internalFunction()
{
    // ...
}

// ERROR -- multiply defined symbol!
void externalFunction()
{
    // ...
}

```

Technically speaking, *declarations* don't have a linkage property at all, because they do not allocate any storage in the executable image; therefore, there is no question as to whether or not the linker should be permitted to cross-reference that storage. A declaration is merely a reference to an entity defined elsewhere. However, it is sometimes convenient to speak about declarations as having internal linkage, because a declaration only applies to the translation unit in which it appears. If we allow ourselves to loosen our terminology in this manner, then declarations *always* have internal linkage—there is no way to cross-reference a single declaration in multiple .cpp files. (If we put a declaration in a header file, then multiple .cpp files can “see” that declaration, but they are in effect each getting a distinct *copy* of the declaration, and each copy has internal linkage within that translation unit.)

This leads us to the real reason why inline function definitions are permitted in header files: it is because inline functions have *internal linkage* by default, just as if they had been declared `static`. If multiple `.cpp` files `#include` a header containing an inline function definition, each translation unit gets a private copy of that function's body, and no "multiply defined symbol" errors are generated. The linker sees each copy as a distinct entity.

3.3.5 Memory Layout of a C/C++ Program

A program written in C or C++ stores its data in a number of different places in memory. In order to understand how storage is allocated and how the various types of C/C++ variables work, we need to understand the memory layout of a C/C++ program.

3.3.5.1 Executable Image

When a C/C++ program is built, the linker creates an *executable file*. Most UNIX-like operating systems, including many game consoles, employ a popular executable file format called the *executable and linking format* (ELF). Executable files on those systems therefore have a `.elf` extension. The Windows executable format is similar to the ELF format; executables under Windows have a `.exe` extension. Whatever its format, the executable file always contains a partial *image* of the program as it will exist in memory when it runs. I say a "partial" image because the program generally allocates memory at runtime in addition to the memory laid out in its executable image.

The executable image is divided into contiguous blocks called *segments* or *sections*. Every operating system lays things out a little differently, and the layout may also differ slightly from executable to executable on the same operating system. But the image is usually comprised of at least the following four segments:

1. *Text segment*. Sometimes called the *code segment*, this block contains executable machine code for all functions defined by the program.
2. *Data segment*. This segment contains all *initialized* global and static variables. The memory needed for each global variable is laid out exactly as it will appear when the program is run, and the proper initial values are all filled in. So when the executable file is loaded into memory, the initialized global and static variables are ready to go.
3. *BSS segment*. "BSS" is an outdated name which stands for "block started by symbol." This segment contains all of the *uninitialized* global and

static variables defined by the program. The C and C++ languages explicitly define the initial value of any uninitialized global or static variable to be zero. But rather than storing a potentially very large block of zeros in the BSS section, the linker simply stores a *count* of how many zero bytes are required to account for all of the uninitialized globals and statics in the segment. When the executable is loaded into memory, the operating system reserves the requested number of bytes for the BSS section and fills it with zeros prior to calling the program's entry point (e.g., `main()` or `WinMain()`).

4. *Read-only data segment.* Sometimes called the *rodata* segment, this segment contains any read-only (constant) global data defined by the program. For example, all floating-point constants (e.g., `const float kPi = 3.141592f;`) and all global object instances that have been declared with the `const` keyword (e.g., `const Foo gReadOnlyFoo;`) reside in this segment. Note that integer constants (e.g., `const int kMaxMonsters = 255;`) are often used as *manifest constants* by the compiler, meaning that they are inserted directly into the machine code wherever they are used. Such constants occupy storage in the text segment, but they are not present in the read-only data segment.

Global variables (variables defined at *file scope* outside any function or class declaration) are stored in either the data or BSS segments, depending on whether or not they have been initialized. The following global will be stored in the data segment, because it has been initialized:

foo.cpp

```
F32 gInitializedGlobal = -2.0f;
```

and the following global will be allocated and initialized to zero by the operating system, based on the specifications given in the BSS segment, because it has not been initialized by the programmer:

foo.cpp

```
F32 gUninitializedGlobal;
```

We've seen that the `static` keyword can be used to give a global variable or function definition *internal linkage*, meaning that it will be "hidden" from other translation units. The `static` keyword can also be used to declare a global variable *within a function*. A function-static variable is *lexically scoped* to the function in which it is declared (i.e., the variable's name can only be "seen" inside the function). It is initialized the first time the function is called (rather than before `main()` is called, as with file-scope statics). But in terms of memory layout in the executable image, a function-static variable acts identically

to a file-static global variable—it is stored in either the data or BSS segment based on whether or not it has been initialized.

```
void readHitchhikersGuide(U32 book)
{
    static U32 sBooksInTheTrilogy = 5; // data segment
    static U32 sBooksRead;             // BSS segment
    // ...
}
```

3.3.5.2 Program Stack

When an executable program is loaded into memory and run, the operating system reserves an area of memory for the *program stack*. Whenever a function is called, a contiguous area of stack memory is pushed onto the stack—we call this block of memory a *stack frame*. If function `a()` calls another function `b()`, a new stack frame for `b()` is pushed on top of `a()`'s frame. When `b()` returns, its stack frame is popped, and execution continues wherever `a()` left off.

A stack frame stores three kinds of data:

1. It stores the *return address* of the calling function so that execution may continue in the calling function when the called function returns.
2. The contents of all relevant *CPU registers* are saved in the stack frame. This allows the new function to use the registers in any way it sees fit, without fear of overwriting data needed by the calling function. Upon return to the calling function, the state of the registers is restored so that execution of the calling function may resume. The return value of the called function, if any, is usually left in a specific register so that the calling function can retrieve it, but the other registers are restored to their original values.
3. The stack frame also contains all *local variables* declared by the function; these are also known as *automatic variables*. This allows each distinct function invocation to maintain its own private copy of every local variable, even when a function calls itself recursively. (In practice, some local variables are actually allocated to CPU registers rather than being stored in the stack frame, but for the most part such variables operate as if they were allocated within the function's stack frame.)

Pushing and popping stack frames is usually implemented by adjusting the value of a single register in the CPU, known as the stack pointer. Figure 3.10 illustrates what happens when the functions shown below are executed.

```
void c()  
{  
    U32 localC1;  
    // ...  
}  
  
F32 b()  
{  
    F32 localB1;  
    I32 localB2;  
  
    // ...  
  
    c();  
  
    // ...  
  
    return localB1;  
}
```

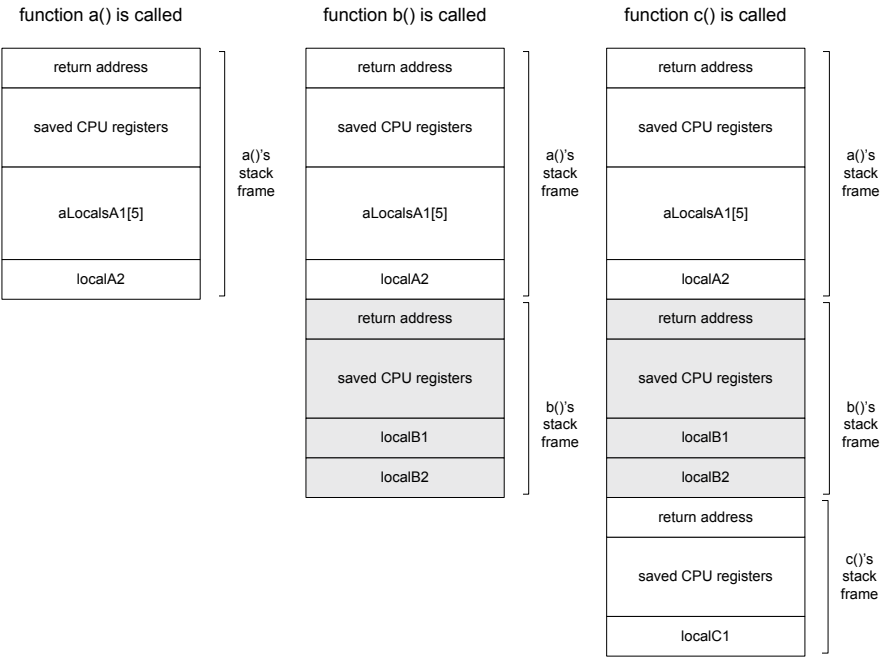


Figure 3.10. Stack frames.

```
void a()
{
    U32 aLocalsA1[5];

    // ...

    F32 localA2 = b();

    // ...
}
```

When a function containing automatic variables returns, its stack frame is abandoned and all automatic variables in the function should be treated as if they no longer exist. Technically, the memory occupied by those variables is still there in the abandoned stack frame—but that memory will very likely be overwritten as soon as another function is called. A common error involves returning the address of a local variable, like this:

```
U32* getMeaningOfLife()
{
    U32 anInteger = 42;
    return &anInteger;
}
```

You *might* get away with this if you use the returned pointer immediately and don't call any other functions in the interim. But more often than not, this kind of code will crash—sometimes in ways that can be difficult to debug.

3.3.5.3 Dynamic Allocation Heap

Thus far, we've seen that a program's data can be stored as global or static variables or as local variables. The globals and statics are allocated within the executable image, as defined by the data and BSS segments of the executable file. The locals are allocated on the program stack. Both of these kinds of storage are *statically* defined, meaning that the size and layout of the memory is known when the program is compiled and linked. However, a program's memory requirements are often not fully known at compile time. A program usually needs to allocate additional memory *dynamically*.

To allow for dynamic allocation, the operating system maintains a block of memory for each running process from which memory can be allocated by calling `malloc()` (or an OS-specific function like `HeapAlloc()` under Windows) and later returned for reuse by the process at some future time by calling `free()` (or an OS-specific function like `HeapFree()`). This memory block is

known as *heap memory*, or the *free store*. When we allocate memory dynamically, we sometimes say that this memory resides *on the heap*.

In C++, the global `new` and `delete` operators are used to allocate and free memory to and from the free store. Be wary, however—individual classes may overload these operators to allocate memory in custom ways, and even the *global* `new` and `delete` operators can be overloaded, so you cannot simply assume that `new` is always allocating from the global heap.

We will discuss dynamic memory allocation in more depth in Chapter 7. For additional information, see http://en.wikipedia.org/wiki/Dynamic_memory_allocation.

3.3.6 Member Variables

C structs and C++ classes allow variables to be grouped into logical units. It's important to remember that a `class` or `struct` *declaration* allocates no memory. It is merely a description of the layout of the data—a cookie cutter which can be used to stamp out *instances* of that `struct` or `class` later on. For example:

```
struct Foo // struct declaration
{
    U32  mUnsignedValue;
    F32  mFloatValue;
    bool mBooleanValue;
};
```

Once a `struct` or `class` has been declared, it can be allocated (defined) in any of the ways that a primitive data type can be allocated; for example,

- as an automatic variable, on the program stack;

```
void someFunction()
{
    Foo localFoo;
    // ...
}
```

- as a global, file-static or function-static;

```
Foo gFoo;
static Foo sFoo;

void someFunction()
{
    static Foo sLocalFoo;
    // ...
}
```


- dynamically allocated from the heap. In this case, the pointer or reference variable used to hold the address of the data can itself be allocated as an automatic, global, static or even dynamically.

```

Foo* gpFoo = nullptr; // global pointer to a Foo

void someFunction()
{
    // allocate a Foo instance from the heap
    gpFoo = new Foo;

    // ...

    // allocate another Foo, assign to local pointer
    Foo* pAnotherFoo = new Foo;

    // ...

    // allocate a POINTER to a Foo from the heap
    Foo** ppFoo = new Foo*;
    (*ppFoo) = pAnotherFoo;
}

```

3.3.6.1 Class-Static Members

As we've seen, the `static` keyword has many different meanings depending on context:

- When used at file scope, `static` means “restrict the visibility of this variable or function so it can only be seen inside this .cpp file.”
- When used at function scope, `static` means “this variable is a global, not an automatic, but it can only be seen inside this function.”
- When used inside a `struct` or `class` declaration, `static` means “this variable is not a regular member variable, but instead acts just like a global.”

Notice that when `static` is used inside a class declaration, it does not control the *visibility* of the variable (as it does when used at file scope)—rather, it differentiates between regular per-instance member variables and per-class variables that act like globals. The *visibility* of a class-static variable is determined by the use of `public:`, `protected:` or `private:` keywords in the class declaration. Class-static variables are automatically included within the namespace of the class or struct in which they are declared. So the name of the class or struct must be used to disambiguate the variable whenever it is used outside that class or struct (e.g., `Foo::sVarName`).

Like an extern declaration for a regular global variable, the declaration of a class-static variable within a class allocates no memory. The memory for the class-static variable must be defined in a .cpp file. For example:

foo.h

```
class Foo
{
public:
    static F32 sClassStatic; // allocates no
                           // memory!
};
```

foo.cpp

```
F32 Foo::sClassStatic = -1.0f; // define memory and
                               // initialize
```

3.3.7 Object Layout in Memory

It’s useful to be able to visualize the memory layout of your classes and structs. This is usually pretty straightforward—we can simply draw a box for the struct or class, with horizontal lines separating data members. An example of such a diagram for the struct Foo listed below is shown in Figure 3.11.

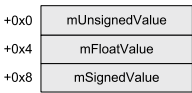


Figure 3.11. Memory layout of a simple struct.

```
struct Foo
{
    U32    mUnsignedValue;
    F32    mFloatValue;
    I32    mSignedValue;
};
```

The sizes of the data members are important and should be represented in your diagrams. This is easily done by using the width of each data member to indicate its size in bits—i.e., a 32-bit integer should be roughly four times the width of an eight-bit integer (see Figure 3.12).

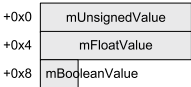


Figure 3.12. A memory layout using width to indicate member sizes.

```
struct Bar
{
    U32    mUnsignedValue;
    F32    mFloatValue;
    bool   mBooleanValue; // diagram assumes this is 8 bits
};
```

3.3.7.1 Alignment and Packing

As we start to think more carefully about the layout of our structs and classes in memory, we may start to wonder what happens when small data members are interspersed with larger members. For example:

```
struct InefficientPacking
{
    U32    mU1; // 32 bits
    F32    mF2; // 32 bits
    U8     mB3; // 8 bits
    I32    mI4; // 32 bits
    bool   mB5; // 8 bits
    char*  mP6; // 32 bits
};
```

You might imagine that the compiler simply packs the data members into memory as tightly as it can. However, this is not usually the case. Instead, the compiler will typically leave “holes” in the layout, as depicted in Figure 3.13. (Some compilers can be requested not to leave these holes by using a preprocessor directive like `#pragma pack`, or via command-line options; but the default behavior is to space out the members as shown in Figure 3.13.)

Why does the compiler leave these “holes”? The reason lies in the fact that every data type has a natural *alignment*, which must be respected in order to permit the CPU to read and write memory effectively. The *alignment* of a data object refers to whether its *address in memory* is a multiple of its *size* (which is generally a power of two):

- An object with 1-byte alignment resides at any memory address.
- An object with 2-byte alignment resides only at even addresses (i.e., addresses whose least significant nibble is 0x0, 0x2, 0x4, 0x8, 0xA, 0xC or 0xE).
- An object with 4-byte alignment resides only at addresses that are a multiple of four (i.e., addresses whose least significant nibble is 0x0, 0x4, 0x8 or 0xC).
- A 16-byte aligned object resides only at addresses that are a multiple of 16 (i.e., addresses whose least significant nibble is 0x0).

Alignment is important because many modern processors can actually only read and write properly aligned blocks of data. For example, if a program requests that a 32-bit (4-byte) integer be read from address 0x6A341174, the memory controller will load the data happily because the address is 4-byte aligned (in this case, its least significant nibble is 0x4). However, if a request is made to load a 32-bit integer from address 0x6A341173, the memory controller now has to read *two* 4-byte blocks: the one at 0x6A341170 and the one

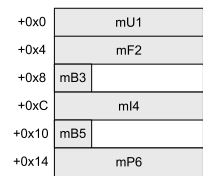


Figure 3.13. Inefficient struct packing due to mixed data member sizes.

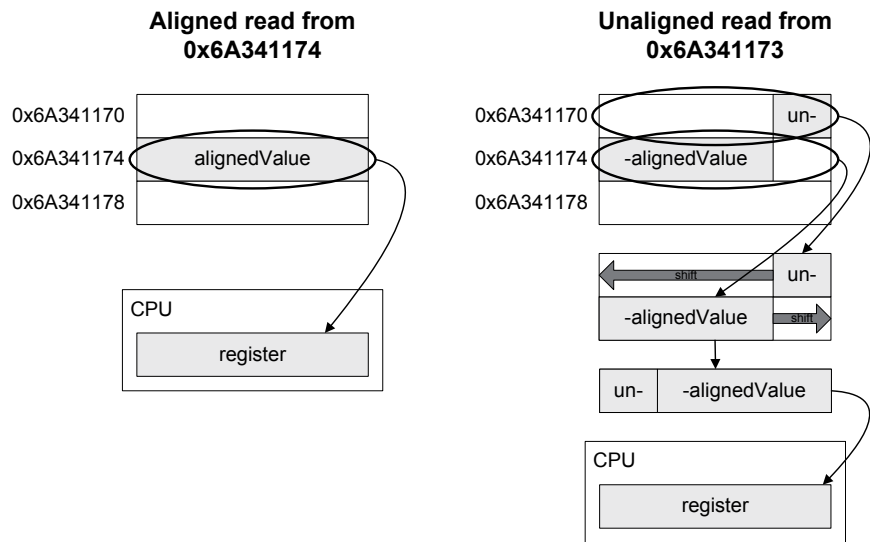


Figure 3.14. Aligned and unaligned reads of a 32-bit integer.

at 0x6A341174. It must then mask and shift the two parts of the 32-bit integer and logically OR them together into the destination register on the CPU. This is shown in Figure 3.14.

Some microprocessors don’t even go this far. If you request a read or write of unaligned data, you might just get garbage. Or your program might just crash altogether! (The PlayStation 2 is a notable example of this kind of intolerance for unaligned data.)

Different data types have different alignment requirements. A good rule of thumb is that a data type should be aligned to a boundary equal to the width of the data type in bytes. For example, 32-bit values generally have a 4-byte alignment requirement, 16-bit values should be 2-byte aligned, and 8-bit values can be stored at any address (1-byte aligned). On CPUs that support SIMD vector math, the vector registers each contain four 32-bit floats, for a total of 128 bits or 16 bytes. And as you would guess, a four-float SIMD vector typically has a 16-byte alignment requirement.

This brings us back to those “holes” in the layout of struct Inefficient Packing shown in Figure 3.13. When smaller data types like 8-bit bools are interspersed with larger types like 32-bit integers or floats in a structure or class, the compiler introduces padding (holes) in order to ensure that everything is properly aligned. It’s a good idea to think about alignment and packing when declaring your data structures. By simply rearranging the members of struct InefficientPacking from the example above, we can eliminate some of the wasted padding space, as shown below and in Figure 3.15:

+0x0	mU1		
+0x4	mF2		
+0x8	mI4		
+0xC	mP6		
+0x10	mB3	mB5	(pad)

Figure 3.15. More efficient packing by grouping small members together.

```

struct MoreEfficientPacking
{
    U32    mU1; // 32 bits (4-byte aligned)
    F32    mF2; // 32 bits (4-byte aligned)
    I32    mI4; // 32 bits (4-byte aligned)
    char*  mP6; // 32 bits (4-byte aligned)
    U8     mB3; // 8 bits (1-byte aligned)
    bool   mB5; // 8 bits (1-byte aligned)
};

```

You'll notice in Figure 3.15 that the size of the structure as a whole is now 20 bytes, not 18 bytes as we might expect, because it has been padded by two bytes at the end. This padding is added by the compiler to ensure proper alignment of the structure in an *array context*. That is, if an array of these structs is defined and the first element of the array is aligned, then the padding at the end guarantees that *all subsequent elements* will also be aligned properly.

The alignment of a structure as a whole is equal to the largest alignment requirement among its members. In the example above, the largest member alignment is 4-byte, so the structure as a whole should be 4-byte aligned. I usually like to add explicit padding to the end of my structs to make the wasted space visible and explicit, like this:

```

struct BestPacking
{
    U32    mU1;      // 32 bits (4-byte aligned)
    F32    mF2;      // 32 bits (4-byte aligned)
    I32    mI4;      // 32 bits (4-byte aligned)
    char*  mP6;      // 32 bits (4-byte aligned)
    U8     mB3;      // 8 bits (1-byte aligned)
    bool   mB5;      // 8 bits (1-byte aligned)
    U8     _pad[2]; // explicit padding
};

```

3.3.7.2 Memory Layout of C++ Classes

Two things make C++ classes a little different from C structures in terms of memory layout: *inheritance* and *virtual functions*.

When class B inherits from class A, B's data members simply appear immediately after A's in memory, as shown in Figure 3.16. Each new derived class simply tacks its data members on at the end, although alignment requirements may introduce padding between the classes. (Multiple inheritance does some whacky things, like including multiple copies of a single base class in the memory layout of a derived class. We won't cover the details here, because game programmers usually prefer to avoid multiple inheritance altogether anyway.)

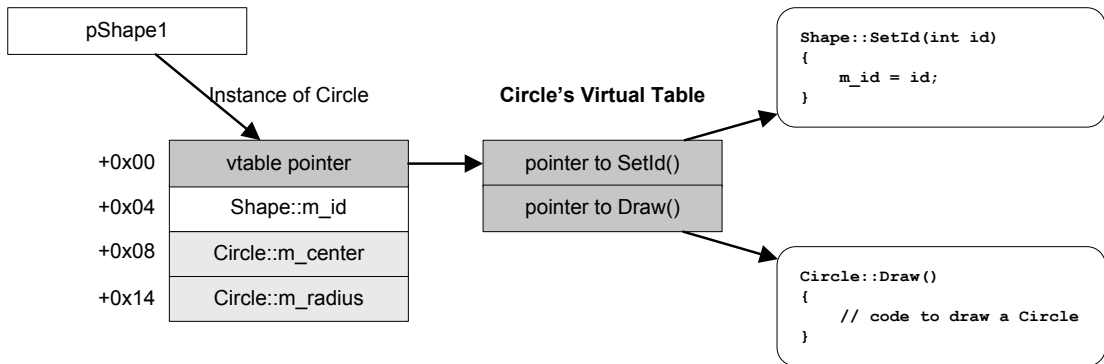


Figure 3.16. Effect of inheritance on class layout.

If a class contains or inherits one or more *virtual functions*, then four additional bytes (or eight bytes if the target hardware uses 64-bit addresses) are added to the class layout, typically at the very beginning of the class' layout. These four or eight bytes are collectively called the *virtual table pointer* or *vpointer*, because they contain a pointer to a data structure known as the *virtual function table* or *vtable*. The vtable for a particular class contains pointers to all the virtual functions that it declares or inherits. Each concrete class has its own virtual table, and every instance of that class has a pointer to it, stored in its vpointer.

The virtual function table is at the heart of polymorphism, because it allows code to be written that is ignorant of the specific concrete classes it is dealing with. Returning to the ubiquitous example of a `Shape` base class with derived classes for `Circle`, `Rectangle` and `Triangle`, let's imagine that `Shape` defines a virtual function called `Draw()`. The derived classes all override this function, providing distinct implementations named `Circle::Draw()`, `Rectangle::Draw()` and `Triangle::Draw()`. The virtual table for any class derived from `Shape` will contain an entry for the `Draw()` function, but that entry will point to different function implementations, depending on the concrete class. `Circle`'s vtable will contain a pointer to `Circle::Draw()`, while `Rectangle`'s virtual table will point to `Rectangle::Draw()`, and `Triangle`'s vtable will point to `Triangle::Draw()`. Given an arbitrary pointer to a `Shape` (`Shape* pShape`), the code can simply dereference the vtable pointer, look up the `Draw()` function's entry in the vtable, and call it. The result will be to call `Circle::Draw()` when `pShape` points to an instance of `Circle`, `Rectangle::Draw()` when `pShape` points to a `Rectangle`, and `Triangle::Draw()` when `pShape` points to a `Triangle`.

These ideas are illustrated by the following code excerpt. Notice that the base class `Shape` defines two virtual functions, `SetId()` and `Draw()`, the latter of which is declared to be *pure virtual*. (This means that `Shape` provides no default implementation of the `Draw()` function, and derived classes *must* override it if they want to be instantiable.) Class `Circle` derives from `Shape`, adds some data members and functions to manage its center and radius, and overrides the `Draw()` function; this is depicted in Figure 3.17. Class `Triangle` also derives from `Shape`. It adds an array of `Vector3` objects to store its three vertices and adds some functions to get and set the individual vertices. Class `Triangle` overrides `Draw()` as we'd expect, and for illustrative purposes it also overrides `SetId()`. The memory image generated by the `Triangle` class is shown in Figure 3.18.

Figure 3.17. `pShape1` points to an instance of class `Circle`.

```
class Shape
{
public:
    virtual void SetId(int id) { m_id = id; }
    int GetId() const { return m_id; }

    virtual void Draw() = 0; // pure virtual -- no impl.

private:
    int m_id;
};

class Circle : public Shape
{
public:
    void SetCenter(const Vector3& c) { m_center=c; }
    Vector3 GetCenter() const { return m_center; }

    void SetRadius(float r) { m_radius = r; }
    float GetRadius() const { return m_radius; }

    virtual void Draw()
    {
        // code to draw a circle
    }

private:
    Vector3 m_center;
    float m_radius;
};
```

```

class Triangle : public Shape
{
public:
    void SetVertex(int i, const Vector3& v);
    Vector3 GetVertex(int i) const { return m_vtx[i]; }

    virtual void Draw()
    {
        // code to draw a triangle
    }

    virtual void SetId(int id)
    {
        // call base class' implementation
        Shape::SetId(id);

        // do additional work specific to Triangles...
    }

private:
    Vector3 m_vtx[3];
};

// -----

void main(int, char**)
{
    Shape* pShape1 = new Circle;
    Shape* pShape2 = new Triangle;

    pShape1->Draw();
    pShape2->Draw();

    // ...
}

```

3.4 Computer Hardware Fundamentals

Programming in a high-level language like C++, C# or Python is an efficient way to build software. But the higher-level your language is, the more it shields you from the underlying details of the hardware on which your code runs. To become a truly proficient programmer, it's important to understand the architecture of your target hardware. This knowledge can help you to op-

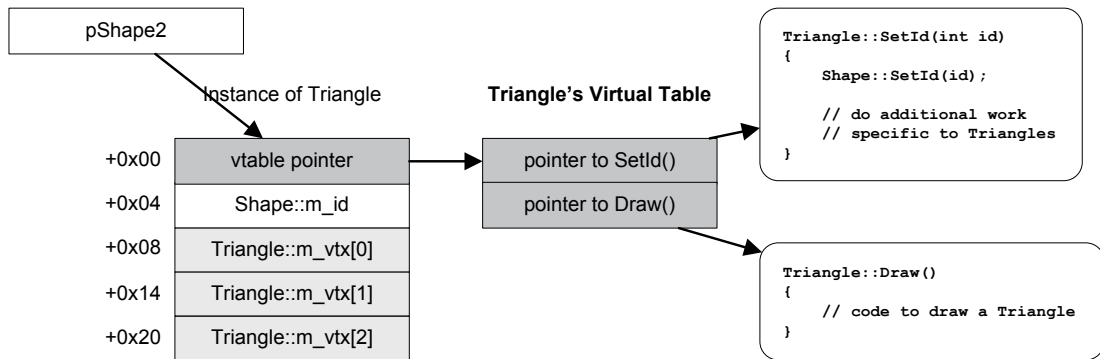


Figure 3.18. `pShape2` points to an instance of class `Triangle`.

timize your code. It's also crucial for concurrent programming—and all programmers must understand concurrency if they hope to take full advantage of the ever-increasing degree of parallelism in modern computing hardware.

3.4.1 Learning from the Simpler Computers of Yesteryear

In the following pages, we'll discuss the design of a simple, generic CPU, rather than diving into the specifics of any one particular processor. However, some readers may find it helpful to ground our somewhat theoretical discussion by reading about the specifics of a real CPU. I myself learned how computers work when I was a teenager, by programming my Apple II computer and the Commodore 64. Both of these machines had a simple CPU known as the 6502, which was designed and manufactured by MOS Technology Inc. I also learned some of what I know by reading about and working with the common ancestor to Intel's entire x86 line of CPUs, the 8086 (and its cousin the 8088). Both of these processors are great for learning purposes due to their simplicity. This is especially true of the 6502, which is the simplest CPU I've ever worked with. Once you understand how a 6502 and/or an 8086 works, modern CPUs will be a lot easier to understand.

To that end, here are a few great resources on the details of 6502 and 8086 architecture and programming:

- Chapters 1 and 2 of the book "Inside the Apple IIe" by Gary B. Little [33] provides a great overview of 6502 assembly language programming. The book is available online at <http://www.apple2scans.net/files/InsidetheIle.pdf>.

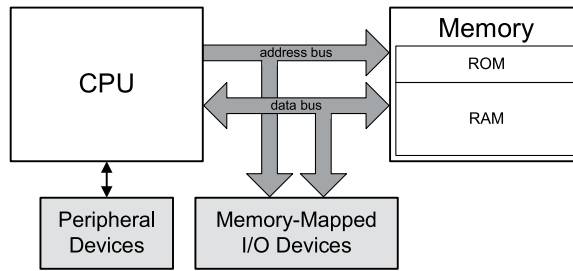


Figure 3.19. A simple serial von Neumann computer architecture.

- <http://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html> gives a nice overview of the x86 instruction set architecture.
- You should also definitely check out *Michael Abrash's Graphics Programming Black Book* [1] for loads of useful information about assembly programming on the 8086, plus incredible tips for software optimization and graphics programming back in the early days of gaming.

3.4.2 Anatomy of a Computer

The simplest computer consists of a *central processing unit* (CPU) and a bank of *memory*, connected to one another on a circuit board called the *motherboard* via one or more *buses*, and connected to external peripheral devices by means of a set of *I/O ports* and/or *expansion slots*. This basic design is referred to as the *von Neumann architecture* because it was first described in 1945 by the mathematician and physicist John von Neumann and his colleagues while working on the classified ENIAC project. A simple serial computer architecture is illustrated in Figure 3.19.

3.4.3 CPU

The CPU is the “brains” of the computer. It consists of the following components:

- an *arithmetic/logic unit* (ALU) for performing integer arithmetic and bit shifting,
- a *floating-point unit* (FPU) for doing floating-point arithmetic (typically using the IEEE 754 floating-point standard representation),
- virtually all modern CPUs also contain a *vector processing unit* (VPU) which is capable of performing floating-point and integer operations on multiple data items in parallel,

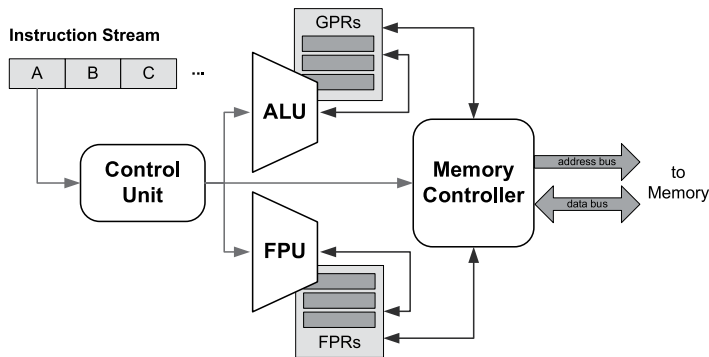


Figure 3.20. Typical components of a serial CPU.

- a *memory controller* (MC) or *memory management unit* (MMU) for interfacing with on-chip and off-chip memory devices,
- a bank of *registers* which act as temporary storage during calculations (among other things), and
- a *control unit* (CU) for decoding and dispatching machine language instructions to the other components on the chip, and routing data between them.

All of these components are driven by a periodic square wave signal known as the *clock*. The frequency of the clock determines the rate at which the CPU performs operations such as executing instructions or performing arithmetic. The typical components of a serial CPU are shown in Figure 3.20.

3.4.3.1 ALU

The ALU performs unary and binary arithmetic operations such as negation, addition, subtraction, multiplication and division, and also performs logical operations such as AND, OR, exclusive OR (abbreviated XOR or EOR), bitwise complement and bit shifting. In some CPU designs, the ALU is physically split into an arithmetic unit (AU) and a logic unit (LU).

An ALU typically performs integer operations only. Floating-point calculations require very different circuitry, and are usually performed by a physically separate floating-point unit (FPU). Early CPUs like the Intel 8088/8086 had no on-chip FPU; if floating-point math support was desired, these CPUs had to be augmented with a separate FPU co-processor chip such as the Intel 8087. In later CPU designs, an FPU was typically included on the main CPU die.

3.4.3.2 VPU

A vector processing unit (VPU) acts a bit like a combination ALU/FPU, in that it can typically perform both integer and floating-point arithmetic. What differentiates a VPU is its ability to apply arithmetic operators to *vectors* of input data—each of which typically consists of between two and 16 floating-point values or up to 64 integer values of various widths—rather than to scalar inputs. Vector processing is also known as *single instruction multiple data* or SIMD, because a single arithmetic operator (e.g., multiplication) is applied to multiple pairs of inputs simultaneously. See Section 4.10 for more details.

Today’s CPUs don’t actually contain an FPU *per se*. Instead, all floating-point calculations, even those involving scalar `float` values, are performed by the VPU. Eliminating the FPU reduces the transistor count on the CPU die, allowing those transistors to be used to implement larger caches, more-complex out-of-order execution logic, and so on. And as we’ll see in Section 4.10.6, optimizing compilers will typically convert math performed on `float` variables into *vectorized* code that uses the VPU anyway.

3.4.3.3 Registers

In order to maximize performance, an ALU or FPU can usually only perform calculations on data that exists in special high-speed memory cells called *registers*. Registers are typically physically separate from the computer’s main memory, located on-chip and in close proximity to the components that access them. They’re usually implemented using fast, high-cost multi-ported static RAM or SRAM. (See Section 3.4.5 for more information about memory technologies.) A bank of registers within a CPU is called a *register file*.

Because registers aren’t part of main memory,¹ they typically don’t have addresses but they do have names. These could be as simple as R0, R1, R2 etc., although early CPUs tended to use letters or short mnemonics as register names. For example, the Intel 8088/8086 had four 16-bit general-purpose registers named AX, BX, CX and DX. The 6502 by MOS Technology, Inc. performed all of its arithmetic operations using a register known as the *accumulator*² (A), and used two auxiliary registers called X and Y for other operations such as indexing into an array.

¹Some early computers did use main RAM to implement registers. For example, the 32 registers in the IBM 7030 Stretch (IBM’s first transistor-based supercomputer) were “overlaid” on the first 32 addresses in main RAM. In some early ALU designs, one of its inputs would come from a register while the other came from main RAM. These designs were practical because back then, RAM access latencies were low relative to the overall performance of the CPU.

²The term “accumulator” arose because early ALUs used to work one bit at a time, and would therefore *accumulate* the answer by masking and shifting the individual bits into the result register.

Some of the registers in a CPU are designed to be used for general calculations. They're appropriately named *general-purpose registers* (GPR). Every CPU also contains a number of *special-purpose registers* (SPR). These include:

- the *instruction pointer* (IP),
- the *stack pointer* (SP),
- the *base pointer* (BP) and
- the *status register*.

Instruction Pointer

The *instruction pointer* (IP) contains the address of the currently-executing instruction in a machine language program (more on machine language in Section 3.4.7.2).

Stack Pointer

In Section 3.3.5.2 we saw how a program's *call stack* serves both as the primary mechanism by which functions call one another, and as the means by which memory is allocated for local variables. The *stack pointer* (SP) contains the address of the top of the program's call stack. The stack can grow up or down in terms of memory addresses, but for the purpose of this discussion let's assume it grows downward. In this case, a data item may be *pushed* onto the stack by subtracting the size of the item from the value of the stack pointer, and then writing the item at the new address pointed to by SP. Likewise, an item can be *popped* off the stack by reading it from the address pointed to by SP, and then adding its size to SP.

Base Pointer

The *base pointer* (BP) contains the base address of the current function's *stack frame* on the call stack. Many of a function's local variables are allocated within its stack frame, although the optimizer may assign others exclusively to a register for the duration of the function. Stack-allocated variables occupy a range of memory addresses at a unique offset from the base pointer. Such a variable can be located in memory by simply subtracting its unique offset from the address stored in BP (assuming the stack grows down).

Status Register

A special register known as the *status register*, *condition code register* or *flags*

register contains bits that reflect the results of the most-recent ALU operation. For instance, if the result of a subtraction is zero, the *zero bit* (typically named “Z”) is set within the status register, otherwise the bit is cleared. Likewise, if an add operation resulted in an overflow, meaning that a binary 1 must be “carried” to the next word of a multi-word addition, the *carry bit* (often named “C”) is set, otherwise it is cleared.

The flags in the status register can be used to control program flow via *conditional branching*, or they can be used to perform subsequent calculations, such as adding the carry bit to the next word in a multi-word addition.

Register Formats

It’s important to understand that the FPU and VPU typically operate on their own private sets of registers, rather than making use of the ALU’s general-purpose integer registers. One reason for this is speed—the closer the registers are to the compute unit that uses them, the less time is needed to access the data they contain. Another reason is that the FPU’s and VPU’s registers are typically *wider* than the ALU’s GPRs.

For example, a 32-bit CPU has GPRs that are 32 bits wide each, but an FPU might operate on 64-bit double-precision floats, or even 80-bit “extended” double-precision values, meaning that its registers have to be 64 or 80 bits wide, respectively. Likewise, each register used by a VPU needs to contain a *vector* of input data, meaning that these registers must be much wider than a typical GPR. For example, Intel’s SSE2 (streaming SIMD extensions) vector processor can be configured to perform calculations on vectors containing either four single-precision (32-bit) floating-point values each, or two double-precision (64-bit) values each. Hence SSE2 vector registers are each 128 bits wide.

The physical separation of registers between ALU and FPU is one reason why conversions between `int` and `float` were very expensive, back in the days when FPUs were commonplace. Not only did the bit pattern of each value have to be converted back and forth between its two’s complement integer format and its IEEE 754 floating-point representation, but the data also had to be transferred physically between the general-purpose integer registers and the FPU’s registers. However, today’s CPUs no longer contain an FPU per se—all floating-point math is typically performed by a vector processing unit. A VPU can handle both integer and floating-point math, and conversions between the two are much less expensive, even when moving data from an integer GPU into a vector register or vice-versa. That said, it’s still a good idea to avoid converting data between `int` and `float` formats

where possible, because even a low-cost conversion is still more expensive than no conversion.

3.4.3.4 Control Unit

If the CPU is the “brains” of the computer, then the *control unit* (CU) is the “brains” of the CPU. Its job is to manage the flow of data within the CPU, and to orchestrate the operation of all of the CPU’s other components.

The CU runs a program by reading a stream of *machine language* instructions, decoding each instruction by breaking it into its opcode and operands, and then issuing work requests and/or routing data to the ALU, FPU, VPU, the registers and/or the memory controller as dictated by the current instruction’s opcode. In pipelined and superscalar CPUs, the CU also contains complex circuitry to handle branch prediction and the scheduling of instructions for out-of-order execution. We’ll look at the operation of the CU in more detail in Section 3.4.7.

3.4.4 Clock

Every digital electronic circuit is essentially a state machine. In order for it to change states, the circuit must be driven to do so by a digital signal. Such a signal might be provided by *changing the voltage* on a line in the circuit from 0 volts to 3.3 volts, or vice-versa.

State changes within a CPU are typically driven by a periodic square wave signal known as the *system clock*. Each rising or falling edge of this signal is known as a *clock cycle*, and the CPU can perform at least one primitive operation on every cycle. To a CPU, time therefore appears to be *quantized*.³

The rate at which a CPU can perform its operations is governed by the *frequency* of the system clock. Clock speeds have increased significantly over the past few decades. Early CPUs developed in the 1970s, like the MOS Technology’s 6502 and Intel’s 8086/8088 CPUs, had clocks that ran in the 1–2 MHz range (millions of cycles per second). Today’s CPUs, like the Intel Core i7, are typically clocked in the 2–4 GHz range (*billions* of cycles per second).

It’s important to realize that one CPU instruction doesn’t necessarily take one clock cycle to execute. Not all instructions are created equal—some instructions are very simple, while others are more complex. Some instructions are implemented under the hood as a combination of simpler micro-operations (μ -ops) and therefore take many more cycles to execute than their simpler counterparts.

³Contrast this to an *analog* electronic circuit, where time is treated as *continuous*. For example, an old-school signal generator can produce a true sine wave that smoothly varies between, say, –5 volts and 5 volts over time.

Also, while early CPUs could truly execute some instructions in a single clock cycle, today's pipelined CPUs break even the simplest instruction down into multiple *stages*. Each stage in a pipelined CPU takes one clock cycle to execute, meaning that a CPU with an N -stage pipeline has a minimum *instruction latency* of N clock cycles. A simple pipelined CPU can retire instructions at a *rate* of one instruction per clock cycle, because a new instruction is fed into the pipeline each clock tick. But if you were to trace one particular instruction through the pipeline, it would take N cycles to move from start to finish through the pipeline. We'll discuss pipelined CPUs in more depth in Section 4.2.

3.4.4.1 Clock Speed versus Processing Power

The “processing power” of a CPU or computer can be defined in various ways. One common measure is the *throughput* of the machine—the number of operations it can perform during a given interval of time. Throughput is expressed either in units of millions of instructions per second (MIPS) or floating-point operations per second (FLOPS).

Because instructions or floating-point operations don't generally complete in exactly one cycle, and because different instructions take differing numbers of cycles to run, the MIPS or FLOPS metrics of a CPU are just averages. As such, you cannot simply look at the clock frequency of a CPU and determine its processing power in MIPS or FLOPS. For example, a serial CPU running at 3 GHz, in which one floating-point multiply takes six cycles to complete on average, could theoretically achieve 0.5 GFLOPS. But many factors including pipelining, superscalar designs, vector processing, multicore CPUs and other forms of parallelism conspire to obfuscate the relationship between clock speed and processing power. Thus the only way to determine the true processing power of a CPU or computer is to measure it—typically by running standardized benchmarks.

3.4.5 Memory

The *memory* in a computer acts like a bank of mailboxes at the post office, with each box or “cell” typically containing a single *byte* of data (an eight-bit value).⁴ Each one-byte memory cell is identified by its *address*—a simple numbering

⁴Actually, early computers often accessed memory in units of “words” that were larger than 8 bits. For example, the IBM 701 (produced in 1952) addressed memory in units of 36-bit words, and the PDP-1 (made in 1959) could access up to 4096 18-bit memory words. Eight-bit bytes were popularized by the Intel 8008 in 1972. Seven bits are required to encode both lower- and uppercase English letters. By extending this to eight bits, a wide range of special characters could also be supported.

scheme ranging from 0 to $N - 1$, where N is the size of addressable memory in bytes.

Memory comes in two basic flavors:

- *read-only memory* (ROM), and
- read/write memory, known for historical reasons⁵ as *random access memory* (RAM).

ROM modules retain their data even when power is not applied to them. Some types of ROM can be programmed only once. Others, known as *electronically erasable programmable ROM* or EEPROM, can be reprogrammed over and over again. (Flash drives are one example of EEPROM memory.)

RAM can be further divided into *static* RAM (SRAM) and *dynamic* RAM (DRAM). Both static and dynamic RAM retain their data as long as power is applied to them. But unlike static RAM, dynamic RAM also needs to be “refreshed” periodically (by reading the data and then re-writing it) in order to prevent its contents from disappearing. This is because DRAM memory cells are built from MOS capacitors that gradually lose their charge, and reading such memory cells is destructive to the data they contain.

RAM can also be categorized by various other design characteristics, such as:

- whether it is *multi-ported*, meaning that it be accessed simultaneously by more than one component within the CPU;
- whether it operates by being synchronized to a clock (SDRAM) or asynchronously;
- and whether or not it supports double data rate access (DDR), meaning the RAM can be read or written on both rising and falling clock edges.

3.4.6 Buses

Data is transferred between the CPU and memory over connections known as *buses*. A bus is just a bundle of parallel digital “wires” called *lines*, each of which can represent a single bit of data. When the line carries a voltage signal⁶ it represents a binary one, and when the line has no voltage applied

⁵Random-access memory was so named because earlier memory technologies used delay loops to store data, meaning that it could only be read in the order it was written. RAM technology improved on this situation by permitting data to be accessed at random, i.e., in any order.

⁶Early *transistor-transistor* logic (TTL) devices operated at a supply voltage of 5 volts, so a 5-volt signal would represent a binary 1. Most of today’s digital electronic devices utilize *complementary metal oxide semiconductor* logic (CMOS) which can operate at a lower supply voltage, typically between 1.2 and 3.3 volts.

(0 volts) it represents a binary zero. A bundle of n single-bit lines arranged in parallel can transmit an n -bit number (i.e., any number in the range 0 through $2^n - 1$).

A typical computer contains two buses: An *address bus* and a *data bus*. The CPU loads data from a memory cell into one of its registers by supplying an address to the memory controller via the address bus. The memory controller responds by presenting the bits of the data item stored in the cell(s) in question onto the data bus, where it can be “seen” by the CPU. Likewise, the CPU writes a data item to memory by broadcasting the destination address over the address bus and placing the bit pattern of the data item to write onto the data bus. The memory controller responds by writing the given data into the corresponding memory cell(s). We should note that the address and data buses are sometimes implemented as two physically separate sets of wires, and sometimes as a single set of wires which are multiplexed between address and data bus functions during different phases of the memory access cycle.

3.4.6.1 Bus Widths

The *width* of the *address bus*, measured in bits, controls the range of possible addresses that can be accessed by the CPU (i.e., the size of *addressable memory* in the machine). For example, a computer with a 16-bit address bus can access at most 64 KiB of memory, using addresses in the range 0x0000 through 0xFFFF. A computer with a 32-bit address bus can access a 4 GiB memory, using addresses in the range 0x00000000 through 0xFFFFFFFF. And a machine with a 64-bit address bus can access a staggering 16 EiB (exbibytes) of memory. That’s $2^{64} = 16 \times 1024^6 \approx 1.8 \times 10^{19}$ bytes!

The width of the *data bus* determines how much data can be transferred between CPU registers and memory at a time. (The data bus is typically the same width as the general-purpose registers in the CPU, although this isn’t always the case.) An 8-bit data bus means that data can be transferred one byte at a time—loading a 16-bit value from memory would require two separate memory cycles, one to fetch the least-significant byte and one to fetch the most-significant byte. At the other end of the spectrum, a 64-bit data bus can transfer data between memory and a 64-bit register as a single memory operation.

It’s possible to access data items that are narrower than the width of a machine’s data bus, but it’s typically more costly than accessing items whose widths match that of the data bus. For example, when reading a 16-bit value on a 64-bit machine, a full 64 bits worth of data must still be read from memory. The desired 16-bit field then has to be masked off and possibly shifted into place within the destination register. This is one reason why the C language

does not define an `int` to be a specific number of bits wide—it was purposefully defined to match the “natural” size of a word on the target machine, in an attempt to make source code more portable. (Ironically this policy has actually resulted in source code often being *less* portable due to implicit assumptions about the width of an `int`.)

3.4.6.2 Words

The term “word” is often used to describe a multi-byte value. However, the number of bytes comprising a word is not universally defined. It depends to some degree on context.

Sometimes the term “word” refers to the smallest multi-byte value, namely 16 bits or two bytes. In that context, a double word would be 32 bits (four bytes) and a quad word would be 64 bits (eight bytes). This is the way the term “word” is used in the Windows API.

On the other hand, the term “word” is also used to refer to the “natural” size of data items on a particular machine. For example, a machine with 32-bit registers and a 32-bit data bus operates most naturally with 32-bit (four byte) values, and programmers and hardware folks will sometimes say that such a machine as a *word size* of 32 bits. The takeaway here is to be aware of context whenever you hear the term “word” being used to refer to the size of a data item.

3.4.6.3 *n*-Bit Computers

You may have encountered the term “*n*-bit computer.” This usually means a machine with an *n*-bit data bus and/or registers. But the term is a bit ambiguous, because it might also refer to a computer whose address bus is *n* bits wide. Also, on some CPUs the data bus and register widths don’t match. For example, the 8088 had 16-bit registers and a 16-bit address bus, but it only had an 8-bit data bus. Hence it acted like a 16-bit machine internally, but its 8-bit data bus caused it to behave like an 8-bit machine in terms of memory accesses. Again, be aware of context when talking about an *n*-bit machine.

3.4.7 Machine and Assembly Language

As far as the CPU is concerned, a “program” is nothing more than a sequential stream of relatively simple *instructions*. Each instruction tells the control unit (CU), and ultimately the other components within the CPU such as the memory controller, ALU, FPU or VPU, to perform an operation. An instruction might move data around within the computer or within the CPU itself, or it might transform that data in some manner (e.g., by performing an arith-

metic or logical operation on the data). Normally the instructions in a program are executed sequentially, although some instructions can alter this sequential flow of control by “jumping” to a new spot within the program’s overall instruction stream.

3.4.7.1 Instruction Set Architecture (ISA)

CPU designs vary widely from one manufacturer to the next. The set of all instructions supported by a given CPU, along with various other details of the CPU’s design like its addressing modes and the in-memory instruction format, is called its *instruction set architecture* or ISA. (This is not to be confused with a programming language’s *application binary interface* or ABI, which defines higher-level protocols like calling conventions.) We won’t attempt to cover the details of any one CPU’s ISA here, but the following categories of instruction types are common to pretty much every ISA:

- *Move*. These instructions move data between registers, or between memory and a register. Some ISAs break the “move” instruction into separate “load” and “store” instructions.
- *Arithmetic operations*. These of course include addition, subtraction, multiplication and division, but may also include other operations like unary negation, inversion, square root, and so on.
- *Bitwise operators*. These include AND, OR, exclusive OR (abbreviated XOR or EOR) and bitwise complement.
- *Shift/rotate operators*. These instructions allow the bits within a data word to be shifted left or right, with or without affecting the carry bit in the status register, or rotated (where the bits rolling off one end of the word “wrap around” to the other end).
- *Comparison*. These instructions allow two values to be compared, in order to determine if one is less than, greater than or equal to the other. In most CPUs, the comparison instructions use the ALU to subtract the two input values, thereby setting the appropriate bits in the status register, but the result of the subtraction is simply discarded.
- *Jump and branch*. These allow program flow to be altered by storing a new address into the instruction pointer. This can be done either unconditionally (in which case it is called a “jump” instruction) or conditionally based on the state of various flags in the status register (in which case it is often called a “branch”). For example, a “branch if zero” instruction alters the contents of the IP if and only if the “Z” bit in the status register is set.

- *Push and pop.* Most CPUs provide special instructions for pushing the contents of a register onto the program stack and for popping the current value at the top of the stack into a register.
- *Function call and return.* Some ISAs provide explicit instructions for calling a function (also known as a procedure or subroutine) and returning from it. However, function call and return semantics can also be provided by a combination of push, pop and jump instructions.
- *Interrupts.* An “interrupt” instruction triggers a digital signal within the CPU that causes it to jump temporarily to a pre-installed *interrupt service routine* routine which is often not part of the program being run. Interrupts are used to notify the operating system or a user program of events such as an input becoming available on a peripheral device. Interrupts can also be triggered by user programs in order to “call” into the operating system kernel’s routines. See section 4.4.2 for more details.
- *Other instruction types.* Most ISAs support a variety of instruction types that don’t fall into one of the categories listed above. For example, the “no-op” instruction (often called NOP) is an instruction that has no effect other than to introduce a short delay. NOP instructions also consume memory, and on some ISAs they are used to align subsequent instructions properly in memory.

We can’t possibly list all instruction types here, but if you’re curious you can always read through the ISA documentation of a real processor like the Intel x86 (which is available at <http://intel.ly/2woVFQ8>).

3.4.7.2 Machine Language

Computers can only deal with numbers. As such, each instruction in a program’s instruction stream must be encoded numerically. When a program is encoded in this way, we say it is written in *machine language*, or ML for short. Of course, machine language isn’t a single language—it’s really a multitude of languages, one for each distinct CPU/ISA.

Every machine language instruction is comprised of three basic parts:

- an *opcode*, which tells the CPU which operation to perform (add, subtract, move, jump, etc.),
- zero or more *operands* which specify the inputs and/or outputs of the instruction, and
- some kind of *options field*, specifying things like the *addressing mode* of the instruction and possibly other flags.

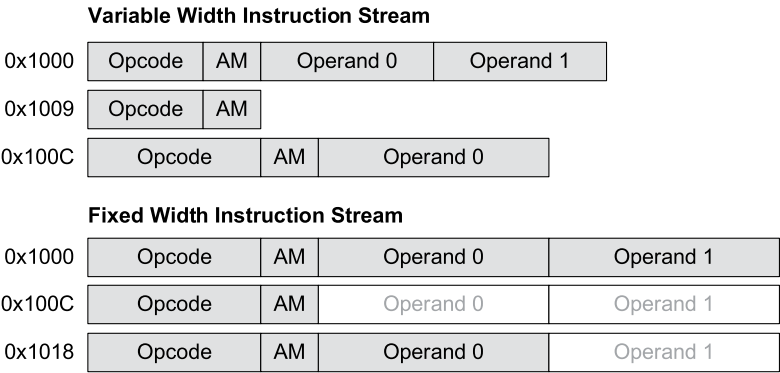


Figure 3.21. Two hypothetical machine language instruction encoding schemes. Top: In a variable-width encoding scheme, different instructions may occupy different numbers of bytes in memory. Bottom: A fixed-width instruction encoding uses the same number of bytes for every instruction in an instruction stream.

Operands come in many flavors. Some instructions might take the names (encoded as numeric ids) of one or more registers as operands. Others might expect a literal value as an operand (e.g., “load the value 5 into register R2,” or “jump to address 0x0102ED5C”). The way in which an instruction’s operands are interpreted and used by the CPU is known as the instruction’s *addressing mode*. We’ll discuss addressing modes in more detail in Section 3.4.7.4.

The opcode and operands (if any) of an ML instruction are packed into a contiguous sequence of bits called an *instruction word*. A hypothetical CPU might encode its instructions as shown in Figure 3.21 with perhaps the first byte containing the opcode, addressing mode and various options flags, followed by some number of bytes for the operands. Each ISA defines the width of an instruction word (i.e., the number of bits occupied by each instruction) differently. In some ISAs, all instructions occupy a fixed number of bits; this is typical of *reduced instruction set computers* (RISC). In other ISAs, different types of instructions may be encoded into differently-sized instruction words; this is common in *complex instruction set computers* (CISC).

Instruction words can be as small as four bits in some microcontrollers, or they may be many bytes in size. Instruction words are often multiples of 32 or 64 bits, because this matches the width of the CPU’s registers and/or data bus. In *very long instruction word* (VLIW) CPU designs, parallelism is achieved by allowing multiple operations to be encoded into a single very wide instruction word, for the purpose of executing them in parallel. Instructions in a VLIW ISA can therefore be upwards of hundreds of bytes in width.


```
ReturnZero:
    ; else return 0;
    xor    eax, eax    ; set EAX to zero
    ret    ; (EAX is the return value)
```

Let's break this down. The `cmp` instruction compares the values in registers EAX and EBX (which we're assuming contain the values `a` and `b` from the C snippet). Next, the `jle` instruction branches to the label `ReturnZero`, but it does so if and *only if* the value in EAX is less than or equal to EBX. Otherwise it falls through.

If EAX is *greater* than EBX ($a > b$), we fall through to the `add` instruction, which calculates $a + b$ and stores the result back into EAX, which we're assuming serves as the return value. We issue a `ret` instruction, and control is returned to the calling function.

If EAX is less than or equal to EBX ($a \leq b$), the branch is taken and we continue execution immediately after the label `ReturnZero`. Here, we use a little trick to set EAX to zero by XORing it with itself. Then we issue a `ret` instruction to return that zero to the caller.

For more details on Intel assembly language programming, see <http://www.cs.virginia.edu/~evans/cs216/guides/x86.html>.

3.4.7.4 Addressing Modes

A seemingly simple instruction like “move” (which transfers data between registers and memory) has many different variants. Are we moving a value from one register to another? Are we loading a literal value like 5 into a register? Are we loading a value from memory into a register? Or are we storing a value in a register out to memory? All of these variants are called *addressing modes*. We won't go over all the possible addressing modes here, but the following list should give you a good feel for the kinds of addressing modes you will encounter on a real CPU:

- *Register addressing*. This mode allows values to be transferred from one register to another. In this case, the operands of the instruction specify which registers are involved in the operation.
- *Immediate addressing*. This mode allows a literal or “immediate” value to be loaded into a register. In this case, the operands are the target register and the immediate value to be loaded.
- *Direct addressing*. This mode allows data to be moved to or from memory. In this case, the operands specify the direction of the move (to or from memory) and the memory address in question.

- *Register indirect addressing.* In this mode, the target memory address is taken from a register, rather than being encoded as a literal value in the operands of the instruction. This is how pointer dereferences are implemented in languages like C and C++. The value of the pointer (an address) is loaded into a register, and then a register-indirect “move” instruction is used to dereference the pointer and either load the value it points to into a target register, or store a value in a source register into that memory address.
- *Relative addressing.* In this mode, the target memory address is specified as an operand, and the value stored in a specified register is used as an offset from that target memory address. This is how indexed array accesses are implemented in a language like C or C++.
- *Other addressing modes.* There are numerous other variations and combinations, some of which are common to virtually all CPUs, and some of which are specific to certain CPUs.

3.4.7.5 Further Reading on Assembly Language

In the preceding sections, we’ve had just a tiny taste of assembly language. For an easy-to-digest description of x86 assembly programming, see <http://flint.cs.yale.edu/cs421/papers/x86-asm/asm.html>. For more on calling conventions and ABIs, see https://en.wikibooks.org/wiki/X86_Disassembly/Functions_and_Stack_Frames.

3.5 Memory Architectures

In a simple von Neumann computer architecture, memory is treated as a single homogeneous block, all of which is equally accessible to the CPU. In reality, a computer’s memory is almost never architected in such a simplistic manner. For one thing, the CPU’s registers are a form of memory, but registers are typically referenced by name in an assembly language program, rather than being addressed like ordinary ROM or RAM. Moreover, even “regular” memory is typically segregated into blocks with different characteristics and different purposes. This segregation is done for a variety of reasons, including cost management and optimization of overall system performance. In this section, we’ll investigate some of the memory architectures commonly found in today’s personal computers and gaming consoles, and explore some of the key reasons why they are architected the way they are.

3.5.1 Memory Mapping

An n -bit address bus gives the CPU access to a theoretical *address space* that is 2^n bytes in size. An individual memory device (ROM or RAM) is always addressed as a contiguous block of memory cells. So a computer's address space is typically divided into various contiguous segments. One or more of these segments correspond to ROM memory modules, and others correspond to RAM modules. For example on the Apple II, the 16-bit addresses in the range 0xC100 through 0xFFFF were assigned to ROM chips (containing the computer's firmware), while the addresses from 0x0000 through 0xBFFF were assigned to RAM. Whenever a physical memory device is assigned to a range of addresses in a computer's address space, we say that the address range has been *mapped* to the memory device.

Of course a computer may not have as much memory installed as could be theoretically addressed by its address bus. A 64-bit address bus can access 16 EiB of memory, so we'd be unlikely to ever fully populate such an address space! (At 160 TiB, even HP's prototype supercomputer called "The Machine" isn't anywhere close to that amount of physical memory.) Therefore, it's commonplace for some segments of a computer's address space to remain unassigned.

3.5.1.1 Memory-Mapped I/O

Address ranges needn't all map to memory devices—an address range might also be mapped to other *peripheral devices*, such as a joystick or a network interface card (NIC). This approach is called *memory-mapped I/O* because the CPU can perform I/O operations on a peripheral device by reading or writing to addresses, just as if they were ordinary RAM. Under the hood, special circuitry detects that the CPU is reading from or writing to a range of addresses that have been mapped to a non-memory device, and converts the read or write request into an I/O operation directed at the device in question. As a concrete example, the Apple II mapped I/O devices into the address range 0xC000 through 0xC0FF, allowing programs to do things like control bank-switched RAM, read and control voltages on the pins of a game controller socket on the motherboard, and perform other I/O operations by merely reading from and writing to the addresses in this range.

Alternatively, a CPU might communicate with non-memory devices via special registers known as *ports*. In this case, whenever the CPU requests that data be read from or written to a port register, the hardware converts the request into an I/O operation on the target device. This approach is called *port-mapped I/O*. In the Arduino line of microcontrollers, port-mapped I/O gives a

program direct control over the digital inputs and outputs at some of the pins of the chip.

3.5.1.2 Video RAM

Raster-based display devices typically read a hard-wired range of physical memory addresses in order to determine the brightness/color of each pixel on the screen. Likewise, early character-based displays would determine which character glyph to display at each location on the screen by reading ASCII codes from a block of memory. A range of memory addresses assigned for use by a video controller is known as *video RAM* (VRAM).

In early computers like the Apple II and early IBM PCs, video RAM used to correspond to memory chips on the motherboard, and memory addresses in VRAM could be read from and written to by the CPU in exactly the same way as any other memory location. This is also the case on game consoles like the PlayStation 4 and the Xbox One, where both the CPU and GPU share access to a single, large block of unified memory.

In personal computers, the GPU often lives on a separate circuit board, plugged into an expansion slot on the motherboard. Video RAM is typically located on the video card so that the GPU can access it as quickly as possible. A bus protocol such as PCI, AGP or PCI Express (PCIe) is used to transfer data back and forth between “main RAM” and VRAM, via the expansion slot’s bus. This physical separation between main RAM and VRAM can be a significant performance bottleneck, and is one of the primary contributors to the complexity of rendering engines and graphics APIs like OpenGL and DirectX 11.

3.5.1.3 Case Study: The Apple II Memory Map

To illustrate the concepts of memory mapping, let’s look at a simple real-world example. The Apple II had a 16-bit address bus, meaning that its address space was only 64 KiB in size. This address space was mapped to ROM, RAM, memory-mapped I/O devices and video RAM regions as follows:

0xC100	-	0xFFFF	ROM (Firmware)
0xC000	-	0xC0FF	Memory-Mapped I/O
0x6000	-	0xBFFF	General-purpose RAM
0x4000	-	0x5FFF	High-res video RAM (page 2)
0x2000	-	0x3FFF	High-res video RAM (page 1)
0x0C00	-	0x1FFF	General-purpose RAM
0x0800	-	0x0BFF	Text/lo-res video RAM (page 2)
0x0400	-	0x07FF	Text/lo-res video RAM (page 1)

0x0200 - 0x03FF	General-purpose and reserved RAM
0x0100 - 0x01FF	Program stack
0x0000 - 0x00FF	Zero page (mostly reserved for DOS)

We should note that the addresses in the Apple II memory map corresponded directly to memory chips on the motherboard. In today's operating systems, programs work in terms of *virtual* addresses rather than *physical* addresses. We'll explore virtual memory in the next section.

3.5.2 Virtual Memory

Most modern CPUs and operating systems support a memory remapping feature known as a *virtual memory system*. In these systems, the memory addresses used by a program don't map directly to the memory modules installed in the computer. Instead, whenever a program reads from or writes to an address, that address is first *remapped* by the CPU via a look-up table that's maintained by the OS. The remapped address might end up referring to an actual cell in memory (with a totally different numerical address). It might also end up referring to a block of data on-disk. Or it might turn out not to be mapped to any physical storage at all. In a virtual memory system, the addresses used by programs are called *virtual addresses*, and the bit patterns that are actually transmitted over the address bus by the memory controller in order to access a RAM or ROM module are called *physical addresses*.

Virtual memory is a powerful concept. It allows programs to make use of more memory than is actually installed in the computer, because data can overflow from physical RAM onto disk. Virtual memory also improves the stability and security of the operating system, because each program has its own private "view" of memory and is prevented from stomping on memory blocks that are owned by other programs or the operating system itself. We'll talk more about how the operating system manages the virtual memory spaces of running programs in Section 4.4.5.

3.5.2.1 Virtual Memory Pages

To understand how this remapping takes place, we need to imagine that the entire addressable memory space (that's 2^n byte-sized cells if the address bus is n bits wide) is conceptually divided into equally-sized contiguous chunks known as *pages*. Page sizes differ from OS to OS, but are always a power of two—a typical page size is 4 KiB or 8 KiB. Assuming a 4 KiB page size, a 32-bit address space would be divided up into 1,048,576 distinct pages, numbered from 0x0 to 0xFFFFF, as shown in Table 3.2.

From Address	To Address	Page Index
0x00000000	0x00000FFF	Page 0x0
0x00001000	0x00001FFF	Page 0x1
0x00002000	0x00002FFF	Page 0x2
...		
0x7FFF2000	0x7FFF2FFF	Page 0x7FFF2
0x7FFF3000	0x7FFF3FFF	Page 0x7FFF3
...		
0xFFFFE000	0xFFFFEFFF	Page 0xFFFFE
0xFFFFF000	0xFFFFF7FF	Page 0xFFFFF

Table 3.2. Division of a 32-bit address space into 4 KiB pages.

The mapping between virtual addresses and physical addresses is always done at the granularity of pages. One hypothetical mapping between virtual and physical addresses is illustrated in Figure 3.22.

3.5.2.2 Virtual to Physical Address Translation

Whenever the CPU detects a memory read or write operation, the address is split into two parts: the *page index* and an *offset* within that page (measured in bytes). For a page size of 4 KiB, the offset is just the lower 12 bits of the address, and the page index is the upper 20 bits, masked off and shifted to the right by 12 bits. For example, the virtual address 0x1A7C6310 corresponds to an offset of 0x310 and a page index of 0x1A7C6.

The page index is then looked up by the CPU's *memory management unit* (MMU) in a *page table* that maps virtual page indices to physical ones. (The page table is stored in RAM and is managed by the operating system.) If the page in question happens to be mapped to a physical memory page, the virtual page index is translated into the corresponding physical page index, the bits of this physical page index are shifted to the left as appropriate and ORed together with the bits of the original page offset, and voila! We have a physical address. So to continue our example, if virtual page 0x1A7C6 happens to map to physical page 0x73BB9, then the translated physical address would end up being 0x73BB9310. This is the address that would actually be transmitted over the address bus. Figure 3.23 illustrates the operation of the MMU.

If the page table indicates that a page is not mapped to physical RAM (either because it was never allocated, or because that page has since been

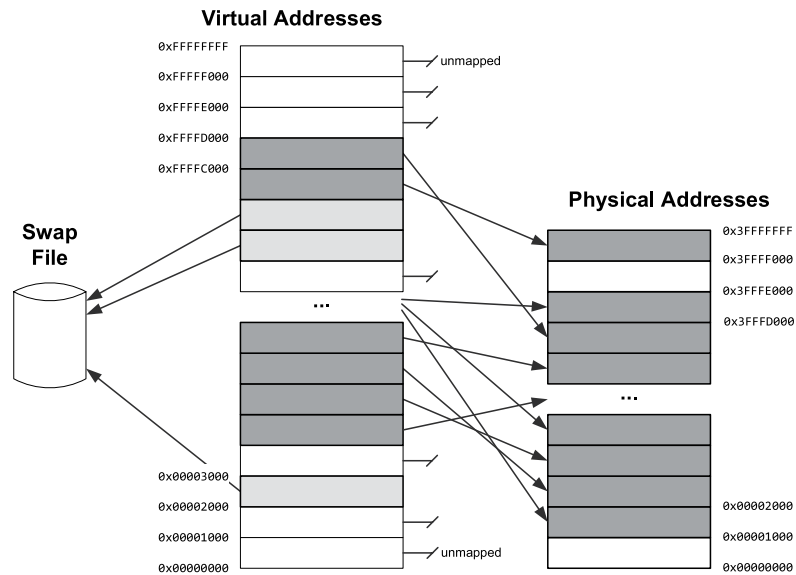


Figure 3.22. Page-sized address ranges in a virtual memory space are remapped either to physical memory pages, a swap file on disk, or they may remain unmapped.

swapped out to a disk file), the MMU raises an *interrupt*, which tells the operating system that the memory request can't be fulfilled. This is called a *page fault*. (See Section 4.4.2 for more on interrupts.)

3.5.2.3 Handling Page Faults

For accesses to unallocated pages, the OS normally responds to a page fault by crashing the program and generating a core dump. For accesses to pages that have been swapped out to disk, the OS temporarily suspends the currently-running program, reads the page from the swap file into a physical page of RAM, and then translates the virtual address to a physical address as usual. Finally it returns control back to the suspended program. From the point of view of the program, this operation is entirely seamless—it never “knows” whether a page was already in memory or had to be swapped in from disk.

Normally pages are swapped out to disk only when the load on the memory system is high, and physical pages are in short supply. The OS tries to swap out only the least-frequently-used pages of memory to avoid programs continually “thrashing” between memory-based and disk-based pages.

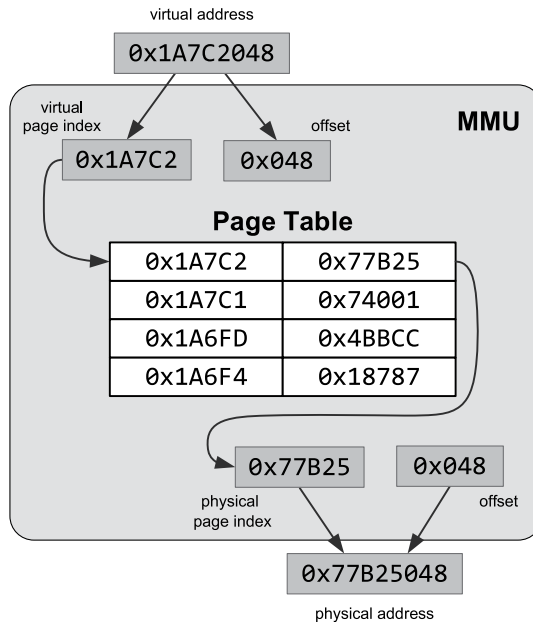


Figure 3.23. The MMU intercepts a memory read operation, and breaks the virtual address into a virtual page index and an offset. The virtual page index is converted to a physical page index via the page table, and the physical address is constructed from the physical page index and the original offset. Finally, the instruction is executed using the remapped physical address.

3.5.2.4 The Translation Lookaside Buffer (TLB)

Because page sizes tend to be small relative to the total size of addressable memory (typically 4 KiB or 8 KiB), the page table can become very large. Looking up physical addresses in the page table would be time-consuming if the entire table had to be scanned for every memory access a program makes.

To speed up access, a caching mechanism is used, based on the assumption that an average program will tend to reuse addresses within a relatively small number of pages, rather than read and write randomly across the entire address range. A small table known as the *translation lookaside buffer* (TLB) is maintained within the MMU on the CPU die, in which the virtual-to-physical address mappings of the most recently-used addresses are cached. Because this buffer is located in close proximity to the MMU, accessing it is very fast.

The TLB acts a lot like a general-purpose *memory cache hierarchy*, except that it is only used for caching page table entries. See Section 3.5.4 for a discussion of how cache hierarchies work.

3.5.2.5 Further Reading on Virtual Memory

See <https://www.cs.umd.edu/class/sum2003/cmsc311/Notes/Memory/virtual.html> and <https://gabrieletolomei.wordpress.com/miscellanea/operating-systems/virtual-memory-paging-and-swapping> for two good discussions of virtual memory implementation details.

This paper by Ulrich Drepper entitled, “What Every Programmer Should Know About Memory” is also a must-read for all programmers: <https://www.akkadia.org/drepper/cpumemory.pdf>.

3.5.3 Memory Architectures for Latency Reduction

The speed with which data can be accessed from a memory device is an important characteristic. We often speak of *memory access latency*, which is defined as the length of time between the moment the CPU requests data from the memory system and the moment that that data is actually received by the CPU. Memory access latency is primarily dependent on three factors:

1. the technology used to implement the individual memory cells,
2. the number of read and/or write ports supported by the memory, and
3. the physical distance between those memory cells and the CPU core that uses them.

The access latency of *static RAM* (SRAM) is generally much lower than that of *dynamic RAM* (DRAM). SRAM achieves its lower latency by utilizing a more complex design which requires more transistors per bit of memory than does DRAM. This in turn makes SRAM more expensive than DRAM, both in terms of financial cost per bit, and in terms of the amount of real estate per bit that it consumes on the die.

The simplest memory cell has a single *port*, meaning only one read or write operation can be performed by it at any given time. *Multi-ported* RAM allows multiple read and/or write operations to be performed simultaneously, thereby reducing the latency caused by contention when multiple cores, or multiple components within a single core, attempt to access a bank of memory simultaneously. As you’d expect, a multi-ported RAM requires more transistors per bit than a single-ported design, and hence it costs more and uses more real estate on the die than a single-ported memory.

The physical distance between the CPU and a bank of RAM also plays a role in determining the access latency of that memory. This is because electronic signals travel at a finite speed within the computer. In theory, electronic

signals are comprised of electromagnetic waves, and hence travel at close⁷ to the speed of light. Additional latencies are introduced by the various switching and logic circuits that a memory access signal encounters on its journey through the system. As such, the closer a memory cell is to the CPU core that uses it, the lower its access latency tends to be.

3.5.3.1 The Memory Gap

In the early days of computing, memory access latency and instruction execution latency were on roughly equal footing. For example, on the Intel 8086, register-based instructions could execute in two to four cycles, and a main memory access also took roughly four cycles. However, over the past few decades both raw clock speeds and the effective instruction throughput of CPUs have been improving at a much faster rate than memory access speeds. Today, the access latency of main memory is extremely high relative to the latency of executing a single instruction: Whereas a register-based instruction still takes between one and 10 cycles to complete on an Intel Core i7, an access to main RAM can take on the order of 500 cycles to complete! This ever-increasing discrepancy between CPU speeds and memory access latencies is often called the *memory gap*. Figure 3.24 illustrates the trend of an ever-increasing memory gap over time.

Programmers and hardware designers have together developed a wide range of techniques to work around the problems caused by high memory access latency. These techniques usually focus on one or more of the following:

1. *reducing* average memory latency by placing smaller, faster memory banks closer to the CPU core, so that frequently-used data can be accessed more quickly;
2. *“hiding”* memory access latency by arranging for the CPU to do other useful work while waiting for a memory operation to complete; and/or
3. *minimizing* accesses to main memory by arranging a program’s data in as efficient a manner as possible with respect to the work that needs to be done with that data.

In this section, we’ll have a closer look at memory architectures for average latency reduction. We’ll discuss the other two techniques (latency “hiding” and

⁷The speed that an electronic signal travels within a transmission medium such as copper wire or optical fiber is always somewhat lower than the speed of light in a vacuum. Each interconnect material has its own characteristic *velocity factor* (VF) ranging from less than 50% to 99% the speed of light in a vacuum.

the minimization of memory accesses via judicious data layout) while investigating parallel hardware design and concurrent programming techniques in Chapter 4.

3.5.3.2 Register Files

A CPU's register file is perhaps the most extreme example of a memory architecture designed to minimize access latency. Registers are typically implemented using multi-ported static RAM (SRAM), usually with dedicated ports for read and write operations, allowing these operations to occur in parallel rather than serially. What's more, the register file is typically located immediately adjacent to the circuitry for the ALU that uses it. Furthermore, registers are accessed pretty much directly by the ALU, whereas accesses to main RAM typically have to pass through the virtual address translation system, the memory cache hierarchy and cache coherence protocol (see Section 3.5.4), the address and data buses, and possibly also through crossbar switching logic. These facts explain why registers can be accessed so quickly, and also why the cost of register RAM is relatively high as compared to general-purpose RAM. This higher cost is justified because registers are by far the most frequently-used memory in any computer, and because the total size of the register file is very small when compared to the size of main RAM.

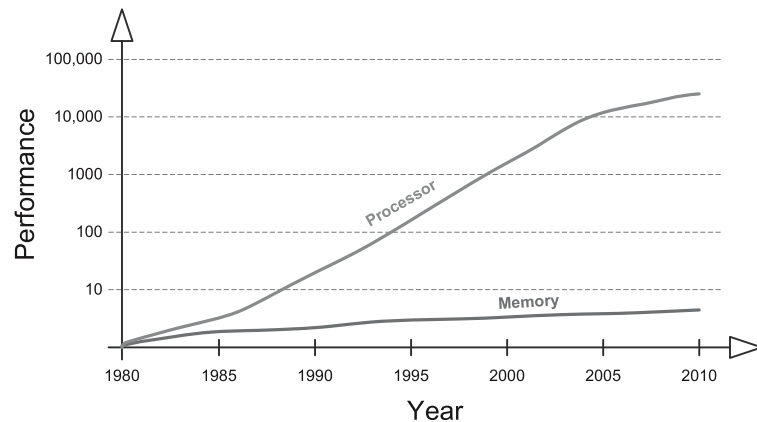


Figure 3.24. The ever-increasing difference between CPU performance and the performance of memory is called the *memory gap*. (Adapted from [23] *Computer Architecture: A Quantitative Approach* by John L. Hennessy and David A. Patterson.)

3.5.4 Memory Cache Hierarchies

A *memory cache hierarchy* is one of the primary mechanisms for mitigating the impacts of high memory access latencies in today's personal computers and game consoles. In a cache hierarchy, a small but fast bank of RAM called the *level 1 (L1) cache* is placed very near to the CPU core (on the same die). Its access latency is almost as low as the CPU's register file, because it is so close to the CPU core. Some systems provide a larger but somewhat slower *level 2 (L2) cache*, located further away from the core (usually also on-chip, and often shared between two or more cores on a multicore CPU). Some machines even have larger but more-distant L3 or L4 caches. These caches work in concert to automatically retain copies of the most frequently-used data, so that accesses to the very large but very slow bank of main RAM located on the system motherboard are kept to a minimum.

A caching system improves memory access performance by keeping *local copies* in the cache of those chunks of data that are most frequently accessed by the program. If the data requested by the CPU is already in the cache, it can be provided to the CPU very quickly—on the order of tens of cycles. This is called a *cache hit*. If the data is not already present in the cache, it must be fetched into the cache from main RAM. This is called a *cache miss*. Reading data from main RAM can take hundreds of cycles, so the cost of a cache miss is very high indeed.

3.5.4.1 Cache Lines

Memory caching takes advantage of the fact that memory access patterns in real software tend to exhibit two kinds of *locality of reference*:

1. *Spatial locality*. If memory address N is accessed by a program, it's likely that nearby addresses such as $N + 1$, $N + 2$ and so on will also be accessed. Iterating sequentially through the data stored in an array is an example of a memory access pattern with a high degree of spatial locality.
2. *Temporal locality*. If memory address N is accessed by a program, it's likely that that same address will be accessed again in the near future. Reading data from a variable or data structure, performing a transformation on it, and then writing an updated result into the same variable or data structure is an example of a memory access pattern with a high degree of temporal locality.

To take advantage of locality of reference, memory caching systems move data into the cache in contiguous blocks called *cache lines* rather than caching data items individually.

For example, let's say the program is accessing the data members of an instance of a class or struct. When the first member is read, it might take hundreds of cycles for the memory controller to reach out into main RAM to retrieve the data. However, the cache controller doesn't just read that one member—it actually reads a larger contiguous block of RAM into the cache. That way, subsequent reads of the other data members of the instance will likely result in low-cost cache hits.

3.5.4.2 Mapping Cache Lines to Main RAM Addresses

There is a simple one-to-many correspondence between memory addresses in the cache and memory addresses in main RAM. We can think of the address space of the cache as being “mapped” onto the main RAM address space in a repeating pattern, starting at address 0 in main RAM and continuing on up until all main RAM addresses have been “covered” by the cache.

As a concrete example, let's say that our cache is 32 KiB in size, and that cache lines are 128 bytes each. The cache can therefore hold 256 cache lines ($256 \times 128 = 32,768 \text{ B} = 32 \text{ KiB}$). Let's further assume that main RAM is 256 MiB in size. So main RAM is 8192 times as big as the cache, because $(256 \times 1024) / 32 = 8192$. That means we need to overlay the address space of the cache onto the main RAM address space 8192 times in order to cover all possible physical memory locations. Or put another way, a single line in the cache maps to 8192 distinct line-sized chunks of main RAM.

Given any address in main RAM, we can find its address in the cache by taking the main RAM address *modulo* the size of the cache. So for a 32 KiB cache and 256 MiB of main RAM, the cache addresses 0x0000 through 0x7FFF (that's 32 KiB) map to main RAM addresses 0x0000 through 0x7FFF. But this range of cache addresses *also* maps to main RAM addresses 0x8000 through 0xFFFF, 0x10000 through 0x17FFF, 0x18000 through 0x1FFFF and so on, all the way up to the last main RAM block at addresses 0xFFF8000 through 0xFFFFFFF. Figure 3.25 illustrates the mapping between main RAM and cache RAM.

3.5.4.3 Addressing the Cache

Let's consider what happens when the CPU reads a single byte from memory. The address of the desired byte in main RAM is first converted into an address within the cache. The cache controller then checks whether or not the cache line containing that byte already exists in the cache. If it does, it's a cache hit, and the byte is read from the cache rather than from main RAM. If it doesn't, it's a cache miss, and the line-sized chunk of data is read from main RAM and

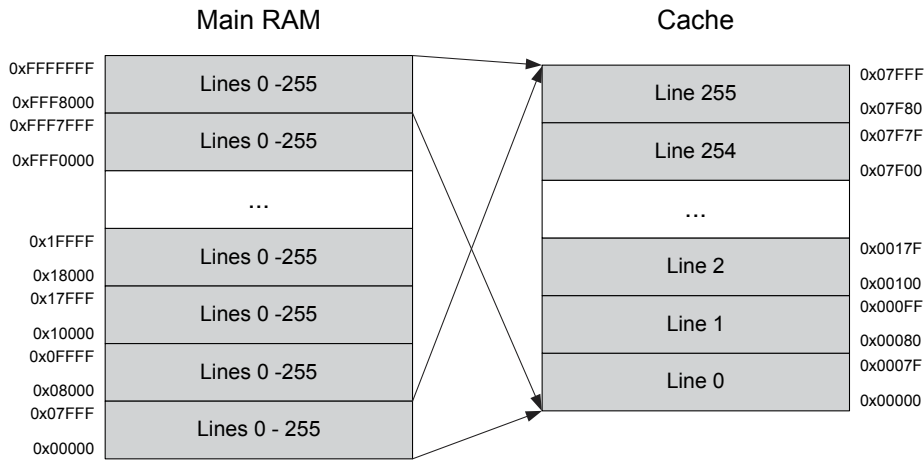


Figure 3.25. Direct mapping between main memory addresses and cache lines.

loaded into the cache so that subsequent reads of nearby addresses will be fast.

The cache can only deal with memory addresses that are *aligned* to a multiple of the cache line size (see Section 3.3.7.1 for a discussion of memory alignment). Put another way, the cache can really only be addressed in units of lines, not bytes. Hence we need to convert our byte's address into a *cache line index*.

Consider a cache that is 2^M bytes in total size, containing lines that are 2^n in size. The n least-significant bits of the main RAM address represent the *offset* of the byte within the cache line. We strip off these n least-significant bits to convert from units of bytes to units of lines (i.e., we divide the address by the cache line size, which is 2^n). Finally we split the resulting address into two pieces: The $(M - n)$ least-significant bits become the *cache line index*, and all the remaining bits tell us from *which* cache-sized block in main RAM the cache line came from. The block index is known as the *tag*.

In the case of a cache miss, the cache controller loads a line-sized chunk of data from main RAM into the corresponding line within the cache. The cache also maintains a small table of tags, one for each cache line. This allows the caching system to keep track of from *which* main RAM block each line in the cache came. This is necessary because of the many-to-one relationship between memory addresses in the cache and memory addresses in main RAM. Figure 3.26 illustrates how the cache associates a tag with each active line within it.

Returning to our example of reading a byte from main RAM, the complete

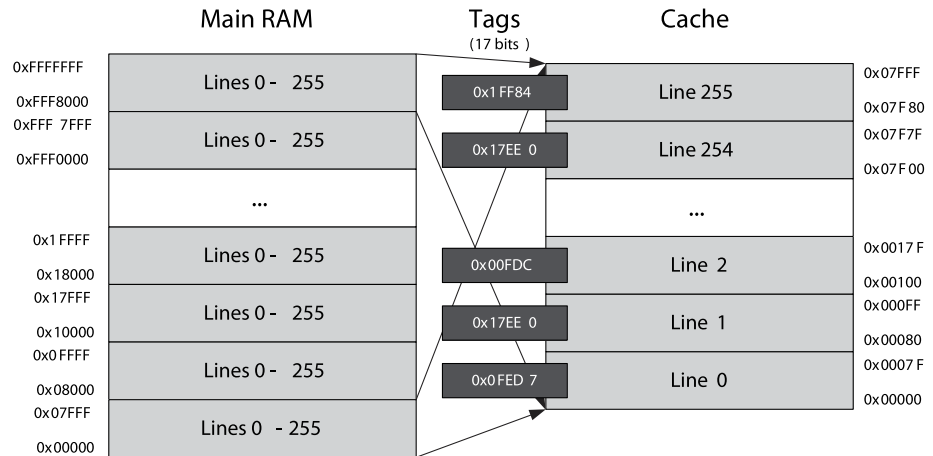


Figure 3.26. A *tag* is associated with each line in the cache, indicating from which cache-sized block of main RAM the corresponding line came.

sequence of events is as follows: The CPU issues a read operation. The main RAM address is converted into an offset, line index and tag. The corresponding tag in the cache is checked, using the line index to find it. If the tag in the cache matches the requested tag, it's a cache hit. In this case, the line index is used to retrieve the line-sized chunk of data from the cache, and the offset is used to locate the desired byte within the line. If the tags do not match, it's a cache miss. In this case, the appropriate line-sized chunk of main RAM is read into the cache, and the corresponding tag is stored in the cache's tag table. Subsequent reads of nearby addresses (those that reside within the same cache line) will therefore result in much faster cache hits.

3.5.4.4 Set Associativity and Replacement Policy

The simple mapping between cache lines and main RAM addresses described above is known as a *direct-mapped* cache. It means that each address in main RAM maps to only *one* line in the cache. Using our 32 KiB cache with 128-byte lines as an example, the main RAM address 0x203 maps to cache line 4 (because 0x203 is 515, and $\lfloor 515/128 \rfloor = 4$). However, in our example there are 8192 unique cache-line-sized blocks of main RAM that all map to cache line 4. Specifically, cache line 4 corresponds to main RAM addresses 0x200 through 0x27F, but also to addresses 0x8200 through 0x827F, and 0x10200 through 0x1027f, and 8189 *other* cache-line-sized address ranges as well.

When a cache miss occurs, the CPU must load the corresponding cache line from main memory into the cache. If the line in the cache contains no valid

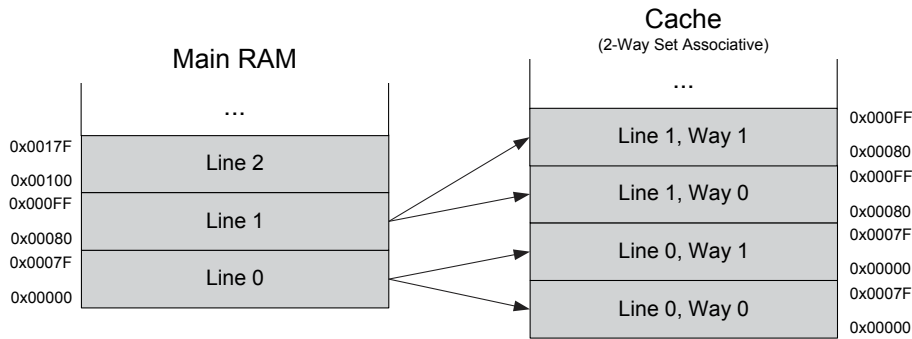


Figure 3.27. A 2-way set associative cache.

data, we simply copy the data into it and we're done. But if the line already contains data (from a different main memory block), we must overwrite it. This is known as *evicting* the cache line.

The problem with a direct-mapped cache is that it can result in pathological cases. For example, two unrelated main memory blocks might keep evicting one another in a ping-pong fashion. We can obtain better average performance if each main memory address can map to two or more distinct lines in the cache. In a *2-way set associative* cache, each main RAM address maps to two cache lines. This is illustrated in Figure 3.27. Obviously a 4-way set associative cache performs even better than a 2-way, and an 8-way or 16-way cache can outperform a 4-way cache and so on.

Once we have more than one “cache way,” the cache controller is faced with a dilemma: When a cache miss occurs, which of the “ways” should we evict and which ones should we allow to stay resident in the cache? The answer to this question differs between CPU designs, and is known as the CPU's *replacement policy*. One popular policy is *not most-recently used* (NMRU). In this scheme, the most-recently used “way” is kept track of, and eviction always affects the “way” or “ways” that are *not* the most-recently used one. Other policies include *first in first out* (FIFO), which is the only option in a direct-mapped cache, *least-recently used* (LRU), *least-frequently used* (LFU), and *pseudorandom*. For more on cache replacement policies, see <https://ece752.ece.wisc.edu/lect11-cache-replacement.pdf>.

3.5.4.5 Multilevel Caches

The *hit rate* is a measure of how often a program hits the cache, as opposed to incurring the large cost of a cache miss. The higher the hit rate, the better the program will perform (all other things being equal). There is a fundamental

trade-off between cache latency and hit rate. The larger the cache, the higher the hit rate—but larger caches cannot be located as close to the CPU, so they tend to be slower than smaller ones.

Most game consoles employ at least two *levels* of cache. The CPU first tries to find the data it's looking for in the *level 1* (L1) cache. This cache is small but has a very low access latency. If the data isn't there, it tries the larger but higher-latency *level 2* (L2) cache. Only if the data cannot be found in the L2 cache do we incur the full cost of a main memory access. Because the latency of main RAM can be so high relative to the CPU's clock rate, some PCs even include *level 3* (L3) and *level 4* (L4) caches.

3.5.4.6 Instruction Cache and Data Cache

When writing high-performance code for a game engine or for any other performance-critical system, it is important to realize that both data and code are cached. The *instruction cache* (I-cache, often denoted I\$) is used to preload executable machine code before it runs, while the *data cache* (D-cache, or D\$) is used to speed up read and write operations performed by that machine code. In a level 1 (L1) cache, the two caches are always physically distinct, because it is undesirable for an instruction read to cause valid data to be bumped out of the cache or vice versa. So when optimizing our code, we must consider both D-cache and I-cache performance (although as we'll see, optimizing one tends to have a positive effect on the other). Higher-level caches (L2, L3, L4) typically do not make this distinction between code and data, because their larger size tends to mitigate the problems of code evicting data or data evicting code.

3.5.4.7 Write Policy

We haven't talked yet about what happens when the CPU *writes* data to RAM. How the cache controller handles writes is known as its *write policy*. The simplest kind of cache is called a *write-through cache*; in this relatively simple cache design, all writes to the cache are mirrored to main RAM immediately. In a *write-back* (or *copy-back*) cache design, data is first written into the cache and the cache line is only flushed out to main RAM under certain circumstances, such as when a dirty cache line needs to be evicted in order to read in a new cache line from main RAM, or when the program explicitly requests a flush to occur.

3.5.4.8 Cache Coherency: MESI, MOESI and MESIF

When multiple CPU cores share a single main memory store, things get more complicated. It's typical for each core to have its own L1 cache, but multiple cores might share an L2 cache, as well as sharing main RAM. See Figure 3.28

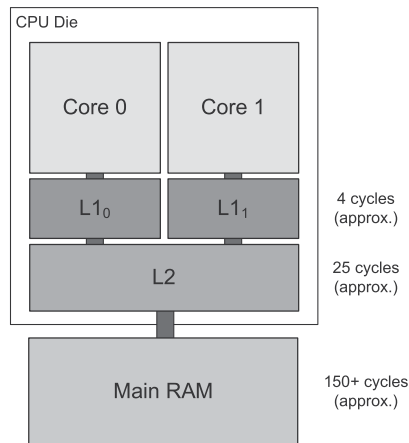


Figure 3.28. Level 1 and level 2 caches.

for an illustration of a two-level cache architecture with two CPU cores sharing one main memory store and an L2 cache.

In the presence of multiple cores, it's important for the system to maintain *cache coherency*. This amounts to making sure that the data in the caches belonging to multiple cores match one another and the contents of main RAM. Coherency doesn't have to be maintained at every moment—all that matters is that the running program can never *tell* that the contents of the caches are out of sync.

The three most common cache coherency protocols are known as MESI (modified, exclusive, shared, invalid), MOESI (modified, owned, exclusive, shared, invalid) and MESIF (modified, exclusive, shared, invalid and forward). We'll discuss the MESI protocol in more depth when we talk about multicore computing architectures in Section 4.9.4.2.

3.5.4.9 Avoiding Cache Misses

Obviously cache misses cannot be totally avoided, since data has to move to and from main RAM eventually. The trick to writing high performance software in the presence of a memory cache hierarchy is to arrange your data in RAM, and design your data manipulation algorithms, in such a way that the minimum number of cache misses occur.

The best way to avoid D-cache misses is to organize your data in *contiguous* blocks that are as *small* as possible and then access them *sequentially*. When the data is contiguous (i.e., you don't "jump around" in memory a lot), a single cache miss will load the maximum amount of relevant data in one go. When

the data is small, it is more likely to fit into a single cache line (or at least a minimum number of cache lines). And when you access your data sequentially, you avoid evicting and reloading cache lines multiple times.

Avoiding I-cache misses follows the same basic principle as avoiding D-cache misses. However, the implementation requires a different approach. The easiest thing to do is to keep your high-performance loops as small as possible in terms of code size, and avoid calling functions within your innermost loops. If you do opt to call functions, try to keep their code size small too. This helps to ensure that the entire body of the loop, including all called functions, will remain in the I-cache the entire time the loop is running.

Use inline functions judiciously. Inlining a small function can be a big performance boost. However, too much inlining bloats the size of the code, which can cause a performance-critical section of code to no longer fit within the cache.

3.5.5 Nonuniform Memory Access (NUMA)

When designing a multiprocessor game console or personal computer, system architects must choose between two fundamentally different memory architectures: *uniform memory access* (UMA) and *nonuniform memory access* (NUMA).

In a UMA design, the computer contains a single large bank of main RAM which is visible to all CPU cores in the system. The physical address space looks the same to each core, and each can read from and write to all memory locations in main RAM. A UMA architecture typically makes use of a cache hierarchy to mitigate memory access latency issues.

One problem with a UMA architecture is that the cores often contend for access to main RAM and any shared caches. For example, the PS4 contains eight cores arranged into two clusters. Each core has its own private L1 cache, but each cluster of four cores shares a single L2 cache, and all cores share main RAM. As such, the cores often contend with one another for access to the L2 cache and main RAM.

One way to address core contention problems is to employ a *non-uniform memory access* (NUMA) design. In a NUMA system, each core is provided with a relatively small bank of high-speed dedicated RAM called a *local store*. Like an L1 cache, a local store is typically located on the same die as the core itself, and is only accessible by that core. But unlike an L1 cache, access to the local store is explicit. A local store might be mapped to part of a core's address space, with main RAM mapped to a different range of addresses. Alternatively, certain cores might *only* be able to see the physical addresses within its local store, and might rely on a *direct memory access controller* (DMAC) to

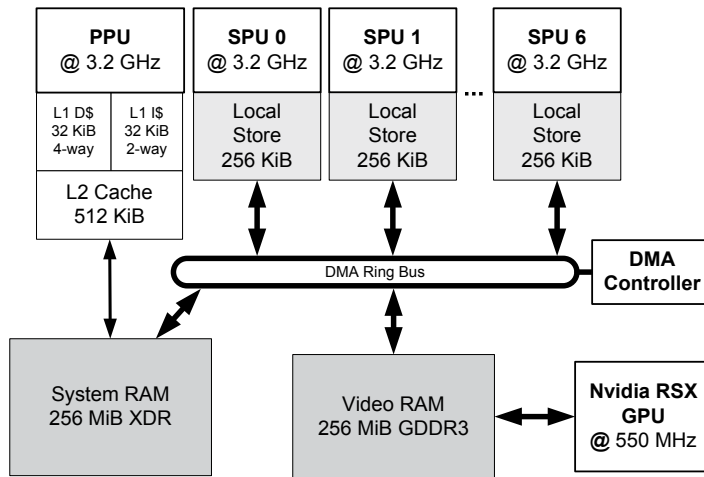


Figure 3.29. Simplified view of the PS3's cell broadband architecture.

transfer data between the local store and main RAM.

3.5.5.1 SPU Local Stores on the PS3

The PlayStation 3 is a classic example of a NUMA architecture. The PS3 contains a single main CPU known as the *Power processing unit* (PPU), eight⁸ co-processors known as *synergistic processing units* (SPUs), and an NVIDIA RSX graphics processing unit (GPU). The PPU has exclusive access to 256 MiB of main system RAM (with an L1 and L2 cache), the GPU has exclusive access to 256 MiB of video RAM (VRAM), and each SPU has its own private 256 KiB local store.

The physical address spaces of main RAM, video RAM and the SPUs' local stores are all totally isolated from one another. This means that, for example, the PPU cannot directly address memory in VRAM or in any of the SPUs' local stores, and any given SPU cannot directly address main RAM, video RAM, or any of the other SPUs' local stores, only its own local store. The memory architecture of the PS3 is illustrated in Figure 3.29.

3.5.5.2 PS2 Scratchpad (SPR)

Looking even farther back to the PlayStation 2, we can learn about another memory architecture that was designed to improve overall system perfor-

⁸Only six of the SPUs are available for use by game applications—one SPU is reserved for use by the operating system, and one is entirely off-limits to account for inevitable flaws in the fabrication process.

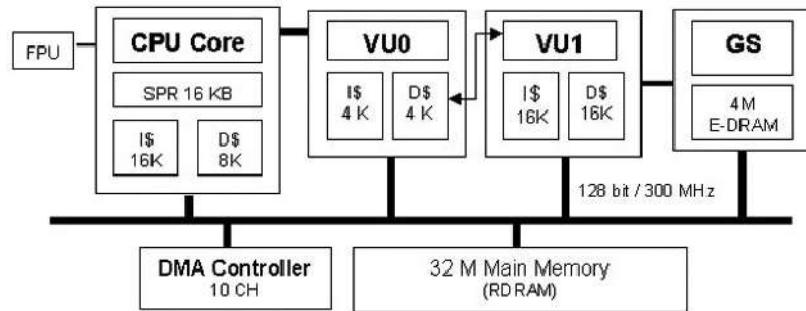


Figure 3.30. Simplified view of the PS2's memory architecture, illustrating the 16 KiB scratchpad (SPR), the L1 caches of the main CPU (EE) and the two vector units (VU0 and VU1), the 4 MiB bank of video RAM accessible to the graphics synthesizer (GS), the 32 MiB bank of main RAM, the DMA controller and the system buses.

mance. The main CPU on the PS2, called the emotion engine (EE), has a special 16 KiB area of memory called the *scratchpad* (abbreviated SPR), in addition to a 16 KiB L1 instruction cache (I-cache) and an 8 KiB L1 data cache (D-cache). The PS2 also contains two vector coprocessors known as VU0 and VU1, each with their own L1 I- and D-caches, and a GPU known as the *graphics synthesizer* (GS) connected to a 4 MiB bank of video RAM. The PS2's memory architecture is illustrated in Figure 3.30.

The scratchpad is located on the CPU die, and therefore enjoys the same low latency as L1 cache memory. But unlike the L1 cache, the scratchpad is memory-mapped so that it appears to the programmer to be a range of regular main RAM addresses. The scratchpad on the PS2 is itself uncached, meaning that reads and writes from and to it are direct; they bypass the EE's L1 cache entirely.

The scratchpad's main benefit isn't actually its low access latency—it's the fact that the CPU can access scratchpad memory without making use of the system buses. As such, reads and writes from and to the scratchpad can happen while the system's address and data bus are being used for other purposes. For example, a game might set up a chain of DMA requests to transfer data between main RAM and the two vector processing units (VUs) in the PS2. While these DMAs are being processed by the DMAC, and/or while the VUs are busy performing calculations (both of which would make heavy use of the system buses), the EE can be performing calculations on data that resides within the scratchpad, without interfering with the DMAs or the operation of the VUs. Moving data into and out of the scratchpad can be done via regular memory move instructions (or `memcpy()` in C/C++), but this task

can also be accomplished via DMA requests. The PS2's scratchpad therefore gives the programmer a lot of flexibility and power in order to maximize a game engine's data throughput.



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

4

Parallelism and Concurrent Programming

Computing performance—typically measured in millions of instructions per second (MIPS) or floating-point operations per second (FLOPS)—has been improving at a staggeringly rapid and consistent rate over the past four decades. In the late 1970s, the Intel 8087 floating-point coprocessor could muster only about 50 kFLOPS (5×10^4 FLOPS), while at roughly the same time a Cray-1 supercomputer the size of a large refrigerator could operate at a rate of roughly 160 MFLOPS (1.6×10^8 FLOPS). Today, the CPU in game consoles like the Playstation 4 or the Xbox One produces roughly 100 GFLOPS (10^{11} FLOPS) of processing power, and the fastest supercomputer, currently China's Sunway TaihuLight, has a LINPACK benchmark score of 93 PFLOPS (peta-FLOPS, or a staggering 9.3×10^{16} floating-point operations per second). This is an improvement of seven orders of magnitude for personal computers, and eight orders of magnitude for supercomputers.

Many factors contributed to this rapid rate of improvement. In the early days, the move from vacuum tubes to solid-state transistors permitted the miniaturization of computing hardware. The number of transistors that could be etched onto a single chip rose dramatically as new kinds of transistors, new types of digital logic, new substrate materials and new manufacturing processes were developed. These advances also contributed to improvements in

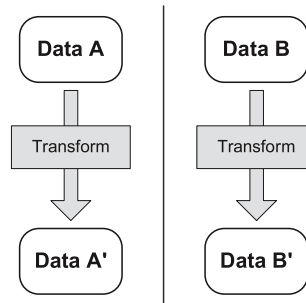


Figure 4.1. Two flows of control operating on *independent* data are not considered concurrent because they are not prone to data races.

power consumption and dramatic increases in CPU clock speeds. And starting in the 1990s, computer hardware manufacturers have increasingly turned to *parallelism* as a means of improving computing performance.

Writing software that runs *correctly* and *efficiently* on parallel computing hardware is significantly more difficult than writing software for the serial computers of yesteryear. It requires a deep understanding of how the hardware actually works. What's more, taking full advantage of the multicore CPUs found in modern computing platforms requires an approach to software design called *concurrent programming*. In a concurrent software system, multiple flows of control cooperate to solve a common problem. These flows of control must be carefully coordinated. Many of the techniques that work well in serial programs break down when applied to concurrent programs. As such, it's important for modern programmers (in all industries, including games) to have a solid understanding of parallel computing hardware, and to be well versed in concurrent programming techniques.

4.1 Defining Concurrency and Parallelism

4.1.1 Concurrency

A concurrent piece of software utilizes *multiple flows of control* to solve a problem. These flows of control might be implemented as multiple *threads* running within the context of a single process, or multiple cooperating processes running either on a single computer or multiple computers. Multiple flows of control can also be implemented within a process using other techniques such as *fibers* or *coroutines*.

The primary factor that distinguishes *concurrent programming* from *sequential programming* is the reading and/or writing of *shared data*. As shown in

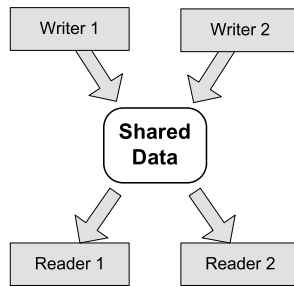


Figure 4.2. Two flows of control both reading from a shared data file and/or writing to a shared data file are examples of *concurrency*.

Figure 4.1, if we have two or more flows of control, each operating on a totally independent block of data, then this is not technically an example of concurrency—it’s just “computing at the same time.”

The central problem of concurrent programming is how to coordinate multiple readers and/or multiple writers of a shared data file, in such a way as to ensure predictable, correct results. At the heart of this problem is a special kind of *race condition* known as a *data race*, in which two or more flows of control “compete” to be the first to read, modify and write a chunk of shared data. The crux of the concurrency problem is to identify and eliminate data races. Two examples of concurrency are illustrated in Figure 4.2.

We’ll explore the techniques programmers use to avoid data races and thereby write reliable concurrent programs starting in Section 4.5. But before we do that, let’s take a look at how parallel computer hardware can both provide an effective platform for running concurrent software, and improve the execution speed of sequential programs as well.

4.1.2 Parallelism

In computer engineering, the term *parallelism* refers to any situation in which two or more distinct hardware components are operating simultaneously. In other words, *parallel* computer hardware can perform more than one task at a time. In contrast, *serial* computer hardware is capable of doing only one thing at a time.

Prior to 1989, consumer-grade computing devices were exclusively serial machines. Examples include the MOS Technology 6502 CPU, which was used in the Apple II and Commodore 64 personal computers, and the Intel 8086, 80286 and 80386 CPUs which were at the heart of early IBM PCs and clones.

Today, parallel computing hardware is ubiquitous. One obvious example of hardware parallelism is a multicore CPU, such as the Intel Core™ i7 or the AMD Ryzen™ 7. But parallelism can be employed at a wide range of granularities. For example, a single CPU might contain multiple ALUs and therefore be capable of performing multiple independent calculations in parallel. And at the other end of the spectrum, a cluster of computers working in tandem to solve a common problem is also an example of hardware parallelism.

4.1.2.1 Implicit versus Explicit Parallelism

One way to classify the various forms of parallelism in computer hardware design is to consider the *purpose* of parallelism in each. In other words, what problem does parallelism solve in a given design? Thinking along these lines, we can divide parallelism coarsely into two categories:

- implicit parallelism, and
- explicit parallelism.

Implicit parallelism refers to the use of parallel hardware components within a CPU for the purpose of improving the performance of a *single* instruction stream. This is also known as *instruction level parallelism* (ILP), because the CPU executes instructions from a single stream (a single thread) but each instruction is executed with some degree of hardware parallelism. Examples of implicit parallelism include:

- pipelining,
- superscalar architectures, and
- very long instruction word (VLIW) architectures.

We'll explore implicit parallelism in Section 4.2. GPUs also make extensive use of implicit parallelism; we'll take a closer look at the design and programming of GPUs in Section 4.11.

Explicit parallelism refers to the use of duplicated hardware components within a CPU, computer or computer system, for the purpose of running *more than one instruction stream* simultaneously. In other words, explicitly parallel hardware is designed to run *concurrent* software more efficiently than would be possible on a serial computing platform. The most common examples of explicit parallelism are:

- hyperthreaded CPUs,
- multicore CPUs,

- multiprocessor computers,
- computer clusters,
- grid computing, and
- cloud computing.

We'll investigate these explicitly parallel architectures further in Section 4.3.

4.1.3 Task versus Data Parallelism

Another way to understand parallelism is to divide it into two broad categories based on the kind of work being done in parallel.

- *Task parallelism*. When multiple heterogeneous operations are performed in parallel, we call this task parallelism. Performing animation calculations on one core while performing collision checks on another would be an example of this form of parallelism.
- *Data parallelism*. When a *single* operation is performed on multiple data items in parallel, it is called data parallelism. Calculating 1000 skinning matrices by running 250 matrix calculations on each of four cores would be an example of data parallelism.

Most real concurrent programs make use of both task and data parallelism to varying degrees.

4.1.4 Flynn's Taxonomy

Yet another way to classify the varying degrees of parallelism we'll encounter in computing hardware is to use *Flynn's Taxonomy*. Proposed by Michael J. Flynn of Stanford University in 1966, this approach breaks down parallelism into a two-dimensional space. Along one axis, we have the number of parallel flows of control (which Flynn refers to as the number of instructions running in parallel at any given moment). On the other axis, we have the number of distinct data streams being operated on by each instruction in the program. The space is thus divided into four quadrants:

- *Single instruction, single data (SISD)*: A single instruction stream operating on a single data stream.
- *Multiple instruction, multiple data (MIMD)*: Multiple instruction streams operating on multiple independent data streams.
- *Single instruction, multiple data (SIMD)*: A single instruction stream operating on multiple data streams (i.e., performing the same sequence of operations on multiple independent streams of data simultaneously).

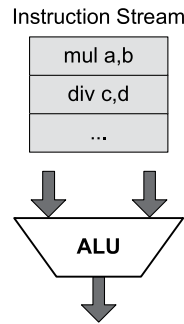


Figure 4.3. Example of SISD. A single ALU performs the multiply first, followed by the divide.

- *Multiple instruction, single data (MISD)*: Multiple instruction streams all operating on a single data stream. (MISD is rarely used in games, but one common application is to provide fault tolerance via redundancy.)

4.1.4.1 Single versus Multiple Data

It's important to realize here that a "data stream" isn't just an array of numbers. Most arithmetic operators are binary—they operate on two inputs to produce a single output. When applied to binary arithmetic, the term "single data" refers to a single *pair* of inputs, with a single output. As an example, let's have a look at how two binary arithmetic operations, a multiply ($a \times b$) and a divide (c/d), might be accomplished under each of the four Flynn categories:

- In a SISD architecture, a single ALU performs the multiply first, followed by the divide. This is illustrated in Figure 4.3.
- In a MIMD architecture, two ALUs perform operations in parallel, operating on two independent instruction streams. This is shown in Figure 4.4.
- The MIMD classification also applies to the case in which a single ALU processes two independent instruction streams via *time-slicing*, as shown in Figure 4.5.
- In a SIMD architecture, a single "wide ALU" known as a vector processing unit (VPU) performs the multiply first, followed by the divide, but each instruction operates on a pair of four-element input vectors and produces a four-element output vector. Figure 4.6 illustrates this approach.

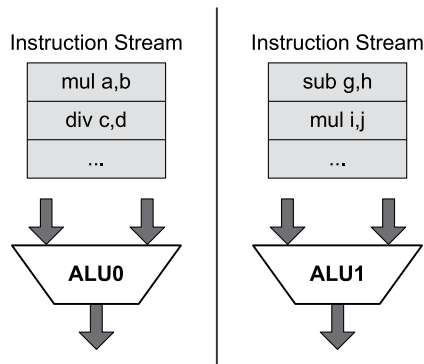


Figure 4.4. Example of MIMD. Two ALUs perform operations in parallel.

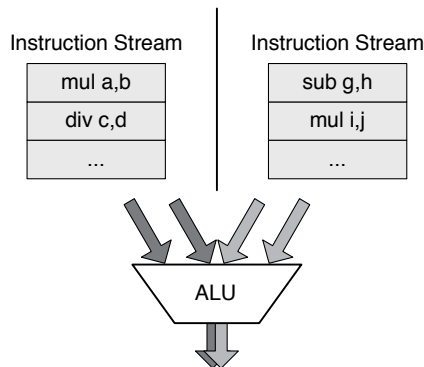


Figure 4.5. Example of time-sliced MIMD. A single ALU performs operations on behalf of two independent instruction streams, perhaps by alternating between them.

- In a MISD architecture, two ALUs process the same instruction stream (multiply first, followed by divide) and ideally produce identical results. Illustrated in Figure 4.7, this architecture is primarily useful for implementing fault tolerance via redundancy. ALU 1 acts as a “hot spare” for ALU 0 and vice-versa, meaning that if one of the ALUs experiences a failure, the system can seamlessly switch to the other.

4.1.4.2 GPU Parallelism: SIMT

In recent years, a fifth classification has been added to Flynn’s taxonomy to account for the design of graphics processing units (GPU). *Single instruction multiple thread* (SIMT) is essentially a hybrid between SIMD and MIMD, used primarily in the design of GPUs. It mixes SIMD processing (a single instruc-

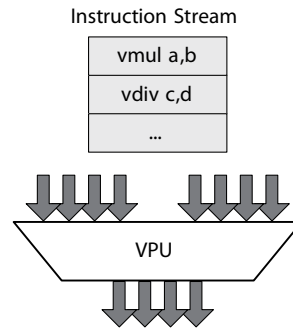


Figure 4.6. Example of SIMD. A single vector processing unit (VPU) performs the multiply first, followed by the divide, but each instruction operates on a pair of four-element input vectors and produces a four-element output vector.

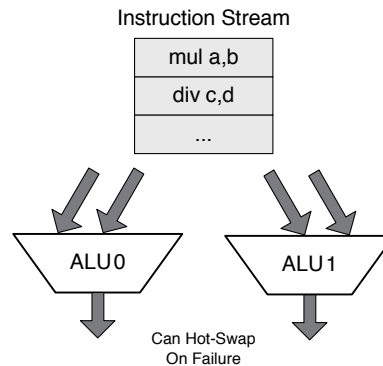


Figure 4.7. Example of MISD. Two ALUs process the same instruction stream (multiply first, followed by divide) and ideally produce identical results.

tion operating on multiple data streams simultaneously) with multithreading (more than one instruction stream sharing a processor via time-slicing).

The term “SIMT” was coined by NVIDIA, but it can be used to describe the design of any GPU. The term *manycore* is also often used to refer to a SIMT design (i.e., a GPU consisting of a relatively large number of lightweight SIMD cores) whereas the term *multicore* refers to MIMD designs (i.e., a CPU with a relatively smaller number of heavyweight general purpose cores). We’ll take a closer look at the SIMT design employed by GPUs in Section 4.11.

4.1.5 Orthogonality of Concurrency and Parallelism

We should stress here that concurrent software doesn't *require* parallel hardware, and parallel hardware isn't *only* for running concurrent software. For example, a concurrent multithreaded program can run on a single, serial CPU core via preemptive multitasking (see Section 4.4.4). Likewise, instruction level parallelism is intended to improve the performance of a single thread, and therefore benefits both concurrent and sequential software. So while they are closely related, concurrency and parallelism are really orthogonal concepts.

As long as our system involves *multiple readers* and/or *multiple writers* of a shared data object, we have a concurrent system. Concurrency can be achieved via preemptive multitasking (on serial *or* parallel hardware) or via true parallelism (in which each thread executes on a distinct core)—the techniques we'll learn in this chapter will be applicable either way.

4.1.6 Roadmap of the Chapter

In the following sections, we'll first turn our attention to implicit parallelism, and how best to optimize our software to take advantage of it. Next, we'll review the most common forms of explicit parallelism. Then we'll explore the various concurrent programming techniques used to harness explicitly parallel computing platforms. Finally, we'll round out the discussion of parallel programming by discussing SIMD vector processing, and how it applies to GPU design and general-purpose GPU programming (GPGPU) techniques.

4.2 Implicit Parallelism

In Section 4.1.2.1 we said that *implicit parallelism* is the use of parallel computing hardware for the purpose of improving the execution speed of a single thread. CPU manufacturers began using implicit parallelism in their consumer products in the late 1980s, in an attempt to make *existing code* run faster on each new generation of CPUs within a given product line.

There are a number of ways to apply parallelism to the problem of improving a CPU's single-threaded performance. The most common are *pipelined* CPU architectures, *superscalar* designs, and *very long instruction word* (VLIW) architectures. We'll begin by having a look at how a pipelined CPU functions, and then address the other two variants of implicit parallelism.

Clock Cycle	Fetch	Dec	Exec	Mem	WB
0	A				
1		A			
2			A		
3				A	
4					A
5	B				
6		B			

Figure 4.8. In a non-pipelined CPU, instruction stages are idle much of the time.

4.2.1 Pipelining

In order for a single machine language instruction to be executed by the CPU, it must pass through a number of distinct *stages*. Every CPU's design is a bit different—some CPU designs employ a large number of granular stages, while others utilize a smaller number of coarse-grained stages. However every CPU implements the following basic stages in one way or another:

- *Fetch*. The instruction to be executed is read from the memory location pointed to by the instruction pointer register (IP).
- *Decode*. The instruction word is decomposed into its opcode, addressing mode and optional operands.
- *Execute*. Based on the opcode, the appropriate functional unit within the CPU is selected (ALU, FPU, memory controller, etc.). The instruction is dispatched to the selected component for processing, along with any relevant operand data. The functional unit then performs its operation.
- *Memory access*. If the instruction involves reading or writing memory, the memory controller performs the appropriate operation during this stage.
- *Register write-back*. The functional unit executing the instruction (ALU, FPU, etc.) writes its results back into the destination register.

Figure 4.8 traces the path of two instructions named “A” and “B” through the five execution phases of a serial CPU. You’ll notice right away that this diagram contains a lot of blank space: While one stage is busy doing its thing to an instruction, all the other stages are twiddling their thumbs, doing nothing.

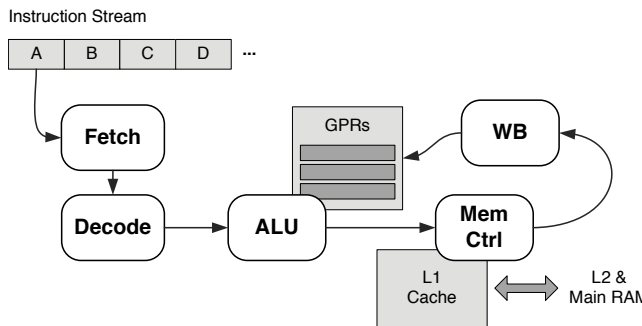


Figure 4.9. Components of a pipelined scalar CPU.

Clock Cycle	Fetch	Dec	Exec	Mem	WB
0	A				
1	B	A			
2	C	B	A		
3	D	C	B	A	
4	E	D	C	B	A
5	F	E	D	C	B
6	G	F	E	D	C

Figure 4.10. Ideal flow of instructions through a pipelined CPU.

Each of the stages of instruction execution is actually handled by different hardware within the CPU, as shown in Figure 4.9. The control unit (CU) and memory controller handle the instruction fetch stage. A different circuit within the CU then handles the decode stage. The ALU, FPU or VPU handles the lion's share of the execute stage. The memory stage is performed by the memory controller. And finally, the write-back stage primarily involves the registers. This division of labor amongst different circuits within the CPU is the key to making the CPU more efficient: We just need to keep all the stages' hardware busy all the time.

The solution is known as *pipelining*. Instead of waiting for each instruction to complete all five stages before starting to execute the next one, we begin the execution of a new instruction on *every* clock cycle. Multiple instructions are therefore "in flight" simultaneously. This process is illustrated in Figure 4.10.

Pipelining is a bit like doing laundry. If you have a large number of loads to do, it wouldn't be very efficient to wait until each load has been washed and

dried before starting the next one—while the washer is busy, the dryer would be sitting idle, and vice versa. It’s much better to keep both machines busy at all times, by starting the second load washing just as soon as the first load goes into the dryer, and so on.

Pipelining is a form of parallelism known as *instruction-level parallelism* (ILP). For the most part, ILP is designed to be transparent to the programmer. Ideally, at a given clock speed, a program that runs properly on a scalar CPU should be able to run correctly—but faster—on a pipelined CPU, as long as the two processors support the same instruction set architecture (ISA) of course. In theory, a CPU with a pipeline that is N stages deep can execute a program N times faster than its serial counterpart. However, as we’ll explore in Section 4.2.4, pipelining doesn’t always perform as well as we would expect, thanks to various kinds of *dependencies* between instructions in the instruction stream. Programmers who are interested in writing high-performance code therefore cannot remain oblivious to ILP. We must embrace it, understand it, and sometimes adjust the design of our code and/or data in order to get the most out of a pipelined CPU.

4.2.2 Latency versus Throughput

The *latency* of a pipeline is the amount of time required to completely process a single instruction. This is just the sum of the latencies of all the stages in the pipeline. Denoting latencies with the time variable T , we can write:

$$T_{\text{pipeline}} = \sum_{i=0}^{N-1} T_i \quad (4.1)$$

for a pipeline with N stages.

The *throughput* or *bandwidth* of a pipeline is a measure of how many instructions it can process per unit time. The throughput of a pipeline is determined by the latency of its slowest stage—much as a chain is only as strong as its weakest link. The throughput can be thought of as a frequency f , measured in instructions per second. It can be written as follows:

$$f = \frac{1}{\max(T_i)}. \quad (4.2)$$

4.2.3 Pipeline Depths

We said that each stage in a CPU can potentially have a different latency (T_i), and that the stage with the longest latency dictates the throughput of the entire processor. On each clock cycle, the other stages sit idle waiting for the longest

stage to complete. Ideally, then, we'd like all of the stages in our CPU to have roughly the same latency.

This goal can be achieved by increasing the total number of stages in the pipeline: If one stage is taking much longer than the others, it can be broken into two or more shorter stages in an attempt to make all the latencies roughly equal. However, we can't just keep subdividing stages forever. The larger the number of stages, the higher the overall instruction latency will be. This increases the cost of pipeline stalls (see Section 4.2.4). Therefore CPU manufacturers try to strike a balance between increasing throughput via deeper pipelines, and keeping the overall instruction latency in check. As a result, real CPU pipelines range from 4 or 5 stages at minimum, to something on the order of 30 stages at most.

4.2.4 Stalls

Sometimes the CPU is unable to issue a new instruction on a particular clock cycles. This is called a *stall*. On such a clock cycle, the first stage in the pipeline lies idle. On the next clock cycle, the second stage will be idle, and so on. A stall can therefore be thought of as a "bubble" of idle time that propagates through the pipeline at a rate of one stage per clock cycle. These bubbles are sometimes called *delay slots*.

4.2.5 Data Dependencies

Stalls are caused by *dependencies* between instructions in the instruction stream being executed. For example, consider the following sequence of instructions:

```
mov  ebx,5      ;; load the value 5 into register EBX
imul eax,10     ;; multiply the contents of EAX by 10
                ;; (result stored in EAX)
add  eax,7      ;; add 7 to EAX (result stored in EAX)
```

Ideally, we'd like to issue the `mov`, `imul` and `add` instructions on three consecutive clock cycles, to keep the pipeline as busy as possible. But in this case, the results of the `imul` instruction are used by the `add` instruction that follows it, so the CPU must wait until the `imul` has made it all the way through the pipeline before issuing the `add`. If the pipeline contains five stages, that means four cycles are wasted (see Figure 4.11). These kinds of dependencies between instructions are called *data dependencies*.

There are actually three kinds of dependencies between instructions that can cause stalls:

- data dependencies,

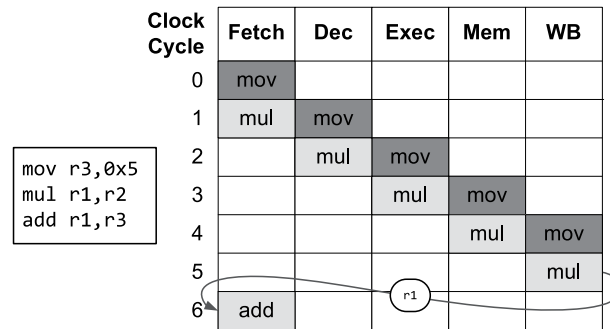


Figure 4.11. A data dependency between instructions causes a pipeline stall.

- control dependencies (also known as branch dependencies), and
- structural dependencies (also known as resource dependencies).

First we'll discuss how to avoid data dependencies, then we'll have a look at branch dependencies and how to mitigate their effects. Finally, we'll introduce superscalar CPU architectures and discuss how they can give rise to structural dependencies in a pipelined CPU.

4.2.5.1 Instruction Reordering

To mitigate the effects of a data dependency, we need to find some other instructions for the CPU to execute while it waits for the dependent instruction to make its way through the pipeline. This can often be accomplished by *re-ordering* the instructions in the program (while taking care not to change the *behavior* of the program in the process). For any given pair of interdependent instructions, we want to find some nearby instructions that are *not* dependent on them, and move those instructions up or down so that they end up running *between* the dependent instruction pair, thus filling the "bubble" with useful work.

Instruction reordering may of course be done by hand, by an adventurous programmer who doesn't mind diving into assembly language programming. Thankfully, however, this is often not necessary: Today's optimizing compilers are very good at reordering instructions automatically to reduce or eliminate the impact of data dependencies.

As programmers, of course we shouldn't blindly trust the compiler to optimize our code perfectly—when writing high-performance code, it's always a good idea to have a look at the disassembly and verify that the compiler did a reasonable job. But that said, we should also remember the 80/20 rule

(Section 2.3) and only spend time optimizing the 20% or less of our code that actually has a noticable impact on overall performance.

4.2.5.2 Out-of-Order Execution

Compilers and programmers aren't the only ones capable of reordering a sequence of machine language instructions to prevent stalls. Many of today's CPUs support a feature known as *out-of-order execution*, which enables them to dynamically detect data dependencies between instructions, and automatically resolve them.

To accomplish this feat, the CPU looks ahead in the instruction stream and analyzes the instructions' register usage in order to detect dependencies between them. When a dependency is found, the "look-ahead window" is searched for another instruction that is *not* dependent on any of the currently-executing instructions. If one is found, it is issued (out of order!) to keep the pipeline busy. The details of how out-of-order execution works is beyond our scope here. Suffice it to say that as programmers, we cannot rely on the CPU to execute the instructions in the same order we (or the compiler) wrote them.

Both the compiler's optimizer and the CPU's out-of-order execution logic take great care to ensure that the behavior of the program doesn't change as a result of instruction reordering. However, as we'll see in Section 4.9.3, compiler optimizations and out-of-order execution *can* cause bugs in a concurrent program (i.e., a program consisting of multiple threads that share data). This is one of the many reasons why concurrent programming requires more care than serial programming.

4.2.6 Branch Dependencies

What happens when a pipelined CPU encounters a conditional branch instruction (e.g., an `if` statement, or the conditional expression at the end of a `for` or `while` loop)? To answer this question, let's consider the following C/C++ code:

```
int SafeIntegerDivide(int a, int b, int defaultVal)
{
    return (b != 0) ? a / b : defaultVal;
}
```

If we looked at the disassembly for this function, it might look something like this on an Intel x86 CPU:

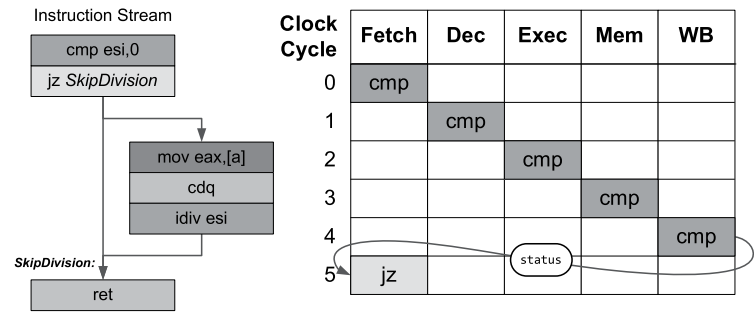


Figure 4.12. A dependency between a comparison instruction and a conditional branch instruction is called a branch dependency.

```
; function preamble omitted for clarity...

; first, put the default into the return register
mov  eax,dword ptr [defaultVal]

mov  esi,dword ptr [b] ; check (b != 0)
cmp  esi,0
jz   SkipDivision

mov  eax, dword ptr[a] ; divisor (a) must be in EDX:EAX
cdq                               ; ... so sign-extend into EDX
idiv esi                        ; quotient lands in EAX

SkipDivision:
; function postamble omitted for clarity...
ret                               ; EAX is the return value
```

The dependency here is between the `cmp` (compare) instruction and the `jz` (jump if equal to zero) instruction. The CPU cannot issue the conditional jump until it knows the results of the comparison. This is called a *branch dependency* (also known as a *control dependency*). See Figure 4.12 for an illustration of a branch dependency.

4.2.6.1 Speculative Execution

One way CPUs deal with branch dependencies is via a technique known as *speculative execution*, also known as *branch prediction*. Whenever a branch instruction is encountered, the CPU tries to *guess* at which branch is going to be taken. It continues to issue the instructions from the selected branch, in the hopes that its guess was correct. Of course, the CPU won't know for sure whether it guessed correctly until the dependent instruction pops out at the end of the pipeline. If the guess ends up being wrong, the CPU has executed

instructions that shouldn't have been executed at all. So the pipeline must be *flushed* and restarted at the first instruction of the correct branch. This is called a *branch penalty*.

The simplest guess a CPU can make is to assume that branches are never taken. The CPU just keeps executing instructions in sequential order, and only jumps the instruction pointer to a new location when its guess is proven wrong. This approach is I-cache friendly, in the sense that the CPU always prefers the branch whose instructions are most likely to be in the cache.

Another slightly more advanced approach to branch prediction is to assume that backward branches are always taken and forward branches are never taken. A backward branch is the kind found at the end of a `while` or `for` loop, so such branches tend to be more prevalent than forward branches.

Most high-quality CPUs include *branch prediction* hardware that can improve the quality these “static” guesses significantly. A branch predictor can track the results of a branch instruction over multiple iterations of a loop and discover patterns that help it make better guesses on subsequent iterations.

PS3 game programmers had to deal with the poor performance of “branchy” code all the time, because the branch predictors on the Cell processor were frankly pretty terrible. But the AMD Jaguar CPU found in the PS4 and Xbox One has highly advanced branch prediction hardware, so game programmers can breathe a little easier when writing code for the PS4.

4.2.6.2 Predication

Another way to mitigate the effects of branch dependencies is to simply avoid branching altogether. Consider again the `SafeIntegerDivide()` function, although we'll modify it slightly to work in terms of floating-point values instead of integers:

```
float SafeFloatDivide(float a, float b, float d)
{
    return (b != 0.0f) ? a / b : d;
}
```

This simple function calculates one of two answers, depending on the results of the conditional test `b != 0`. Instead of using a conditional branch statement to return one of these two answers, we can instead arrange for our conditional test to generate a *bit mask* consisting of all zeros (0x0U) if the condition is false, and all ones (0xFFFFFFFFU) if it is true. We can then execute *both* branches, generating two alternate answers. Finally, we use the mask to produce the final answer that we'll return from the function.

The following pseudocode illustrates the idea behind predication. (Note that this code won't run as-is. In particular, you can't mask a float with an unsigned int and obtain a float result—you'd need to use a union to reinterpret the bit patterns of the floats as if they were unsigned integers when applying the mask.)

```
int SafeFloatDivide_pred(float a, float b, float d)
{
    // convert Boolean (b != 0.0f) into either 1U or 0U
    const unsigned condition = (unsigned)(b != 0.0f);

    // convert 1U -> 0xFFFFFFFFU
    // convert 0U -> 0x00000000U
    const unsigned mask = 0U - condition;

    // calculate quotient (will be QNaN if b == 0.0f)
    const float q = a / b;

    // select quotient when mask is all ones, or default
    // value d when mask is all zeros (NOTE: this won't
    // work as written -- you'd need to use a union to
    // interpret the floats as unsigned for masking)
    const float result = (q & mask) | (d & ~mask);
    return result;
}
```

Let's take a closer look at how this works:

- The test `b != 0.0f` produces a `bool` result. We convert this into an unsigned integer by simply casting it. This results in either the value 1U (corresponding to `true`) or 0U (corresponding to `false`).
- We convert this unsigned result into a bit mask by subtracting it from 0U. Zero minus zero is still zero, and zero minus one is `-1`, which is `0xFFFFFFFFU` in 32-bit unsigned integer arithmetic.
- Next, we go ahead and calculate the quotient. We run this code *regardless* of the result of the non-zero test, thereby side-stepping any branch dependency issues.
- We now have our two answers ready to go: The quotient `q` and the default value `d`. We want to apply the mask in order to “select” one or the other value. But to do this, we need to *reinterpret* the floating-point bit patterns of `q` and `d` as if they were unsigned integers. The most portable way to accomplish this in C/C++ is to use a union containing

two members, one of which interprets a 32-bit value as a float, the other of which interprets it as an unsigned.

- The mask is applied as follows: We bitwise AND the quotient q with the mask, producing a bit pattern that matches q if the mask is all ones, but is all zeros if the mask is all zeros. We bitwise AND the default value d with the *complement* of the mask, yielding all zeros if the mask is all ones, or the bit pattern of d if the mask is all zeros. Finally, we bitwise OR these two values together, effectively selecting *either* the value of q or the value of d .

The use of a mask to select one of two possible values like this is called *predication* because we run both code paths (the one that returns a / b and the one that returns d) but each code path is *predicated* on the results of the test $(a \neq 0)$, via the mask. Because we are selecting one of two possible values, this is also often called a *select* operation.

Going to all this trouble to avoid a branch may seem like overkill. And it can be—its usefulness depends on the relative cost of a branch versus the predicated alternative on your target hardware. Predication really shines when a CPU’s ISAs provide special machine language instructions for performing a select operation. For example, the PowerPC ISA offers an integer select instruction `isel`, a floating-point select instruction `fsel`, and even a SIMD vector select instruction `vecsel`, and their use can definitely result in performance improvements on PowerPC based platforms (like the PS3).

It’s important to realize that predication only works when both branches can be executed *safely*. Performing a divide by zero operation in floating-point generates a quiet not-a-number (QNaN), but an integer divide by zero throws an exception that will crash your game (unless it is caught). That’s why we converted this example to floating-point before applying predication to it.

4.2.7 Superscalar CPUs

The pipelined CPU we described in Section 4.2.1 is what is called a *scalar* processor. This means that it can start executing at most one instruction per clock cycle. Yes, multiple instructions are “in flight” at any given moment, but only one *new* instruction is sent down the pipeline every clock cycle.

At its core, parallelism is about making use of multiple hardware components simultaneously. So one way to double the throughput of a CPU (at least in theory!) would be to *duplicate* most of the components on the chip, in such a way that *two* instructions could be launched each clock cycle. This is called a *superscalar* architecture.

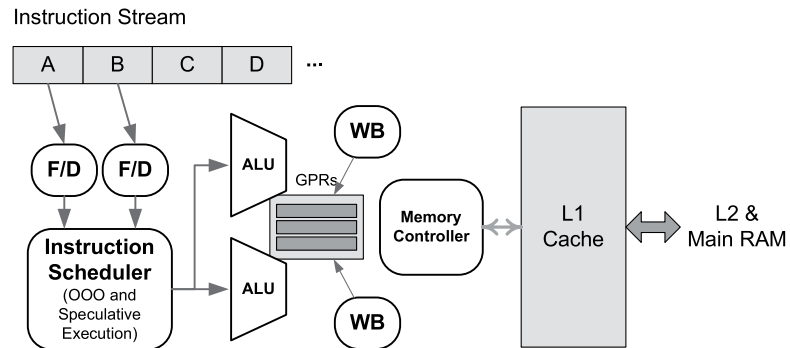


Figure 4.13. A pipelined superscalar CPU contains multiple execution components (ALUs, FPUs and/or VPUs) fed by a single instruction scheduler which typically supports out-of-order and speculative execution.

In a superscalar CPU, two (or more) instances of the circuitry that manages each stage of the pipeline¹ is present on-chip. The CPU still fetches instructions from a single instruction stream, but instead of issuing the one instruction pointed to by the IP during each clock cycle, the next *two* instructions are fetched and dispatched during each clock cycle. Figure 4.13 illustrates the hardware components found in a two-way superscalar CPU, and Figure 4.14 traces the path of ten instructions, “A” through “N,” as they move through this CPU’s two parallel pipelines.

4.2.7.1 Complexity of Superscalar Designs

Implementing a superscalar CPU isn’t quite as simple as “copying and pasting” two identical CPU cores onto a die. Although it’s reasonable to envision a superscalar CPU as two parallel instruction pipelines, these two pipelines are fed from a single instruction stream. Some kind of control logic is therefore required at the front end of these parallel pipes. Just as on a CPU that supports out-of-order execution, a superscalar CPU’s control logic looks ahead in the instruction stream in an attempt to identify *dependencies* between instructions, and then issues instructions out of order in an attempt to mitigate their effects.

In addition to data and branch dependencies, a superscalar CPU is prone to a third kind of dependency known as a *resource dependency*. This kind of dependency arises when two or more consecutive instructions all require the

¹Technically pipelining and superscalar designs are two independent forms of parallelism. A pipelined CPU needn’t be superscalar. Likewise, a superscalar CPU needn’t be pipelined, although the majority of them are.

Clock Cycle	F ₀	F ₁	D ₀	D ₁	E ₀	E ₁	M ₀	M ₁	W ₀	W ₁
0	A	B								
1	C	D	A	B						
2	E	F	C	D	A	B				
3	G	H	E	F	C	D	A	B		
4	I	J	G	H	E	F	C	D	A	B
5	K	L	I	J	G	H	E	F	C	D
6	M	N	K	L	I	J	G	H	E	F

Figure 4.14. Best-case execution of 14 instructions “A” through “N” on a superscalar pipelined CPU over seven clock cycles.

same functional unit within the CPU. For example, let’s imagine we have a superscalar CPU with two integer ALUs but only one FPU. Such a processor is capable of issuing two integer arithmetic instructions on each clock. But if two floating-point arithmetic instructions are encountered in the instruction stream, they cannot both be issued on the same clock cycle because the resource required by the second (the FPU) will already be in use by the first. As such, the control logic that manages instruction dispatch on a superscalar CPU is even more complex than that found on a scalar CPU that supports out-of-order execution.

4.2.7.2 Superscalar and RISC

A two-way superscalar CPU requires roughly two times the silicon real-estate of a comparable scalar CPU design. In order to free up transistors, most superscalar CPUs are therefore *reduced instruction set* (RISC) processors. The ISA of a RISC processor provides a comparatively small set of instructions, each with a very focused purpose. More complex operations are performed by building up sequences of these simpler instructions. In contrast, the ISA of a *complex instruction set computer* (CISC) offers a much wider variety of instructions, each of which may be capable of performing more complex operations.

4.2.8 Very Long Instruction Word (VLIW)

We saw in Section 4.2.7.1 that a superscalar CPU contains highly complex instruction dispatch logic. This logic takes up valuable real-estate on the CPU

die. Also, CPUs are only capable of looking ahead in the instruction stream by a relatively small number of instructions when analyzing dependencies and looking for opportunities for out-of-order and/or superscalar instruction dispatch. This limits the effectiveness of the dynamic optimizations a CPU can perform.

A somewhat simpler way to implement instruction level parallelism is to design a CPU that has multiple compute elements (ALUs, FPUs, VPU) on-chip, but leaves the task of *dispatching* instructions to those compute elements entirely to the programmer and/or compiler. That way, all the complex instruction-dispatch logic can be eliminated, and those transistors devoted instead to implementing more compute elements or a larger cache. As a side benefit, programmers and compilers ought to be better at optimizing the dispatching of the instructions in their programs than the CPU could ever be, because they can select instructions for dispatch from a much wider window (typically an entire function's worth of instructions).

To allow programmers and/or compilers to dispatch instructions to multiple compute elements on each clock cycle, the instruction word is extended so that it contains two or more "slots," each corresponding to a compute element on the chip. For example, if our hypothetical CPU contained two integer ALUs and two FPUs, a programmer or compiler would need to be able to encode up to two integer and two floating-point operations within each instruction word. We call this a *very long instruction word* (VLIW) design. The VLIW architecture is illustrated in Figure 4.15.

We can look to the Playstation 2 for a concrete example of VLIW architecture: The PS2 contained two coprocessors called *vector units* (VU0 and VU1), each of which was capable of dispatching two instructions per clock cycle. Each instruction word was comprised of two slots called *low* and *high*. It was often a challenge to fill both slots effectively when hand-coding in assembly language, although tools were developed that helped programmers to convert a one-instruction-per-clock program into an efficient two-instructions-per-clock format.

There are trade-offs between the superscalar and VLIW approaches. Because it lacks the complex scheduling, out-of-order execution and branch prediction logic of a superscalar CPU, a VLIW processor is much simpler, and can therefore potentially make heavier use of parallelism than its superscalar counterparts. However, it can be very tough to transform a serial program into a form that takes full advantage of the parallelism in a VLIW. This makes the job of the programmer and/or compiler more difficult. That said, a number of advances have been made to overcome some of these limitations, including variable-width VLIW designs. For example, see <http://researcher.watson>.

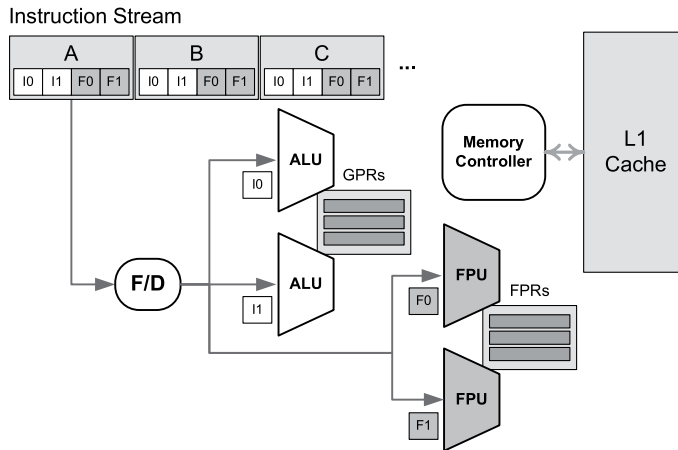


Figure 4.15. A pipelined VLIW CPU architecture consisting of two integer ALUs and two floating-point FPUs. Each very long instruction word consists of two integer and two floating-point operations, which are dispatched to the corresponding functional units. Notice the absence of the complex instruction scheduling logic that would be present in a superscalar CPU.

ibm.com/researcher/view_group_subpage.php?id=2834.

4.3 Explicit Parallelism

Explicit parallelism is designed to make concurrent software run more efficiently. Hence all explicitly parallel hardware designs permit more than one instruction stream to be processed in parallel. We'll list a few common explicitly parallel designs below, increasing in granularity from *hyperthreading* at the most fine-grained end of the spectrum to *cloud computing* at the most coarse-grained end.

4.3.1 Hyperthreading

As we saw in Section 4.2.5.2, some pipelined CPUs are capable of issuing instructions *out of order* as a means of reducing pipeline stalls. Normally a pipelined CPU executes instructions in program order; but sometimes the next instruction in the instruction stream cannot be issued due to a dependency on an in-flight instruction. This creates a *delay slot* into which another instruction could theoretically be issued. An OOO CPU can “look ahead” in the instruction stream and select an instruction to issue out-of-order into such a delay slot.

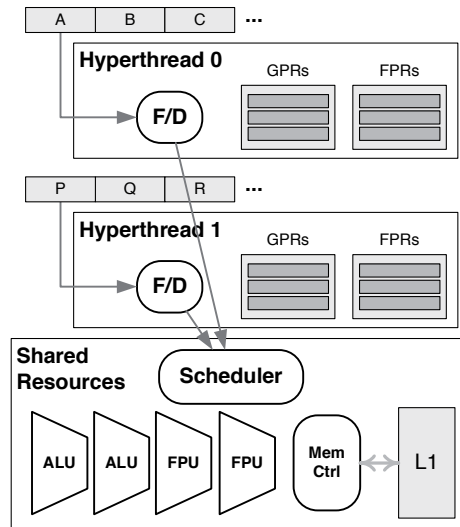


Figure 4.16. A hyperthreaded CPU containing two front ends (each consisting of a fetch/decode unit and a register file), but with a single back end containing ALUs, FPUs, a memory controller, an L1 cache, and an out-of-order instruction scheduler. The scheduler issues instructions from both front-end threads to the shared back-end components.

With only a single instruction stream, the CPU's options are somewhat limited when selecting an instruction to issue into a delay slot. But what if the CPU could select its instructions from two separate instruction streams at once? This is the principle behind a *hyperthreaded* (HT) CPU core.

Technically speaking, an HT core consists of two register files and two instruction decode units, but with a single “back end” for executing instructions, and a single shared L1 cache. This design enables an HT core to run two independent threads, while requiring fewer transistors than a dual core CPU, thanks to the shared back end and L1 cache. Of course, this sharing of hardware components also results in lower instruction throughput relative to a comparable dual core CPU, because the threads contend for these shared resources. Figure 4.16 illustrates the key components in a typical hyperthreaded CPU design.

4.3.2 Multicore CPUs

A CPU *core* can be defined as a self-contained unit capable of executing instructions from at least one instruction stream. Every CPU design we've looked at until now could therefore qualify as a “core.” When more than one core is included on a single CPU die, we call it a *multicore* CPU.

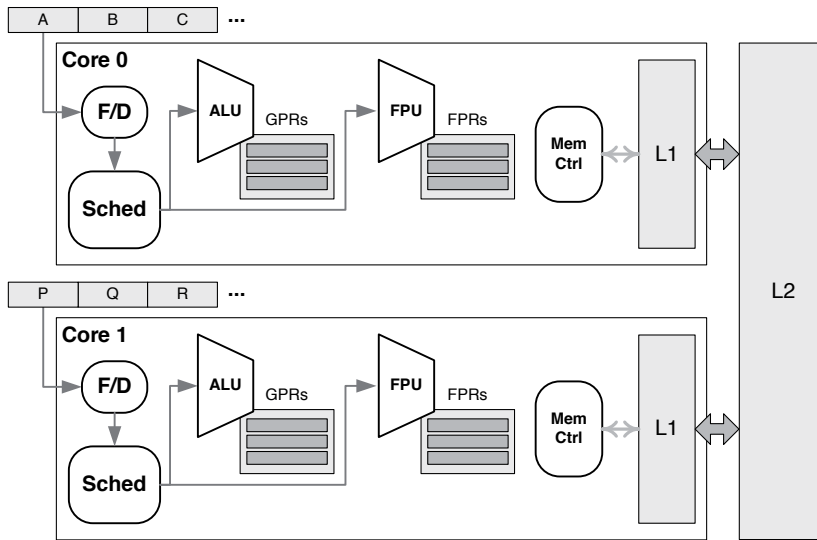


Figure 4.17. A simple multicore CPU design.

The specific design within each core can be any of the designs we’ve looked at thus far—each core might employ a simple serial design, a pipelined design, a superscalar architecture, a VLIW design, or might be a hyperthreaded core. Figure 4.17 illustrates a simple example of a multicore CPU design.

The PlayStation 4 and Xbox One game consoles both contain multicore CPUs. Each contains an *accelerated processing unit* (APU) consisting of two quad-core AMD Jaguar modules, integrated onto a single die with a GPU, memory controller and video codec. (Of these eight cores, seven are available for use by game applications. However, roughly half of the bandwidth on the seventh core is reserved for operating system use.) The Xbox One X also contains an eight-core APU, but its cores are based on proprietary technology developed in partnership with AMD, rather than on the Jaguar microarchitecture like its predecessor. Figure 4.18 shows a block diagram of the PS4 hardware architecture, and Figure 4.19 presents a block diagram of the Xbox One hardware architecture.

4.3.3 Symmetric versus Asymmetric Multiprocessing

The *symmetry* of a parallel computing platform has to do with how the CPU cores in the machine are treated by the operating system. In *symmetric multiprocessing* (SMP), the available CPU cores in the machine (provided by any combination of hyperthreading, multicore CPUs or multiple CPUs on a single

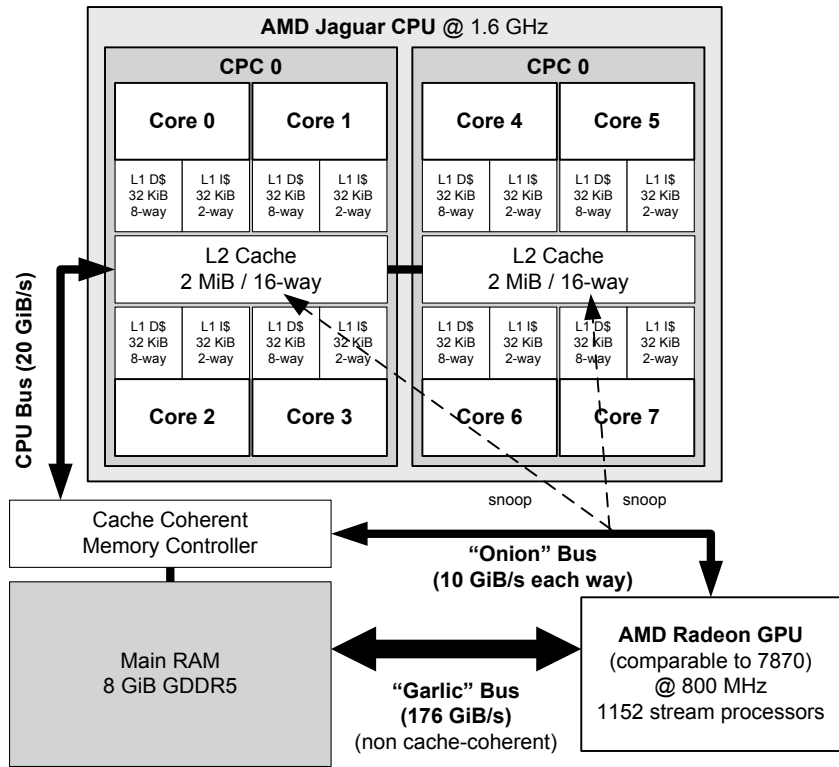


Figure 4.18. Simplified view of the PS4's architecture.

motherboard) are homogeneous in terms of design and ISA, and are treated equally by the operating system. Any thread can be scheduled to execute on any core. (Note, however, that it is possible in such systems to specify an *affinity* for a thread, causing it to be more likely, or even guaranteed, to be scheduled on a particular core.)

The PlayStation 4 and the Xbox One are examples of SMP. Both of these consoles contain eight cores, of which seven are available for use by the programmer, and the application is free to run threads on any of the available cores.

In *asymmetric multiprocessing* (AMP), the CPU cores are not necessarily homogeneous, and the operating system does not treat them equally. In AMP, one "master" CPU core typically runs the operating system, and the other cores are treated as "slaves" to which workloads are distributed by the master core.

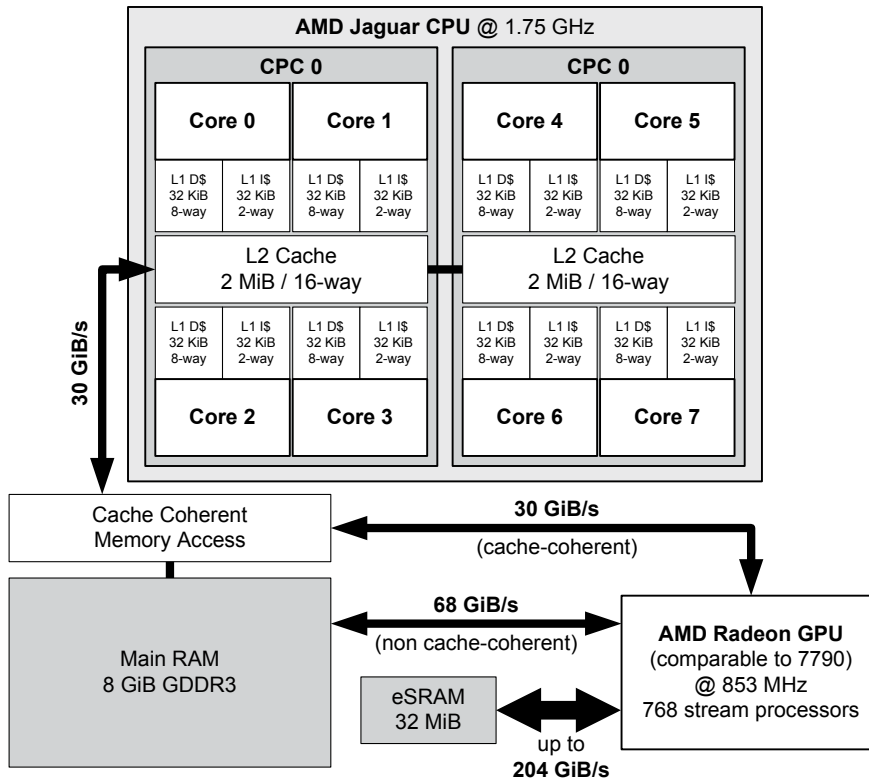


Figure 4.19. Simplified view of the Xbox One's architecture.

The *cell broadband engine* (CBE) used in the PlayStation 3 is an example of AMP; it employs a main CPU known as the “power processing unit” (PPU) which is based on the PowerPC ISA, along with eight coprocessors known as “synergistic processing units” (SPUs) which are based around a completely different ISA. (See Section 3.5.5 for more on the PS3’s hardware architecture.)

4.3.4 Distributed Computing

Yet another way to achieve parallelism in computing is to make use of multiple stand-alone computers working in concert. This is known as *distributed computing* in the most general sense. There are various ways to architect a distributed computing system, including:

- computer clusters,
- grid computing, and

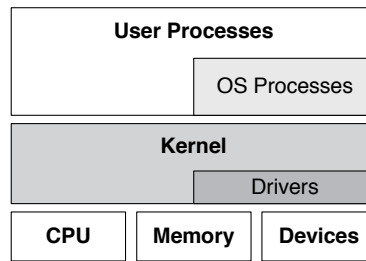


Figure 4.20. The kernel and device drivers sit directly on top of the hardware, and run in privileged mode. All other operating system software and all user programs are implemented on top of the kernel and driver layer, and run in a somewhat restricted user mode.

- cloud computing.

We'll focus exclusively on parallelism within a single computer in this book, but you can read more about distributed computing by searching for the above terms online.

4.4 Operating System Fundamentals

Now that we have a solid understanding of the basics of parallel computer hardware, let's turn our attention to the services provided by the operating system that make concurrent programming possible.

4.4.1 The Kernel

Modern operating systems handle a wide variety of tasks, across a wide range of granularities. These range from the handling of keyboard and mouse events or the scheduling of programs for *preemptive multitasking* at one end of the spectrum, to managing a printer queue or the network stack at the other end. The “core” of the operating system—the part that handles all of the most fundamental and lowest-level operations—is called the *kernel*. The rest of the operating system, and all user programs, are built atop the services provided by the kernel. This architecture is illustrated in Figure 4.20.

4.4.1.1 Kernel Mode versus User Mode

The kernel and its device drives run in a special mode called *protected mode*, *privileged mode* or *kernel mode*, while all other programs in the system (including all other parts of the operating system that aren't part of the kernel) operate in *user mode*. As the name suggests, software running in privileged mode has

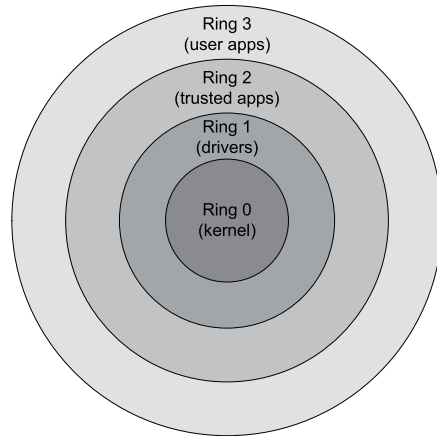


Figure 4.21. Example of CPU protection rings, showing four rings. The kernel runs in ring 0, device drivers run in ring 1, trusted programs with I/O permissions run in ring 2, and all other user programs run in ring 3.

full access to all of the hardware in the computer, whereas user mode software is *restricted* in various ways in order to ensure stability of the computer system as a whole. Software running in user mode can access low-level services only by making a special *kernel call*—a request for the kernel to perform a low-level operation on the user program’s behalf. This ensures that a program can’t inadvertently or maliciously destabilize the system.

In practice, operating systems may implement multiple *protection rings*. The kernel runs in ring 0, which is the most trusted ring and has all possible privileges within the system. Device drivers might run in ring 1, trusted programs with I/O permissions might run in ring 2, while all other “untrusted” user programs run in ring 3. But this is just one example—the number of rings varies from CPU to CPU and from OS to OS, as does the assignment of subsystems to the various rings. The protection ring concept is illustrated in Figure 4.21.

4.4.1.2 Kernel Mode Privileges

Kernel mode (ring 0) software has access to *all* of the machine language instructions defined by the CPU’s ISA. This includes a powerful subset of instructions called *privileged* instructions. These privileged instructions might allow certain registers to be modified that are normally off-limits (e.g., to control virtual memory mapping, or to mask and unmask interrupts). Or they might allow certain regions of memory to be accessed, or allow other nor-

mally restricted operations to be performed. Examples of privileged instructions on the Intel x86 processor include `wrmsr` (write to model-specific register) and `cli` (clear interrupts). By restricting the use of these powerful instructions only to “trusted” software like the kernel, system stability and security is improved.

With these privileged ML instructions, the kernel can implement security measures. For example, the kernel typically locks down certain pages of virtual memory so that they cannot be written to by a user program. Both the kernel’s software and all of its internal record-keeping data are kept in protected memory pages. This ensures that a user program won’t stomp on the kernel, and thereby crash the entire system.

4.4.2 Interrupts

An *interrupt* is a signal sent to the CPU in order to notify it of an important low-level event, such as a keypress on the keyboard, a signal from a peripheral device, or the expiration of a timer. When such an event occurs, an *interrupt request* (IRQ) is raised. If the operating system wishes to respond to the event, it pauses (interrupts) whatever processing had been going on, and calls a special kind of function called an *interrupt service routine* (ISR). The ISR function performs some operation in response to the event (ideally doing so as quickly as possible) and then control is returned back to whatever program had been running prior to the interrupt having been raised.

There are two kinds of interrupt: *hardware interrupts* and *software interrupts*. A hardware interrupt is requested by placing a non-zero voltage onto one of the pins of the CPU. Hardware interrupts might be raised by devices such as a keyboard or mouse, or by a periodic timer circuit on the motherboard or within the CPU itself. Because it’s triggered by an external device, a hardware interrupt can happen at any time—even right in the middle of executing a CPU instruction. As such, there may be a tiny delay between the moment when a hardware interrupt is physically raised and when the CPU is in a suitable state to handle it.

A software interrupt is triggered by software rather than by a voltage on a CPU pin. It has the same basic effect as a hardware interrupt, in that it causes the operation of the CPU to be interrupted and a service routine to be called. A software interrupt can be triggered explicitly by executing an “interrupt” machine language instruction. Or one may be triggered in response to an erroneous condition detected by the CPU while running a piece of software—these are called *traps* or sometimes *exceptions* (although the latter term should not be confused with language-level exception handling). For example, if an ALU is

instructed to perform a divide-by-zero operation, a software interrupt will be raised. The operating system normally handles such interrupts by crashing the program in question and producing a *core dump* file. However, a debugger attached to the program could catch this interrupt and instead cause the program to break into the debugger for inspection.

4.4.3 Kernel Calls

In order for user software to perform a privileged operation, such as mapping or unmapping physical memory pages in the virtual memory system or accessing a raw network socket, the user program must make a request to the kernel. The kernel responds by performing the operation in a safe manner on behalf of the user program. Such a request is called a *kernel call* or *system call*.

On most systems, calling into the kernel is accomplished via a software interrupt.² In the case of an interrupt-triggered system call, the user program places any input arguments in a specific place (either in memory or in registers) and then issues a “software interrupt” instruction with an integer argument that specifies which kernel operation is being requested. This causes the CPU to be put into a mode with elevated privileges, staves the state of the calling program, and then causes the appropriate kernel interrupt service routine to be called. Presuming that the kernel allows the request to proceed, it performs the requested operation (in privileged mode) and then control is returned to the caller (after first restoring its execution state). This switch from a user mode program into the kernel is an example of *context switching*. See Section 4.4.6.5 for more on context switching.

On most modern operating systems, the user program doesn’t execute a software interrupt or system call instruction manually, for example via inline assembly code. That would be messy and error-prone. Instead, a user program calls a *kernel API* function, which in turn marshalls the arguments and trips the software interrupt. This is why system calls appear to be regular function calls from the point of view of the user program.

4.4.4 Preemptive Multitasking

The earliest minicomputers and personal computers ran a single program at a time. They were inherently *serial* computers, capable of reading a program from a single instruction stream, and executing one instruction from this stream at a time. The *disk operating systems* (DOS) in those days weren’t much

²On some systems, a special variant of the `call` instruction is used to call into the kernel. For example, on MIPS processors this instruction is named `syscall`.

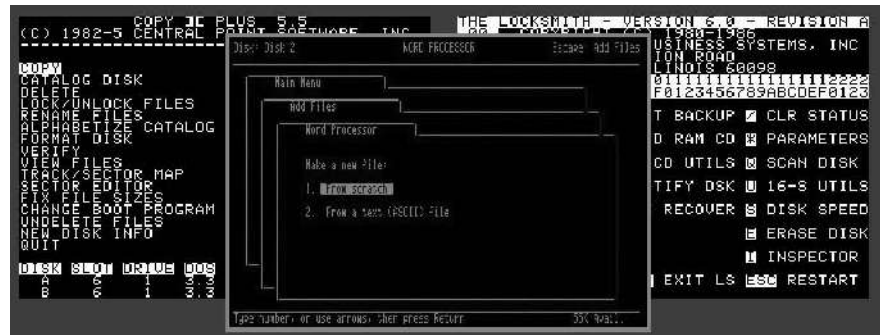


Figure 4.22. Three full-screen programs running on the Apple II computer. Programs on the Apple II were always full-screen because it could only run one program at a time. From left to right: Copy II Plus, the AppleWorks word processor, and The Locksmith.

more than glorified device drivers, allowing programs to interface with devices such as tape, floppy and hard disk drives. The entire computer would be devoted to running a single program at a time. Figure 4.22 shows a few full-screen programs running on an Apple II computer.

As operating systems and computer hardware became more advanced, it became possible to run more than one program on a serial computer at a time. On shared mainframe computer systems, a technique known as *multiprogramming* would allow one program to run while another was waiting for a time-consuming request to be satisfied by a peripheral device. Classic Mac OS and versions of Windows prior to Windows NT and Windows 95 used a technique known as *cooperative multitasking*, in which only one program would be running on the machine at a time, but each program would regularly *yield* the CPU so that another program could get a chance to run. In this way, each program ended up with a periodic “slice” of CPU time. Technically, this technique is known as *time division multiplexing* (TDM) or *temporal multithreading* (TMT). Informally it’s called *time-slicing*.

Cooperative multitasking suffered from one big problem: Time-slicing required the cooperation of each and every program in the system. One “rogue” program could consume all of the CPU’s time if it failed to yield to other programs periodically. The PDP-6 Monitor and Multics operating systems solved this problem by introducing a technique known as *preemptive multitasking*. This technology was later adopted by the UNIX operating system and all of its variants, along with later versions of Mac OS and Windows.

In preemptive multitasking, programs still share the CPU by time-slicing. However the scheduling of programs is controlled by the operating system, not via cooperation between the programs themselves. As a result, each pro-

gram gets a regular, consistent and reliable time slice on the CPU. The time slice during which one particular program is allowed to run on the CPU is sometimes called the program's *quantum*. To implement preemptive multitasking, the operating system responds to a regularly-timed hardware interrupt in order to periodically *context switch* between the different programs running on the system. We'll see how context switching works in more depth in the next section (4.4.6.5).

We should note here that preemptive multitasking is used even on multicore machines, because typically the number of threads is greater than the number of cores. For example, if we were to have 100 threads and only four CPU cores, then the kernel would use preemptive multitasking to time-slice between 25 threads on each core.

4.4.5 Processes

A *process* is the operating system's way of managing a *running instance* of a program contained in an *executable file* (.exe on Windows, .elf on Linux). A process only exists while its program is actually running—when an instance of a program exits, is killed, or crashes, the OS destroys the process associated with that instance. Multiple processes can be running on a computer system at any given time. This might include multiple instances of the *same* program.

Programmers interact with processes via an API provided by the operating system. The details of this API differ from OS to OS, but the key concepts are roughly the same across all of them. A complete discussion of any one operating system's process API is beyond the scope of this book, but for the purposes of illustrating the concepts we'll focus primarily on the API style of UNIX-like operating systems such as Linux, BSD and MacOS. But we'll make note of situations in which Windows or game console operating systems deviate significantly from the core ideas of a UNIX-like process API.

4.4.5.1 Anatomy of a Process

Under the hood, a process consists of:

- a *process id (PID)* that uniquely identifies the process within the operating system;
- a set of *permissions*, such as which user “owns” each process and to which user group it belongs;
- a reference to the process's *parent process*, if any,
- a *virtual memory space* containing the process's “view” of physical memory (see Section 4.4.5.2 for more information);

- the values of all defined *environment variables*;
- the set of all open *file handles* in use by the process;
- the current *working directory* for the process,
- resources for managing *synchronization* and *communication* between processes in the system, such as message queues, pipes, and semaphores;
- one or more *threads*.

A *thread* encapsulates a running instance of a single stream of machine language instructions. By default, a process contains a single thread. But as we'll discuss in depth in Section 4.4.6, more than one thread can be created within a process, allowing more than one instruction stream to run *concurrently*. The kernel schedules all of the threads in the system (from all currently-running processes) to run on the available cores. It uses preemptive multitasking to time-slice between threads when there are more threads than there are cores.

We should stress here that threads are the *fundamental unit of program execution* within an operating system, not processes. A process merely provides an *environment* within which its thread(s) can run, including a virtual memory map and a set of *resources* that are used by and shared between all threads within that process. Whenever a thread is scheduled to run on a core, its process becomes active, and that process's resources and environment become available for use by the thread while it runs. So when we say that a *thread* is running on a core, remember that it is always doing so within the *context* of exactly one process.

4.4.5.2 Virtual Memory Map of a Process

You'll recall from Section 3.5.2 that a program generally never works directly with physical memory addresses.³ Rather, the program accesses memory in terms of *virtual addresses*, and the CPU and operating system cooperate to *remap* these virtual addresses into physical addresses. We said that the remapping of virtual to physical addresses happens in terms of contiguous blocks of addresses called *pages*, and that a *page table* is used by the OS to map virtual page indices to physical page indices.

Every process has its own virtual page table. This means that every process has its own custom *view* of memory. This is one of the primary ways in which the operating system provides a secure and stable execution environment. Two processes cannot corrupt each other's memory, because the physical pages owned by one process are simply not mapped into the other process's address space (unless they explicitly share pages). Also, pages owned by the

³User programs always work in terms of virtual memory addresses, but the kernel can work directly with physical addresses.

kernel are protected from inadvertent or deliberate corruption by a user process because they are mapped to a special range of addresses known as *kernel space* which can only be accessed by code running in kernel mode.

A process's virtual page table effectively defines its *memory map*. The memory map typically contains:

- the text, data and BSS sections as read in from the program's executable file;
- a view of any shared libraries (DLLs, PRXs) used by the program;
- a *call stack* for each thread;
- a region of memory called the *heap* for dynamic memory allocation;
- possibly some pages of memory that are *shared* with other processes; and
- a range of *kernel space* addresses which are inaccessible to the process (but become accessible whenever a kernel call executes).

Text, Data and BSS Sections

When a program is first run, the kernel creates a process internally and assigns it a unique PID. It then sets up a virtual page map for the process—in other words, it creates the virtual address space of the process. It then allocates physical pages as necessary and maps them into the virtual address space by adding entries to the process's page table.

The kernel reads the executable file (text, data and BSS sections) into memory by allocating virtual pages and loading the data into them. This allows the program's code and global data to be “visible” within the process's virtual address space. The machine code in an executable file is actually *relocatable*, meaning that its addresses are specified as relative offsets rather than absolute memory addresses. These relative addresses are *fixed up* by the operating system, meaning they are converted back into real (virtual) addresses, prior to running the program. (For more on the format of an executable file, see Section 3.3.5.1.)

Call Stack

Every running thread needs a call stack (see Section 3.3.5.2). When a process is first run, the kernel creates a single default thread for it. The kernel allocates physical memory pages for this thread's call stack, and maps them into the process' virtual address space so the stack can be “seen” by the thread. The values of the stack pointer (SP) and base pointer (BP) are initialized to point to the bottom of the empty stack. Finally, the thread starts executing at the

entry point of the program. (In C/C++ this is typically `main()`, or `WinMain()` under Windows.)

Heap

Processes can allocate memory *dynamically* via `malloc()` and `free()` in C, or global `new` and `delete` in C++. These requests come from a region of memory called the *heap*. Physical pages of memory are allocated on demand by the kernel in order to fulfill dynamic allocation requests; these pages are mapped into the virtual address space of the process as memory is allocated by it, and pages whose contents have been completely freed are unmapped and returned to the system.

Shared Libraries

All non-trivial programs depend on external libraries. A library can be *statically* linked into a program, meaning that a *copy* of the library's code is placed into the executable file itself. Most operating systems also support the concept of *shared libraries*. In this case, the program contains only *references* to the library's API functions, not a copy of the library's machine code. Shared libraries are called *dynamic link libraries* (DLL) under Windows. On the PlayStation 4, the OS supports a kind of dynamically linked library called a PRX. (Interestingly, the name PRX comes from the PlayStation 3, where it stood for **P**PU **R**elocatable **E**xecutable, in reference to the main processor in the PS3 which was called the PPU.)

Shared libraries generally work as follows: The first time a shared library is needed by a process, the OS loads that library into physical memory, and maps a view of it into the process's virtual address space. The addresses of the functions and global variables provided by the shared library are patched into the program's machine code, allowing it to call them as if they had been statically linked into the executable.

The benefit of shared libraries only becomes evident when a second process is run that uses the same shared library. Rather than loading a *copy* of the library's code and global variables, the already-loaded physical pages are simply mapped into the virtual address space of the new process. This saves memory and speeds up the process of running all but the first process that uses a given shared library.

Shared libraries have other benefits, too. For example, a shared library can be updated, say to fix some bugs, and in theory all programs that use that shared library will immediately benefit (without having to be relinked and redistributed to users). That said, in practice updating shared libraries can

inadvertently cause compatibility problems amongst the programs that use them. This leads to a proliferation of different versions of each shared library within the system—a situation affectionately known as “DLL hell” amongst Windows developers. To work around these problems, Windows moved to a system of manifests that help to guarantee compatibility between shared libraries and the programs that use them.

Kernel Pages

On most operating systems, the address space of a process is actually divided into two large contiguous blocks—user space and kernel space. For example, on 32-bit Windows, user space corresponds to the address range from address 0x0 through 0x7FFFFFFF (the lower 2 GiB of the address space), while kernel space corresponds to addresses between 0x80000000 and 0xFFFFFFFF (the upper 2 GiB of the space). On 64-bit Windows, user space corresponds to the 8 TiB range of addresses from 0x0 through 0x7FFF'FFFFFFFF, and the gigantic 248 TiB range from 0xFFFF0800'00000000 through 0xFFFFFFFF'FFFFFFFF is reserved for use by the kernel (although not all of it is actually used).

User space is mapped through a virtual page table that is unique to each process. However, kernel space uses a separate virtual page table that is shared between all processes. This is done so that all processes in the system have a consistent “view” of the kernel’s internal data.

Normally, user processes are prevented from accessing the kernel’s pages—if they try to do so, a page fault will occur and the program will crash. However, when a user process makes a system call, a *context switch* (see Section 4.4.6.5) is performed into the kernel. This puts the CPU into privileged mode, allowing the kernel to access the kernel space address ranges (as well as the virtual pages of the current process). The kernel runs its code in privileged mode, updates its internal data structures as necessary, and finally puts the CPU back into user mode and returns control to the user program. For more details on how user and kernel space memory mapping works under Windows, search for “virtual address spaces” on <https://docs.microsoft.com>.

It’s interesting (and a bit frightening!) to note that the recently-discovered “Meltdown” and “Spectre” exploits make use of a CPU’s *out-of-order* and *speculative* execution logic (respectively) to trick it into accessing data located in memory pages that are normally protected from a user-mode process. For more on these exploits and how operating systems are protecting themselves against them, see <https://meltdownattack.com/>.

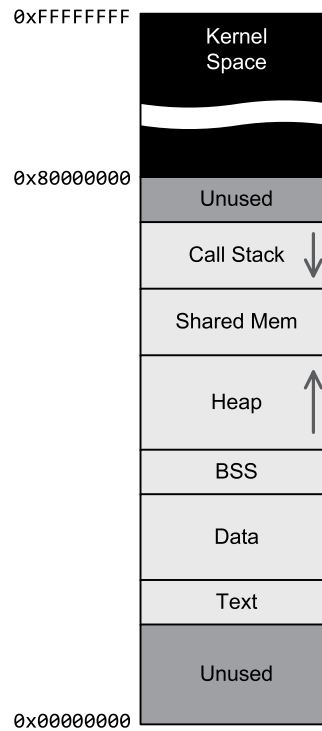


Figure 4.23. A process's memory map as it might look under 32-bit Windows.

Example Process Memory Map

Figure 4.23 depicts the memory map of a process as it might look under 32-bit Windows. All of the process's virtual pages are mapped into user space—the lower 2 GiB of the address space. The executable files' text, data and BSS segments are mapped at a low memory address, followed by the heap in a higher range, followed by any shared memory pages. The call stack is mapped at the high end of the user address space. Finally, the operating system's kernel pages are mapped into the upper 2 GiB of the address space.

The actual addresses for each of the regions in the memory map aren't predictable. This is partly because each program's segments are of different sizes and hence will map to different address ranges. Also, the numeric values of the addresses actually *change* between runs of the same executable program, thanks to a security measure known as *address space layout randomization* (ASLR).

4.4.6 Threads

A *thread* encapsulates a running instance of a single stream of machine language instructions. Each thread within a process is comprised of:

- a *thread id (TID)* which is unique within its process, but may or may not be unique across the entire operating system;
- the thread's *call stack*—a contiguous block of memory containing the stack frames of all currently-executing functions;
- the values of all special- and general-purpose *registers*⁴ including the *instruction pointer (IP)*, which points at the current instruction in the thread's instruction stream, the *base pointer (BP)* and *stack pointer (SP)* which define the current function's stack frame;
- a block of general-purpose memory associated with each thread, known as *thread local storage (TLS)*.

By default, a process contains a single *main thread* and hence executes a single instruction stream. This thread begins executing at the entry point of the program—typically the `main()` function. However, all modern operating systems are capable of executing more than one *concurrent* instruction stream within the context of a single process.

You can think of a thread as the *fundamental unit of execution* within the operating system. A thread provides the minimum resources required to execute an instruction stream—a call stack and a set of registers. The process merely provides the *environment* in which one or more threads execute. This is illustrated in Figure 4.24.

4.4.6.1 Thread Libraries

All operating systems that support multithreading provide a collection of system calls for creating and manipulating threads. A few portable thread libraries are also available, the best known of which are the IEEE POSIX 1003.1c standard thread library (pthread) and the C11 and C++11 standard thread libraries. The Sony PlayStation 4 SDK provides a set of thread functions prefixed with `sce` that map pretty much directly to the POSIX thread API.

The various thread APIs differ in their details, but all of them support the following basic operations:

1. *Create*. A function or class constructor that spawns a new thread.

⁴Technically a thread's execution context only encompasses the values of registers that are visible in *user mode*; it excludes the values of certain privileged-mode registers.

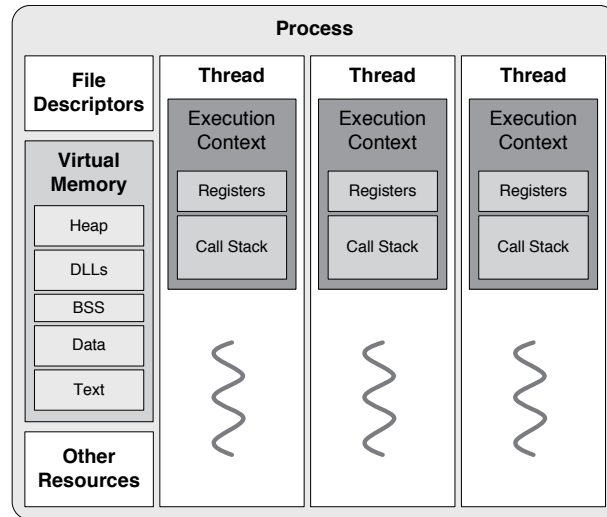


Figure 4.24. A process encapsulates the resources required to run one or more threads. Each thread encapsulates an *execution context* comprised of the contents of the CPU's registers and a call stack.

2. *Terminate*. A function that terminates the calling thread.
3. *Request to exit*. A function that allows one thread to request another thread to exit.
4. *Sleep*. A function that puts the current thread to sleep for a specified length of time.
5. *Yield*. A function that yields the remainder of the thread's time slice so other threads can get a chance to run.
6. *Join*. A function that puts the calling thread to sleep until another thread or group of threads has terminated.

4.4.6.2 Thread Creation and Termination

When an executable file is run, the process created by the OS to encapsulate it automatically contains a single thread, and execution of this thread begins at the *entry point* of the program—in C/C++ this is the special function `main()`. This “main thread” can spawn new threads if desired, by making calls to an operating system specific function such as `pthread_create()` (POSIX threads), `CreateThread()` (Windows), or by instantiating an instance of a

thread class such as `std::thread` (C++11). The new thread begins execution at an entry point function whose address is provided by the caller.

Once created, a thread will continue to exist until it terminates. The execution of a thread can terminate in a number of ways:

- It can end “naturally” by *returning* from its entry point function. (In the special case of the main thread, returning from `main()` not only ends the thread, but also ends the entire process.)
- It can call a function such as `pthread_exit()` to *explicitly* terminate its execution *before* having returned from its entry point function.
- It can be *killed* externally by another thread. In this case, the external thread makes a *request* to cancel the thread in question, but the thread may not respond immediately to the request, or it may ignore the request entirely. The cancelability of a thread is determined when that thread is created.
- It can be forcibly killed because its process has ended. (A process terminates when the main thread returns from the `main()` entry point function, when any thread calls `exit()` to kill the process explicitly, or when an external actor kills the process.)

4.4.6.3 Joining Threads

It’s common for one thread to spawn one or more child threads, do some useful work of its own, and then wait for the child threads to be done with their work before continuing. For example, let’s say the main thread wants to perform 1000 computations, and let’s assume further that this program is running on a quad-core machine. The most efficient approach would be to divide the work into four equally-sized chunks, and spawn four threads to do the processing in parallel. Once the computations are complete, let’s assume that the main thread wants to perform a checksum on the results. The resulting code might look something like this:

```
ComputationResult g_aResult[1000];

void Compute(void* arg)
{
    uintptr_t startIndex = (uintptr_t)arg;
    uintptr_t endIndex = startIndex + 250;
    for (uintptr_t i = startIndex; i < endIndex; ++i)
    {
        g_aResult[i] = ComputeOneResult(...);
    }
}
```

```

    }
}

void main()
{
    pthread_t  tid[4];

    for (int i = 0; i < 4; ++i)
    {
        const uintptr_t startIndex = i * 250;
        pthread_create(&tid[i], nullptr,
                      Compute, (void*)startIndex);
    }

    // perhaps do some other useful work...

    // wait for computations to be done
    for (int i = 0; i < 4; ++i)
    {
        pthread_join(&tid[i], nullptr);
    }

    // all threads are done, so we can do our checksum
    unsigned checksum = Sha1(g_aResult,
                             1000*sizeof(ComputationResult));

    // ...
}

```

4.4.6.4 Polling, Blocking and Yielding

Normally a thread runs until it terminates. But sometimes a running thread needs to wait for some future event to occur. For example, a thread might need to wait for a time-consuming operation to complete, or for some resource to become available. In such a situation, we have three options:

1. The thread can *poll*,
2. it can *block*, or
3. it can *yield* while polling.

Polling

Polling involves a thread sitting in a tight loop, waiting for a condition to be-

come true. It's a bit like kids in the back seat on a road trip, repeatedly asking, "Are we there yet? Are we there yet?" Here's an example:

```
// wait for condition to become true
while (!CheckCondition())
{
    // twiddle thumbs
}

// the condition is now true and we can continue...
```

It should be obvious that this approach, while simple, has the potential of burning CPU cycles unnecessarily. This approach is sometimes called a *spin-wait* or a *busy-wait*.

Blocking

If we expect our thread to wait for a relatively long period of time for a condition to become true, busy-waiting is not a good option. Ideally we'd like to put our thread to sleep so that it doesn't waste CPU resources, and rely on the kernel to wake it back up when the condition becomes true at some future time. This is called *blocking* the thread.

A thread blocks by making a special kind of operating system call known as a *blocking function*. If the condition is already true at the moment a blocking function is called, the function won't actually block—it will simply return immediately. But if the condition is false, the kernel will put the thread to sleep, and add the thread and the condition on which it is waiting into a table. Later, when the condition becomes true, the kernel uses this internal table to identify and wake up any threads that are waiting on that condition.

There are all sorts of OS functions that block. Here are a few examples:

- *Opening a file.* Most functions that open a file such as `fopen()` will block the calling thread until the file has actually been opened (which may take hundreds or even thousands of cycles). Some functions, like `open()` under Linux, offer a non-blocking option (`O_NONBLOCK`) to support asynchronous file I/O.
- *Explicit sleeping.* Some functions explicitly put the calling thread to sleep for a specified length of time. Variants include `usleep()` (Linux), `Sleep()` (Windows) `std::this_thread::sleep_until()` (C++11 standard library) and `pthread_sleep()` (POSIX threads).
- *Joining with another thread.* A function such as `pthread_join()` blocks the calling thread until the thread being waited on has terminated.

- *Waiting for a mutex lock.* Functions like `pthread_mutex_wait()` attempt to obtain an exclusive lock on a resource via an operating system object known as a *mutex* (see Section 4.6). If no other thread holds the lock, the function grants the lock to the calling thread and returns immediately; otherwise, the calling thread is put to sleep until the lock can be obtained.

Operating system calls aren't the only functions that can block. Any user-space function that ultimately calls a blocking OS function is itself considered to be a blocking function. It's a good idea to document such a function, so that the programmers who use it will know that it has the potential to block.

Yielding

This technique falls part-way between polling and blocking. The thread polls the condition in a loop, but on each iteration it relinquishes the remainder of its time slice by calling `pthread_yield()` (POSIX), `Sleep(0)` or `SwitchToThread()` (Windows), or an equivalent system call.

Here's an example:

```
// wait for condition to become true
while (!CheckCondition())
{
    // yield the remainder of my time slice
    pthread_yield(NULL);
}

// the condition is now true and we can continue...
```

This approach tends to result in fewer wasted cycles and better power consumption than a pure busy-wait loop.

Yielding the CPU still involves a kernel call, and is therefore quite expensive. Some CPUs provide a lightweight “pause” instruction. (For example, on an Intel x86 ISA with SSE2, the `_mm_pause()` intrinsic emits such an instruction.) This kind of instruction reduces the power consumption of a busy-wait loop by simply waiting for the CPU's instruction pipeline to empty out before allowing execution to continue:

```
// wait for condition to become true
while (!CheckCondition())
{
    // Intel SSE2 only:
    // reduce power consumption by pausing for ~40 cycles
    _mm_pause();
}
```

```
}  
  
// the condition is now true and we can continue...
```

See <https://software.intel.com/en-us/comment/1134767> and <http://software.intel.com/en-us/forums/topic/309231> for an in-depth discussion of how and why to use a pause instruction in a busy-wait loop.

4.4.6.5 Context Switching

Every thread maintained by the kernel exists in one of three states:⁵

- *Running*. The thread is actively running on a core.
- *Runnable*. The thread is able to run, but it is waiting to receive a time slice on a core.
- *Blocked*. The thread is asleep, waiting for some condition to become true.

A *context switch* occurs whenever the kernel causes a thread to transition from one of these states to another.

A context switch always happens in privileged mode on the CPU—in response to the hardware interrupt that drives preemptive multitasking (i.e., transitions between Running and Runnable), in response to an explicit blocking kernel call made by a running thread (i.e., a transition from Running or Runnable to Blocked), or in response to a waited-on condition becoming true, thus “waking” a sleeping thread (i.e., transitioning it from Blocked to Runnable). The kernel’s thread state machine is illustrated in Figure 4.25.

When a thread is in the Running state, it is actively making use of a CPU core. The core’s registers contain information pertinent to the execution of that thread, such as its instruction pointer (IP), stack and base pointers (SP and BP), and the contents of various general-purpose registers (GPRs). The thread also maintains a call stack, which stores local variables and return addresses for the currently-running function and the entire stack of functions that ultimately called it. Together, this information is known as the thread’s *execution context*.

Whenever a thread transitions away from the Running state to either Runnable or Blocked, the contents of the CPU’s registers are saved to a memory block that has been reserved for the thread by the kernel. Later, when a Runnable thread transitions back to the Running state, the kernel repopulates the CPU’s registers with that thread’s saved register contents.

⁵Some operating systems make use of additional states, but such states are implementation details that we can safely ignore for our purposes.

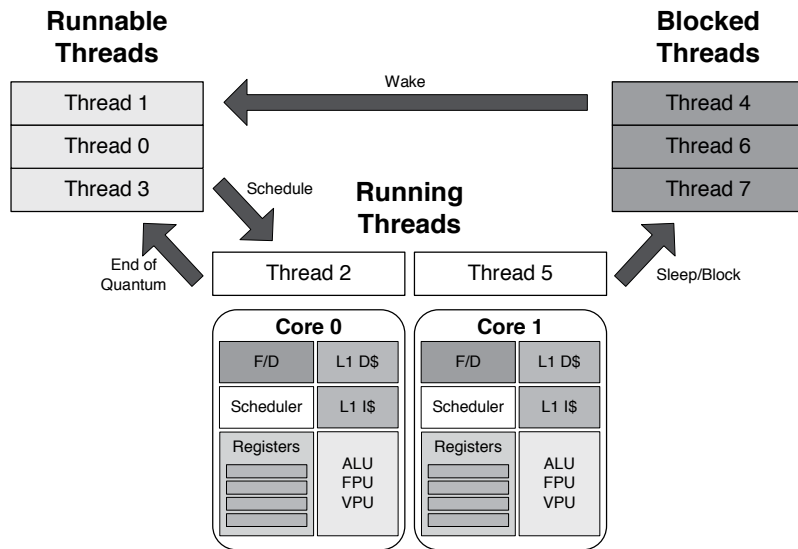


Figure 4.25. Every thread can be in one of three states: Running, Runnable or Blocked.

We should note here that a thread's call stack need not be saved or restored explicitly during a context switch. This is because each thread's call stack already resides in a distinct region within its process' virtual memory map. The act of saving and restoring the contents of the CPU's registers includes saving and restoring the stack and base pointers (SP and BP), and thereby effectively saves and restores the thread's call stack "for free."

During a context switch, if the incoming thread resides in a different process from that of the outgoing thread, the kernel also needs to save off the state of the outgoing process' virtual memory map, and set up the virtual memory map of the incoming process. You'll recall from Section 3.5.2 that a virtual memory map is defined by a virtual page table. The act of saving and restoring a virtual memory map therefore involves saving and restoring a pointer to this page table, which is usually maintained in a special privileged CPU register. The translation lookaside buffer (TLB) must also be flushed whenever an inter-process context switch occurs (see Section 3.5.2.4). These additional steps make context switching between processes more expensive than context switching between threads within a single process.

4.4.6.6 Thread Priorities and Affinity

For the most part, the kernel handles the job of scheduling threads to run on the available cores in the machine. However, programmers do have two ways

to affect how threads are scheduled: *priority* and *affinity*.

A thread's priority controls how it is scheduled relative to other Runnable threads in the system. Higher-priority threads generally take precedence over lower-priority threads. Different operating systems offer different numbers of priority levels. For example, Windows threads can belong to one of six priority classes, and there are seven distinct priority levels within each class. These two values are combined to produce a total of 32 distinct "base priorities" which are used when scheduling threads.

The simplest thread scheduling rule is this: As long as at least one higher-priority Runnable thread exists, no lower-priority threads will be scheduled to run. The idea behind this approach is that most threads in the system will be created at some default priority level, and hence will share the processing resources fairly. But once in a while, a higher-priority thread can become Runnable. When it does, it runs as close to immediately as possible, hopefully exits after a relatively short period of time, and thereby returns control to all of the lower-priority threads.

Such a simple priority-based scheduling algorithm can lead to a situation in which a small number of high-priority threads run continually, thereby preventing any lower-priority threads from running. This is known as *starvation*. Some operating systems attempt to mitigate the ill effects of starvation by introducing exceptions to the simple scheduling rule that aim to give at least some CPU time to starving lower-priority threads.

Another way in which programmers can control thread scheduling is via a thread's *affinity*. This setting requests that the kernel either lock a thread to a particular core, or that it should at least *prefer* one or more cores over the others when scheduling the thread.

4.4.6.7 Thread Local Storage

We said that all threads within a process share the process's resources, including its virtual memory space. There is one exception to this rule—each thread is given a private memory block known as *thread local storage* (TLS). This allows threads to keep track of data that shouldn't be shared with other processes. For example, each thread might maintain a private memory allocator. We can think of the TLS memory block as being a part of the thread's execution context.

In practice, TLS memory blocks are usually visible to all threads within a process. They're typically not protected, the way operating system virtual memory pages are. Instead, the OS grants each thread its own TLS block, all mapped into the process's virtual address space at different numerical addresses, and a system call is provided that allows any one thread to obtain

	ID	Managed ID	Category	Name	Location	Priority
SeqDotNetMandelbrot.vshost.exe (id = 54424) : C:\Users\Pavel\Documents\MatrixWorkshops\SeqDotNetMandelbrot\SeqDotNetMa						
	0x00	0x00	Unknown Thread	[Thread Destroyed]	<not available>	
	0x00005E8C	0x00	Worker Thread	<No Name>	<not available>	Highest
	0x0000F6EC	0x00000003	Worker Thread	<No Name>	<not available>	Normal
	0x00008B50	0x00000006	Worker Thread	vshost.RunParkingWindow	[Managed tc	Normal
	0x00008BF0	0x00000008	Main Thread	Main Thread	DotNetManr	Normal
	0x0000EC54	0x00000007	Worker Thread	.NET SystemEvents	[Managed tc	Normal
	0x000067F8	0x00000009	Worker Thread	<No Name>	DotNetManr	Normal
	0x000107C4	0x0000000A	Worker Thread	<No Name>	DotNetManr	Normal
	0x00009B44	0x0000000B	Worker Thread	<No Name>	DotNetManr	Normal
	0x00007158	0x0000000C	Worker Thread	<No Name>	DotNetManr	Normal
	0x00007008	0x0000000D	Worker Thread	<No Name>	DotNetManr	Normal
	0x00004DA4	0x0000000F	Worker Thread	<No Name>	<not available>	Normal
	0x000039B8	0x0000000E	Worker Thread	<No Name>	<not available>	Normal
	0x0001171C	0x00000010	Worker Thread	<No Name>	DotNetManr	Normal

Figure 4.26. The Threads window in Visual Studio is the primary interface for debugging multithreaded programs.

the address of its private TLS block.

4.4.6.8 Thread Debugging

All good debuggers nowadays provide tools for debugging multithreaded applications. In Microsoft Visual Studio, the *Threads Window* is the central tool for this purpose. Whenever you break into the debugger, this window lists all the threads currently in existence within the application. Double-clicking on a thread makes its execution context active within the debugger. Once a thread's context has been activated, you can walk up and down its call stack via the Call Stack window, and view local variables within each function's scope via the Watch windows. This works even if the thread is in its Runnable or Blocked state. The Visual Studio Threads window is shown in Figure 4.26.

4.4.7 Fibers

In preemptive multitasking, thread scheduling is handled automatically by the kernel. This is often convenient, but sometimes programmers find it desirable to have control over the scheduling of workloads in their programs. For example, when implementing a *job system* for a game engine (discussed in Section 8.6.4), we might want to allow jobs to explicitly yield the CPU to other jobs, without worrying about the possibility of preemption “pulling the rug out” from under our jobs as they run. In other words, sometimes we want to use *cooperative* rather than preemptive multitasking.

Some operating systems provide just such a cooperative multitasking mechanism: They are known as *fibers*. A fiber is a lot like a thread, in that it represents a running instance of a stream of machine language instructions. A fiber has a call stack and register state (an execution context), just like a thread. However, the big difference is that a fiber is never scheduled directly by the kernel. Instead, fibers run within the context of a thread, and are scheduled cooperatively, by each other.

In this section, we'll talk about Windows fibers specifically. Some other operating systems, such as Sony's PlayStation 4 SDK, provide very similar fiber APIs.

4.4.7.1 Fiber Creation and Destruction

How do we convert a thread-based process into a fiber-based one? Every process starts with a single thread when it first runs; hence processes are thread-based by default. When a thread calls the function `ConvertThreadToFiber()`, a new fiber is created within the context of the calling thread. This "bootstraps" the process so that it can create and schedule more fibers. Other fibers are created by calling `CreateFiber()` and passing it the address of a function that will serve as its entry point. Any running fiber can cooperatively schedule a different fiber to run within its thread by calling `SwitchToFiber()`. When a fiber is no longer needed, it can be destroyed by calling `DeleteFiber()`.

4.4.7.2 Fiber States

A fiber can be in one of two states: Active or Inactive. When a fiber is in its Active state, it is assigned to a thread, and executes on its behalf. When a fiber is in its Inactive state, it is sitting on the sidelines, not consuming the resources of any thread, just waiting to be activated. Windows calls an Active fiber the "selected" fiber for a given thread.

An Active fiber can deactivate itself and make another fiber active by calling `SwitchToFiber()`. This is the only way that fibers can switch between the Active and Inactive states.

Whether or not an Active fiber is actively executing on a CPU core is determined by the state of its enclosing thread. When an Active fiber's thread is in the Running state, that fiber's machine language instructions are being executed on a core. When an Active fiber's thread is in the Runnable or Blocked state, its instructions of course cannot execute, because the entire thread is sitting on the sidelines, either waiting to be scheduled on a core or waiting for a condition to become true.

It's important to understand that fibers don't themselves have a Blocked state, the way threads do. In other words, it's not possible to put a fiber to sleep waiting on a condition. Only its thread can be put to sleep. Because of this restriction, whenever a fiber needs to wait for a condition to become true, it either busy-waits or it calls `SwitchToFiber` in order to yield control to another fiber while it waits. Making a blocking OS call from within a fiber is usually a pretty big no-no. Doing so would put the fiber's enclosing thread to sleep, thereby preventing that fiber from doing anything—including scheduling other fibers to run cooperatively.

4.4.7.3 Fiber Migration

A fiber can migrate from thread to thread, but only by passing through its Inactive state. As an example, consider a fiber `F` that is running within the context of thread `A`. Fiber `F` calls `SwitchToFiber(G)` to activate a different fiber named `G` inside thread `A`. This puts fiber `F` into its Inactive state (meaning it is no longer associated with any thread). Now let's assume that another thread named `B` is running fiber `H`. If fiber `H` calls `SwitchToFiber(F)`, then fiber `F` has effectively migrated from thread `A` to thread `B`.

4.4.7.4 Debugging with Fibers

Because fibers are provided by the OS, debugging tools and profiling tools should be able to “see” them, just the way they can “see” threads. For example, when debugging on the PS4 using SN Systems' Visual Studio debugger plugin for Clang, fibers automatically show up in the Threads window as if they were threads. You can double-click a fiber to activate it within the Watch and Call Stack windows, and then walk up and down its call stack just as you normally would with a thread.

If you're considering using fibers in your game engine, it's a good idea to check out your debugger's capabilities on your target platform before you commit a lot of time and effort to a fiber-based design. If your debugger and/or target platform doesn't provide good tools for debugging fibers, that could be a deal breaker.

4.4.7.5 Further Reading on Fibers

You can read more about Windows fibers here: [https://msdn.microsoft.com/en-us/library/windows/desktop/ms682661\(v=vs.85\).aspx](https://msdn.microsoft.com/en-us/library/windows/desktop/ms682661(v=vs.85).aspx).

4.4.8 User-Level Threads and Coroutines

Both threads and fibers tend to be rather “heavy weight” because these facilities are provided by the kernel. This implies that most functions that you’d call to manipulate threads or fibers involve a context switch into kernel space—not a cheap operation. But there are lighter-weight alternatives to threads and fibers. These mechanisms allow programmers to code in terms of multiple independent flows of control, each with its own execution context, but without the high cost of making kernel calls. Collectively, these facilities are known as *user-level threads*.

User-level threads are implemented entirely in user space. The kernel knows nothing about them. Each user-level thread is represented by an ordinary data structure that keeps track of the thread’s id, possibly a human-readable name, and execution context information (the contents of CPU registers and a call stack). A user-level thread library provides API functions for creating and destroying threads, and context switching between them. Each user-level thread runs within the context of a “real” thread or fiber that has been provided by the operating system.

The trick to implementing a user-level thread library is figuring out how to implement a context switch. If you think about it, a context switch mostly boils down to swapping the contents of the CPU’s registers. After all, the registers contain all of the information needed to describe a thread’s execution context—including the instruction pointer and the call stack. So by writing some clever assembly language code, it’s possible to implement a context switch. And once you have a context switch, the rest of your user-level thread library is nothing more than data management.

User-level threads aren’t supported very well in C and C++, but some portable and non-portable solutions do exist. POSIX provided a collection of functions for managing lightweight thread execution contexts via its `ucontext.h` header file (<https://en.wikipedia.org/wiki/Setcontext>), but this API has since been deprecated. The C++ Boost library provides a portable user-level thread library. (Search for “context” on <http://www.boost.org/> for documentation on this library.)

4.4.8.1 Coroutines

Coroutines are a particular type of user-level thread that can prove very useful for writing inherently asynchronous programs, like web servers—and games! A coroutine is a generalization of the concept of a subroutine. Whereas a subroutine can only exit by returning control to its caller, a coroutine can also exit by *yielding* to another coroutine. When a coroutine yields, its execution

context is maintained in memory. The next time the coroutine is *called* (by being yielded *to* by some other coroutine) it continues from where it left off.

Subroutines call each other in a heirarchical fashion. Subroutine A calls B, which calls C, which returns to B, which returns to A. But coroutines call each other symmetrically. Coroutine A can yield to B, which can yield to A, ad infinitum. This back and forth calling pattern doesn't lead to an infinitely deepening call stack, because each coroutine maintains its own private execution context (call stack and register contents). Thus yielding from coroutine A to coroutine B acts more like a context switch between threads than like a function call. But because coroutines are implemented with *user-level* threads, these context switches are very efficient.

Here's a pseudocode example of a system in which one coroutine continually produces data that is consumed by another coroutine:

```
Queue g_queue;

coroutine void Produce()
{
    while (true)
    {
        while (!g_queue.IsFull())
        {
            CreateItemAndAddToQueue(g_queue);
        }
        YieldToCoroutine(Consume);

        // continues from here on next yield...
    }
}

coroutine void Consume()
{
    while (true)
    {
        while (!g_queue.IsEmpty())
        {
            ConsumeItemFromQueue(g_queue);
        }
        YieldToCoroutine(Produce);

        // continues from here on next yield...
    }
}
```

Coroutines are most often provided by high-level languages like Ruby, Lua

and Google's Go. It's also possible to use coroutines in C or C++. The C++ Boost library provides a solid implementation of coroutines, but Boost requires you to compile and link against a pretty huge codebase. If you want something leaner, you may want to try rolling your own coroutine library. The following blog post by Malte Skarupke demonstrates that doing so isn't quite as onerous a task as you might at first imagine: <https://probablydance.com/2013/02/20/handmade-coroutines-for-windows/>.

4.4.8.2 Kernel Threads versus User Threads

The term “kernel thread” has two very different meanings, and this can become a major source of confusion as you read more about multithreading. So let's demystify the term. The two definitions are as follows:

1. On Linux, a “kernel thread” is a special kind of thread created for internal use by the kernel itself, which runs only while the CPU is in privileged mode. The kernel also creates threads for use by processes (via an API such as `pthread` or C++11's `std::thread`). These threads run in user space within the context of a process. In this sense of the term, any thread that runs in privileged mode is a kernel thread, and any thread that runs in user mode (in the context of a single-threaded or multithreaded process) is a “user thread.”
2. The term “kernel thread” can also be used to refer to any thread that is *known to* and *scheduled by* the kernel. Using this definition, a kernel thread can execute in either kernel space or user space, and the term “user thread” only applies to a flow of control that is managed entirely by a user-space program *without* the kernel being involved at all, such as a coroutine.

Using definition #2, a fiber blurs the line between “kernel thread” and “user thread.” On the one hand, the kernel is aware of fibers and maintains a separate call stack for each one. On the other hand, a fiber is not scheduled by the kernel—it can run only when another fiber or thread explicitly hands control to it via a call such as `SwitchToFiber()`.

4.4.9 Further Reading on Processes and Threads

We've covered the basics of processes, threads and fibers in the preceding sections, but really we've only just scratched the surface. For more information, check out some of the following websites:

- For an introduction to threads, see https://www.cs.uic.edu/~jbell/CourseNotes/OperatingSystems/4_Threads.html.
- The full pthread API docs are available online; just search for “pthread documentation.”
- For documentation on the Windows thread API, search for “Process and Thread Functions” on <https://msdn.microsoft.com/>.
- For more information on thread scheduling, search for Nikita Ishkov’s “A Complete Guide to Linux Process Scheduling” online.
- For a great introduction to Go’s implementation of coroutines (which are known as “goroutines”), watch this presentation by Rob Pike: <https://www.youtube.com/watch?v=f6kdp27TYZs>.

4.5 Introduction to Concurrent Programming

There’s a wide range of explicitly parallel computing hardware out there, but how can we take advantage of it as programmers? The answer lies in *concurrent programming* techniques. In concurrent software, a workload is broken down into two or more flows of control that can run semi-independently. As we saw in Section 4.1, in order for a system to qualify as concurrent, it must involve multiple readers and/or multiple writers of *shared* data.

Rob Pike, a Distinguished Engineer at Google Inc. who specializes in distributed and concurrent systems and programming languages, defines concurrency as “the composition of independently executing computations.” This definition underscores the idea that the multiple flows of control in a concurrent system normally operate semi-independently, but their computations are *composed by sharing data* and by *synchronizing* their operations in various ways.

Concurrency can take many forms. Some examples include:

- a piped chain of commands running under Linux or Windows, such as `cat render.cpp | grep "light"`,
- a single process comprised of multiple threads that share a virtual memory space and operate on a common dataset,
- a thread group comprised of thousands of threads running on a GPU, all cooperating to render a scene,
- a multiplayer video game, sharing a common game state between clients running on multiple PCs or game consoles.

4.5.1 Why Write Concurrent Software?

Concurrent programs are sometimes written because a model of multiple semi-independent flows of control simply matches the problem better than a single flow-of-control design. A concurrent design might also be chosen to best make use of a multicore computing platform, even if the problem at hand might be more naturally suited to a sequential design.

4.5.2 Concurrent Programming Models

In order for the various threads within a concurrent program to cooperate, they need to *share data*, and they need to *synchronize* their activities. In other words, they need to *communicate*. There are two basic ways in which concurrent threads can communicate:

- *Message passing.* In this communication mode, concurrent threads pass *messages* between one another in order to share data and synchronize their activities. The messages might be sent across a network, passed between processes using a pipe, or transmitted via a message queue in memory that is accessible to both sender and receiver. This approach works both for threads running on a single computer (either within a single process or across multiple processes), and for threads within processes running on physically distinct computers (e.g., a computer cluster or a grid of machines spread across the globe).
- *Shared memory.* In this communication mode, two or more threads are granted access to the same block of physical memory, and can therefore operate directly on any data objects residing in that memory area. Direct access to shared memory only works when all threads are running on a single computer with a bank of physical RAM that can be “seen” by all CPU cores. Threads within a single process always share a virtual address space, so they can share memory “for free.” Threads within different processes can also share memory by mapping certain physical memory pages into all of the processes’ virtual address spaces.

It’s interesting to note that the *illusion* of shared memory between physically separate computers can be implemented on top of a message-passing system—this technique is known as *distributed shared memory*. Likewise, a message-passing mechanism can be implemented on top of a shared memory architecture, by implementing a message queue that resides in the shared memory pool.

Each approach has its pros and cons. Physically-shared memory is the most efficient way to share a large amount of data, because that data doesn't have to be copied for transmission between threads. On the other hand, as we'll see in Sections 4.5.3 and 4.7, the *sharing* of resources of any kind (memory or other resources) brings with it a host of synchronization problems that tend to be difficult to reason about, and are very tricky to account for in a manner that guarantees correctness of the program. A message-passing design tends to lessen the impacts of (but not eliminate) these kinds of problems.

In this book, we'll focus primarily on shared memory concurrency. We do this for two reasons: First, this is the kind of concurrency you're most likely to encounter as a game programmer, because game engines are typically implemented as single-process multithreaded programs. (Networked multiplayer games are a notable exception to this rule, since they make heavy use of message passing.) Second, shared memory concurrency is a more difficult topic to get your head around. Once you understand concurrency within a shared memory environment, message passing techniques should prove relatively easy to learn.

4.5.3 Race Conditions

A *race condition* is defined as any situation in which the behavior of a program is dependent on timing. In other words, in the presence of a race condition, the behavior of the program can *change* when the relative sequence of events occurring across the system changes, due to variability in the lengths of time taken by the various flows of control to perform their tasks.

4.5.3.1 Critical Races

Sometimes race conditions are innocuous—the behavior of the program may change somewhat depending on timing, but the race causes no ill effects. On the other hand, a *critical race* is a race condition that has the potential to cause *incorrect* program behavior.

The kinds of bugs caused by critical races often seem “strange” or even “impossible” to programmers who aren't experienced with them. Examples include:

- intermittent or seemingly random bugs or crashes,
- incorrect results,
- data structures that get into corrupted states,
- bugs that magically disappear when you switch to a debug build,

- bugs that are around for a while, and then go away for a few days, only to return again (usually the night before E3!),
- bugs that go away when logging (a.k.a., “`printf()` debugging”) is added to the program in an attempt to discover the source of the problem.

Programmers often call these kinds of issues Heisenbugs.

4.5.3.2 Data Races

A *data race* is a critical race condition in which two or more flows of control interfere with one another while reading and/or writing a block of shared data, resulting in data corruption. Data races are *the* central problem of concurrent programming. Writing concurrent programs always boils down to eliminating data races, either by carefully controlling access to shared data, or by replacing shared data with private, independent copies of data (thereby transforming a concurrency problem into a sequential programming problem).

To better understand data races, consider the following simple snippet of C/C++ code:

```
int g_count = 0;

inline void IncrementCount()
{
    ++g_count;
}
```

If you compiled this code for an Intel x86 CPU and viewed the disassembly, it would look something like this:

```
mov    eax,[g_count]    ; read g_count into register EAX
inc     eax              ; increment the value
mov     [g_count],eax    ; write EAX back into g_count
```

This is an example of a *read-modify-write* (RMW) operation.

Now imagine that two threads A and B were both to call the `IncrementCount()` function concurrently (either in parallel or via preemptive multithreading). Under normal operation, if each thread called the function exactly once, we’d expect the final value of `g_count` to be 2, because either thread A increments `g_count` and then thread B increments it, or vice-versa. This is illustrated in Table 4.1.

Next let’s consider the case of our two threads running on a single-core machine with preemptive multitasking. Let’s say that thread A runs first, and

Thread A		Thread B		Value of g_count
Action	EAX	Action	EAX	
	?		?	0
Read	0		?	0
Increment	1		?	0
Write	1		?	1
	1	Read	1	1
	1	Increment	2	1
	1	Write	2	2

Table 4.1. Example of correct operation of a simple two-threaded concurrent program. First thread A reads the contents of a shared variable, increments the value, and writes the results back into the shared variable. At a later time, thread B does the same steps. The final value of the shared variable is 2 as expected.

Thread A		Thread B		Value of g_count
Action	EAX	Action	EAX	
	?		?	0
Read	0		?	0
	0	Read	0	0
	0	Increment	1	0
	0	Write	1	1
Increment	1		1	1
Write	1		1	1

Table 4.2. Example of a race condition. Thread A reads the value of the shared variable, but then is preempted by thread B, which also reads the (same) value. By the time both threads have incremented the value and written their results back to the shared variable, the global variable contains the incorrect value 1 instead of the expected value of 2.

has just finished executing the first `mov` instruction when a context switch to thread B occurs. Instead of thread A executing its `inc` instruction, thread B runs its first `mov` instruction. After some time thread B's quantum expires, and the kernel context switches back to thread A, which continues where it left off and executes the `inc` instruction. Table 4.2 illustrates what happens. Hint: It's not good! The final value of `g_count` is no longer 2 as it should be.

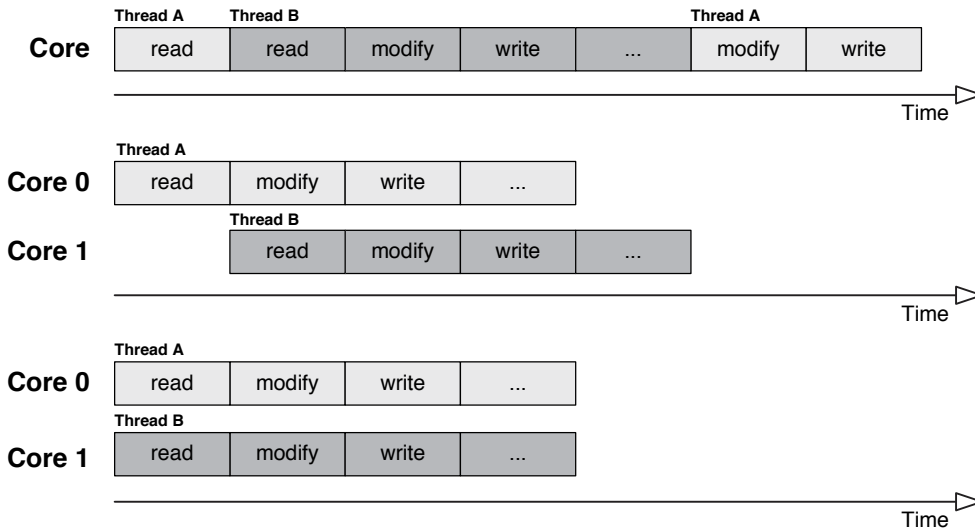


Figure 4.27. Three ways in which a data race can occur within a read-modify-write operation. Top: Two threads racing on a single CPU core. Middle: Two threads overlapping on two separate cores and offset by one instruction. Bottom: Two threads overlapping in perfect synchronization on two cores.

If we run our two threads on parallel hardware, a similar bug can occur, although for a slightly different reason. As in the single-core case, we might get lucky: The two read-modify-write operations might not overlap at all, and the result will be correct. However, if the two read-modify-write operations overlap, either offset from one another or in perfect synchronization, both threads can end up loading the same value of `g_count` into their respective EAX registers. Both will increment the value, and both will write it to memory. One thread will overwrite the results of the other, but it doesn't really matter—because they both loaded the same initial value, the final value of `g_count` will end up being incorrect, just as it was in the single-core scenario. The three data race scenarios (preemption, offset overlap, and perfect synchronization) are illustrated in Figure 4.27.

4.5.4 Critical Operations and Atomicity

Whenever one operation is interrupted by another, we have the potential for a data race bug. However, not all interruptions actually cause bugs. For example, if a thread is performing an operation on a chunk of data that can only be “seen” by that one thread, no data race can occur. Such an operation can be interrupted at any moment by any other operation without consequence.

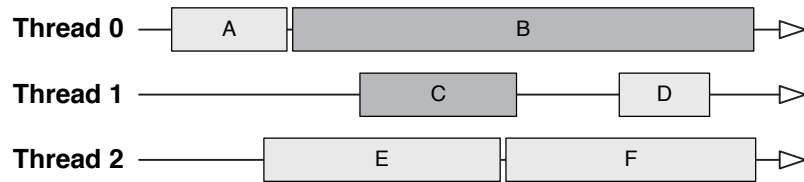


Figure 4.28. Because each step in an algorithm takes a finite amount of time to be performed, it becomes difficult to answer questions about the relative ordering of the steps in a multithreaded program. For example, does operation B happen before or after operation C?

Likewise, if an operation on one data object is interrupted by an operation on a different object, there's no way for those two operations to interfere with one another;⁶ and hence no data race bugs are possible. Data race bugs only occur when an operation on a shared object is interrupted by another operation on that *same* object. So we can only talk meaningfully about data races in relation to a particular shared data object.

Let's use the term *critical operation* to refer to any operation that can possibly read or mutate one particular shared object. To guarantee that the shared object is free from data race bugs, we must ensure that none of its critical operations can interrupt one another. When a critical operation is made uninterruptible in this manner, it is called an *atomic operation*. Alternatively, we can say that such an operation has the property of *atomicity*.

4.5.4.1 Invocation and Response

When we first learn to program, we're usually taught that the *time* required to perform each step in an algorithm is not relevant to the correctness of the algorithm—all that matters is that the steps are performed in the proper *order*. This simple model works well for sequential (single-threaded) programs. But in the presence of multiple threads, it's not possible to define the order of a set of operations when those operations each have a finite duration. This idea is illustrated in Figure 4.28.

In a concurrent system, the only way to define the notion of *order* is to restrict ourselves to talking about *instantaneous events*. Given any pair of instantaneous events, there are only three possibilities: event A happens before event B, event A happens after event B, or the two events are simultaneous. (Perfectly simultaneous events are rare, but they can occur in a multicore computer in which some or all cores share a synchronized clock.)

Any operation with a finite duration can be broken down into two instantaneous events—its *invocation* (the moment at which the operation begins) and

⁶This is only strictly true if the two objects reside on different cache lines.

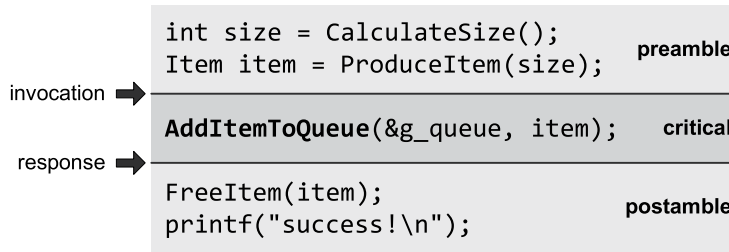


Figure 4.29. Any code snippet that includes a critical operation can be partitioned into three sections. The critical operation itself is bounded above by its invocation, and below by its response.

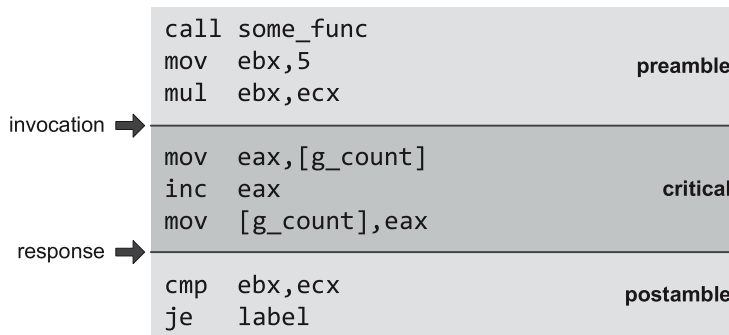


Figure 4.30. Another example, this time in assembly language, of a code snippet partitioned into three sections, bounded by the invocation and response of a critical operation.

its *response* (the moment at which it is considered to be complete). When we look at any code snippet that includes a critical operation on some shared data object, we can thus divide it into three sections, with the instantaneous invocation and response events demarking the boundaries between them. Note that we're talking here about the order in which events occur *as they were written* in the source code—this is known as *program order*.

- *Preamble section:* All code that occurs before the critical operation's invocation, in program order.
- *Critical section:* The code that comprises the critical operation itself.
- *Postamble section:* All code that occurs after the critical operation's response, in program order.

This notion of partitioning a block of code into three sections is illustrated in Figures 4.29 and 4.30.

4.5.4.2 Atomicity Defined

As we saw in Section 4.5.3.2, a data race bug can occur when a critical operation is interrupted by another critical operation on the same shared object. This can happen:

- when one thread preempts another on a single core, or
- when two or more critical operations overlap across multiple cores.

Thinking in terms of invocation and response, we can pin down the general notion of interruption a bit more precisely: An *interruption* occurs whenever the invocation and/or response of one operation occurs *between* the invocation and response of another operation. But as we've said, not all interruptions lead to data race bugs. A critical operation on a particular shared object can only be affected by a data race if its invocation and response are interrupted by another critical operation on that *same* object. Therefore, we can define the *atomicity* of a critical operation as follows:

A critical operation can be said to have executed *atomically* if its invocation and response are not interrupted by another critical operation on that same object.

We should stress here that it's perfectly fine for a critical operation to be interrupted by other noncritical operations, or by critical operations affecting other unrelated data objects. Only when two critical operations on the *same* object interrupt one another does a data race bug occur. Figure 4.31 illustrates various cases—one in which a critical operation succeeds in executing atomically, and three in which it does not.

We can guarantee that a critical operation will be executed atomically if we can make it appear, from the point of view of all other threads in the system, to have occurred *instantaneously*. In other words, it must appear as though the invocation and response of the operation are simultaneous, or that the critical operation itself has a zero duration. That way, there can be no possibility of another critical operation's invocation or response "sneaking in" between the invocation and response of the operation in question.

4.5.4.3 Making an Operation Atomic

But how can we transform a critical operation into an atomic operation? The easiest and most reliable way to accomplish this is to use a special object called a *mutex*. A mutex is an object provided by the operating system that acts like a padlock, in that it can be locked and unlocked by a thread. Given two critical

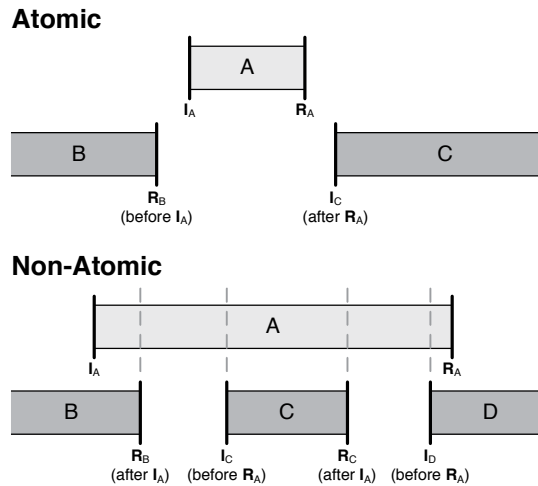


Figure 4.31. Top: Critical operation A can be said to have executed atomically, because it was not interrupted by any other invocations or responses from critical operations on the same shared object. Bottom: Three scenarios in which critical operation A executed nonatomically because it was interrupted by the invocation and/or response of another critical operation on the same object.

operations on a particular shared data object, we guard the invocation of each operation with the acquisition of the mutex, and release the mutex at each one's response. Because the OS guarantees that a mutex can only be acquired by one thread at a time, we can thus be certain that the invocation or response of one operation can never happen in between the invocation and response of the other. From the point of view of the global ordering of the events in a concurrent system, a critical operation guarded with a mutex lock *appears* to be instantaneous.

Mutexes are part of a collection of concurrency tools provided by the operating system known as *thread synchronization primitives*. We'll explore thread synchronization primitives in Section 4.6.

4.5.4.4 Atomicity as Serialization

Consider a group of threads, all attempting to perform a single operation on a shared data object. Without atomicity, these operations might happen simultaneously, or they might overlap in all sorts of unpredictable ways over time. This is illustrated in Figure 4.32.

However, making the operation atomic guarantees that only one thread will ever be performing it at any given moment in time. This has the effect of *serializing* the operations—what used to be a jumble of overlapping opera-

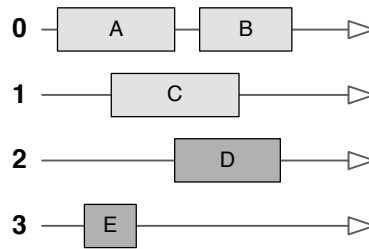


Figure 4.32. Without atomicity, operations performed by multiple threads can overlap in unpredictable ways over time.

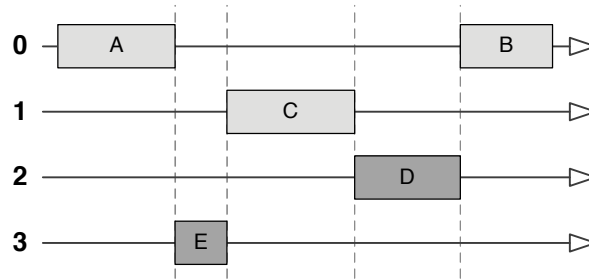


Figure 4.33. By wrapping each critical operation in a mutex lock-unlock pair, we force the operations to execute *sequentially*.

tions is transformed into an orderly sequential sequence of atomic operations. Making an operation atomic gives us no control over what the order will end up being; all we can say for certain is that the operations will be performed in *some* sequential order. This idea is illustrated in Figure 4.33.

4.5.4.5 Data-Centric Consistency Models

The concepts of atomicity and the serialization of operations within a concurrent system are part of a larger topic known as *data-centric consistency models*. A consistency model is a *contract* between a data store, such as a shared data object in a concurrent system or a database in a distributed system, and a collection of threads that share that data store. It makes reasoning about the behavior of the data store easier—as long as the threads follow the rules of the contract, the programmer can be certain that the data store will behave in a consistent and predictable manner, and its data will not become corrupted.

A data store that provides a guarantee of atomicity can be said to be *linearizable*. The topic of data-centric consistency is a bit beyond our scope here, but you can read more about it online. Here are a few good places to start:

- Search for “consistency model” and “linearizability” on Wikipedia;
- <https://www.cse.buffalo.edu/~stevko/courses/cse486/spring13/lectures/26-consistency2.pdf>;
- <http://www.cs.cmu.edu/~srini/15-446/S09/lectures/10-consistency.pdf>.

4.6 Thread Synchronization Primitives

Every operating system that supports concurrency provides a suite of tools known as *thread synchronization primitives*. These tools provide two services to concurrent programmers:

1. The ability to share resources between threads by making critical operations *atomic*.
2. The ability to *synchronize* the operation of two or more threads:
 - a. by enabling a thread to *go to sleep* while it waits for a resource to become available or for one or more other threads to complete a task, and
 - b. by enabling a running thread to *notify* one or more sleeping threads by waking them up.

We should note here that while these thread synchronization primitives are robust and relatively easy to use, they are generally quite expensive. This is because these tools are provided by the kernel. Interacting with any of them therefore requires a *kernel call*, which involves a context switch into protected mode. Such context switches can cost upwards of 1000 clock cycles. Because of their high cost, some concurrent programmers prefer to implement their own atomicity and synchronization tools, or they turn to *lock-free programming* to improve the efficiency of their concurrent software. Nevertheless, a solid understanding of these synchronization primitives is an important part of any concurrent programmer’s toolkit.

4.6.1 Mutexes

A mutex is an operating system object that allows critical operations to be made atomic. A mutex can be in one of two states: *unlocked* or *locked*. (These two states are sometimes called *released* and *acquired*, or *signaled* and *nonsignaled*, respectively.)

The most important property of a mutex is that it guarantees that only one thread will ever be holding a lock on it at any given time. So, if we wrap all critical operations for a particular shared data object in a mutex lock, those operations become atomic relative to one another. In other words, the operations become *mutually exclusive*. This is where the name “mutex” comes from—it’s short for “mutual exclusion.”

A mutex is represented either by a regular C++ object or by a handle to an opaque kernel object. Its API is typically comprised of the following functions:

1. `create()` or `init()`. A function call or class constructor that creates the mutex.
2. `destroy()`. A function call or destructor that destroys the mutex.
3. `lock()` or `acquire()`. A blocking function that locks the mutex on behalf of the calling thread, but puts the thread to sleep (see Section 4.4.6.4) if the lock is currently held by another thread.
4. `try_lock()` or `try_acquire()`. A non-blocking function that attempts to lock the mutex, but returns immediately if the lock cannot be acquired.
5. `unlock()` or `release()`. A non-blocking function that releases the lock on the mutex. In most operating systems, only the thread that locked a mutex is permitted to unlock it.

When a mutex is locked by a thread in the system, we say it is in a *non-signaled* state. When the thread releases the lock, the mutex becomes *signaled*. If one or more other threads is asleep (blocked) waiting on the mutex, the act of signaling it causes the kernel to select one of these waiting threads and wake it up. In some operating systems, it’s possible for a thread to explicitly wait for a kernel object such as a mutex to become signaled. Under Windows, the `WaitForSingleObject()` and `WaitForMultipleObjects()` OS calls serve this purpose.

4.6.1.1 POSIX

Now that we have an understanding of how mutexes work, let’s take a look at a few examples. The POSIX thread library exposes kernel mutex objects via a C-style functional interface. Here’s how we’d use it to turn our shared counter example from Section 4.5.3.2 into an atomic operation:

```
#include <pthread.h>

int g_count = 0;
pthread_mutex_t g_mutex;
```



```

inline void IncrementCount()
{
    pthread_mutex_lock(&g_mutex);
    ++g_count;
    pthread_mutex_unlock(&g_mutex);
}

```

Note that in the interest of clarity and brevity, we've omitted the code, normally executed by the main thread, that calls `pthread_mutex_init()` to initialize the mutex prior to spawning the threads that will use it, and that calls `pthread_mutex_destroy()` to destroy the mutex once all other threads have exited.

4.6.1.2 C++ Standard Library

Starting with C++11, the C++ standard library exposes kernel mutexes via the class `std::mutex`. Here's how we'd use it to make the increment of a shared counter atomic:

```

#include <mutex>

int g_count = 0;
std::mutex g_mutex;

inline void IncrementCount()
{
    g_mutex.lock();
    ++g_count;
    g_mutex.unlock();
}

```

The constructor and destructor of the `std::mutex` class handles the initialization and destruction of the underlying kernel mutex object, making it a little bit easier to use than `pthread_mutex_t`.

4.6.1.3 Windows

Under Windows, a mutex is represented by an opaque kernel object and referenced through a handle. A mutex is “locked” by waiting for it to become signaled, using the general-purpose `WaitForSingleObject()` function. Unlocking a mutex is accomplished by calling `ReleaseMutex()`. Rewriting our simple example using Windows mutexes, and again omitting the details of the creation and destruction of the mutex object, we arrive at the following code:

```
#include <windows.h>

int g_count = 0;
HANDLE g_hMutex;

inline void IncrementCount()
{
    if (WaitForSingleObject(g_hMutex, INFINITE)
        == WAIT_OBJECT_0)
    {
        ++g_count;
        ReleaseMutex(g_hMutex);
    }
    else
    {
        // learn to deal with failure...
    }
}
```

4.6.2 Critical Sections

In most operating systems, a mutex can be shared between processes. As such, it is a data structure that is managed internally by the kernel. This means that all operations performed on a mutex involve a kernel call, and hence a context switch into protected mode on the CPU. This makes mutexes relatively expensive, even when no other threads are contending for the lock.

Some operating systems provide less-expensive alternatives to a mutex. For example, Microsoft Windows provides a locking mechanism known as a *critical section*. The terminology and API look a bit different to that of a mutex, but a critical section under Windows is really just a low-cost mutex.

The API of a critical section looks like this:

1. `InitializeCriticalSection()`. Constructs a critical section object.
2. `DeleteCriticalSection()`. Destroys an initialized critical section object.
3. `EnterCriticalSection()`. A blocking function that locks a critical section on behalf of the calling thread, but busy-waits or puts the thread to sleep if the lock is currently held by another thread.
4. `TryEnterCriticalSection()`. A non-blocking function that attempts to lock a critical section, but returns immediately if the lock cannot be acquired.

5. `LeaveCriticalSection()`. A non-blocking function that releases the lock on a critical section object.

Here's how we'd implement an atomic increment using the Windows critical section API:

```
#include <windows.h>

int g_count = 0;
CRITICAL_SECTION g_critsec;

inline void IncrementCount()
{
    EnterCriticalSection(&g_critsec);
    ++g_count;
    LeaveCriticalSection(&g_critsec);
}
```

As before, we've omitted some details. The main thread normally initializes the critical section prior to spawning the threads that use it, and would of course clean it up once the threads have all exited.

How is the low cost of a critical section achieved? When a thread first attempts to enter (lock) a critical section that is already locked by another thread, an inexpensive *spin lock* is used to wait until the other thread has left (unlocked) that critical section. A spin lock does not require a context switch into the kernel, making it a few thousand clock cycles cheaper than a mutex. Only if the thread busy-waits for too long is the thread put to sleep, as it would be with a regular mutex. This less-expensive approach works because, unlike a mutex, a critical section cannot be shared across process boundaries. We'll discuss spin locks in more depth in Section 4.9.7.

Some other operating systems provide "cheap" mutex variants as well. For example, Linux supports a thing called a "futex" that acts somewhat like a critical section under Windows. Its use is beyond our scope here, but you can read more about futexes at <https://www.akkadia.org/drepper/futex.pdf>.

4.6.3 Condition Variables

In concurrent programming, we often need to send signals between threads in order to synchronize their activities. One example of this is the ubiquitous producer-consumer problem which we introduced in Section 4.4.8.1. In this problem, we have two threads: A *producer* thread calculates or otherwise generates some data, and that data is read and put to use by a *consumer* thread. Obviously the consumer thread cannot consume the data until the producer

has produced it. Hence the producer thread needs a way to notify the consumer that its data is ready for consumption.

We could consider using a global Boolean variable as a signalling mechanism. The following code snippet illustrates the idea, using POSIX threads. (Some details have been omitted for clarity.)

```

Queue          g_queue;
pthread_mutex_t g_mutex;
bool           g_ready = false;

void* ProducerThread(void*)
{
    // keep on producing forever...
    while (true)
    {
        pthread_mutex_lock(&g_mutex);

        // fill the queue with data
        ProduceDataInto(&g_queue);

        g_ready = true;
        pthread_mutex_unlock(&g_mutex);

        // yield the remainder of my timeslice
        // to give the consumer a chance to run
        pthread_yield();
    }
    return nullptr;
}

void* ConsumerThread(void*)
{
    // keep on consuming forever...
    while (true)
    {
        // wait for the data to be ready
        while (true)
        {
            // read the value into a local,
            // making sure to lock the mutex
            pthread_mutex_lock(&g_mutex);
            const bool ready = g_ready;
            pthread_mutex_unlock(&g_mutex);

            if (ready)
                break;
        }
    }
}

```

```

    }

    // consume the data
    pthread_mutex_lock(&g_mutex);
    ConsumeDataFrom(&g_queue);
    g_ready = false;
    pthread_mutex_unlock(&g_mutex);

    // yield the remainder of my timeslice
    // to give the producer a chance to run
    pthread_yield();
}
return nullptr;
}

```

Besides the fact that this example is somewhat contrived, there's one big problem with it: The consumer thread spins in a tight loop, polling the value of `g_ready`. As we discussed in Section 4.4.6.4, busy-waiting like this wastes valuable CPU cycles.

Ideally, we'd like a way to *block* the consumer thread (put it to sleep) while the producer does its work, and then wake it up when the data is ready to be consumed. This can be accomplished by making use of a new kind of kernel object called a *condition variable* (CV).

A condition variable isn't actually a variable that stores a condition. Rather, it's a queue of waiting (sleeping) threads, combined with a mechanism that allows a running thread to wake up the sleeping threads at a time of its choosing. (Perhaps "wait queue" would have been a better name for these things.) The sleep and wake operations are performed in an atomic way with the help of a mutex provided by the program, plus a little help from the kernel.

The API for a condition variable typically looks something like this:

1. `create()` or `init()`. A function call or class constructor that creates a condition variable.
2. `destroy()`. A function call or destructor that destroys a condition variable.
3. `wait()`. A blocking function that puts the calling thread to sleep.
4. `notify()`. A non-blocking function that wakes up any threads that are currently asleep waiting on the condition variable.

Let's rewrite our simple producer-consumer example using a CV:

```

Queue          g_queue;
pthread_mutex_t g_mutex;

```

```

bool          g_ready = false;
pthread_cond_t g_cv;

void* ProducerThreadCV(void*)
{
    // keep on producing forever...
    while (true)
    {
        pthread_mutex_lock(&g_mutex);

        // fill the queue with data
        ProduceDataInto(&g_queue);

        // notify and wake up the consumer thread
        g_ready = true;
        pthread_cond_signal(&g_cv);
        pthread_mutex_unlock(&g_mutex);
    }
    return nullptr;
}

void* ConsumerThreadCV(void*)
{
    // keep on consuming forever...
    while (true)
    {
        // wait for the data to be ready
        pthread_mutex_lock(&g_mutex);
        while (!g_ready)
        {
            // go to sleep until notified... the mutex
            // will be released for us by the kernel
            pthread_cond_wait(&g_cv, &g_mutex);

            // when it wakes up, the kernel makes sure
            // that this thread holds the mutex again
        }

        // consume the data
        ConsumeDataFrom(&g_queue);
        g_ready = false;
        pthread_mutex_unlock(&g_mutex);
    }
    return nullptr;
}

```

The consumer thread calls `pthread_cond_wait()` to go to sleep until

`g_ready` becomes true. The producer works for a while producing its data. When the data is ready, the producer sets the global `g_ready` flag to true, and then wakes up the sleeping consumer by calling `pthread_cond_signal()`. The consumer then consumes the data. In this example, the consumer and producer ping-pong back and forth like this indefinitely.

You probably noticed that the consumer thread locks its mutex before entering the while loop that checks the `g_ready` flag. When it waits on the condition variable, it apparently goes to sleep *while holding the mutex lock*! Normally this would be a big no-no: If a thread goes to sleep while holding a lock, it will almost certainly lead to a *deadlock* situation (see Section 4.7.1). However, this is not a problem when using a condition variable. That's because the kernel actually does a little slight of hand, *unlocking the mutex* after the thread has been safely put to bed. Later, when the sleeping thread is woken back up, the kernel does some more slight of hand to ensure that the lock will once again be held by the freshly awoken thread.

You may have noticed another oddity: The consumer thread still uses a while loop to check the value of `g_ready`, despite also using a condition variable to wait for the flag to become true. The reason that this loop is necessary is that threads can sometimes be awoken spuriously by the kernel. As a result, when the call to `pthread_cond_wait()` returns, the value of `g_ready` might *not* actually be true yet. So we must keep polling in a loop until the condition really is true.

4.6.4 Semaphores

Just as a mutex acts like an atomic Boolean flag, a *semaphore* acts like an atomic counter whose value is never allowed to drop below zero. We can think of a semaphore as a special kind of mutex that allows *more than one thread* to acquire it simultaneously.

A semaphore can be used to permit a group of threads to share a limited set of resources. For example, let's suppose that we're implementing a rendering system that allows text and 2D images to be rendered to offscreen buffers, for the purposes of drawing the game's heads-up display (HUD) and in-game menus. Due to memory constraints, let's further assume that we can only afford to allocate four of these buffers. A semaphore can be used to ensure that no more than four threads are permitted to render into these buffers at any given moment.

The API of a semaphore is typically comprised of the following functions:

1. `init()`. Initializes a semaphore object, and sets its counter to a specified

initial value.

2. `destroy()`. Destroys a semaphore object.
3. `take()` or `wait()`. If the counter value encapsulated by a given semaphore is greater than zero, this function decrements the counter and returns immediately. If its counter value is currently zero, this function blocks (puts the thread to sleep) until the semaphore's counter rises above zero again.
4. `give()`, `post()` or `signal()`. Increments the encapsulated counter value by one, thereby opening up a "slot" for another thread to `take()` the semaphore. If a thread is currently asleep waiting on the semaphore when `give()` is called, that thread will wake up from its call to `take()` or `wait()`.⁷

So, to implement a resource pool that can be accessed by up to N threads at a time, we would simply create a semaphore and initialize its counter to N . A thread gains access to the resource pool by calling `take()`, and releases its hold on the resource pool when it is done by calling `give()`.

We say that a semaphore is *signaled* whenever its count is greater than zero, and it is *nonsignaled* when its counter is equal to zero. This is why the functions that take and give a semaphore are named `wait()` and `signal()`, respectively, in some APIs: If the semaphore isn't signaled when a thread calls this function, the thread will *wait* for the semaphore to become signaled.

4.6.4.1 Mutex versus Binary Semaphore

A semaphore whose initial value is set to 1 is called a *binary semaphore*. One might think that a binary semaphore is identical to a mutex. Certainly both objects permit only one thread to acquire it at a time. However, these two synchronization objects are not equivalent, and are typically used for quite different purposes.

The key difference between a mutex and a binary semaphore is that a mutex can only be unlocked by the thread that locked it. A semaphore's counter, on the other hand, can be incremented by one thread and later decremented by another thread. This implies that a binary semaphore can be "unlocked" by a different thread than the one that "locked" it. Or really, we should say that a binary semaphore can be *given* by a different thread than the one that *takes* it.

This seemingly subtle difference between mutexes and binary semaphores leads to very different use cases for these two kinds of synchronization object.

⁷When you read about semaphores, you may discover some authors using the function names `p()` and `v()` instead of `wait()` and `signal()`. These letters come from the Dutch names for these two operations.

A mutex is used to make an operation *atomic*. But a binary semaphore is typically used to send a *signal* from one thread to another.

Consider again our producer-consumer example, in which the producer needs to notify the consumer when the data it produces is ready for consumption. This notification mechanism can be implemented using two binary semaphores, one that allows the producer to wake the consumer, and one that allows the consumer to wake the producer. We can think of these semaphores as representing the number of elements that are used and free within a buffer that's shared between the two threads, although in this simple example the buffer can only hold a single item. As such, we'll call the semaphores `g_semUsed` and `g_semFree`, respectively. Here's what the code would look like, using POSIX semaphores:

```
Queue g_queue;
sem_t g_semUsed; // initialized to 0
sem_t g_semFree; // initialized to 1

void* ProducerThreadSem(void*)
{
    // keep on producing forever...
    while (true)
    {
        // produce an item (can be done non-
        // atomically because it's local data)
        Item item = ProduceItem();

        // decrement the free count
        // (wait until there's room)
        sem_wait(&g_semFree);

        AddItemToQueue(&g_queue, item);

        // increment the used count
        // (notify consumer that there's data)
        sem_post(&g_semUsed);
    }
    return nullptr;
}

void* ConsumerThreadSem(void*)
{
    // keep on consuming forever...
    while (true)
    {
        // decrement the used count
```

```

        // (wait for the data to be ready)
        sem_wait(&g_semUsed);

        Item item = RemoveItemFromQueue(&g_queue);

        // increment the free count
        // (notify producer that there's room)
        sem_post(&g_semFree);

        // consume the item (can be done non-
        // atomically because it's local data)
        ConsumeItem(item);
    }
    return nullptr;
}

```

4.6.4.2 Implementing a Semaphore

It turns out that one can implement a semaphore in terms of a mutex, a condition variable and an integer. In that sense, a semaphore is a “higher-level” construct than either a mutex or a condition variable. Here’s what the implementation looks like:

```

class Semaphore
{
private:
    int m_count;
    pthread_mutex_t m_mutex;
    pthread_cond_t m_cv;

public:
    explicit Semaphore(int initialCount)
    {
        m_count = initialCount;
        pthread_mutex_init(&m_mutex, nullptr);
        pthread_cond_init(&m_cv, nullptr);
    }

    void Take()
    {
        pthread_mutex_lock(&m_mutex);
        // put the thread to sleep as long as
        // the count is zero
        while (m_count == 0)
            pthread_cond_wait(&m_cv, &m_mutex);
    }
}

```

```

        --m_count;
        pthread_mutex_unlock(&m_mutex);
    }

    void Give()
    {
        pthread_mutex_lock(&m_mutex);
        ++m_count;
        // if the count was zero before the
        // increment, wake up a waiting thread
        if (m_count == 1)
            pthread_cond_signal(&m_cv);
        pthread_mutex_unlock(&m_mutex);
    }

    // aliases for other commonly-used function names
    void Wait() { Take(); }
    void Post() { Give(); }
    void Signal() { Give(); }
    void Down() { Take(); }
    void Up() { Give(); }
    void P() { Take(); } // Dutch "proberen" = "test"
    void V() { Give(); } // Dutch "verhogen" =
                        // "increment"
};

```

4.6.5 Windows Events

Windows provides a mechanism called an *event object* that is similar in function to a condition variable, but much simpler to use. Once an event object has been created, a thread can go to sleep by calling `WaitForSingleObject()`, and that thread can be awoken by another thread by calling `SetEvent()`. Rewriting our producer-consumer example using event objects yields the following very simple implementation:

```

#include <windows.h>

Queue g_queue;
Handle g_hUsed; // initialized to false (nonsignaled)
Handle g_hFree; // initialized to true (signaled)

void* ProducerThreadEv(void*)
{
    // keep on producing forever...
    while (true)
    {
        // produce an item (can be done non-

```

```

        // atomically because it's local data)
        Item item = ProduceItem();

        // wait until there's room
        WaitForSingleObject(&g_hFree);

        AddItemToQueue(&g_queue, item);

        // notify consumer that there's data
        SetEvent(&g_hUsed);
    }
    return nullptr;
}

void* ConsumerThreadEv(void*)
{
    // keep on consuming forever...
    while (true)
    {
        // wait for the data to be ready
        WaitForSingleObject(&g_hUsed);

        Item item = RemoveItemFromQueue(&g_queue);

        // notify producer that there's room
        SetEvent(&g_hFree);

        // consume the item (can be done non-
        // atomically because it's local data)
        ConsumeItem(item);
    }
    return nullptr;
}

void MainThread()
{
    // create event in the nonsignalled state
    g_hUsed = CreateEvent(nullptr, false,
                          false, nullptr);
    g_hFree = CreateEvent(nullptr, false,
                          true, nullptr);

    // spawn our threads
    CreateThread(nullptr, 0x2000, ConsumerThreadEv,
                0, 0, nullptr);
    CreateThread(nullptr, 0x2000, ProducerThreadEv,
                0, 0, nullptr);
}

```

```
    // ...  
}
```

4.7 Problems with Lock-Based Concurrency

In Section 4.5.3.2, we learned that *data races* can lead to incorrect program behavior in a concurrent system. We saw that the solution to this problem is to make operations on *shared* data objects *atomic*. One way to achieve atomicity is to wrap these operations in *locks*, which are often implemented using operating-system provided thread synchronization primitives such as mutexes.

However, atomicity is only part of the concurrency story. There are other problems that can plague a concurrent system, even when all shared-data operations have been carefully protected by locks. In the following sections, we'll briefly explore the most common of these problems.

4.7.1 Deadlock

Deadlock is a situation in which no thread in the system can make progress, resulting in a hang. When a deadlock occurs, all threads are in their Blocked states, waiting on some resource to become available. But because no threads are Runnable, none of those resources can ever become available, so the entire program hangs.

For a deadlock to occur, we need at least two threads and two resources. For example, Thread 1 holds Resource A but is waiting for Resource B; and at the same time, Thread 2 holds Resource B but is waiting for Resource A. This situation is illustrated by the following code snippet:

```
void Thread1()  
{  
    g_mutexA.lock(); // holds lock for Resource A  
    g_mutexB.lock(); // sleeps waiting for Resource B  
    // ...  
}  
  
void Thread2()  
{  
    g_mutexB.lock(); // holds lock for Resource B  
    g_mutexA.lock(); // sleeps waiting for Resource A  
    // ...  
}
```

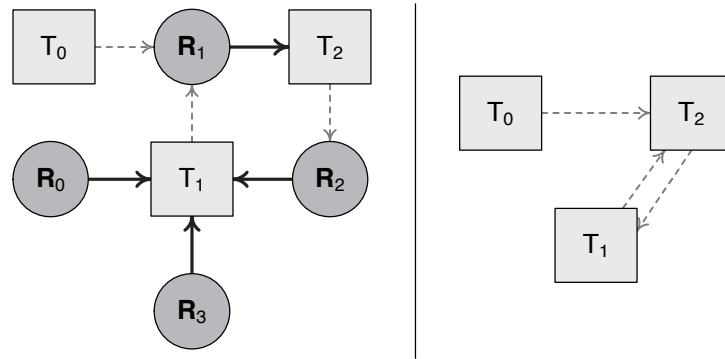


Figure 4.34. Left: Dark arrows show resources currently held by threads. Light dashed arrows show threads waiting on resources to become available. Right: We can eliminate the resources and simply draw the dependencies (light dashed arrows) between threads. A cycle in such a thread dependency graph indicates a deadlock.

Other more complex kinds of deadlock can of course occur as well, involving more threads and more resources. But the key factor that defines any deadlock situation is a *circular* dependency between threads and their resources.

To analyze a system for the possibility of deadlock, we can draw a graph of our threads, our resources, and the dependencies between them, as shown in Figure 4.34. In this graph, we've used squares to represent threads and circles to represent resources (or more precisely, the mutex locks that protect them). Solid arrows connect resources to the threads that currently hold locks on them. Dashed arrows connect threads to the resources they are waiting for. We can actually eliminate the resource nodes in the graph for simplicity if we like, leaving only the dashed lines connecting threads that are waiting on other threads. If such a dependency graph ever contains a *cycle*, we have a deadlock.

Actually a cycle in the dependency graph isn't quite enough to produce a deadlock. Strictly speaking, there are four necessary and sufficient conditions for deadlock, known as the *Coffman conditions*:

1. *Mutual exclusion*. A single thread can be granted exclusive access to a single resource via a mutex lock.
2. *Hold and wait*. A thread must be holding one lock when it goes to sleep waiting on another lock.
3. *No lock preemption*. No one (not even the kernel) is allowed to forcibly

break a lock held by a sleeping thread.

4. *Circular wait.* There must exist a cycle in the thread dependency graph.

Avoiding deadlock always boils down to preventing one or more of the Coffman conditions from holding true. Since violating conditions #1 and #3 would be “cheating,” solutions usually focus on avoiding conditions #2 and #4.

Hold and wait can be avoided by reducing the number of locks. In our simple example, if Resource A and Resource B were both protected by a single lock L, then deadlock could not occur. Either Thread 1 would obtain the lock, and gain exclusive access to both resources while Thread 2 waits, or Thread 2 would obtain the lock while Thread 1 waits.

The circular wait condition can be avoided by imposing a global order to all lock-taking in the system. In our simple two-threaded example, if we were to ensure that Resource A’s lock is always taken before Resource B’s lock, deadlock would be avoided. This would work because one thread will always obtain a lock on Resource A before attempting to take any other locks. This effectively puts all other contending threads to sleep, thereby ensuring that the attempt to take a lock on Resource B will always succeed.

4.7.2 Livelock

Another solution to the deadlock problem is for threads to *try* to take locks without going to sleep (using a function such as `pthread_mutex_trylock()`). If a lock cannot be obtained, the thread sleeps for some short period of time and then *retries* the lock.

When threads use an explicit strategy, such as timed retries, in order to avoid or resolve deadlocks, a new problem can arise: The threads can end up spending all of their time just trying to resolve the deadlock, rather than performing any real work. This is known as *livelock*.

As a simple example of livelock, consider again our example of two threads 1 and 2 contending over two resources A and B. Whenever a thread is unable to obtain a lock, it releases any locks it already holds and waits for a fixed timeout before trying again. If both threads use the same timeout, we can get into a situation in which the same degenerate situation simply repeats over and over. Our threads become “stuck” forever trying to resolve the conflict, and neither one ever gets a chance to do its real job. Livelock is akin to a stalemate in chess.

Livelock can be avoided by using an asymmetric deadlock resolution algorithm. For example, we could ensure that only one thread, either chosen at

random or based on priority, ever takes action to resolve a deadlock when one is detected.

4.7.3 Starvation

Starvation is defined as any situation in which one or more threads fail to receive any execution time on the CPU. Starvation can happen when one or more higher-priority threads fail to relinquish control of the CPU, thereby preventing lower-priority threads from running. Livelock is another kind of starvation, in which a deadlock-resolution algorithm effectively starves all threads of their ability to do “real” work.

Priority-based starvation is normally avoided by ensuring that high-priority threads never run for very long. Ideally a multithreaded program would consist of a pool of lower-priority threads that normally share the system’s CPU resources fairly. Once in a while, a higher-priority thread runs, takes care of its business quickly, and then ends, returning the CPU resources to the lower-priority threads.

4.7.4 Priority Inversion

Mutex locks can lead to a situation known as *priority inversion*, in which a low-priority thread acts as though it is the highest-priority thread in the system. Consider two threads, L and H, with a low and high priority, respectively. Thread L takes a mutex lock and then is preempted by H. If H attempts to take this same lock, then H will be put to sleep because L already holds the lock. This permits L to run even though it is lower priority than H—in violation of the principle that lower-priority threads should not run while a higher-priority thread is runnable.

Priority inversion can also occur if a medium-priority thread M preempts thread L while it is holding a lock needed by H. In this case, L goes to sleep while M runs, preventing it from releasing the lock. When H runs, it is therefore unable to obtain the lock; it goes to sleep, and now M’s priority has effectively been inverted with that of thread H.

The consequences of priority inversion may be negligible. For example, if the lower-priority thread immediately relinquishes the lock, the duration of the priority inversion may be short and it may go unnoticed or produce only minor ill effects. However, in extreme cases, priority inversion can lead to deadlock or other kinds of system failure. For example, a priority inversion can cause a high-priority thread to miss a critical deadline.

Solutions to the priority inversion problem include:

- Avoiding locks that can be taken by both low- and high-priority threads. (This solution is often not feasible.)
- Assigning a very high priority to the mutex itself. Any thread that takes the mutex has its priority temporarily raised to that of the mutex, thus ensuring it cannot be preempted while holding the lock.
- Random priority boosting. In this approach, threads that are actively holding locks are randomly boosted in priority until they exit their critical sections. This approach is used in the Windows scheduling model.

4.7.5 The Dining Philosophers

The famous *dining philosophers* problem is a great illustration of the problems of deadlock, livelock and starvation. It describes a situation in which five philosophers sit around a circular table, each with a plate of spaghetti in front of her. Between each philosopher sits a single chopstick. The philosophers wish to alternate between thinking (which they can do without any chopsticks), and eating (a task for which a philosopher requires two chopsticks). The problem is to define a pattern of behavior for the philosophers that ensures they can all alternate between thinking and eating without experiencing deadlock, livelock or starvation. (Obviously the philosophers represent threads, and the chopsticks represent mutex locks.)

You can read about this well-known problem online, so we won't devote a lot of space to discussing it here. However, it will be instructive to consider a few of the most common solutions to the problem:

- *Global order.* A dependency cycle can occur if a philosopher were always to pick up the chopstick to her left first (or always the one to her right). One solution to this problem is to assign each chopstick a unique global index. Whenever a philosopher wishes to eat, he or she always picks up the chopstick with the lowest index first. This prevents the dependency cycle, and hence avoids deadlock.
- *Central arbitor.* In this solution, a central arbitor (the "waiter") either grants a philosopher two chopsticks or none. This avoids the hold and wait problem by ensuring that no philosopher ever gets into a situation in which he or she holds only one chopstick, thereby preventing deadlock.
- *Chandra-Misra.* In this solution, chopsticks are marked either dirty or clean, and the philosophers send each other messages requesting chopsticks. You can read more about this solution by searching for "chandy-misra" online.

- $N - 1$ *philosophers*. For a table with N philosophers and N chopsticks, an integer semaphore can be used to limit the number of philosophers who are permitted to pick up chopsticks at any given time to $N - 1$. This solves the deadlock and livelock problems, because even in degenerate situations, at least one philosopher will succeed in obtaining two chopsticks. It does, however, permit one of the philosophers to starve, unless an additional “fairness” criterion is introduced.

4.8 Some Rules of Thumb for Concurrency

The solutions to the dining philosophers problem we described in the preceding section hint at some general principles and rules of thumb which we can apply to virtually any concurrent programming problem. Let’s take a brief look at a few of them.

4.8.1 Global Ordering Rules

In a concurrent program, the *order* in which events occur is not dictated by the order of the instructions in the program, as it is in a single-threaded program. If ordering is required in a concurrent system, that ordering must be imposed *globally*, across all threads.

This is one reason why a doubly linked list is not a concurrent data structure. A doubly linked list is designed the way it is in order to provide fast ($O(N)$) insertion and deletion of elements at any location within the list. Underlying this design is an assumption that *program order* is equivalent to *data order*—that when an ordered sequence of operations is performed on a list, the resulting list will contain correspondingly ordered elements. For example, let’s say we are given a linked list that contains the ordered elements { A, B, C }. If we execute the following two operations:

1. insert D before C,
2. insert E before C,

the assumption is that the resulting list will contain the ordered elements { A, B, D, E, C }.

In a single-threaded program, this assumption holds true. But in a multi-threaded system, program order no longer dictates data order. If one thread performs the insertion of D before C, and another thread performs the insertion of E before C, we have a race condition that could lead to any of the following results:

- { A, B, D, E, C },
- { A, B, E, D, C }, or
- { A, B, *corrupted data* }.

A corrupted list can occur if the data structure's operations aren't properly protected with critical sections. For example, both D's and E's "next" pointers might end up pointing to C.

A global ordering rule is the only viable solution to this problem. We first need to ask ourselves why the order of the elements in the list matters, and if it even matters at all. If order is unimportant, we can use a singly-linked list and always append the elements—an operation that can be implemented reliably either with locks or in a lock-free manner. And if a global ordering *is* required, we need to identify a stable and deterministic ordering criterion that does not depend on the order in which program events happen to occur. For example, we might sort the list alphabetically, or by priority, or some other useful criterion. The answers to these questions in turn dictate what kind of data structure we'll want to use. Attempting to use a doubly-linked list in a concurrent way (i.e., giving multiple threads mutable access to the list) is like trying to fit a square peg into a round hole.

4.8.2 Transaction-Based Algorithms

In the central arbiter solution to the dining philosophers problem, the arbiter or "waiter" hands out chopsticks in pairs: Either a philosopher receives all of the resources he needs (two chopsticks), or he receives none of them. This is known as a *transaction*.

A transaction can be more precisely defined as an indivisible bundle of resources and/or operations. Threads in a concurrent system submit transaction requests to a central arbiter of some kind. A transaction either succeeds in its entirety, or it fails in its entirety (because some other thread's transaction is actively being processed when the request arrives). If the transaction fails, its thread keeps resubmitting the transaction request until it succeeds (possibly waiting for a short time between retries).

Transaction-based algorithms are common in concurrent and distributed systems programming. And as we'll see in Section 4.9, the concept of a transaction underlies most *lock-free* data structures and algorithms.

4.8.3 Minimizing Contention

The most efficient concurrent system would be one in which all threads run without ever having to wait for a lock. This ideal can never be fully achieved,

of course, but concurrent systems programmers do attempt to minimize the amount of *contention* between threads.

As an example, consider a group of threads that are producing data and storing it into a central repository. Every time one of these threads attempts to store its data in the repository, it contends with all of the other threads for this shared resource. A simple solution that can sometimes work is to give each thread its own private repository. The threads can now produce data independently of one another, with no contention. When all of the threads are done producing their output, a central thread can collate the results.

The analog to this approach in the case of the dining philosophers problem would be to give each philosopher two chopsticks from the outset. Doing so would of course remove all concurrency from the problem—without any shared resources, there is no concurrency. In real-world concurrent systems we can't remove *all* sharing of resources, but we can certainly look for ways to *minimize* resource sharing in order to minimize lock contention.

4.8.4 Thread Safety

Generally speaking, a class or functional API is called *thread-safe* when its functions can be safely called by any thread in a multithreaded process. For any one function, thread safety is typically achieved by entering a critical section at the top of the function's body, performing some work, and then leaving the critical section just prior to returning. A class whose functions are all thread-safe is sometimes called a *monitor*. (The term "monitor" is also used to refer to a class that uses condition variables internally to permit clients to sleep while waiting for its protected resources to become available.)

Thread-safety is a convenient feature for a class or interface to offer. But it also imposes overhead that is sometimes unnecessary. For example, this overhead would be wasteful if the interface is being used in a single-threaded program, or if it's being used exclusively by one thread in a multithreaded program.

Thread-safety can also become a problem when an interface function needs to be called reentrantly. For example, if a class provides two thread-safe functions `A()` and `B()`, then these functions cannot call one another because each one independently enters and leaves the critical section. One solution to this problem is to use reentrant locks (see Section 4.9.7.3). Another is to implement "unsafe" versions of the functions in an interface, and then "wrap" each of them in thread-safe variant that simply enters a critical section, calls the "unsafe" function, and then leaves the critical section. That way, we can call the "unsafe" functions internally to the system, but the external interface of the

system remains thread-safe.

In my view, it's a bad idea to attempt to handle concurrent programming by simply creating classes and APIs that are 100% thread-safe. Doing so leads to interfaces that are unnecessarily heavy-weight, and it encourages programmers to ignore the fact that they are working in a concurrent environment. Instead, we should recognize and embrace the presence of concurrency in our software, and aim to design data structures and algorithms that address this concurrency explicitly. The goal should be to produce a software system that minimizes contention and dependencies between threads, and minimizes the use of locks. Achieving a completely lock-free system is a lot of work and is tough to get right. (See Section 4.9.) But striving in the *direction* of lock-freedom (i.e., minimizing the use of locks) is a far better strategy than over-using locks in a futile attempt to create “water-tight” interfaces that allow programmers to be oblivious of concurrency.

4.9 Lock-Free Concurrency

Thus far, all of our solutions to the problem of race conditions in a concurrent system have revolved around using *mutex locks* to make critical operations atomic, and possibly leveraging *condition variables* and the kernel's ability to *put threads to sleep* to avoid them wasting valuable CPU cycles in a busy-wait loop. During that discussion, we mentioned that there's another way to avoid race conditions that can potentially be more efficient. That approach is known as *lock-free* concurrency.

Contrary to popular belief, the term “lock-free” doesn't actually refer to the elimination of mutex locks, although that is certainly a component of the approach. In fact, “lock-free” refers to the practice of preventing threads from going to sleep while waiting on a resource to become available. In other words, in lock-free programming we never allow a thread to *block*. So perhaps the term “blocking-free” would have been more descriptive.

Lock-free programming is actually just one of a collection of *non-blocking* concurrent programming techniques. When a thread is blocked, it ceases to make progress. The goal of all of these techniques is to provide guarantees about the *progress* that can be made by the threads in the system, and by the system as a whole. We can organize these techniques into the following categories, listed in order of increasing strength of the progress guarantees they provide:

- *Blocking*. A blocking algorithm is one in which a thread can be put to sleep while waiting for shared resources to become available. A block-

ing algorithm is prone to deadlock, livelock, starvation and priority inversion.

- *Obstruction freedom.* An algorithm is obstruction-free if we can guarantee that a single thread will always complete its work in a bounded number of steps, when all of the other threads in the system are suddenly suspended. The single thread that continues to run is said to be performing a *solo execution* in this scenario, and an obstruction-free algorithm is sometimes called *solo-terminating* because the solo thread eventually terminates while all other threads are suspended. No algorithm that uses a mutex lock or spin lock can be obstruction-free, because if any thread were to be suspended while it is holding a lock, the solo thread might become stuck forever waiting for that lock.
- *Lock freedom.* The technical definition of lock freedom is that in any infinitely-long run of the program, an infinite number of operations will be completed. Intuitively, a lock-free algorithm guarantees that *some* thread in the system can always make progress; in other words, if one thread is arbitrarily suspended, all others can still make progress. This again precludes the use of mutex or spin locks, because if a thread holding a lock is suspended, it can cause other threads to block. A lock-free algorithm is typically transaction-based: A transaction may fail if another thread interrupts it, in which case the transaction is rolled back and retried until it succeeds. This approach avoids deadlock, but it can allow some threads to starve. In other words, certain threads might get stuck in a loop of failing and retrying their transactions indefinitely, while other threads' transactions always succeed.
- *Wait freedom.* A wait-free algorithm provides all the guarantees of lock freedom, but also guarantees starvation freedom. In other words, all threads can make progress, and no thread is allowed to starve indefinitely.

The term “lock-free programming” is sometimes used loosely to refer to *any* algorithm that avoids blocking, but technically speaking the correct term for obstruction-free, lock-free and wait-free algorithms as a whole is “*non-blocking algorithm*.”

The topic of non-blocking algorithms is vast, and is still an open area of research. A complete discussion of the topic would require an entire book of its own. In this chapter, we'll introduce some of the basic principles of lock-free programming. We'll begin by exploring the true causes of data race bugs. Then we'll have a look at how mutexes are actually implemented under the

hood, and learn how to implement our own inexpensive *spin locks*. Finally, we'll present an implementation of a simple lock-free linked list. This discussion ought to be sufficient to give you a flavor for what lock-free data structures and algorithms tend to look like, and should provide a solid jumping off point for further reading and experimentation with lock-free techniques.

4.9.1 Causes of Data Race Bugs

In Section 4.5.3.2, we said that data race bugs occur when a critical operation is interrupted by another critical operation on the same shared data. As it turns out, there are two other ways in which data race bugs can arise, and if we're going to implement our own spin locks or write lock-free algorithms, we'll need to understand them all. Data race bugs can be introduced into a concurrent program:

- via the *interruption* of one critical operation by another,
- by the *instruction reordering* optimizations performed by the compiler and CPU, and
- as a result of hardware-specific *memory ordering semantics*.

Let's break these down a bit further:

- Threads interrupt one another all the time as a result of preemptive multitasking and/or by running them on multiple cores. However, when a *critical* operation is interrupted by another critical operation on the *same* shared data object, a data race bug can occur.
- Optimizing compilers often reorder instructions in an attempt to minimize pipeline stalls. Likewise, the out-of-order execution logic within the CPU can cause instructions to be executed in an order that differs from program order. Instruction reordering is guaranteed not to alter the observable behavior of a single-threaded program. But it *can* alter the way in which two or more threads *cooperate* to share data, thereby introducing bugs into a concurrent program.
- Thanks to aggressive optimizations within a computer's memory controller, the *effects* of a read or write instruction can sometimes become *delayed* relative to other reads and/or writes in the system. As with compiler optimizations and OOO execution, these memory controller optimizations are designed never to alter the observable behavior of a single-threaded program. However, these optimizations *can* change the order of a critical pair of reads and/or writes in a concurrent system,

thereby preventing threads from sharing data in a predictable way. In this book, we'll refer to these kinds of concurrency bugs as *memory ordering bugs*.

In order to guarantee that a critical operation is free from data races, and is therefore atomic, we must ensure that none of these three things can happen.

4.9.2 Implementing Atomicity

First, let's tackle the problem of making a critical operation atomic (i.e., uninterruptible). Before, we waved our hands a bit and said that by wrapping our critical operations in a *mutex* lock/unlock pair, we can magically transform them into atomic operations. But how does a mutex actually work?

4.9.2.1 Atomicity by Disabling Interrupts

To prevent other threads from interrupting our operation, we could try disabling interrupts just prior to performing the operation, making sure to reenabling them after the operation has been completed. That would certainly prevent the kernel from context-switching to another thread in the middle of our atomic operation. However, this approach only works on a single-core machine using preemptive multitasking.

Interrupts are disabled by executing a machine language instruction (such as `cli`, "clear interrupt enable bit," on an Intel x86 architecture). But this kind of instruction only affects the core that executed it. The other cores would continue to run their threads (with interrupts, and therefore preemptive multithreading, still enabled), and those threads could still interrupt our operation. So this approach has only limited applicability in the real world.

4.9.2.2 Atomic Instructions

The term "atomic" suggests the notion of breaking an operation down into smaller and smaller pieces, until we reach a granularity that is indivisible. This idea raises the question: Are there any machine language instructions that are *naturally* atomic? In other words, does the CPU guarantee that some instructions are uninterruptible?

The answer to these questions is "yes," but with a few caveats. There are most certainly some machine language instructions that can *never* be assumed to execute atomically. Other instructions are atomic, but only when operating on certain kinds of data. Some CPUs permit virtually any instruction to be *forced* to execute atomically by specifying a prefix on the instruction in assembly language. (The Intel x86 ISA's `lock` prefix is one example.)

This is good news for concurrent programmers. In fact, it is the existence of these *atomic instructions* that permits us to implement atomicity tools such as mutexes and spin locks, which in turn permit us to make larger-scale operations atomic.

Different CPUs and ISAs provide different sets of atomic instructions, governed by different rules. But we can generalize all atomic instructions as falling into two categories:

- atomic reads and writes, and
- atomic read-modify-write (RMW) instructions.

4.9.2.3 Atomic Reads and Writes

On most CPUs, we can be reasonably certain that a read or write of a four-byte-aligned 32-bit integer will be atomic. That being said, every processor is different, so it's important to always consult your CPU's ISA documentation before relying on the atomicity of any particular instruction.

Some CPUs also support atomic reads and writes of smaller or larger objects, such as single bytes or 64-bit integers, again assuming they are aligned to a multiple of their own size. This is true because on most CPUs, reading and writing an aligned integer whose width in bits is equal to or smaller than the width of a register (or sometimes the width of a cache line) can be performed in a single *memory access cycle*. Because a CPU performs its operation in lock-step with a discrete clock, a memory cycle can't be interrupted, even by another core. As a result, the read or write is effectively atomic.

Misaligned reads and writes usually don't have this atomicity property. This is because in order to read or write a misaligned object, the CPU usually composes two aligned memory accesses. As such, the read or write might be interrupted, and we cannot assume it will be atomic. (See Section 3.3.7.1 for more details on alignment.)

4.9.2.4 Atomic Read-Modify-Write

Atomic reads and writes aren't enough to implement atomic operations in a general sense. In order to implement a locking mechanism like a mutex, we need to be able to read a variable's contents from memory, perform some operation on that variable, and then write the results back to memory, all without being interrupted.

All modern CPUs support concurrency by providing at least one atomic read-modify-write (RMW) instruction. In the following sections, we'll have a look at a few different kinds of atomic RMW instructions. Each has its pros and cons. All of them can be used to implement a mutex or spin lock.

4.9.2.5 Test and Set

The simplest RMW instruction is known as *test-and-set* (TAS). The TAS instruction doesn't actually test and set a value. Rather, it atomically sets a Boolean variable to 1 (true) and returns its *previous* value (so that the value can be tested).

```
// pseudocode for the test-and-set instruction
bool TAS(bool* pLock)
{
    // atomically...
    const bool old = *pLock;
    *pLock = true;
    return old;
}
```

The test-and-set instruction can be used to implement a simple kind of lock called a *spin lock*. Here's some pseudocode illustrating the idea. In this example, we use a hypothetical *compiler intrinsic* called `_tas()` to emit a TAS machine language instruction into our code. Different compilers will provide different intrinsics for this instruction, if the target CPU supports it. For example, under Visual Studio the TAS intrinsic is named `_interlockedbittestandset()`.

```
void SpinLockTAS(bool* pLock)
{
    while (_tas(pLock) == true)
    {
        // someone else has lock -- busy-wait...
        PAUSE();
    }

    // when we get here, we know that we successfully
    // stored a value of true into *pLock AND that it
    // previously contained false, so no one else has
    // the lock -- we're done
}
```

Here, the `PAUSE()` macro indicates the use of a compiler intrinsic, such as Intel's `SSE2 _mm_pause()`, to reduce power consumption during the busy-wait loop. See Section 4.4.6.4 for details on why it's advisable to use a pause instruction inside a busy-wait loop where possible.

It's important to stress here that the above example is intended for illustrative purposes only. It is not 100% correct because it lacks proper memory

fencing. We'll present a complete working example of a spin lock in Section 4.9.7.

4.9.2.6 Exchange

Some ISAs like Intel x86 offer an atomic *exchange* instruction. This instruction swaps the contents of two registers, or a register and a location in memory. On x86, it is atomic by default when exchanging a register with memory (meaning it acts as if the instruction had been preceded with the `lock` prefix).

Here's how to implement a spin lock using an atomic *exchange* instruction. In this example, we're using Visual Studio's `_InterlockedExchange()` *compiler intrinsic* to emit the Intel x86 `xchg` instruction into our code. (Again, note that without proper memory fencing this example is incomplete and won't work reliably. See Section 4.9.7 for a complete implementation.)

```
void SpinLockXCHG(bool* pLock)
{
    bool old = true;
    while (true)
    {
        // emit the xchg instruction for 8-bit words
        _InterlockedExchange8(old, pLock);
        if (!old)
        {
            // if we get back false,
            // then the lock succeeded
            break;
        }
        PAUSE();
    }
}
```

Under Microsoft Visual Studio, all of the “interlocked” functions named with a leading underscore are *compiler intrinsics*—they simply emit the appropriate assembly language instruction directly into your code. The Windows SDK provides a set of similarly-named functions *without* the leading underscore—those functions are implemented in terms of the intrinsic where possible, but they're much more expensive because they involve making a kernel call.

4.9.2.7 Compare and Swap

Some CPUs provide an instruction known as *compare-and-swap* (CAS). This instruction checks the existing value in a memory location, and atomically swaps it with a new value *if and only if* the existing value matches an expected value

provided by the programmer. It returns true if the operation succeeded, meaning that the memory location contained the expected value. It returns false if the operation failed because the location's contents were not as expected.

CAS can operate on values larger than a Boolean. Variants are typically provided at least for 32- and 64-bit integers, although smaller word sizes may also be supported.

The behavior of the CAS instruction is illustrated by the following pseudocode:

```
// pseudocode for compare and swap
bool CAS(int* pValue, int expectedValue, int newValue)
{
    // atomically...
    if (*pValue == expectedValue)
    {
        *pValue = newValue;
        return true;
    }
    return false;
}
```

To implement any atomic read-modify-write operation using CAS, we generally apply the following strategy:

1. Read the old value of the variable we're attempting to update.
2. Modify the value in whatever way we see fit.
3. When writing the result, use the CAS instruction instead of a regular write.
4. Iterate until the CAS succeeds.

The CAS instruction allows us to detect data races, by comparing the value that's actually in the memory location at the time of the write with its value *before* the invocation of our read-modify-write operation. In the absence of a race, the CAS instruction acts just like a write instruction. But if the value in memory changes between the read and the write, we know that some other thread beat us to the punch. In that case, we back off and try again.

Implementing a spin lock via compare-and-swap would look something like this, again using a hypothetical compiler intrinsic called `_cas()` to emit the CAS instruction. (This example once again omits the memory fences that would be required to make it work reliably on all hardware—see Section 4.9.7 for a fully-functional spin lock.)

```

void SpinLockCAS(int* pValue)
{
    const int kLockValue = -1; // 0xFFFFFFFF
    while (!_cas(pValue, 0, kLockValue))
    {
        // must be locked by someone else -- retry
        PAUSE();
    }
}

```

And here's how we'd implement an atomic increment using CAS.

```

void AtomicIncrementCAS(int* pValue)
{
    while (true)
    {
        const int oldValue = *pValue; // atomic read
        const int newValue = oldValue + 1;
        if (_cas(pValue, oldValue, newValue))
        {
            break; // success!
        }
        PAUSE();
    }
}

```

On the Intel x86 ISA, the CAS instruction is called `cmpxchg`, and it can be emitted with Visual Studio's `_InterlockedCompareExchange()` compiler intrinsic.

4.9.2.8 ABA Problem

We should mention that the CAS instruction suffers from an inability to detect one specific kind of data race. Consider an atomic RMW write operation in which the read sees the value A. Before we're able to issue the CAS instruction, another thread preempts us, or runs on another core, and writes two values into the location we're trying to atomically update: First it writes the value B, and then it writes the value A again. When our CAS instruction does finally execute, it won't be able to tell the difference between the first A it read and the A that was written by the other thread. Hence it will "think" that no data race occurred, when in fact one did. This is known as the *ABA problem*.

4.9.2.9 Load Linked/Store Conditional

Some CPUs break the compare-and-swap operation into a *pair* of instructions known as *load linked* and *store conditional* (LL/SC). The *load linked* instruction

reads the value of a memory location atomically, and also stores the address in a special CPU register known as the *link register*. The *store conditional* instruction writes a value into the given address, but only if the address matches the contents of the link register. It returns true if the write succeeded, or false if it failed.

Any write operation on the bus (including a store conditional) clears the link register to zero. This means that an LL/SC instruction pair is capable of detecting data races, because if any write occurs between the LL and SC instructions, the SC will fail.

An LL/SC instruction pair is used in much the same way a regular read is paired with a CAS instruction. Specifically, an atomic read-modify-write operation would be implemented using the following strategy:

1. Read the old value of the variable via an LL instruction.
2. Modify the value in whatever way we see fit.
3. Write the result using the SC instruction.
4. Iterate until the SC succeeds.

Here's how we'd implement an atomic increment using LL/SC (using the hypothetical compiler intrinsics `_ll()` and `_sc()` to emit the LL and SC instructions, respectively):

```
void AtomicIncrementLLSC(int* pValue)
{
    while (true)
    {
        const int oldValue = _ll(*pValue);
        const int newValue = oldValue + 1;
        if (_sc(pValue, newValue))
        {
            break; // success!
        }
        PAUSE();
    }
}
```

Because the link register is cleared by *any* bus write, the SC instruction may fail spuriously. But that doesn't affect the correctness of an atomic RMW implemented with LL/SC—it just means that our busy-wait loop might end up executing a few more iterations than we'd expect.

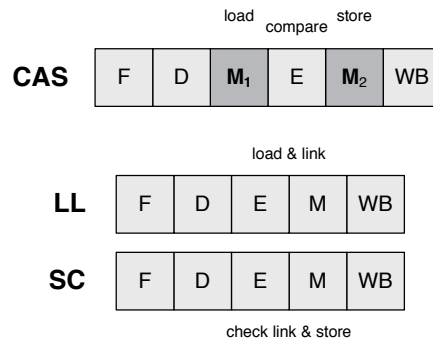


Figure 4.35. The compare and swap (CAS) instruction reads a memory location, performs a comparison, and then conditionally writes to that same location. It therefore requires two memory access stages, making it more difficult to implement within a simple pipelined CPU architecture than the linked/store conditional (LL/SC) instruction pair, each of which requires only a single memory access stage.

4.9.2.10 Advantages of LL/SC over CAS

The LL/SC instruction pair offers two distinct advantages over the single CAS instruction.

First, because the SC instruction fails whenever *any* write is performed on the bus, an LL/SC pair is not prone to the ABA problem.

Second, an LL/SC pair is more pipeline-friendly than a CAS instruction. The simplest pipeline is comprised of five stages: fetch, decode, execute, memory access and register write-back. But a CAS instruction requires two memory access cycles: One to read the memory location so it can be compared to the expected value, and one to write the result if the comparison passes. This means that a pipeline that supports CAS has to include an additional memory access stage that goes unused most of the time. On the other hand, the LL and SC instructions each only require a single memory access cycle, so they fit more naturally into a pipeline with only one memory access stage. A comparison of CAS and LL/SC from a pipelining perspective is shown in Figure 4.35.

4.9.2.11 Strong and Weak Compare-Exchange

The C++11 standard library provides portable functions for performing atomic compare-exchange operations. These may be implemented by a CAS instruction on some target hardware, or by an LL/SC instruction pair on other hardware. Because of the possibility of spurious failures of the store-conditional instruction, C++11 provides two varieties of compare-exchange: strong and weak. Strong compare-exchange “hides” spurious SC failures from the pro-

grammer, while weak compare-exchange does not. Search for “Strong Compare and Exchange Lawrence Crowl” online for a paper on the rationale behind strong and weak compare-exchange functions in C++11.

4.9.2.12 Relative Strength of Atomic RMW Instructions

It’s interesting to note that the TAS instruction is *weaker* than the CAS and LL/SC instructions in terms of achieving *consensus* between multiple threads in a concurrent system. Consensus in this context refers to an agreement between threads about the value of a shared variable (even if some threads in the system experience a failure).

Because the TAS instruction only operates on a Boolean value, it can only solve a problem known as the *wait-free consensus problem* for two concurrent threads. The CAS instruction operates on a 32-bit value, so it can solve this problem for any number of threads.

The topic of wait-free consensus is well beyond our scope here; it’s mostly of interest when building fault-tolerant systems. If you’re interested in fault tolerance, you can read more about the consensus problem by searching for “consensus (computer science)” on Wikipedia.

4.9.3 Barriers

Interruptions aren’t the only cause of data race bugs. Compilers and CPUs also conspire to introduce subtle bugs into our concurrent programs by means of the *instruction reordering* optimizations they perform, as described in Section 4.2.5.1.

The cardinal rule of compiler optimizations and out-of-order execution is that their optimizations shall have no visible effects on the behavior of *a single thread*. However, neither the compiler nor the CPU’s control logic has any knowledge of what other threads might be running in the system, or what they might be doing. As a result, the cardinal rule isn’t sufficient to prevent instruction reordering optimizations from introducing bugs into concurrent programs.

The thread synchronization primitives provided by the operating system (mutexes et al.) are carefully crafted to avoid the concurrency bugs that can be caused by instruction reordering optimizations. But now that we’re investigating how mutexes are implemented, let’s take a look at how to avoid these problems manually.

4.9.3.1 How Instruction Reordering Causes Concurrency Bugs

As an example of the kinds of problems instruction reordering can cause in concurrent software, consider again the producer-consumer problem from Section 4.6.3. We've simplified the example, and removed the mutexes so we can expose the bugs that can be introduced by instruction reordering.

```
int32_t g_data = 0;
int32_t g_ready = 0;

void ProducerThread()
{
    // produce some data
    g_data = 42;

    // inform the consumer
    g_ready = 1;
}

void ConsumerThread()
{
    // wait for the data to be ready
    while (!g_ready)
        PAUSE();

    // consume the data
    ASSERT(g_data == 42);
}
```

On a CPU on which aligned reads and writes of 32-bit integers are atomic, this example doesn't actually require mutexes. However, there's nothing to prevent the compiler or the CPU's out-of-order execution logic from reordering the producer's write of 1 into `g_ready` so that it occurs *before* the write of 42 into `g_data`. Likewise, in theory the compiler could reorder the consumer's check that `g_data` is equal to 42 so that it happens *before* the while loop. So even though all of our reads and writes are atomic, this code may not behave reliably.

Instruction reordering really happens at the assembly language level, so it may be a lot more subtle than a reordering of the *statements* in a C/C++ program. For example, the following C/C++ code:

```
A = B + 1;
B = 0;
```

would produce the following Intel x86 assembly code:

```
mov    eax, [B]
add    eax, 1
mov    [A], eax
mov    [B], 0
```

The compiler could very well reorder the instructions as follows, without producing any noticeable effect in a single-threaded execution:

```
mov    eax, [B]
mov    [B], 0    ;; write to B before A!
add    eax, 1
mov    [A], eax
```

If a second thread were waiting for B to become zero before reading the value of A, it would cease to function correctly if this compiler optimization were to be applied.

Jeff Preshing wrote a great blog post on this topic, available at <http://preshing.com/20120625/memory-ordering-at-compile-time/>. (This is where the assembly language example above comes from.) I highly recommend all of Jeff's posts on concurrent programming, so do be sure to check them out.

4.9.3.2 Volatile in C/C++ (and Why It Doesn't Help Us)

How can we prevent the compiler from reordering critical sequences of reads and writes? In C and C++, the `volatile` type qualifier guarantees that consecutive reads or writes of a variable cannot be “optimized away” by the compiler, so this sounds like a promising idea. However, it doesn't work reliably for a number of reasons.

The `volatile` qualifier in C/C++ was really designed to make memory-mapped I/O and signal handlers work reliably. As such, the only guarantee it provides is that the contents of a variable marked `volatile` won't be cached in a register—the variable's value will be re-read directly from memory every time it's accessed. Some compilers do guarantee that instructions will not be reordered across a read or write of a `volatile` variable, but not all of them do, and some only provide this guarantee when targeting certain CPUs, or only when a particular command-line option is passed to the compiler. The C and C++ standards don't require this behavior, so we certainly cannot rely on it when writing portable code. (See <https://msdn.microsoft.com/en-us/magazine/dn973015.aspx> for an in-depth discussion of this topic.)

Moreover, the `volatile` keyword in C/C++ does nothing to prevent the CPU's out-of-order execution logic from reordering the instructions at run-time. And it cannot prevent cache coherency related issues either (see Section 4.9.3). So, at least in C and C++, the `volatile` keyword won't help us to write reliable concurrent software.⁸

4.9.3.3 Compiler Barriers

One reliable way of preventing the compiler from reordering read and write instructions across critical operation boundaries is to explicitly instruct it not to do so. This can be accomplished by inserting a special pseudoinstruction into our code known as a *compiler barrier*.

Different compilers express barriers with different syntax. With GCC, a compiler barrier can be inserted via some inline assembly syntax as shown below; under Microsoft Visual C++, the compiler intrinsic `_ReadWriteBarrier()` has the same effect.

```
int32_t  g_data = 0;
int32_t  g_ready = 0;

void ProducerThread()
{
    // produce some data
    g_data = 42;

    // dear compiler: please don't reorder
    // instructions across this barrier!
    asm volatile("" ::: "memory")

    // inform the consumer
    g_ready = 1;
}

void ConsumerThread()
{
    // wait for the data to be ready
    while (!g_ready)
        PAUSE();

    // dear compiler: please don't reorder
    // instructions across this barrier!
    asm volatile("" ::: "memory")
```

⁸In some languages including Java and C#, the `volatile` type qualifier *does* guarantee atomicity, and can be used to implement concurrent data structures and algorithms. See Section 4.9.6 for more on this topic.

```
// consume the data
ASSERT(g_data == 42);
}
```

There are other ways to prevent the compiler from reordering instructions. For example, most function calls serve as an implicit compiler barrier. This makes sense, because the compiler doesn't know anything⁹ about the side effects of a function call. As such, it can't assume that the state of memory will be the same before and after the call, which means most optimizations aren't permitted across a function call. Some optimizing compilers do make an exception to this rule for inline functions.

Unfortunately, compiler barriers don't prevent the CPU's out-of-order execution logic from reordering instructions at runtime. Some ISAs provide a special instruction for this purpose (e.g., PowerPC's `isync` instruction). In Section 4.9.4.5, we'll learn about a collection of machine language instructions known as *memory fences* which serve as instruction reordering barriers for both the compiler and the CPU, but more importantly also prevent *memory reordering* bugs. So atomic instructions and fences are all we really need to write reliable mutexes, as well as spin locks and other *lock-free* algorithms.

4.9.4 Memory Ordering Semantics

In Section 4.9.1, we said that in addition to the compiler or CPU actually reordering the machine language instructions in our programs, it's possible for read and write instructions to become *effectively reordered* in a concurrent system. Specifically, in a multicore machine with a multilevel memory cache, two or more cores can sometimes *disagree* about the apparent order in which a sequence of read and write instructions occurred, even when the instructions were *actually* executed in the order we intended. Obviously, such disagreements can cause subtle bugs in concurrent software.

These mysterious and vexing problems can only occur on a multicore machine with a multilevel cache. A single CPU core always "sees" the effects of its own read and write instructions in the order they were executed; only when there are two or more cores can disagreements arise. What's more, different CPUs have different memory ordering behavior, meaning that these

⁹Function calls only serve as implicit barriers when the compiler is unable to "see" the definition of the function, such as when the function is defined in a separate translation unit. Link time optimizations (LTO) can introduce concurrency bugs by providing a way for the compiler's optimizer to see the definitions of functions it otherwise could not have seen, thereby effectively eliminating these implicit barriers.

strange effects can differ from machine to machine when running the exact same source program!

Thankfully, all is not lost. Every CPU is governed by a strict set of rules known as its *memory ordering semantics*. These rules provide various guarantees about how reads and writes propagate between cores, and they provide programmers with the tools necessary to enforce a particular ordering, when the default semantics are insufficient.

Some CPUs offer only weak guarantees by default, while others provide stronger guarantees and hence require less intervention on the part of the programmer. So, if we can understand how to overcome memory ordering issues on a CPU with the weakest memory ordering semantics, we can be pretty sure those techniques will also work on CPUs with stronger default semantics.

4.9.4.1 Memory Caching Revisited

In order to understand how these mysterious memory reordering effects can occur, we need to look more closely at how a multilevel memory cache works.

In Section 3.5.4, we described in detail how a memory cache avoids the very high memory access latency of main RAM by keeping frequently-used data in the cache. This means that as long as a data object is present in the cache, the CPU will always try to work with that cached copy, rather than reaching out to access the copy in main RAM.

Let's briefly review how this works by considering the following simple (and totally contrived) function:

```
constexpr int      COUNT = 16;
alignas(64) float  g_values[COUNT];
float              g_sum = 0.0f;

void CalculateSum()
{
    g_sum = 0.0f;
    for (int i = 0; i < COUNT; ++i)
    {
        g_sum += g_values[i];
    }
}
```

The first statement sets `g_sum` to zero. Presuming that the contents of `g_sum` aren't already in the L1 cache, the cache line containing it will be read into L1 at this point. Likewise, on the first iteration of the loop, the cache line containing all of the elements of the `g_values` array will be loaded into L1. (They should all fit presuming our cache lines are at least 64 bytes wide, because we aligned

the array to a 64-byte boundary with C++11's `alignas` specifier.) Subsequent iterations will read the copy of `g_values` that resides in the L1 cache, rather than reading them from main RAM.

During each iteration, `g_sum` is updated. The compiler might optimize this by keeping the sum in a register until the end of the loop. But whether or not this optimization is performed, we know that the `g_sum` variable will be written to at some point during this function. When that happens, again the CPU will write to the *copy* of `g_sum` that exists in the L1 cache, rather than writing directly to main RAM.

Eventually, of course, the “master” copy of `g_sum` has to be updated. The memory cache hardware does this automatically by triggering a *write-back* operation that copies the cache line from L1 back to main RAM. But the write-back doesn't usually happen right away; it's typically deferred until the modified variable is read again.¹⁰

4.9.4.2 Multicore Cache Coherency Protocols

In a multicore machine, memory caching gets a lot more complicated. Figure 4.36 illustrates a simple dual-core machine, in which each core has its own private L1 cache, and the two cores share an L2 cache and a large bank of main RAM. To keep the following discussion as simple as possible, let's ignore the L2 cache and treat it as being roughly equivalent to main RAM.

Suppose that the simplified producer-consumer example shown in Section 4.9.3.3 is running on this dual-core machine. The producer thread runs on Core 1, and the consumer thread runs on Core 2. Let's further assume for the purposes of this discussion that none of the instructions in either thread have been reordered.

```
int32_t  g_data = 0;
int32_t  g_ready = 0;

void ProducerThread() // running on Core 1
{
    g_data = 42;
    // assume no instruction reordering across this line
    g_ready = 1;
}
```

¹⁰Some memory cache hardware does allow cached writes to *write-through* to main RAM immediately. We can safely ignore write-through caches for the purposes of the present discussion.

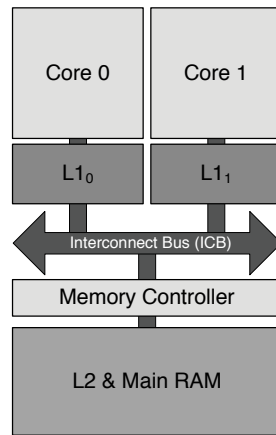


Figure 4.36. A dual core machine with a local L1 cache per core, connected to a memory controller via the interconnect bus (ICB). The memory controller implements a cache coherency protocol such as MESI to ensure that both cores have a consistent view of the contents of memory within the CPU's cache coherency domain.

```

void ConsumerThread() // running on Core 2
{
    while (!g_ready)
        PAUSE();
    // assume no instruction reordering across this line
    ASSERT(g_data == 42);
}
  
```

Now consider what happens when the producer (on Core 1) writes to `g_ready`. In the name of efficiency, this write causes Core 1's L1 cache to be updated, but it won't trigger a *write-back* to main RAM until some time later. This means that for some finite amount of time after the write has occurred, the most up-to-date value of `g_ready` don't exist anywhere but inside Core 1's L1 cache.

Let's say that the consumer (running on Core 2) attempts to read `g_ready` at some time after the producer set it to 1. Like Core 1, Core 2 prefers to read from the cache whenever possible, to avoid the high cost of reading main RAM. Core 2's local L1 cache does not contain a copy of `g_ready`, but Core 1's L1 cache *does*. So ideally Core 2 would like to ask Core 1 for its copy, because that would still be a lot less expensive than getting the data from main RAM. And in this particular case, doing so would also have the distinct advantage of returning the most up-to-date value.

A *cache coherency protocol* is a communication mechanism that permits cores to share data between their local L1 caches in this way. Most CPUs use either the MESI or MOESI protocol.

4.9.4.3 The MESI Protocol

Under the MESI protocol, each cache line can be in one of four states:

- **Modified.** This cache line has been modified (written to) locally.
- **Exclusive.** The main RAM memory block corresponding to this cache line exists *only* in this core's L1 cache—no other core has a copy of it.
- **Shared.** The main RAM memory block corresponding to this cache line exists in *more than one* core's L1 cache, and all cores have an identical copy of it.
- **Invalid.** This cache line no longer contains valid data—the next read will need to obtain the line either from another core's L1 cache, or from main RAM.

The MOESI protocol adds another state named **Owned**, which allows cores to share modified data without writing it back to main RAM first. We'll focus on MESI here in the name of simplicity.

Under the MESI protocol, all cores' L1 caches are connected via a special bus called the *interconnect bus* (ICB). Collectively, the L1 caches, any higher-level caches, and main RAM form what is known as a *cache coherency domain*. The protocol ensures that all cores have a consistent "view" of the data in this domain.

We can get a feel for how the MESI state machine works by returning to our example.

- Let's assume that Core 1 (the producer) first tries to read the current value of `g_ready` for some reason. Presuming that this variable doesn't already exist in any core's L1 cache, the cache line that contains it is loaded into Core 1's L1 cache. The cache line is put into the **Exclusive** state, meaning that no other core has this line.
- Now let's say that Core 2 (the consumer) attempts to read `g_ready`. A *Read* message is sent over the ICB. Core 1 has this cache line, so it responds with a copy of the data. At this point, the cache line is put into the **Shared** state on both cores, indicating that both have an identical copy of the line.
- Next, the producer on Core 1 writes a 1 into `g_ready`. This updates the value in Core 1's L1 cache, and puts its copy of the line into the **Modified** state. An *Invalidate* message is sent across the ICB, causing Core 2's copy of the line to be put into the **Invalid** state. This indicates that Core 2's copy of the line containing `g_ready` is no longer up-to-date.

- The next time that Core 2 (the consumer) tries to read `g_ready`, it finds that its locally-cached copy is **Invalid**. It sends a *Read* message across the ICB and obtains the newly-modified line from Core 1's L1 cache. This causes both cores' cache lines to be put into the **Shared** state once again. It also triggers a *write-back* of the line to main RAM.

A complete discussion of the MESI protocol is beyond our scope, but this example should give you a good feel for how it works to allow multiple cores to share data between their L1 caches while minimizing accesses to main RAM.

4.9.4.4 How MESI Can Go Wrong

Based on the discussion of the MESI protocol in the preceding section, it would seem that the problem of data sharing between L1 caches in a multicore machine has been solved in a watertight way. How, then, can the memory ordering bugs we've hinted at actually happen?

There's a one-word answer to that question: Optimization. On most hardware, the MESI protocol is highly optimized to minimize latency. This means that some operations aren't actually performed immediately when messages are received over the ICB. Instead, they are deferred to save time. As with compiler optimizations and CPU out-of-order execution optimizations, MESI optimizations are carefully crafted so as to be undetectable by a single thread. But, as you might expect, concurrent programs once again get the raw end of this deal.

For example, our producer (running on Core 1) writes 42 into `g_data` and then immediately writes 1 into `g_ready`. Under certain circumstances, optimizations in the MESI protocol can cause the new value of `g_ready` to become visible to other cores within the cache coherency domain *before* the updated value of `g_data` becomes visible. This can happen, for example, if Core 1 already has `g_ready`'s cache line in its local L1 cache, but does *not* have `g_data`'s line yet. This means that the consumer (on Core 2) can potentially see a value of 1 for `g_ready` *before* it sees a value of 42 in `g_data`, resulting in a data race bug.

This state of affairs can be summarized as follows:

Optimizations within a cache coherency protocol can make two read and/or write instructions *appear* to happen, from the point of view of other cores in the system, in an order that is opposite to the order in which the instructions were actually executed.

4.9.4.5 Memory Fences

When the apparent order of two instructions gets reversed by our cache coherency protocol, we say that the first instruction (in program order) has *passed* the second. There are four ways in which one instruction can pass another:

1. A read can pass another read,
2. a read can pass a write,
3. a write can pass another write, or
4. a write can pass a read.

To prevent the memory effects of a read or write instruction passing other reads and/or writes, modern CPUs provide special machine language instructions known as *memory fences*, also known as *memory barriers*.

In theory, a CPU could provide individual fence instructions to prevent each of these four cases from happening. For example, a *ReadRead* fence would only prevent reads from passing other reads, but would not prevent any of the other cases. Also, a fence instruction could be unidirectional or bidirectional. A one-way fence would guarantee that no reads or writes that precede it in program order can end up having an effect after it, but not vice-versa. A bidirectional fence, on the other hand, would prevent “leakage” of memory effects in either direction across the fence. Theoretically, then, we could imagine a CPU that provides twelve distinct fence instructions—a bidirectional, forward, and reverse variant of each of the four basic fence types listed above.

Thankfully real CPUs don’t usually provide all twelve kinds of fences. Instead, an ISA typically specifies a handful of fence instructions which serve as combinations of these theoretical fence types.

The strongest kind of fence is called a *full fence*. It ensures that all reads and writes occurring before the fence in program order will never appear to have occurred after it, and likewise that all reads and writes occurring after it will never appear to have happened before it. In other words, a full fence is a two-way barrier that affects both reads and writes.

A full fence is actually very expensive to realize in hardware. CPU designers don’t like forcing programmers to use an expensive construct when a cheaper one will do, so most CPUs provide a variety of less-expensive fence instructions which provide weaker guarantees than those provided by a full fence.

All fence instructions have two very useful side-effects:

1. They serve as compiler barriers, and

2. they prevent the CPU's out-of-order logic from reordering instructions across the fence.

This means that when we use a fence to prevent the memory ordering bugs that are caused by our CPU's cache coherency protocol, it also serves to prevent instruction reordering. So atomic instructions and memory fences are all we really need to write reliable mutexes and spin locks, and to write other *lock-free* algorithms as well.

4.9.4.6 Acquire and Release Semantics

No matter what the specific fence instructions look like under a particular ISA, we can reason about their effects by thinking in terms of the *semantics* they provide—in other words, the guarantees they enforce about the behavior of reads and writes in the system.

Memory ordering semantics are really properties of read or write instructions, not properties of the fences themselves. The fences merely provide a way for programmers to ensure that a read or write instruction has a particular memory ordering semantic. There are really only three memory ordering semantics we typically need to worry about:

- *Release* semantics. This semantic guarantees that a *write* to shared memory can never be passed by any other read or write that *precedes* it in program order. When this semantic is applied to a shared write, we call it a *write-release*. This semantic operates in the *forward* direction only—it says nothing about preventing memory operations that occur after the write-release from appearing to happen before it.
- *Acquire* semantics. This semantic guarantees that a *read* from shared memory can never be passed by any other read or write that occurs *after* it in program order. When this semantic is applied to a shared read, we call it a *read-acquire*. This semantic operates in the *reverse* direction only—it does nothing to prevent memory operations that occur before the read-acquire from having their effects seen after it.
- *Full fence* semantics. This bidirectional semantic ensures that all memory operations appear to occur in program order across the boundary created by a fence instruction in the code. No read or write that occurs before the fence in program order can appear to have occurred after the fence, and likewise no read or write that is after the fence in program order can appear to have occurred before it.

The individual fence instructions provided by any particular ISA typically provide at least one of these three memory ordering semantics. The details of how each fence instruction actually provides these semantic guarantees are CPU-specific, and for the most part as concurrent programmers, we don't care. As long as we can express the concept of write-release, read-acquire and full fence in our source code, we should be able to write a reliable spin lock or code other lock-free algorithms.

4.9.4.7 When to Use Acquire and Release Semantics

A *write-release* is most often used in a producer scenario—in which a thread performs two consecutive writes (e.g., writing to `g_data` and then `g_ready`), and we need to ensure that all other threads will see the two writes in the correct order. We can enforce this ordering by making the second of these two writes a write-release. To implement this, a fence instruction that provides *release semantics* is placed *before* the write-release instruction. Technically speaking, when a core executes a fence instruction with release semantics, it waits until all prior writes have been fully committed to memory within the cache coherency domain before executing the second write (the write-release).

A *read-acquire* is typically used in a consumer scenario—in which a thread performs two consecutive reads in which the second is conditional on the first (e.g., only reading `g_data` after a read of the flag `g_ready` comes back `true`). We enforce this ordering by making sure that the first read is a read-acquire. To implement this, a fence instruction that provides *acquire semantics* is placed *after* the read-acquire instruction. Technically speaking, when a core executes a fence instruction with acquire semantics, it waits until all writes from other cores have been fully flushed into the cache coherency domain before it continues on to execute the second read. This ensures that the second read will never appear to have occurred before the read-acquire.

Here's our producer-consumer example again, in full lock-free glory, using acquire and release fences to impose the necessary memory ordering semantics:

```
int32_t  g_data = 0;
int32_t  g_ready = 0;

void ProducerThread() // running on Core 1
{
    g_data = 42;

    // make the write to g_ready into a write-release
    // by placing a release fence *before* it
    RELEASE_FENCE();
```

```

    g_ready = 1;
}

void ConsumerThread() // running on Core 2
{
    // make the read of g_ready into a read-acquire
    // by placing an acquire fence after it
    while (!g_ready)
        PAUSE();
    ACQUIRE_FENCE();

    // we can now read g_data safely...
    ASSERT(g_data == 42);
}

```

For an excellent in-depth presentation of exactly why acquire and release fences are required under the MESI cache coherency protocol, see <http://www.swedishcoding.com/2017/11/10/multi-core-programming-and-cache-coherency/>.

4.9.4.8 CPU Memory Models

We mentioned in Section 4.9.4 that some CPUs provide stronger memory order semantics by default than others. On a CPU with strong memory semantics, read and/or write instructions behave like a certain kind of fence by default, without the programmer having to specify a fence instruction explicitly. For example, the DEC Alpha has notoriously weak semantics by default, requiring careful fencing in almost all situations. At the other end of the spectrum, an Intel x86 CPU has quite strong memory ordering semantics by default. For a great discussion of weak and strong memory ordering, see <http://preshing.com/20120930/weak-vs-strong-memory-models/>.

4.9.4.9 Fence Instructions on Real CPUs

Now that we understand the theory behind memory ordering semantics, let's take a very brief look at memory fence instructions on some real CPUs.

The Intel x86 ISA specifies three fence instructions: `sfence` provides release semantics, `lfence` provides acquire semantics, and `mfence` acts as a full fence. Certain x86 instructions may also be prefixed by a `lock` modifier to make them behave atomically, and to provide a memory fence prior to execution of the instruction. The x86 ISA is strongly ordered by default, meaning that fences aren't actually required in many cases where they would be on CPUs with weaker default ordering semantics. But there are some cases in which these fence instructions *are* required. For an example, see the post

entitled, “Who ordered memory fences on an x86?” by Bartosz Milewski (<https://bit.ly/2HuXpfo>).

The PowerPC ISA is quite weakly ordered, so explicit fence instructions are usually required to ensure correct semantics. The PowerPC makes a distinction between reads and writes to memory versus reads and writes to I/O devices, and hence it offers a variety of fence instructions that differ primarily in how they handle memory versus I/O. A full fence on PowerPC is provided by the `sync` instruction, but there’s also a “lightweight” fence called `lwsync`, a fence for I/O operations called `eieio` (ensure in-order execution of I/O), and even a pure instruction reordering barrier `isync` that does not provide any memory ordering semantics. You can read more about the PowerPC’s fence instructions here: <https://www.ibm.com/developerworks/systems/articles/powerpc.html>.

The ARM ISA provides a pure instruction reordering barrier called `isb`, two full memory fence instructions `dmb` and `dsb`, a one-way read-acquire instruction `ldar` and a one-way write-release instruction `stlr`. Interestingly, this ISA rolls acquire and release semantics into the read and write instructions themselves, rather than as separate fence instructions. For more information, see <http://infocenter.arm.com/help/index.jsp?topic=/com.arm.doc.den0024a/CJAIAJFI.html>.

4.9.5 Atomic Variables

Using atomic instructions and memory fences directly can be tedious and error-prone, not to mention being totally non-portable. Thankfully, as of C++11, the class template `std::atomic<T>` allows virtually any data type to be converted into an *atomic variable*. (A specialized class called `std::atomic_flag` encapsulates an atomic Boolean variable.) In addition to atomicity, the `std::atomic` family of templates provides its variables with “full fence” memory ordering semantics by default (although weaker semantics can be specified if desired). This frees us to write lock-free code without having to worry about any of the three causes of data race bugs.

Using these facilities, our producer-consumer example can be written as follows:

```
std::atomic<float> g_data;
std::atomic_flag g_ready = false;

void ProducerThread()
{
    // produce some data
    g_data = 42;
```

```

        // inform the consumer
        g_ready = true;
    }

    void ConsumerThread()
    {
        // wait for the data to be ready
        while (!g_ready)
            PAUSE();

        // consume the data
        ASSERT(g_data == 42);
    }

```

Notice that this code looks nearly identical to the erroneous code we first presented when talking about race conditions in Section 4.5.3. By simply wrapping our variables in `std::atomic`, we've converted a concurrent program that is prone to data race bugs into one that is race-free.

Under the hood, the implementations of `std::atomic<T>` and `std::atomic_flag` are, of course, complex. The standard C++ library has to be portable, so its implementation makes use of whichever atomic and barrier machine language instructions happen to be available on the target platform. What's more, the `std::atomic<T>` template can wrap any type imaginable, but of course CPUs don't provide atomic instructions for manipulating arbitrarily-sized data objects. As such, the `std::atomic<T>` template must be specialized by size: When the template is applied to a 32- or 64-bit type, it can be implemented without locks, using atomic machine language instructions directly. But when it is applied to larger types, mutex locks are used to ensure correct atomic behavior. (You can call `is_lock_free()` on any atomic variable to find out whether its implementation really is lock-free on your target hardware.)

4.9.5.1 C++ Memory Order

By default, C++ atomic variables make use of full memory barriers, to ensure that they will work correctly in all possible use cases. However, it is possible to *relax* these guarantees, by passing a *memory order semantic* (an optional argument of type `std::memory_order`) to functions that manipulate atomic variables. The documentation regarding `std::memory_order` is pretty confusing, so let's demystify it. Here are the possible memory order settings, and what they mean:

1. *Relaxed*. An atomic operation performed with relaxed memory order

semantics guarantees *only* atomicity. No barrier or fence is used.

2. *Consume*. A read performed with consume semantics guarantees that no other read or write *within the same thread* can be reordered before this read. In other words, this semantic only serves to prevent *compiler optimizations* and *out-of-order execution* from reordering the instructions—it does nothing to ensure any particular memory ordering semantics within the cache coherency domain. It is typically implemented with an instruction reordering barrier like the PowerPC’s `isync` instruction.
3. *Release*. A write performed with *release* semantics guarantees that no other read or write in this thread can be reordered after it, and the write is guaranteed to be visible by other threads reading the same address. It employs a *release fence* in the CPU’s cache coherency domain to accomplish this.
4. *Acquire*. A read performed with *acquire* semantics guarantees *consume* semantics, and in addition it guarantees that writes to the same address by other threads will be visible to this thread. It does this via an *acquire fence* in the CPU’s cache coherency domain.
5. *Acquire/Release*. This semantic (the default) is the safest, because it applies a *full* memory fence.

It’s important to realize that using a memory ordering specifier doesn’t guarantee that a particular semantic will actually be used on every platform. All it does is to guarantee that the semantics will be *at least* that strong—a stronger semantic might be employed on some target hardware. For example, on Intel x86, the relaxed memory order isn’t possible because the CPU’s default memory ordering semantics are relatively strong. On an Intel CPU, any request for a relaxed read operation will actually end up having *acquire* semantics.

Using these memory order specifiers requires switching from `std::atomic`’s overloaded assignment and cast operators to explicit calls to `store()` and `load()`. Here’s our simple producer-consumer example again, this time using `std::memory_order` specifiers to provide our release and acquire barriers:

```
std::atomic<float> g_data;
std::atomic<bool> g_ready = false;

void ProducerThread()
{
    // produce some data
```



```

    g_data.store(42, std::memory_order_relaxed);

    // inform the consumer
    g_ready.store(true, std::memory_order_release);
}

void ConsumerThread()
{
    // wait for the data to be ready
    while (!g_ready.load(std::memory_order_acquire))
        PAUSE();

    // consume the data
    ASSERT(g_data.load(std::memory_order_relaxed) == 42);
}

```

It’s important to remember the 80/20 rule when using “relaxed” memory ordering semantics like this. These semantics are easy to get wrong, so you should probably only use non-default memory ordering with `std::atomic` when you can prove, via profiling, that the performance improvement is really necessary—and that changing your code to use explicit memory ordering semantics does, in fact, produce the benefits you expect!

A complete discussion of how to use memory ordering semantics in C++11 is beyond our scope, but you can read more by searching online for the article entitled, “Implementing Scalable Atomic Locks for Multi-Core Intel® EM64T and IA32 Architectures” by Michael Chynoweth. This forum discussion offers some interesting insights as well, and illustrates just how complex this kind of programming can get: <https://groups.google.com/forum/#!topic/boost-developers-archive/QLrat5ASrnM>.

4.9.6 Concurrency in Interpreted Programming Languages

Thus far we’ve only discussed concurrency in the context of compiled languages like C and C++, and in assembly language. These languages compile or assemble down to raw machine code that is executed directly by the CPU. As such, atomic operations and locks must be implemented with the aid of special machine language instructions that provide atomic operations and cache coherent memory barriers, with additional help from the kernel (which makes sure threads are put to sleep and awoken appropriately) and the compiler (which respects barrier instructions when optimizing the code).

The story is somewhat different for interpreted programming languages like Java and C#. Programs written in these languages execute in the context of a *virtual machine* (VM): Java programs run inside the Java Virtual Machine

(JVM), and C# programs run within the context of the Common Language Runtime (CLR). A virtual machine is essentially a software emulation of a CPU, reading bytecoded instructions one by one and executing them. A VM also acts a bit like an emulated operating system kernel: It provides its own notion of “threads” (comprised of bytecoded instructions), and it handles all of the details of scheduling those threads itself. Because the operation of a VM is implemented entirely in software, an interpreted language like Java or C# can provide powerful concurrent synchronization facilities that are not as constrained by the hardware as they would be in a compiled language like C or C++.

One example of this principle in action is the `volatile` type qualifier. We said in Section 4.6 that in C/C++ a `volatile` variable is not atomic. However, in Java and C#, the `volatile` type qualifier *does* guarantee atomicity. Operations on volatile variables in these languages cannot be optimized, and they cannot be interrupted by another thread. Moreover, all reads of a volatile variable in Java and C# are effectively performed directly from main memory rather than from the cache, and likewise all writes are effectively written to main RAM rather than to the cache. All of these guarantees can be provided in part because the virtual machine has full control over the execution of the bytecoded instruction streams that comprise each app.

A full discussion of the atomicity and thread synchronization facilities provided by interpreted languages like C# and Java is well beyond the scope of this book. But now that you have a solid understanding of the principles behind atomicity at the lowest levels, understanding these facilities in other higher-level languages should be a snap. For further reading, the following websites are a good start:

- C#: Search for “Parallel Processing and Concurrency in the .NET Framework” on <https://docs.microsoft.com>.
- Java: <https://docs.oracle.com/javase/tutorial/essential/concurrency/>.

4.9.7 Spin Locks

When discussing atomic machine language instructions in Section 4.9.2.2, we presented some code snippets illustrating how a spin lock might be implemented using each of those instructions. But because of instruction reordering and memory ordering semantics, those examples weren’t 100% correct. In this section, we’ll present an industrial-strength spin lock, and then explore a few useful variants.

4.9.7.1 Basic Spin Lock

A spin lock can be implemented using a `std::atomic_flag`, either wrapped in a C++ class or accessed via a simple functional API. The spin lock is acquired by using a TAS instruction to atomically set the flag to `true`, retrying in a while loop until the TAS succeeds. It is unlocked by atomically writing a value of `false` into the flag.

When acquiring a spin lock, it's important to use *read-acquire* memory ordering semantics to read the lock's current contents as part of the TAS operation. This fence guards against the rare scenario in which the lock is observed as having been released when in reality some other thread hasn't *completely* exited its critical section yet. In C++11, this can be accomplished by passing `std::memory_order_acquire` to the `test_and_set()` call. In raw assembly language, we'd place an acquire fence instruction after the TAS instruction.

When releasing a spin lock, it's likewise important to use *write-release* semantics to ensure that all writes performed *after* the call to `Unlock()` aren't observed by other threads as if they had happened *before* the lock was released.

Here's a complete implementation of a TAS-based spin lock, using correct and minimal memory ordering semantics:

```
class SpinLock
{
    std::atomic_flag m_atomic;

public:
    SpinLock() : m_atomic(false) { }

    bool TryAcquire()
    {
        // use an acquire fence to ensure all subsequent
        // reads by this thread will be valid
        bool alreadyLocked = m_atomic.test_and_set(
            std::memory_order_acquire);

        return !alreadyLocked;
    }

    void Acquire()
    {
        // spin until successful acquire
        while (!TryAcquire())
        {
```

```

        // reduce power consumption on Intel CPUs
        // (can substitute with std::this_thread::yield()
        // on platforms that don't support CPU pause, if
        // thread contention is expected to be high)
        PAUSE();
    }
}

void Release()
{
    // use release semantics to ensure that all prior
    // writes have been fully committed before we unlock
    m_atomic.clear(std::memory_order_release);
}
};

```

4.9.7.2 Scoped Locks

It's often inconvenient and error-prone to manually unlock a mutex or spin lock, especially when the function using the lock has multiple return sites. In C++ we can use a simple wrapper class called a *scoped lock* to ensure that a lock is automatically released when a particular scope is exited. It works by simply acquiring the lock in the constructor, and releasing it in the destructor.

```

template<class LOCK>
class ScopedLock
{
    typedef LOCK lock_t;
    lock_t* m_pLock;

public:
    explicit ScopedLock(lock_t& lock) : m_pLock(&lock)
    {
        m_pLock->Acquire();
    }

    ~ScopedLock()
    {
        m_pLock->Release();
    }
};

```

This scoped lock class can be used with any spin lock or mutex that has a conformant interface (i.e., any lock class that supports the functions `Acquire()` and `Release()`). Here's how it might be used:

```

SpinLock g_lock;

int ThreadSafeFunction()
{
    // the scoped lock acts like a "janitor"
    // because it cleans up for us!
    ScopedLock<decltype(g_lock)> janitor(g_lock);

    // do some work...

    if (SomethingWentWrong())
    {
        // lock will be released here
        return -1;
    }

    // so some more work...

    // lock will also be released here
    return 0;
}

```

4.9.7.3 Reentrant Locks

A vanilla spin lock will cause a thread to deadlock if it ever tries to reacquire a lock that it already holds. This can happen whenever two or more *thread-safe* functions attempt to call one another reentrantly from within the same thread. For example, given two functions:

```

SpinLock g_lock;

void A()
{
    ScopedLock<decltype(g_lock)> janitor(g_lock);

    // do some work...
}

void B()
{
    ScopedLock<decltype(g_lock)> janitor(g_lock);

    // do some work...

    // make a call to A() while holding the lock
    A(); // deadlock!
}

```

```

        // do some more work...
    }

```

We can relax this reentrancy restriction if we can arrange for our spin lock class to cache the id of the thread that has locked it. That way, the lock can “know” the difference between a thread trying to reacquire its own lock (which we wish to allow) versus a thread trying to grab a lock that’s already held by a different thread (which should cause the caller to wait). To make sure that calls to `Acquire()` and `Release()` are done in matching pairs, we’ll also want to add a reference count to the class. Here’s a functional implementation, using appropriate memory fencing:

```

class ReentrantLock32
{
    std::atomic<std::size_t> m_atomic;
    std::int32_t m_refCount;

public:
    ReentrantLock32() : m_atomic(0), m_refCount(0) { }

    void Acquire()
    {
        std::hash<std::thread::id> hasher;
        std::size_t tid = hasher(std::this_thread::get_id());

        // if this thread doesn't already hold the lock...
        if (m_atomic.load(std::memory_order_relaxed) != tid)
        {
            // ... spin wait until we do hold it
            std::size_t unlockValue = 0;
            while (!m_atomic.compare_exchange_weak(
                unlockValue,
                tid,
                std::memory_order_relaxed, // fence below!
                std::memory_order_relaxed))
            {
                unlockValue = 0;
                PAUSE();
            }
        }

        // increment reference count so we can verify that
        // Acquire() and Release() are called in pairs
        ++m_refCount;
    }
}

```

```

        // use an acquire fence to ensure all subsequent
        // reads by this thread will be valid
        std::atomic_thread_fence(std::memory_order_acquire);
    }

void Release()
{
    // use release semantics to ensure that all prior
    // writes have been fully committed before we unlock
    std::atomic_thread_fence(std::memory_order_release);

    std::hash<std::thread::id> hasher;
    std::size_t tid = hasher(std::this_thread::get_id());
    std::size_t actual = m_atomic.load(std::memory_order_relaxed);
    assert(actual == tid);

    --m_refCount;
    if (m_refCount == 0)
    {
        // release lock, which is safe because we own it
        m_atomic.store(0, std::memory_order_relaxed);
    }
}

bool TryAcquire()
{
    std::hash<std::thread::id> hasher;
    std::size_t tid = hasher(std::this_thread::get_id());

    bool acquired = false;

    if (m_atomic.load(std::memory_order_relaxed) == tid)
    {
        acquired = true;
    }
    else
    {
        std::size_t unlockValue = 0;
        acquired = m_atomic.compare_exchange_strong(
            unlockValue,
            tid,
            std::memory_order_relaxed, // fence below!
            std::memory_order_relaxed);
    }
    if (acquired)
    {
        ++m_refCount;
    }
}

```

```

        std::atomic_thread_fence(
            std::memory_order_acquire);
    }
    return acquired;
}
};

```

4.9.7.4 Readers-Writer Locks

In a system in which multiple threads can read and write a shared data object, we could control access to the object using a mutex or a spin lock. However, multiple threads should be allowed to *read* the shared object concurrently. It's only when the shared object is being *mutated* that we need to ensure mutual exclusivity. What we'd like is a kind of lock that allows any number of readers to acquire it concurrently. Whenever a writer thread attempts to acquire the lock, it should wait until all readers are finished, and then it should acquire the lock in a special "exclusive" mode that prevents any other readers or writers from gaining access until it has completed its mutation of the shared object. This kind of lock is called a *readers-writer lock* (also known as a *shared-exclusive lock* or a *push lock*).

We can implement a readers-writer lock in a manner similar to how we implemented our reentrant lock. However, instead of storing the thread id in the atomic variable, we'll store a reference count indicating how many readers currently hold the lock. Each time a reader acquires the lock, the count is incremented; each time a reader releases the lock, the count is decremented.

But how, then, can we also provide an "exclusive" lock mode for the writer? All we need to do is to reserve one (very high) reference count value and use it to denote that a writer currently holds the lock. If our reference count is an unsigned 32-bit integer, the value 0xFFFFFFFFU could do nicely as the reserved value. Even easier, we can simply reserve the most-significant bit, meaning that reference counts from 0 to 0x7FFFFFFFU represent reader locks, and the reserved value 0x80000000U represents a writer lock (with no other values being valid).

A readers-writer lock suffers from starvation problems: A writer that holds the lock too long can cause all of the readers to starve, and likewise a lot of readers can cause the writers to starve. A *sequential lock* is one possible alternative that tackles the starvation issue (see <https://en.wikipedia.org/wiki/Seqlock> for details). Check out <https://lwn.net/Articles/262464> for a description of yet another interesting locking technique, used in the Linux kernel, that supports multiple concurrent readers and writers, called *read-copy-update* (RCU).

We'll leave the implementation of a readers-writer lock up to you as an exercise! However, if you'd like to compare notes, you can find a fully-functional implementation on this book's website (www.gameenginebook.com).

4.9.7.5 Lock-Not-Needed Assertions

No matter how you slice it, locks are expensive. Mutexes are expensive even in the absence of contention. In a low-contention scenario, spin locks are relatively cheap, but they still introduce a non-zero cost into any piece of software.

It's often the case that the programmer knows *a priori* that a lock isn't required. In a game engine, for example, each iteration of the game loop is usually performed in distinct stages. If a shared data structure is accessed exclusively by a single thread early in the frame, and that same data is accessed again by a single thread later in the frame, then we don't actually need a lock. Yes, in theory those two threads could overlap, and if they were to do so a lock would definitely be needed. But in practice, given the way our game loop is structured, we might know that such overlap can never occur.

In this kind of situation, we have a few choices. We could put the locks in, just in case. That way, if someone rearranges the order in which things are done during the frame, and ends up making these threads overlap, we'd be covered. Another option is to just ignore the possibility of overlap and not lock anything.

There is a third option which I find more appealing in such scenarios: We can *assert* that a lock isn't required. This has two benefits. First, it can be done very cheaply, and in fact the assertions can be stripped prior to shipping your game. Second, it automatically detects problems if our assumptions about the overlap of the threads proves to be incorrect—or if the assumptions are broken later on by a refactor of the code. There's no standardized name for this kind of assertion, so we'll call them *lock-not-needed assertions* in this book.

So how do we detect that a lock is needed? One way would be to use an atomic Boolean variable, complete with proper memory fencing, and use it like a mutex. Except that instead of actually acquiring a mutex lock, we would simply assert that the Boolean is false, and then set it to true atomically. And instead of releasing a lock, we assert that our Boolean is true, and then set it to false atomically. This approach would work, but it would be just as expensive as an uncontended spin lock. We can do better.

The "trick" is to realize that we only care about *detecting* overlaps between critical operations on a shared object. And that detection needn't be 100% reliable. A 90% hit rate is probably just fine. If two critical operations ever *do* overlap, there may be times when we fail to detect it. But if your game is being run multiple times a day, every day, by a team of 100 or more developers,

plus a QA department consisting of at least another 10 or 20 people, you can be pretty sure *someone* will detect the problem if one exists.

So, instead of an *atomic* Boolean, we'll just use a *volatile* Boolean. As we've stated, the *volatile* keyword doesn't do much to prevent concurrent race bugs. But it does guarantee that reads and writes of the Boolean won't be optimized away, and that's really all we need. We'll get a reasonably good detection rate, and the test is dirt cheap.

```
class UnnecessaryLock
{
    volatile bool    m_locked;

public:
    void Acquire()
    {
        // assert no one already has the lock
        assert(!m_locked);

        // now lock (so we can detect overlapping
        // critical operations if they happen)
        m_locked = true;
    }
    void Release()
    {
        // assert correct usage (that Release()
        // is only called after Acquire())
        assert(m_locked);

        // unlock
        m_locked = false;
    }
};

#if ASSERTIONS_ENABLED
#define BEGIN_ASSERT_LOCK_NOT_NECESSARY(L) (L).Acquire()
#define END_ASSERT_LOCK_NOT_NECESSARY(L)   (L).Release()
#else
#define BEGIN_ASSERT_LOCK_NOT_NECESSARY(L)
#define END_ASSERT_LOCK_NOT_NECESSARY(L)
#endif

// Example usage...

UnnecessaryLock g_lock;
```

```

void EveryCriticalOperation()
{
    BEGIN_ASSERT_LOCK_NOT_NECESSARY(g_lock);

    printf("perform critical op...\n");

    END_ASSERT_LOCK_NOT_NECESSARY(g_lock);
}

```

We could also wrap the locks in a janitor (see Section 3.1.1.6), like this:

```

class UnnecessaryLockJanitor
{
    UnnecessaryLock* m_pLock;
public:
    explicit
    UnnecessaryLockJanitor(UnnecessaryLock& lock)
        : m_pLock(&lock) { m_pLock->Acquire(); }
    ~UnnecessaryLockJanitor() { m_pLock->Release(); }
};

#if ASSERTIONS_ENABLED
#define ASSERT_LOCK_NOT_NECESSARY(J,L) \
    UnnecessaryLockJanitor J(L)
#else
#define ASSERT_LOCK_NOT_NECESSARY(J,L)
#endif

// Example usage...

UnnecessaryLock g_lock;

void EveryCriticalOperation()
{
    ASSERT_LOCK_NOT_NECESSARY(janitor, g_lock);

    printf("perform critical op...\n");
}

```

We implemented this at Naughty Dog and it successfully caught a number of cases of critical operations overlapping when the programmers had assumed they never could do so. So this little gem is tried and true.

4.9.8 Lock-Free Transactions

This is supposed to be a section on lock-free programming, but thus far we've spent all of our time on writing spin locks! Perhaps counterintuitively, the

act of writing a spin lock is an example of lock-free programming, from the perspective of the implementation of the spin lock itself. We've also learned a lot about atomic instructions, compiler barriers and memory fences along the way. So this has been a useful exercise. However, we haven't really explored the principles of lock-free programming per se; for that purpose it will be instructive to look at an example other than a spin lock. The topic of lock-free and non-blocking algorithms is huge. It really deserves its own book, so we won't attempt to cover the topic in depth here. But we can at least get a feel for how lock-free approaches usually work.

The goal of lock-free programming is of course to avoid taking locks that will either put a thread to sleep, or cause it to get caught up in a busy-wait loop inside a spin lock. To perform a critical operation in a lock-free manner, we need to think of each such operation as a *transaction* that can either succeed in its entirety, or fail in its entirety. If it fails, the transaction is simply retried until it does succeed.

To implement any transaction, no matter how complex, we perform the majority of the work *locally* (i.e., using data that's visible only to the current thread, rather than operating directly on the shared data). When all of our ducks are in a row and the transaction is ready to *commit*, we execute a *single atomic instruction*, such as CAS or LL/SC. If this atomic instruction succeeds, we have successfully "published" our transaction globally—it becomes a permanent part of the shared data structure on which we're operating. But if the atomic instruction fails, that means some *other* thread was attempting to commit a transaction at the same time we were.

This fail-and-retry approach works because whenever one thread fails to commit its transaction, we know that it was because some *other* thread managed to *succeed*. As a result, one thread in the system is always making forward progress (just maybe not us). And that is the definition of *lock-free*.

4.9.9 A Lock-Free Linked List

As a concrete example, let's look at a simple lock-free singly-linked list. The only operation we'll support in this discussion is `push_front()`.

To prepare the transaction, we allocate the new `Node` and populate it with data. We also set its next pointer to point to whichever node is currently at the head of the linked list. The transaction is now ready to commit atomically.

The commit itself consists of a call to `compare_exchange_weak()` on the head pointer, which we declared as an atomic pointer to a `Node`. If this call succeeds, we've inserted our new node at the head of the linked list and we're done. But if it fails, we need to retry. This involves re-initializing our

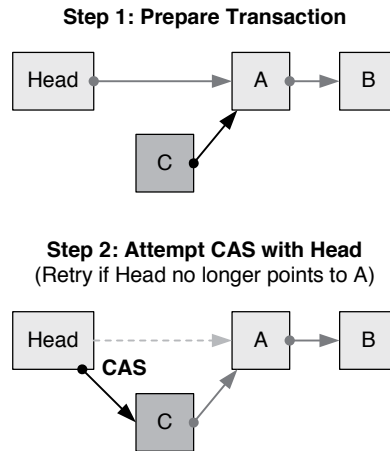


Figure 4.37. A lock-free implementation of insertion at the head of a singly-linked list. Top: The transaction is prepared by setting the next pointer of the new node to point to the current head of the list. Bottom: The transaction is committed by using an atomic CAS operation to swap the head pointer with a pointer to the new node. If the CAS fails, we return to the top and try again until it succeeds.

new node’s next pointer to point to what is now potentially a *brand new* head node (presumably inserted by another thread—this is perhaps why we failed in the first place). This two-stage process is illustrated in Figure 4.37.

In the code below, you won’t see an explicit re-initialization of the node’s next pointer. That’s because `compare_exchange_weak()` does the re-initialization step for us. (How convenient!) Here’s what the code would look like:

```
template< class T >
class SList
{
    struct Node
    {
        T      m_data;
        Node*  m_pNext;
    };
    std::atomic< Node* > m_head { nullptr };

public:
    void push_front(T data)
    {
        // prep the transaction locally
        auto pNode = new Node();
        pNode->m_data = data;
```

```

        pNode->m_pNext = m_head;

        // commit the transaction atomically
        // (retrying until it succeeds)
        while (!m_head.compare_exchange_weak(
            pNode->m_pNext, pNode))
        { }
    }
};

```

4.9.10 Further Reading on Lock-Free Programming

Concurrency is a vast and profound topic, and in this chapter we've only just scratched the surface. As always, the goal of this book is merely to build awareness and serve as a jumping-off point for further learning.

- For a complete discussion of implementing a lock-free singly-linked list, check out Herb Sutter's talk at CppCon 2014, which is where the example above came from. The talk is available on YouTube in two parts:
 - <https://www.youtube.com/watch?v=c1gO9aB9nbs>, and
 - <https://www.youtube.com/watch?v=CmxkPChOcvw>.
- This lecture by Geoff Langdale of CMU provides a great overview: https://www.cs.cmu.edu/~410-s05/lectures/L31_LockFree.pdf.
- Also be sure to check out this presentation by Samy Al Bahra for a clear and easily digestible overview of pretty much every topic under the sun related to concurrent programming: http://concurrencykit.org/presentations/lockfree_introduction/#/.
- Mike Acton's excellent talk on concurrent thinking is a must-read; it is available at http://cellperformance.beyond3d.com/articles/public/concurrency_rabit_hole.pdf.
- These two online books are great resources for concurrent programming: <http://greenteapress.com/semaphores/LittleBookOfSemaphores.pdf> and <https://www.kernel.org/pub/linux/kernel/people/paulmck/perfbook/perfbook.2011.01.02a.pdf>.
- Some excellent articles on lock-free programming and how atomics, barriers and fences work can be found on Jeff Preshing's blog: <http://preshing.com/20120612/an-introduction-to-lock-free-programming>.

- This page has great information about memory barriers on Linux: <https://www.mjmwired.net/kernel/Documentation/memory-barriers.txt#305>

4.10 SIMD/Vector Processing

In Section 4.1.4, we introduced a form of parallelism known as *single instruction multiple data* (SIMD). This refers to the ability of most modern microprocessors to perform a mathematical operation on multiple data items *in parallel*, using a single machine instruction. In this section, we'll explore SIMD techniques in some detail, and conclude the chapter with a brief discussion of how SIMD and multithreading are combined into a form of parallelism known *single instruction multiple thread* (SMT), which forms the basis of all modern GPUs.

Intel first introduced its MMX¹¹ instruction set with their Pentium line of CPUs in 1994. These instructions permitted SIMD calculations to be performed on eight 8-bit integers, four 16-bit integers, or two 32-bit integers packed into special 64-bit MMX registers. Intel followed this up with various revisions of an extended instruction set called *streaming SIMD extensions*, or SSE, the first version of which appeared in the Pentium III processor.

The SSE instruction set utilizes 128-bit registers that can contain integer or IEEE floating-point data. The SSE mode most commonly used by game engines is called *packed 32-bit floating-point mode*. In this mode, four 32-bit `float` values are packed into a single 128-bit register. An operation such as addition or multiplication can thus be performed on four pairs of `floats` in parallel by taking two of these 128-bit registers as its inputs. Intel has since introduced various upgrades to the SSE instruction set, named SSE2, SSE3, SSSE3 and SSE4. In 2007, AMD introduced its own variants named XOP, FMA4 and CVT16.

In 2011, Intel introduced a new, wider SIMD register file and accompanying instruction set named *advanced vector extensions* (AVX). AVX registers are 256 bits wide, permitting a single instruction to operate on pairs of up to eight 32-bit floating-point operands in parallel. The AVX2 instruction set is an extension to AVX. Some Intel CPUs now support AVX-512, an extension to AVX permitting 16 32-bit floats to be packed into a 512-bit register.

¹¹Officially, MMX is a meaningless initialism trademarked by Intel. Unofficially, developers consider it to stand for “multimedia extensions” or “matrix math extensions.”

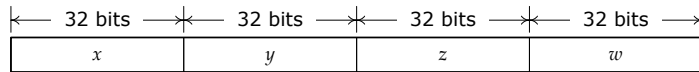


Figure 4.38. The four components of an SSE register in 32-bit floating-point mode.

4.10.1 The SSE Instruction Set and Its Registers

The SSE instruction set includes a wide variety of operations, with many variants for operating on differently-sized data elements within SSE registers. For the purposes of the present discussion, however, we'll stick to the relatively small subset of instructions that deal with packed 32-bit floating-point data. These instructions are denoted with a `ps` suffix, indicating that we're dealing with *packed* data (`p`), and that each element is a *single*-precision `float` (`s`). However most of the upcoming discussions extend intuitively into AVX's 256- and 512-bit modes; see <https://software.intel.com/en-us/articles/introduction-to-intel-advanced-vector-extensions> for an overview of AVX.

The SSE registers are named `XMMi`, where i is an integer between 0 and 15 (e.g., `XMM0`, `XMM1`, and so on). In packed 32-bit floating-point mode, each 128-bit `XMMi` register contains four 32-bit floats. In AVX, the registers are 256 bits wide and are named `YMMi`; in AVX-512, they are 512 bits in width and are named `ZMMi`.

In this chapter, we'll often refer to the individual floats within an SSE register as $[x \ y \ z \ w]$, just as they would be when doing vector/matrix math in homogeneous coordinates on paper (see Figure 4.38). It doesn't usually matter what you call the elements of an SSE register, as long as you're consistent about how you *interpret* each element. The most general approach is to think of an SSE vector `r` as containing the elements $[r_0 \ r_1 \ r_2 \ r_3]$. Most SSE documentation uses this convention, although some documentation uses a $[w \ x \ y \ z]$ convention, so be careful out there!

4.10.1.1 The `__m128` Data Type

In order for SSE instructions to perform arithmetic on packed floating-point data, that data must reside in one of the `XMMi` registers. For long-term storage, packed floating-point data can of course reside in memory, but it must be transferred from RAM into an SSE register prior to being used for any calculations, and the results subsequently transferred back to RAM.

To make working with SSE and AVX data easier, C and C++ compilers provide special data types that represent packed arrays of `float`s. The `__m128` type encapsulates a packed array of four `float`s for use with the SSE intrinsics. (The `__m256` and `__m512` types likewise represent packed arrays of eight

and 16 `float`s, respectively, for use with AVX intrinsics.)

The `__m128` data type and its kin can be used to declare global variables, automatic variables, function arguments and return types, and even class and structure members. Declaring automatic variables and function arguments to be of type `__m128` often results in the compiler treating those values as direct proxies for SSE registers. But using the `__m128` type to declare global variables, structure/class members, and sometimes local variables results in the data being stored as a 16-byte aligned array of `float` in memory. Using a memory-based `__m128` variable in an SSE calculation will cause the compiler to implicitly emit instructions for loading the data from memory into an SSE register prior to performing the calculation on it, and likewise to emit instructions for storing the results of the calculation back into the memory that “backs” each such variable. As such, it’s a good idea to check the disassembly to make sure that you’re not doing unnecessary loads and stores of SSE registers when using the `__m128` type (and its AVX relatives).

4.10.1.2 Alignment of SSE Data

Whenever data that is intended for use in an `XMMi` register is stored in memory, it must be 16-byte (128-bit) aligned. (Likewise, data intended for use with AVX’s 256-bit `YMMi` registers must be 32-byte (256-bit) aligned, and data for use with the 512-bit `ZMMi` registers must be 64-byte (512-bit) aligned.)

The compiler ensures that global and local variables of type `__m128` are aligned automatically. It also pads `struct` and `class` members so that any `__m128` members are aligned properly *relative to* the start of the object, and ensures that the alignment of the entire `struct` or `class` is equal to the worst-case alignment of its members. This means that declaring a global or local variable instance of a `struct` or `class` that includes at least one `__m128` member will be 16-byte aligned by the compiler automatically.

However, all *dynamically allocated* instances of such a `struct` or `class` need to be aligned manually. Likewise, any array of `float` that you intend to use with SSE instructions must be properly aligned; you can ensure this via the C++11 `alignas` specifier. See Section 6.2.1.3 for more information on aligned memory allocations.

4.10.1.3 SSE Compiler Intrinsics

We could work directly with the SSE and AVX assembly language instructions, perhaps using our compiler’s inline assembly syntax. However, writing code like this is not only non-portable, it’s also a big pain in the butt! To make life easier, modern compilers provide *intrinsics*—special syntax that looks and

behaves like a regular C function, but is actually boiled down to inline assembly code by the compiler. Many intrinsics translate into a single assembly language instruction, although some are *macros* that translate into a sequence of instructions.

In order to use SSE and AVX intrinsics, your .cpp file must `#include <xmmintrin.h>` in Visual Studio, or `<x86intrin.h>` when compiling with Clang or gcc.

4.10.1.4 Some Useful SSE Intrinsics

There are a lot of SSE intrinsics, but for the purposes of our discussion here we can start with only five of them:

- `__m128 _mm_set_ps(float w, float z, float y, float x);`
This intrinsic initializes an `__m128` variable with the four floating-point values provided.
- `__m128 _mm_load_ps(const float* pData);`
This intrinsic loads four `float`s from a C-style array into an `__m128` variable. The input array must be 16-byte aligned.
- `void _mm_store_ps(float* pData, __m128 v);`
This intrinsic stores the contents of an `__m128` variable into a C-style array of four `float`s, which must be 16-byte aligned.
- `__m128 _mm_add_ps(__m128 a, __m128 b);`
This intrinsic adds the four pairs of `float`s contained in the variables `a` and `b` in parallel and returns the result.
- `__m128 _mm_mul_ps(__m128 a, __m128 b);`
This intrinsic multiplies the four pairs of `float`s contained in the variables `a` and `b` in parallel and returns the result.

You may have noticed that the arguments `x`, `y`, `z` and `w` are passed to the `_mm_set_ps()` function in reverse order. This strange convention probably arises from the fact that Intel CPUs are *little-endian*. Just as a single floating-point value with the bit pattern `0x12345678` would be stored in memory as the bytes `0x78`, `0x56`, `0x34`, `0x12` in order of increasing addresses, so too are the contents of an SSE register stored in memory in an order that's opposite to the order those components actually appear within the register. In other words, not only are the four bytes comprising each float in an SSE register stored in little-endian order, but so too are the four floats themselves. All of this is just a question of naming convention: There's really no "most-significant"

or “least-significant” float within an SSE register. So we can either treat the in-memory order as the “correct” order and consider `_mm_set_ps()` to be “backwards,” or we can treat the arguments of `_mm_set_ps()` as being in the “correct” order and think of all in-memory vectors as being “backwards.” We’ll stick with the former convention, since it means that we’ll be able to read off our vectors more naturally: A homogeneous vector v consisting of members (v_x, v_y, v_z, v_w) would be stored in a C/C++ array as `float v[] = { vx, vy, vz, vw }`, but would be passed to `_mm_set_ps()` as `w, z, y, x`.

Here’s a small code snippet that loads two four-element floating-point vectors, adds them, and prints the results.

```
#include <xmmintrin.h>

void TestAddSSE()
{
    alignas(16) float A[4];
    alignas(16) float B[4] = { 2.0f, 4.0f, 6.0f, 8.0f };

    // set a = (1, 2, 3, 4) from literal values, and
    // load b = (2, 4, 6, 8) from a floating-point array
    // just to illustrate the two ways of doing this
    // (remember that _mm_set_ps() is backwards!)
    __m128 a = _mm_set_ps(4.0f, 3.0f, 2.0f, 1.0f);
    __m128 b = _mm_load_ps(&B[0]);

    // add the vectors
    __m128 r = _mm_add_ps(a, b);

    // store '__m128 a' into a float array for printing
    _mm_store_ps(&A[0], a);

    // store result into a float array for printing
    alignas(16) float R[4];
    _mm_store_ps(&R[0], r);

    // inspect results
    printf("a = %.1f %.1f %.1f %.1f\n",
           A[0], A[1], A[2], A[3]);
    printf("b = %.1f %.1f %.1f %.1f\n",
           B[0], B[1], B[2], B[3]);
    printf("r = %.1f %.1f %.1f %.1f\n",
           R[0], R[1], R[2], R[3]);
}
```

4.10.1.5 AltiVec vector Types

As a quick aside, the GNU C/C++ compiler *gcc* (used to compile code for the PS3, for example) provides support for the PowerPC's *AltiVec* instruction set, which provides support for SIMD operations, much as SSE does on Intel processors. 128-bit vector types can be declared like regular C/C++ types, but they are preceded by the keyword `vector`. For example, a SIMD variable containing four floats would be declared `vector float`. *gcc* also provides a means of writing *literal* SIMD values into your source code. For example, you can initialize a `vector float` with a value like this:

```
vector float v = (vector float)(-1.0f, 2.0f, 0.5f, 1.0f);
```

The corresponding Visual Studio code is a tad more clunky:

```
// use compiler intrinsic to load "literal" value
// (remember _mm_set_ps() is backwards!)
__m128 v = _mm_set_ps(1.0f, 0.5f, 2.0f, -1.0f);
```

We won't cover AltiVec explicitly in this chapter, but once you understand SSE it'll be very easy to learn.

4.10.2 Using SSE to Vectorize a Loop

SIMD offers the potential to speed up certain kinds of calculations by a factor of four, because a single machine language instruction can perform an arithmetic operation on four floating-point values in parallel. Let's see how this can be done using SSE intrinsics.

First, consider a simple loop that adds two arrays of floats pairwise, storing each result into an output array:

```
void AddArrays_ref(int count,
                  float* results,
                  const float* dataA,
                  const float* dataB,
                  {
    for (int i = 0; i < count; ++i)
    {
        results[i] = dataA[i] + dataB[i];
    }
}
```

We can speed up this loop significantly using SSE intrinsics, like this:

```

void AddArrays_sse(int count,
                  float* results,
                  const float* dataA,
                  const float* dataB)
{
    // NOTE: the caller needs to ensure that the size of
    // all 3 arrays are equal, and a multiple of four!
    assert(count % 4 == 0);

    for (int i = 0; i < count; i += 4)
    {
        __m128 a = _mm_load_ps(&dataA[i]);
        __m128 b = _mm_load_ps(&dataB[i]);
        __m128 r = _mm_add_ps(a, b);
        _mm_store_ps(&results[i], r);
    }
}

```

In this version, we loop over the values four at a time. We load blocks of four floats into SSE registers, add them in parallel, and store the results into a corresponding block of four floats within the result array. This is called *vectorizing* our loop. (In this example, we’re assuming that the sizes of the three arrays are equal, and that the size is a multiple of four. The caller is responsible for *padding* the arrays with one, two or three dummy values each, as necessary, in order to meet this requirement.)

Vectorization can lead to a significant speed improvement. This particular example isn’t quite four times faster, due to the overhead of having to load the values in groups of four and store the results on each iteration; but when I measured these functions running on very large arrays of floats, the non-vectorized loop took roughly 3.8 times as long to do its work as the vectorized loop.

4.10.3 Vectorized Dot Products

Let’s apply vectorization to a somewhat more interesting task: Calculating dot products. Given two arrays of four-element vectors, the goal is to calculate their dot products pairwise, and store the results into an output array of floats.

Here’s a reference implementation without using SSE. In this implementation, each contiguous block of four floats within the `a[]` and `b[]` input arrays is interpreted as one homogeneous four-element vector.

```

void DotArrays_ref(int count,
                  float r[],
                  const float a[],

```

```

        const float b[]))
{
    for (int i = 0; i < count; ++i)
    {
        // treat each block of four floats as a
        // single four-element vector
        const int j = i * 4;

        r[i] = a[j+0]*b[j+0] // ax*bx
              + a[j+1]*b[j+1] // ay*by
              + a[j+2]*b[j+2] // az*bz
              + a[j+3]*b[j+3]; // aw*bw
    }
}

```

4.10.3.1 A First Attempt (That's Slow)

Here's a first attempt at using SSE intrinsics for this task:

```

void DotArrays_sse_horizontal(int count,
                              float r[],
                              const float a[],
                              const float b[])
{
    for (int i = 0; i < count; ++i)
    {
        // treat each block of four floats as a
        // single four-element vector
        const int j = i * 4;

        __m128 va = _mm_load_ps(&a[j]); // ax,ay,az,aw
        __m128 vb = _mm_load_ps(&b[j]); // bx,by,bz,bw

        __m128 v0 = _mm_mul_ps(va, vb);

        // add across the register...
        __m128 v1 = _mm_hadd_ps(v0, v0);
        // (v0w+v0z, v0y+v0x, v0w+v0z, v0y+v0x)
        __m128 vr = _mm_hadd_ps(v1, v1);
        // (v0w+v0z+v0y+v0x, v0w+v0z+v0y+v0x,
        //   v0w+v0z+v0y+v0x, v0w+v0z+v0y+v0x)

        _mm_store_ss(&r[i], vr); // extract vr.x as a float
    }
}

```

This implementation required a new instruction: `_mm_hadd_ps()` (*horizontal*

add). This intrinsic operates on a single input register (x, y, z, w) and calculates two sums: $s = x + y$ and $t = z + w$. It stores these two sums into the four slots of the destination register as (t, s, t, s) . Performing this operation twice allows us to calculate the sum $d = x + y + z + w$. This is called *adding across a register*.

Adding across a register is not usually a good idea because it's a very *slow* operation. Profiling the `DotArrays_sse()` implementation shows that it actually takes a little bit *more* time than the reference implementation. Using SSE here has actually slowed us down!¹²

4.10.3.2 A Better Approach

The key to realizing the power of SIMD parallelism for dot products is to figure out a way to avoid having to add across a register. This can be done, but we'll have to *transpose* our input vectors first. By storing them in transposed order, we can calculate our dot product in just the same way that we calculated it when using floats: We multiply the x components, then add that result to the product of the y components, then add that result to the product of the z components, and finally add that result to the product of the w components. Here's what the code looks like:

```
void DotArrays_sse(int count,
                  float r[],
                  const float a[],
                  const float b[])
{
    for (int i = 0; i < count; i += 4)
    {
        __m128 vaX = _mm_load_ps(&a[(i+0)*4]); // a[0,4,8,12]
        __m128 vaY = _mm_load_ps(&a[(i+1)*4]); // a[1,5,9,13]
        __m128 vaZ = _mm_load_ps(&a[(i+2)*4]); // a[2,6,10,14]
        __m128 vaW = _mm_load_ps(&a[(i+3)*4]); // a[3,7,11,15]

        __m128 vbX = _mm_load_ps(&b[(i+0)*4]); // b[0,4,8,12]
        __m128 vbY = _mm_load_ps(&b[(i+1)*4]); // b[1,5,9,13]
        __m128 vbZ = _mm_load_ps(&b[(i+2)*4]); // b[2,6,10,14]
        __m128 vbW = _mm_load_ps(&b[(i+3)*4]); // b[3,7,11,15]

        __m128 result;
        result = _mm_mul_ps(vaX, vbX);
        result = _mm_add_ps(result, _mm_mul_ps(vaY, vbY));
        result = _mm_add_ps(result, _mm_mul_ps(vaZ, vbZ));
    }
}
```

¹²With SSE4, Intel introduced the intrinsic `_mm_dp_ps()` (and the corresponding `dpps` instruction) which calculates a dot product with somewhat lower latency than the version involving two invocations of `_mm_hadd_ps()`. But all horizontal adds are very expensive, and should be avoided wherever possible.

```

        result = _mm_add_ps(result, _mm_mul_ps(vaW, vbW));
    }
    _mm_store_ps(&r[i], result);
}

```

The MADD Instruction

It's interesting to note that a multiply followed by an add is such a common operation that it has its own name—*madd*. Some CPUs provide a single SIMD instruction for performing a *madd* operation. For example, the PowerPC AltiVec intrinsic `vec_madd()` performs this operation. So in AltiVec, the guts of our `DotArrays()` function could be made just a little bit simpler:

```

vector float result = vec_mul(vaX, vbX);
result = vec_madd(vaY, vbY, result);
result = vec_madd(vaZ, vbZ, result);
result = vec_madd(vaW, vbW, result);

```

4.10.3.3 Transpose as We Go

The above implementation *assumes* that the input data has already been transposed by the caller. In other words, the `a[]` array is assumed to contain the components $\{a_0, a_4, a_8, a_{12}, a_1, a_5, a_9, a_{13}, \dots\}$ and likewise for the `b[]` array. If we want to operate on input data that's in the same format as it was for the reference implementation, we'll have to do the transposition within our function. Here's how:

```

void DotArrays_sse_transpose(int count,
                             float r[],
                             const float a[],
                             const float b[])
{
    for (int i = 0; i < count; i += 4)
    {
        __m128 vaX = _mm_load_ps(&a[(i+0)*4]); // a[0,1,2,3]
        __m128 vaY = _mm_load_ps(&a[(i+1)*4]); // a[4,5,6,7]
        __m128 vaZ = _mm_load_ps(&a[(i+2)*4]); // a[8,9,10,11]
        __m128 vaW = _mm_load_ps(&a[(i+3)*4]); // a[12,13,14,15]

        __m128 vbX = _mm_load_ps(&b[(i+0)*4]); // b[0,1,2,3]
        __m128 vbY = _mm_load_ps(&b[(i+1)*4]); // b[4,5,6,7]
        __m128 vbZ = _mm_load_ps(&b[(i+2)*4]); // b[8,9,10,11]
        __m128 vbW = _mm_load_ps(&b[(i+3)*4]); // b[12,13,14,15]
    }
}

```



```

    _MM_TRANSPOSE4_PS(vaX, vaY, vaZ, vaW);
                                // vaX = a[0,4,8,12]
                                // vaY = a[1,5,9,13]
                                // ...
    _MM_TRANSPOSE4_PS(vbX, vbY, vbZ, vbW);
                                // vbX = b[0,4,8,12]
                                // vbY = b[1,5,9,13]
                                // ...

    __m128 result;
    result = _mm_mul_ps(vaX, vbX);
    result = _mm_add_ps(result, _mm_mul_ps(vaY, vbY));
    result = _mm_add_ps(result, _mm_mul_ps(vaZ, vbZ));
    result = _mm_add_ps(result, _mm_mul_ps(vaW, vbW));

    _mm_store_ps(&r[i], result);
}
}

```

Those two calls to `_MM_TRANSPOSE()` are actually invocations of a somewhat complex macro that uses *shuffle* instructions to move the components of the four input registers around. Thankfully shuffling isn't a particularly expensive operation, so transposing our vectors as we calculate the dot products doesn't introduce too much overhead. Profiling all three implementations of `DotArrays()` shows that our final version (the one that transposes the vectors as it goes) is roughly 3.5 times faster than the reference implementation.

4.10.3.4 Shuffle and Transpose

For the curious reader, here's what the `_MM_TRANSPOSE()` macro looks like:

```

#define _MM_TRANSPOSE4_PS(row0, row1, row2, row3) \
{ __m128 tmp3, tmp2, tmp1, tmp0; \
 \
    tmp0 = _mm_shuffle_ps((row0), (row1), 0x44); \
    tmp2 = _mm_shuffle_ps((row0), (row1), 0xEE); \
    tmp1 = _mm_shuffle_ps((row2), (row3), 0x44); \
    tmp3 = _mm_shuffle_ps((row2), (row3), 0xEE); \
 \
    (row0) = _mm_shuffle_ps(tmp0, tmp1, 0x88); \
    (row1) = _mm_shuffle_ps(tmp0, tmp1, 0xDD); \
    (row2) = _mm_shuffle_ps(tmp2, tmp3, 0x88); \
    (row3) = _mm_shuffle_ps(tmp2, tmp3, 0xDD); }

```

Those crazy hexadecimal numbers are bit-packed four-element fields called

shuffle masks. They tell the `_mm_shuffle()` intrinsic how exactly to shuffle the components. These bit-packed fields are a common source of confusion, possibly because of the naming conventions used in most documentation. But it's actually pretty simple: A shuffle mask is constructed out of four integers, each of which represents one of the components of an SSE register (and hence can have a value between 0 and 3).

```
#define SHUFMASK(p,q,r,s) \
    (p | (q<<2) | (r<<4) | (s<<6))
```

Passing two SSE registers `a` and `b` along with a shuffle mask to the `_mm_shuffle_ps()` intrinsic results in the specified components of `a` and `b` appearing in the output register `r` as follows:

```
__m128 a = ...;
__m128 b = ...;
__m128 r = _mm_shuffle_ps(a, b,
                          SHUFMASK(p,q,r,s));
// r == ( a[p], a[q], b[r], b[s] )
```

4.10.4 Vector-Matrix Multiplication with SSE

Now that we understand how to perform a dot product, we can multiply a four-element vector with a 4×4 matrix. To do it, we simply need to perform four dot products between the input vector and each of the four rows of the input matrix.

We'll start by defining a `Mat44` class that encapsulates four SSE vectors, representing its four rows. We'll use a union so that we can easily access the individual members of the matrix as floats. (This works because instances of our `Mat44` class will always reside in memory, never directly in SSE registers.)

```
union Mat44
{
    float c[4][4]; // components
    __m128 row[4]; // rows
};
```

The function to multiply a vector and a matrix looks like this:

```
__m128 MulVecMat_sse(const __m128& v, const Mat44& M)
{
    // first transpose v
    __m128 vX = _mm_shuffle_ps(v, v, 0x00); // (vx,vx,vx,vx)
```

```

__m128 vY = _mm_shuffle_ps(v, v, 0x55); // (vy,vy,vy,vy)
__m128 vZ = _mm_shuffle_ps(v, v, 0xAA); // (vz,vz,vz,vz)
__m128 vW = _mm_shuffle_ps(v, v, 0xFF); // (vw,vw,vw,vw)

__m128 r =      _mm_mul_ps(vX, M.row[0]);
r = _mm_add_ps(r, _mm_mul_ps(vY, M.row[1]));
r = _mm_add_ps(r, _mm_mul_ps(vZ, M.row[2]));
r = _mm_add_ps(r, _mm_mul_ps(vW, M.row[3]));
return r;
}

```

The shuffles are used to replicate *one* component of **v** (either v_x , v_y , v_z or v_w) across *all four* lanes of an SSE register. This has the effect of transposing **v** prior to performing the dot product with the rows of **M**, which are already transposed. (Remember that vector-matrix multiplication normally involves taking dot products between an input vector and the *columns* of the matrix. Here, we’re transposing **v** into four SSE registers, and then doing our math, component-wise, with the *rows* of the matrix.)

4.10.5 Matrix-Matrix Multiplication with SSE

Multiplying two 4×4 matrices with SSE intrinsics is trivial once we have a function to multiply a vector and a matrix. Here’s what the code looks like:

```

void MulMatMat_sse(Mat44& R, const Mat44& A, const Mat44& B)
{
    R.row[0] = MulVecMat_sse(A.row[0], B);
    R.row[1] = MulVecMat_sse(A.row[1], B);
    R.row[2] = MulVecMat_sse(A.row[2], B);
    R.row[3] = MulVecMat_sse(A.row[3], B);
}

```

4.10.6 Generalized Vectorization

Because an SSE register contains four floating-point values, it’s tempting to think of it as a natural “fit” for the components of a four-element homogeneous vector **v**, and to think that the best use of SSE is for doing 3D vector math. However, this is a very limiting way of thinking about SIMD parallelism.

Most “batched” operations, in which a single computation is performed repeatedly on a large dataset, can be vectorized using SIMD parallelism. If you think about it, the components of a SIMD register really function like parallel “lanes” in which arbitrary processing can be performed. Working with `float`

variables gives us a single lane, but working with 128-bit (four-element) SIMD variables allows us to do that same calculation in parallel across four lanes—in other words, we can perform our calculations four at a time. Working with 256-bit AVX registers gives us eight lanes, allowing us to perform our calculations eight at a time. And AVX-512 gives us 16 lanes, letting us do 16 calculations at a time.

The easiest way to write *vectorized* code is to start out by writing it as a single-lane algorithm (just using `floats`). Once it works, we can convert it to operate N elements at a time, using SIMD registers that have an N -lane capacity. We've already seen this process in action: In Section 4.10.3, we first wrote a loop that performed a large batch of dot products one at a time, and then we converted to use SSE so that we could perform those dot products four at a time.

One nice side-effect of vectorizing your code in this way is that you can reap the benefits of wider SIMD hardware with little adjustment to your code. On a machine with only SSE support, you perform four operations per iteration of your loop; on a machine that supports AVX, you simply change it to do eight operations per iteration; and on an AVX-512 system, you can do 16 operations per iteration.

Interestingly, most optimizing compilers can *vectorize* some kinds of single-lane loops automatically. In fact, when writing the above examples, it took some doing to force the compiler *not* to vectorize my single-lane code, so that I could compare its performance to my SIMD implementation! Once again, it's always a good idea to look at the disassembly when writing optimized code—you may discover the compiler is doing more (or less) than you thought!

4.10.7 Vector Predication

Let's take a look at another (totally contrived) example. This example will reinforce the ideas of generalized vectorization, but it will also serve to illustrate another useful technique: *vector predication*.

Imagine that we needed to take the square roots of a large array of `floats`. We'd start out by writing it as a single-lane loop, like this:

```
#include <cmath>

void SqrtArray_ref(float* __restrict__ r,
                  const float* __restrict__ a,
                  int count)
{
    for (unsigned i = 0; i < count; ++i)
    {
```

```

        if (a[i] >= 0.0f)
            r[i] = std::sqrt(a[i]);
        else
            r[i] = 0.0f;
    }
}

```

Next, let's convert this loop into SSE, performing four square roots at a time:

```

#include <xmmintrin.h>

void SqrtArray_sse_broken(float* __restrict__ r,
                          const float* __restrict__ a,
                          int count)
{
    assert(count % 4 == 0);
    __m128 vz = _mm_set1_ps(0.0f); // all zeros

    for (int i = 0; i < count; i += 4)
    {
        __m128 va = _mm_load_ps(a + i);

        __m128 vr;
        if (_mm_cmpge_ps(va, vz)) // ???
            vr = _mm_sqrt_ps(va);
        else
            vr = vz;

        _mm_store_ps(r + i, vr);
    }
}

```

This seems simple enough: We simply stride through the input array four floats at a time, load groups of four floats into an SSE register, and then perform four parallel square roots with `_mm_sqrt_ps()`.

However, there's one small gotcha in this loop. We need to check whether the input values are greater than or equal to zero, because the square root of a negative number is imaginary (and will therefore produce QNaN). The intrinsic `_mm_cmpge_ps()` compares the values of two SSE registers, component-wise, to see if they are greater than or equal to a vector of values supplied by the caller. However, this function doesn't return a `bool`. How can it? We're comparing four values to four other values, so some of them may pass the test while others fail it. That means we can't just do an `if` check against the results of `_mm_cmpge_ps()`.¹³

¹³The `if` check in the single-lane reference implementation also prevents the compiler from automatically vectorizing the loop.

Does this spell doom for our vectorized implementation? Thankfully not. All SSE comparison intrinsics like `_mm_cmpge_ps()` produce a four-component result, stored in an SSE register. But instead of containing four floating-point values, the result consists of four 32-bit masks. Each mask contains all binary 1s (0xFFFFFFFFU) if the corresponding component in the input register passed the test, and all binary 0s (0x0U) if that component failed the test.

We can use the results of an SSE comparison intrinsic by applying it as a bit mask in order to *select* between one of two possible results. In our example, when the input value passes the test (is greater than or equal to zero), we want to select the square root of that input value; when it fails the test (is negative), we want to select a value of zero. This is called *predication*, and because we're applying it to SIMD vectors, it's called *vector predication*.

In Section 4.2.6.2, we saw how to do predication with floating-point values, using bitwise AND, OR and NOT operators. Here's an excerpt from that example:

```
// ...

// select quotient when mask is all ones, or default
// value d when mask is all zeros (NOTE: this won't
// work as written -- you'd need to use a union to
// interpret the floats as unsigned for masking)
const float result = (q & mask) | (d & ~mask);
return result;
}
```

It's no different here, we just need to use SSE versions of these bitwise operators:

```
#include <xmmintrin.h>

void SqrtArray_sse(float* __restrict__ r,
                  const float* __restrict__ a,
                  int count)
{
    assert(count % 4 == 0);
    __m128 vz = _mm_set1_ps(0.0f);

    for (int i = 0; i < count; i += 4)
    {
        __m128 va = _mm_load_ps(a + i);
```

```

    // always do the quotient, but it may end
    // up producing QNaN in some or all lanes
    __m128 vq = _mm_sqrt_ps(va);

    // now select between vq and vz, depending
    // on whether the input was greater than
    // or equal to zero or not
    __m128 mask = _mm_cmpge_ps(va, zero);

    // (vq & mask) | (vz & ~mask)
    __m128 qmask = _mm_and_ps(mask, vq);
    __m128 znotmask = _mm_andnot_ps(mask, vz);
    __m128 vr = _mm_or_ps(qmask, znotmask);

    _mm_store_ps(r + i, vr);
}
}

```

It's convenient and commonplace to encapsulate this *vector predication* operation in a function, which is typically called *vector select*. PowerPC's AltiVec ISA provides an intrinsic called `vec_sel()` for this purpose. It works like this:

```

// pseudocode illustrating how AltiVec's vec_sel() intrinsic
// works
vector float vec_sel(vector float falseVec,
                      vector float trueVec,
                      vector bool mask)
{
    vector float r;
    for (each lane i)
    {
        if (mask[i] == 0)
            r[i] = falseVec[i];
        else
            r[i] = trueVec[i];
    }
    return r;
}

```

SSE2 provided no vector select instruction, but thankfully one was introduced in SSE4—it is emitted by the intrinsic `_mm_blendv_ps()`.

Let's take a look at how we might implement a vector select operation ourselves. We can write it like this:

```

__m128 __mm_select_ps(const __m128 a,
                     const __m128 b,
                     const __m128 mask)
{
    // (b & mask) | (a & ~mask)
    __m128 bmask = __mm_and_ps(mask, b);
    __m128 anotmask = __mm_andnot_ps(mask, a);
    return __mm_or_ps(bmask, anotmask);
}

```

Or if we're feeling brave, we can accomplish the same thing with exclusive OR:

```

__m128 __mm_select_ps(const __m128 a,
                     const __m128 b,
                     const __m128 mask)
{
    // (((a ^ b) & mask) ^ a)
    __m128 diff = __mm_xor_ps(a, b);
    return __mm_xor_ps(a, __mm_and_ps(mask, diff));
}

```

See if you can figure out how this works. Here are two hints: The exclusive OR operator calculates the *bitwise difference* between two values. Two XORs in a row leave the input value unchanged ($((a \oplus b) \oplus b == a)$).

4.11 Introduction to GPGPU Programming

We said in the preceding section that most optimizing compilers can *vectorize* some code automatically, if the target CPU includes a SIMD vector processing unit, and if the source code meets certain requirements (such as not involving complex branching). Vectorization is also one of the pillars of *general-purpose GPU* programming (GPGPU). In this section, we'll take a brief introductory look at how a GPU differs from a CPU in terms of its hardware architecture, and how the concepts of SIMD parallelism and vectorization enable programmers to write *compute shaders* that are capable of processing large amounts of data in parallel on a GPU.

4.11.1 Data-Parallel Computations

A GPU is a specialized coprocessor designed specifically to accelerate those computations that involve a high degree of *data parallelism*. It does so by combining SIMD parallelism (vectorized ALUs) with MIMD parallelism (by employing a form of preemptive multithreading). NVIDIA coined the term *single instruction multiple thread* (SIMT) to describe this SIMD/MIMD hybrid design,

although the design is not unique to NVIDIA GPUs—although the specifics of GPU designs vary from vendor to vendor and from product line to product line in significant ways, all GPUs employ the general principles of SIMT parallelism in their designs.

GPUs are designed specifically to perform *data-parallel* computations on very large datasets. In order for a computational task to be well-suited to execution on a GPU, the computations performed on any one element of the dataset must be *independent* of the results of computations on other elements. In other words, it must be possible to process the elements in any order.

The simple examples of SIMD vectorization that we presented starting in Section 4.10.3 are all examples of data-parallel computations. Recall this function, which processes two potentially very large arrays of input vectors and produces an output array containing the scalar dot products of those vectors:

```
void DotArrays_ref(unsigned count,
                  float r[],
                  const float a[],
                  const float b[])
{
    for (unsigned i = 0; i < count; ++i)
    {
        // treat each block of four floats as a
        // single four-element vector
        const unsigned j = i * 4;

        r[i] = a[j+0]*b[j+0] // ax*bx
              + a[j+1]*b[j+1] // ay*by
              + a[j+2]*b[j+2] // az*bz
              + a[j+3]*b[j+3]; // aw*bw
    }
}
```

The computation performed by each iteration of this loop is independent of the computations performed by the other iterations. That means that we are free to perform the computations in any order we see fit. Moreover, this property is what allows us to apply SSE or AVX intrinsics to vectorize the loop—instead of performing the computations one-at-a-time, we can use SIMD parallelism to perform four, eight or 16 computations simultaneously, thereby reducing the iteration count by a factor of four, eight or 16, respectively.

Now imagine taking SIMD parallelization to an extreme. What if we had a SIMD VPU with 1024 lanes? In that case, we could divide the total number of iterations by 1024—and when the input arrays contain 1024 elements or fewer, we could literally execute the entire loop in a single iteration!

This is, roughly speaking, what a GPU does. However, it doesn't use SIMDs that are *actually* 1024 lanes wide each. A GPU's *SIMD units* are typically eight or 16 lanes wide, but they process workloads in batches of 32 or 64 elements at a time. What's more, a GPU contains *many* such SIMD units. So a large workload can be dispatched across these SIMD units in parallel, resulting in the GPU being capable of processing literally thousands of data elements in parallel.

Data-parallel computations are just what the doctor ordered when applying a pixel shader (also known as a fragment shader) to millions of pixels, or a vertex shader to hundreds of thousands or even millions of 3D mesh vertices, every frame at 30 or 60 FPS. But modern GPUs expose their computational power to programmers, allowing us to write general-purpose *compute shaders*. As long as the computations we wish to perform on a large dataset have the property of being largely independent of one another, they can probably be executed by a GPU more efficiently than they could be on a CPU.

4.11.2 Compute Kernels

In Section 4.10.3, we saw that in order to vectorize a loop like the one in the `DotArrays_ref()` function, we must rewrite the code. The vectorized version of the function makes use of SSE or AVX intrinsics; our *scalar* data types are replaced by *vector* types such as SSE's `__m128` or AltiVec's `vector float`; and the loop is hard-coded to iterate four, eight or 16 elements at a time.

When writing a GPGPU compute shader, we take a different tack. Instead of hard-coding the loop to operate in fixed-size batches, we leave the loop as a "single-lane" computation using scalar data types. Then, we extract the body of the loop into a separate function known as a *kernel*. Here's what the example above would look like as a kernel:

```
void DotKernel(unsigned i,
               float r[],
               const float a[],
               const float b[])
{
    // treat each block of four floats as a
    // single four-element vector
    const unsigned j = i * 4;

    r[i] = a[j+0]*b[j+0] // ax*bx
          + a[j+1]*b[j+1] // ay*by
          + a[j+2]*b[j+2] // az*bz
          + a[j+3]*b[j+3]; // aw*bw
}
```

```

void DotArrays_gpgpu1(unsigned count,
                      float r[],
                      const float a[],
                      const float b[])
{
    for (unsigned i = 0; i < count; ++i)
    {
        DotKernel(i, r, a, b);
    }
}

```

The `DotKernel()` function is now in a form that's suitable for conversion into a *compute shader*. It processes just one element of the input data, and produces a single output element. This is analogous to how a pixel/fragment shader receives a single input pixel/fragment color and transforms it into a single output color, or to how a vertex shader receives a single input vertex and produces a single output vertex. The GPU effectively runs the `for` loop for us, calling our kernel function once for each element of our dataset.

GPGPU compute kernels are typically written in a special *shading language* which can be compiled into machine code that's understandable by the GPU. Shading languages are usually very close to C in syntax, so converting a C or C++ loop into a GPU compute kernel isn't usually a particularly difficult undertaking. Some examples of shading languages include DirectX's HLSL (high-level shader language), OpenGL's GLSL, NVIDIA's Cg and CUDA C languages, and OpenCL.

Some shading languages require you to move your kernel code into special source files, separate from your C++ application code. OpenCL and CUDA C, however, are extensions to the C++ language itself. As such, they permit programmers to write compute kernels as regular C/C++ functions, with only minor syntactic adjustments, and to invoke those kernels on the GPU with relatively simple syntax.

As a concrete example, here's our `DotKernel()` function written in CUDA C:

```

__global__ void DotKernel_CUDA(int count,
                               float* r,
                               const float* a,
                               const float* b)
{
    // CUDA provides a magic "thread index" to each
    // invocation of the kernel -- this serves as
    // our loop index i

```

```

size_t i = threadIdx.x;

// make sure the index is valid
if (i < count)
{
    // treat each block of four floats as a
    // single four-element vector
    const unsigned j = i * 4;

    r[i] = a[j+0]*b[j+0] // ax*bx
          + a[j+1]*b[j+1] // ay*by
          + a[j+2]*b[j+2] // az*bz
          + a[j+3]*b[j+3]; // aw*bw
}
}

```

You'll notice that the loop index `i` is taken from a variable called `threadIdx` within the kernel function itself. The thread index is a "magic" input provided by the compiler, in much the same way that the `this` pointer "magically" points to the current instance within a C++ class member function. We'll talk more about the thread index in the next section.

4.11.3 Executing a Kernel

Given that we've written a compute kernel, let's see how to execute it on the GPU. The details differ from shading language to shading language, but the key concepts are roughly equivalent. For example, here's how we'd kick off our compute kernel in CUDA C:

```

void DotArrays_gpgpu2(unsigned count,
                      float r[],
                      const float a[],
                      const float b[])
{
    // allocate "managed" buffers that are visible
    // to both CPU and GPU
    int *cr, *ca, *cb;
    cudaMallocManaged(&cr, count * sizeof(float));
    cudaMallocManaged(&ca, count * sizeof(float) * 4);
    cudaMallocManaged(&cb, count * sizeof(float) * 4);

    // transfer the data into GPU-visible memory
    memcpy(ca, a, count * sizeof(float) * 4);
    memcpy(cb, b, count * sizeof(float) * 4);
}

```

```

// run the kernel on the GPU
DotKernel_CUDA <<<1, count>>> (cr, ca, cb, count);

// wait for GPU to finish
cudaDeviceSynchronize();

// return results and clean up
memcpy(r, cr, count * sizeof(float));
cudaFree(cr);
cudaFree(ca);
cudaFree(cb);
}

```

A bit of set-up code is required to allocate the input and output buffers as “managed” memory that is visible to both CPU and GPU, and to copy the input data into them. The CUDA-specific triple angled brackets notation (<<<G,N>>>) then executes the compute kernel on the GPU, by submitting a request to the driver. The call to `cudaDeviceSynchronize()` forces the CPU to wait until the GPU has done its work (in much the same way that `pthread_join()` forces one thread to wait for the completion of another). Finally, we free the GPU-visible data buffers.

Let’s have a closer look at the <<<G,N>>> angled bracket notation. The second argument N within the angled brackets allows us to specify the *dimensions* of our input data. This corresponds to the number of *iterations* of our loop that we want the GPU to perform. It can actually be a one-, two- or three-dimensional quantity, allowing us to process one-, two- or three-dimensional input arrays. However, just like in C/C++, a multidimensional array is really just a one-dimensional array that’s indexed in a special way. For example, in C/C++ a two-dimensional array access written like `[row][column]` is really equivalent to a one-dimensional array access `[row*numColumns + column]`. The same principle applies to multidimensional GPU buffers.

The first argument G in the angled brackets tells the driver how many *thread groups* (known as *thread blocks* in NVIDIA terminology) to use when running this compute kernel. A compute job with a single thread group is constrained to run on a single *compute unit* on the GPU. A compute unit is essentially a *core* within the GPU—a hardware component that is capable of executing an instruction stream. Passing a larger number for G allows the driver to divide the workload up across more than one compute unit.

4.11.4 GPU Threads and Thread Groups

The G argument tells the GPU driver into how many *thread groups* to divide our work. As you might expect, a thread group is comprised of some number

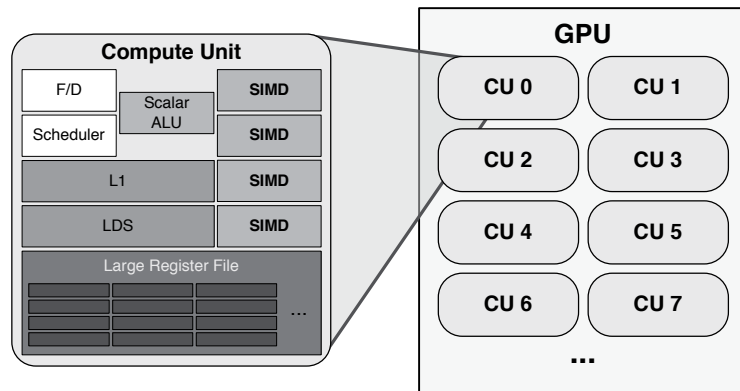


Figure 4.39. A typical GPU is comprised of multiple compute units (CU), each of which acts like a stripped down CPU core. A CU contains an instruction fetch/decode unit, an L1 cache, possibly a block of local data storage RAM (LDS), a scalar ALU, and a number of SIMD units for executing vectorized code.

of *GPU threads*. But what exactly does the term “thread” mean in the context of a GPU?

Every GPU kernel is compiled into an *instruction stream* consisting of a sequence of GPU machine language instructions, in much the same way that a C/C++ function is compiled down into a stream of CPU instructions. So a GPU “thread” is equivalent to a CPU “thread,” in the sense that both kinds of threads represent a stream of machine language instructions that can be executed by one or more cores. However, a GPU executes its threads in a manner that is somewhat different from the way in which a CPU executes its threads. As a result, the term “thread” has a subtly different meaning when applied to GPU compute kernels than it has when applied to running programs on a CPU.

In order to understand this difference in terminology, let’s take a brief glimpse at the architecture of a GPU. We said in Section 4.11.1 that a GPU is comprised of multiple *compute units*, each of which contains some number of *SIMD units*. We can think of a compute unit like a stripped down CPU core: It contains an instruction fetch/decode unit, possibly some memory caches, a regular “scalar” ALU, and usually somewhere in the neighborhood of four SIMD units, which serve much the same function as the vector processing unit (VPU) in a SIMD-enabled CPU. This architecture is illustrated in Figure 4.39.

The SIMD units in a CU have different lane widths on different GPUs, but for the sake of this discussion let’s assume we’re working on an AMD

Radeon™ Graphics Core Next (GCN) architecture, in which the SIMDs are 16 lanes wide. The CU is not capable of speculative or out-of-order execution—it simply reads a stream of instructions and executes them one by one, using the SIMD units to apply each instruction¹⁴ to 16 elements of input data in parallel.

To execute a compute kernel on a CU, the driver first subdivides the input data buffers into blocks consisting of 64 elements each. For each of these 64-element blocks of data, the compute kernel is invoked on one CU. Such an invocation is called a *wavefront* (also known as a *warp* in NVIDIA speak). When executing a wavefront, the CU fetches and decodes the instructions of the kernel one by one. Each instruction is applied to 16 data elements in *lock step* using a SIMD unit. Internally, the SIMD unit consists of a four-*stage* pipeline, so it takes four clock cycles to complete. So rather than allow the stages of this pipeline to sit idle for three clock cycles out of every four, the CU applies the same instruction three more times, to three more blocks of 16 data elements. This is why a wavefront consists of 64 data elements, even though the SIMDs in the CU are only 16 lanes wide.

Because of this somewhat peculiar way in which a CU executes an instruction stream, the term “*GPU thread*” actually refers to a single SIMD *lane*. You can think of a GPU thread, therefore, as a single iteration of the original non-vectorized loop that we started with, before we converted its body into a compute kernel. Alternatively, you can think of a GPU thread as a single invocation of the kernel function, operating on a single input datum and producing a single output datum. The fact that the GPU actually runs multiple GPU threads in parallel (i.e., the fact that it really runs the kernel once per wavefront, but processes 64 data elements at a time) is just an implementation detail. By insulating the programmer from having to think about the details of how the computation is vectorized on any particular GPU, compute kernels (and graphics shaders as well) can be written in a portable manner.

4.II.4.1 From SIMD to SIMT

The term *single instruction multiple thread* (SIMT) was introduced to underscore the fact that a GPU doesn’t just use SIMD parallelism—it also uses a form of preemptive multithreading to time-slice between wavefronts. Let’s take a brief look at why this is done.

A SIMD unit runs a wavefront by applying each instruction in the shader program to 64 data elements at a time, essentially in lock-step. (We can ignore the fact that the wavefront is processed in subgroups of 16 elements each for the purposes of the present discussion.) However, any one instruction in the

¹⁴The compute units on a GPU do contain a scalar ALU and therefore can perform some instructions in “single lane” fashion, operating on a single input datum at a time.

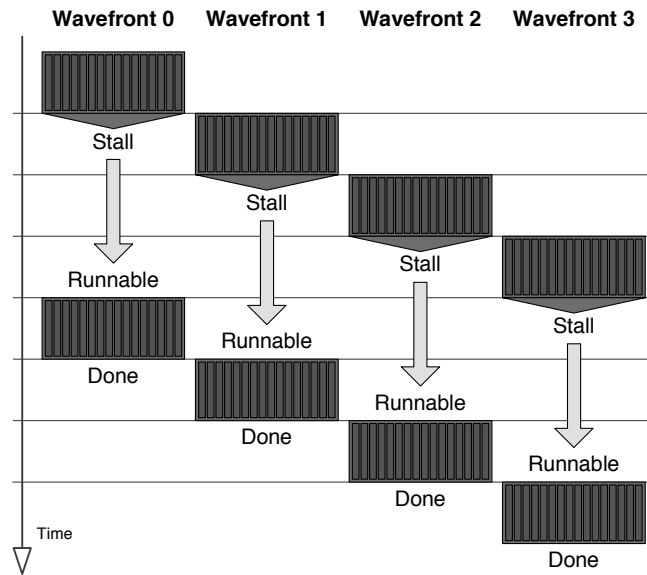


Figure 4.40. Whenever one wavefront stalls, for example due to memory access latency, the SIMD unit context switches to another wavefront to fill the delay slot.

program might end up having to access memory, and that introduces a large stall while the SIMD unit waits for the memory controller to respond.

To fill these large delay slots, a SIMD unit time-slices between multiple wavefronts (taken from a single shader program, or potentially from many unrelated shader programs). Whenever one wavefront stalls, the SIMD unit context switches to another wavefront, thereby keeping the unit busy (as long as there are runnable wavefronts to switch to). This strategy is illustrated in Figure 4.40.

As you might well imagine, the SIMD units in a GPU need to perform context switches at a very high frequency. During a context switch on a CPU, the state of the outgoing thread's registers are saved to memory so they won't be lost, and then the state of the incoming thread's registers are read from memory into the CPU's registers so that it can continue executing where it left off. However, on a GPU it would take too much time to save the state of each wavefront's SIMD registers every time a context switch occurred.

To eliminate the cost of saving registers during context switches, each SIMD unit contains a very large register file. The number of *physical* registers in this register file is many times larger than the number of *logical* registers available to any one wavefront (typically on the order of ten times larger).

This means that the contents of the logical registers for up to ten wavefronts can be maintained at all times in these physical registers. And that in turn implies that context switches can be performed between wavefronts without saving or restoring any registers whatsoever.

4.11.5 Further Reading

Obviously GPGPU programming is a huge topic, and as always we've only just scratched the surface in this book. For more information on GPGPU and graphics shader programming, check out the following online tutorials and resources:

- Introduction to CUDA programming: <https://developer.nvidia.com/how-to-cuda-c-cpp>
- OpenCL learning resources: <https://developer.nvidia.com/opencl>
- HLSL programming guide and reference manual: [https://msdn.microsoft.com/en-us/library/bb509561\(v=VS.85\).aspx](https://msdn.microsoft.com/en-us/library/bb509561(v=VS.85).aspx)
- Introduction to the OpenGL shading language: https://www.khronos.org/opengl/wiki/OpenGL_Shading_Language
- AMD Radeon™ GCN architecture whitepaper: https://www.amd.com/Documents/GCN_Architecture_whitepaper.pdf



Taylor & Francis

Taylor & Francis Group

<http://taylorandfrancis.com>

5

3D Math for Games

A game is a mathematical model of a virtual world simulated in real time on a computer of some kind. Therefore, mathematics pervades everything we do in the game industry. Game programmers make use of virtually all branches of mathematics, from trigonometry to algebra to statistics to calculus. However, by far the most prevalent kind of mathematics you'll be doing as a game programmer is 3D vector and matrix math (i.e., 3D *linear algebra*).

Even this one branch of mathematics is very broad and very deep, so we cannot hope to cover it in any great depth in a single chapter. Instead, I will attempt to provide an overview of the mathematical tools needed by a typical game programmer. Along the way, I'll offer some tips and tricks, which should help you keep all of the rather confusing concepts and rules straight in your head. For an excellent in-depth coverage of 3D math for games, I highly recommend Eric Lengyel's book [32] on the topic. Chapter 3 of Christer Ericson's book [14] on real-time collision detection is also an excellent resource.

5.1 Solving 3D Problems in 2D

Many of the mathematical operations we're going to learn about in the following chapter work equally well in 2D and 3D. This is very good news, because

it means you can *sometimes* solve a 3D vector problem by thinking and drawing pictures in 2D (which is considerably easier to do!) Sadly, this equivalence between 2D and 3D does not hold all the time. Some operations, like the cross product, are only defined in 3D, and some problems only make sense when all three dimensions are considered. Nonetheless, it almost never hurts to start by thinking about a simplified two-dimensional version of the problem at hand. Once you understand the solution in 2D, you can think about how the problem extends into three dimensions. In some cases, you'll happily discover that your 2D result works in 3D as well. In others, you'll be able to find a coordinate system in which the problem really *is* two-dimensional. In this book, we'll employ two-dimensional diagrams wherever the distinction between 2D and 3D is not relevant.

5.2 Points and Vectors

The majority of modern 3D games are made up of three-dimensional objects in a virtual world. A game engine needs to keep track of the positions, orientations and scales of all these objects, animate them in the game world, and transform them into screen space so they can be rendered on screen. In games, 3D objects are almost always made up of triangles, the vertices of which are represented by points. So, before we learn how to represent whole objects in a game engine, let's first take a look at the point and its closely related cousin, the vector.

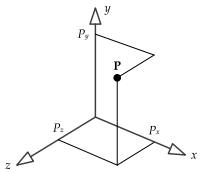


Figure 5.1. A point represented in Cartesian coordinates.

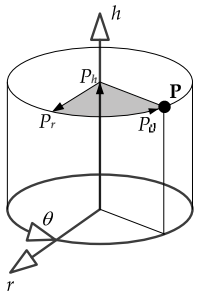


Figure 5.2. A point represented in cylindrical coordinates.

5.2.1 Points and Cartesian Coordinates

Technically speaking, a *point* is a location in n -dimensional space. (In games, n is usually equal to 2 or 3.) The Cartesian coordinate system is by far the most common coordinate system employed by game programmers. It uses two or three mutually perpendicular axes to specify a position in 2D or 3D space. So, a point P is represented by a pair or triple of real numbers, (P_x, P_y) or (P_x, P_y, P_z) (see Figure 5.1).

Of course, the Cartesian coordinate system is not our only choice. Some other common systems include:

- *Cylindrical coordinates.* This system employs a vertical “height” axis h , a radial axis r emanating out from the vertical, and a yaw angle *theta* (θ). In cylindrical coordinates, a point P is represented by the triple of numbers (P_h, P_r, P_θ) . This is illustrated in Figure 5.2.

- *Spherical coordinates.* This system employs a pitch angle ϕ (ϕ), a yaw angle θ (θ) and a radial measurement r . Points are therefore represented by the triple of numbers (P_r, P_ϕ, P_θ) . This is illustrated in Figure 5.3.

Cartesian coordinates are by far the most widely used coordinate system in game programming. However, always remember to select the coordinate system that best maps to the problem at hand. For example, in the game *Crank the Weasel* by Midway Home Entertainment, the main character Crank runs around an art-deco city picking up loot. I wanted to make the items of loot swirl around Crank's body in a spiral, getting closer and closer to him until they disappeared. I represented the position of the loot in cylindrical coordinates relative to the Crank character's current position. To implement the spiral animation, I simply gave the loot a constant angular speed in θ , a small constant linear speed inward along its radial axis r and a very slight constant linear speed upward along the h -axis so the loot would gradually rise up to the level of Crank's pants pockets. This extremely simple animation looked great, and it was much easier to model using cylindrical coordinates than it would have been using a Cartesian system.

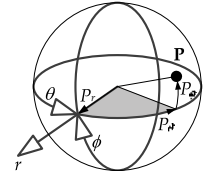


Figure 5.3. A point represented in spherical coordinates.

5.2.2 Left-Handed versus Right-Handed Coordinate Systems

In three-dimensional Cartesian coordinates, we have two choices when arranging our three mutually perpendicular axes: right-handed (RH) and left-handed (LH). In a right-handed coordinate system, when you curl the fingers of your right hand around the z -axis with the thumb pointing toward positive z coordinates, your fingers point from the x -axis toward the y -axis. In a left-handed coordinate system the same thing is true using your left hand.

The only difference between a left-handed coordinate system and a right-

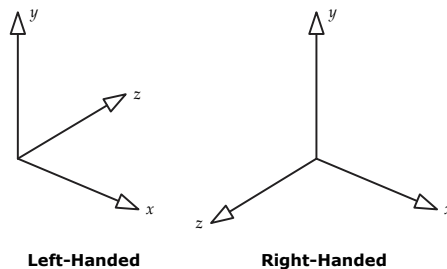


Figure 5.4. Left- and right-handed Cartesian coordinate systems.

handed coordinate system is the direction in which one of the three axes is pointing. For example, if the y -axis points upward and x points to the right, then z comes toward us (out of the page) in a right-handed system, and away from us (into the page) in a left-handed system. Left- and right-handed Cartesian coordinate systems are depicted in Figure 5.4.

It is easy to convert from left-handed to right-handed coordinates and vice versa. We simply flip the direction of any one axis, leaving the other two axes alone. It's important to remember that the rules of mathematics do not change between left-handed and right-handed coordinate systems. Only our *interpretation* of the numbers—our mental image of how the numbers map into 3D space—changes. Left-handed and right-handed conventions apply to visualization only, not to the underlying mathematics. (Actually, handedness does matter when dealing with cross products in physical simulations, because a cross product is not actually a vector—it's a special mathematical object known as a *pseudovector*. We'll discuss pseudovectors in a little more depth in Section 5.2.4.9.)

The mapping between the numerical representation and the visual representation is entirely up to us as mathematicians and programmers. We could choose to have the y -axis pointing up, with z forward and x to the left (RH) or right (LH). Or we could choose to have the z -axis point up. Or the x -axis could point up instead—or down. All that matters is that we decide upon a mapping, and then stick with it consistently.

That being said, some conventions do tend to work better than others for certain applications. For example, 3D graphics programmers typically work with a left-handed coordinate system, with the y -axis pointing up, x to the right and positive z pointing away from the viewer (i.e., in the direction the virtual camera is pointing). When 3D graphics are rendered onto a 2D screen using this particular coordinate system, increasing z -coordinates correspond to increasing *depth* into the scene (i.e., increasing distance away from the virtual camera). As we will see in subsequent chapters, this is exactly what is required when using a z -buffering scheme for depth occlusion.

5.2.3 Vectors

A *vector* is a quantity that has both a *magnitude* and a *direction* in n -dimensional space. A vector can be visualized as a *directed line segment* extending from a point called the *tail* to a point called the *head*. Contrast this to a *scalar* (i.e., an ordinary real-valued number), which represents a magnitude but has no direction. Usually scalars are written in italics (e.g., v) while vectors are written in boldface (e.g., $\mathbf{v}$).

A 3D vector can be represented by a triple of scalars (x, y, z) , just as a point

can be. The distinction between points and vectors is actually quite subtle. Technically, a vector is just an offset *relative to* some known point. A vector can be moved anywhere in 3D space—as long as its magnitude and direction don’t change, it is the same vector.

A vector can be used to represent a point, provided that we fix the tail of the vector to the origin of our coordinate system. Such a vector is sometimes called a *position vector* or *radius vector*. For our purposes, we can interpret any triple of scalars as either a point or a vector, provided that we remember that a *position vector* is constrained such that its tail remains at the origin of the chosen coordinate system. This implies that points and vectors are treated in subtly different ways mathematically. One might say that points are *absolute*, while vectors are *relative*.

The vast majority of game programmers use the term “vector” to refer both to points (position vectors) and to vectors in the strict linear algebra sense (purely directional vectors). Most 3D math libraries also use the term “vector” in this way. In this book, we’ll use the term “direction vector” or just “direction” when the distinction is important. Be careful to always keep the difference between points and directions clear in your mind (even if your math library doesn’t). As we’ll see in Section 5.3.6.1, directions need to be treated differently from points when converting them into homogeneous coordinates for manipulation with 4×4 matrices, so getting the two types of vector mixed up can and will lead to bugs in your code.

5.2.3.1 Cartesian Basis Vectors

It is often useful to define three *orthogonal unit vectors* (i.e., vectors that are mutually perpendicular and each with a length equal to one), corresponding to the three principal Cartesian axes. The unit vector along the x -axis is typically called $\mathbf{i}$, the y -axis unit vector is called $\mathbf{j}$, and the z -axis unit vector is called $\mathbf{k}$. The vectors $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$ are sometimes called Cartesian *basis vectors*.

Any point or vector can be expressed as a sum of scalars (real numbers) multiplied by these unit basis vectors. For example,

$$(5, 3, -2) = 5\mathbf{i} + 3\mathbf{j} - 2\mathbf{k}.$$

5.2.4 Vector Operations

Most of the mathematical operations that you can perform on scalars can be applied to vectors as well. There are also some new operations that apply only to vectors.

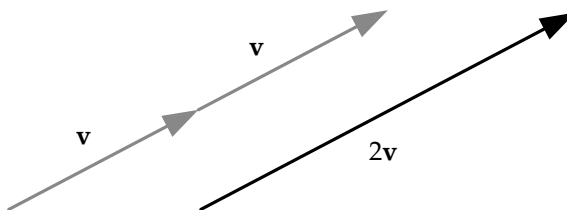


Figure 5.5. Multiplication of a vector by the scalar 2.

5.2.4.1 Multiplication by a Scalar

Multiplication of a vector $\mathbf{a}$ by a scalar s is accomplished by multiplying the individual components of $\mathbf{a}$ by s :

$$s\mathbf{a} = (sa_x, sa_y, sa_z).$$

Multiplication by a scalar has the effect of scaling the magnitude of the vector, while leaving its direction unchanged, as shown in Figure 5.5. Multiplication by -1 flips the direction of the vector (the head becomes the tail and vice versa).

The scale factor can be different along each axis. We call this *nonuniform scale*, and it can be represented as the *component-wise product* of a scaling vector $\mathbf{s}$ and the vector in question, which we'll denote with the $\otimes$ operator. Technically speaking, this special kind of product between two vectors is known as the *Hadamard product*. It is rarely used in the game industry—in fact, nonuniform scaling is one of its *only* commonplace uses in games:

$$\mathbf{s} \otimes \mathbf{a} = (s_x a_x, s_y a_y, s_z a_z). \quad (5.1)$$

As we'll see in Section 5.3.7.3, a scaling vector $\mathbf{s}$ is really just a compact way to represent a 3×3 diagonal scaling matrix $\mathbf{S}$. So another way to write Equation (5.1) is as follows:

$$\mathbf{a}\mathbf{S} = \begin{bmatrix} a_x & a_y & a_z \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 \\ 0 & s_y & 0 \\ 0 & 0 & s_z \end{bmatrix} = \begin{bmatrix} s_x a_x & s_y a_y & s_z a_z \end{bmatrix}.$$

We'll explore matrices in more depth in Section 5.3.

5.2.4.2 Addition and Subtraction

The addition of two vectors $\mathbf{a}$ and $\mathbf{b}$ is defined as the vector whose components are the sums of the *components* of $\mathbf{a}$ and $\mathbf{b}$. This can be visualized by placing

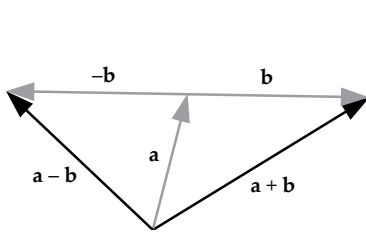


Figure 5.6. Vector addition and subtraction.

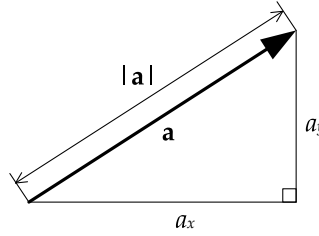


Figure 5.7. Magnitude of a vector (shown in 2D for ease of illustration).

the head of vector **a** onto the tail of vector **b**—the sum is then the vector from the tail of **a** to the head of **b** (see also Figure 5.6):

$$\mathbf{a} + \mathbf{b} = [(a_x + b_x), (a_y + b_y), (a_z + b_z)].$$

Vector subtraction $\mathbf{a} - \mathbf{b}$ is nothing more than addition of **a** and $-\mathbf{b}$ (i.e., the result of scaling **b** by -1 , which flips it around). This corresponds to the vector whose components are the difference between the components of **a** and the components of **b**:

$$\mathbf{a} - \mathbf{b} = [(a_x - b_x), (a_y - b_y), (a_z - b_z)].$$

Vector addition and subtraction are depicted in Figure 5.6.

Adding and Subtracting Points and Directions

You can add and subtract direction vectors freely. However, technically speaking, points cannot be added to one another—you can only add a direction vector to a point, the result of which is another point. Likewise, you can take the difference between two points, resulting in a direction vector. These operations are summarized below:

- direction + direction = direction
- direction – direction = direction
- point + direction = point
- point – point = direction
- point + point = *nonsense*

5.2.4.3 Magnitude

The magnitude of a vector is a scalar representing the length of the vector as it would be measured in 2D or 3D space. It is denoted by placing vertical bars

around the vector's boldface symbol. We can use the Pythagorean theorem to calculate a vector's magnitude, as shown in Figure 5.7:

$$|\mathbf{a}| = \sqrt{a_x^2 + a_y^2 + a_z^2}.$$

5.2.4.4 Vector Operations in Action

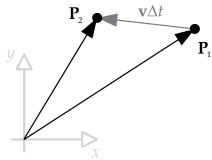


Figure 5.8. Simple vector addition can be used to find a character's position in the next frame, given her position and velocity in the current frame.

Believe it or not, we can already solve all sorts of real-world game problems given just the vector operations we've learned thus far. When trying to solve a problem, we can use operations like addition, subtraction, scaling and magnitude to generate new data out of the things we already know. For example, if we have the current position vector of an AI character $\mathbf{P}_1$, and a vector $\mathbf{v}$ representing her current velocity, we can find her position on the next frame $\mathbf{P}_2$ by scaling the velocity vector by the frame time interval Δt , and then adding it to the current position. As shown in Figure 5.8, the resulting vector equation is $\mathbf{P}_2 = \mathbf{P}_1 + \mathbf{v} \Delta t$. (This is known as *explicit Euler integration*—it's actually only valid when the velocity is constant, but you get the idea.)

As another example, let's say we have two spheres, and we want to know whether they intersect. Given that we know the center points of the two spheres, $\mathbf{C}_1$ and $\mathbf{C}_2$, we can find a direction vector between them by simply subtracting the points, $\mathbf{d} = \mathbf{C}_2 - \mathbf{C}_1$. The magnitude of this vector $d = |\mathbf{d}|$ determines how far apart the spheres' centers are. If this distance is less than the sum of the spheres' radii, they are intersecting; otherwise they're not. This is shown in Figure 5.9.

Square roots are expensive to calculate on most computers, so game programmers should always use the *squared magnitude* whenever it is valid to do

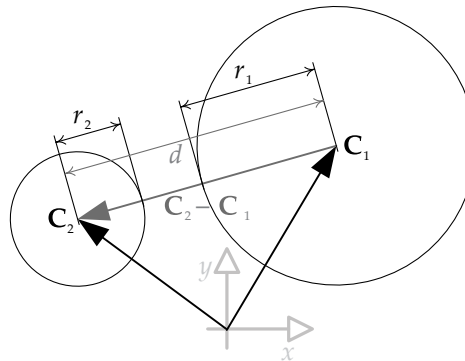


Figure 5.9. A sphere-sphere intersection test involves only vector subtraction, vector magnitude and floating-point comparison operations.

so:

$$|\mathbf{a}|^2 = (a_x^2 + a_y^2 + a_z^2).$$

Using the squared magnitude is valid when comparing the *relative lengths* of two vectors (“is vector $\mathbf{a}$ longer than vector $\mathbf{b}$?”), or when comparing a vector’s magnitude to some other (squared) scalar quantity. So in our sphere-sphere intersection test, we should calculate $d^2 = |\mathbf{d}|^2$ and compare this to the squared sum of the radii, $(r_1 + r_2)^2$ for maximum speed. When writing high-performance software, never take a square root when you don’t have to!

5.2.4.5 Normalization and Unit Vectors

A *unit vector* is a vector with a magnitude (length) of one. Unit vectors are very useful in 3D mathematics and game programming, for reasons we’ll see below.

Given an arbitrary vector $\mathbf{v}$ of length $v = |\mathbf{v}|$, we can convert it to a unit vector $\mathbf{u}$ that points in the same direction as $\mathbf{v}$, but has unit length. To do this, we simply multiply $\mathbf{v}$ by the reciprocal of its magnitude. We call this *normalization*:

$$\mathbf{u} = \frac{\mathbf{v}}{|\mathbf{v}|} = \frac{1}{v} \mathbf{v}.$$

5.2.4.6 Normal Vectors

A vector is said to be *normal* to a surface if it is *perpendicular* to that surface. Normal vectors are highly useful in games and computer graphics. For example, a *plane* can be defined by a point and a normal vector. And in 3D graphics, lighting calculations make heavy use of normal vectors to define the direction of surfaces relative to the direction of the light rays impinging upon them.

Normal vectors are usually of unit length, but they do not need to be. Be careful not to confuse the term “normalization” with the term “normal vector.” A normalized vector is any vector of unit length. A normal vector is any vector that is perpendicular to a surface, whether or not it is of unit length.

5.2.4.7 Dot Product and Projection

Vectors can be multiplied, but unlike scalars there are a number of different kinds of vector multiplication. In game programming, we most often work with the following two kinds of multiplication:

- the *dot product* (a.k.a. scalar product or inner product), and
- the *cross product* (a.k.a. vector product or outer product).

The dot product of two vectors yields a scalar; it is defined by adding the products of the individual components of the two vectors:

$$\mathbf{a} \cdot \mathbf{b} = a_x b_x + a_y b_y + a_z b_z = d \quad (\text{a scalar}).$$

The dot product can also be written as the product of the magnitudes of the two vectors and the cosine of the angle between them:

$$\mathbf{a} \cdot \mathbf{b} = |\mathbf{a}| |\mathbf{b}| \cos \theta.$$

The dot product is *commutative* (i.e., the order of the two vectors can be reversed) and *distributive* over addition:

$$\mathbf{a} \cdot \mathbf{b} = \mathbf{b} \cdot \mathbf{a};$$

$$\mathbf{a} \cdot (\mathbf{b} + \mathbf{c}) = \mathbf{a} \cdot \mathbf{b} + \mathbf{a} \cdot \mathbf{c}.$$

And the dot product combines with scalar multiplication as follows:

$$s\mathbf{a} \cdot \mathbf{b} = \mathbf{a} \cdot s\mathbf{b} = s(\mathbf{a} \cdot \mathbf{b}).$$

Vector Projection

If $\mathbf{u}$ is a unit vector ($|\mathbf{u}| = 1$), then the dot product ($\mathbf{a} \cdot \mathbf{u}$) represents the length of the *projection* of vector $\mathbf{a}$ onto the infinite line defined by the direction of $\mathbf{u}$, as shown in Figure 5.10. This projection concept works equally well in 2D or 3D and is highly useful for solving a wide variety of three-dimensional problems.

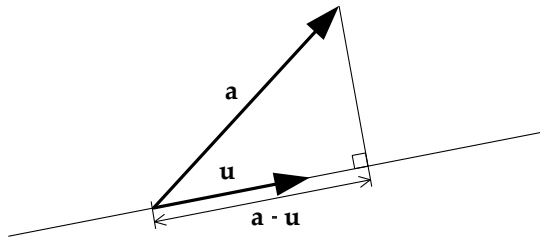


Figure 5.10. Vector projection using the dot product.

Magnitude as a Dot Product

The squared magnitude of a vector can be found by taking the dot product of

that vector with itself. Its magnitude is then easily found by taking the square root:

$$\begin{aligned} |\mathbf{a}|^2 &= \mathbf{a} \cdot \mathbf{a}; \\ |\mathbf{a}| &= \sqrt{\mathbf{a} \cdot \mathbf{a}}. \end{aligned}$$

This works because the cosine of zero degrees is 1, so $|\mathbf{a}| |\mathbf{a}| \cos \theta = |\mathbf{a}| |\mathbf{a}| = |\mathbf{a}|^2$.

Dot Product Tests

Dot products are great for testing if two vectors are collinear or perpendicular, or whether they point in roughly the same or roughly opposite directions. For any two arbitrary vectors $\mathbf{a}$ and $\mathbf{b}$, game programmers often use the following tests, as shown in Figure 5.11:

- *Collinear.* $(\mathbf{a} \cdot \mathbf{b}) = |\mathbf{a}| |\mathbf{b}| = ab$ (i.e., the angle between them is exactly 0 degrees—this dot product equals +1 when $\mathbf{a}$ and $\mathbf{b}$ are *unit vectors*).
- *Collinear but opposite.* $(\mathbf{a} \cdot \mathbf{b}) = -ab$ (i.e., the angle between them is 180 degrees—this dot product equals -1 when $\mathbf{a}$ and $\mathbf{b}$ are unit vectors).
- *Perpendicular.* $(\mathbf{a} \cdot \mathbf{b}) = 0$ (i.e., the angle between them is 90 degrees).
- *Same direction.* $(\mathbf{a} \cdot \mathbf{b}) > 0$ (i.e., the angle between them is less than 90 degrees).
- *Opposite directions.* $(\mathbf{a} \cdot \mathbf{b}) < 0$ (i.e., the angle between them is greater than 90 degrees).

Some Other Applications of the Dot Product

Dot products can be used for all sorts of things in game programming. For example, let's say we want to find out whether an enemy is in front of the player character or behind him. We can find a vector from the player's position $\mathbf{P}$ to the enemy's position $\mathbf{E}$ by simple vector subtraction ($\mathbf{v} = \mathbf{E} - \mathbf{P}$). Let's assume we have a vector $\mathbf{f}$ pointing in the direction that the player is *facing*. (As we'll see in Section 5.3.10.3, the vector $\mathbf{f}$ can be extracted directly from the player's model-to-world matrix.) The dot product $d = \mathbf{v} \cdot \mathbf{f}$ can be used to test whether the enemy is in front of or behind the player—it will be positive when the enemy is in front and negative when the enemy is behind.

The dot product can also be used to find the height of a point above or below a plane (which might be useful when writing a moon-landing game for example). We can define a plane with two vector quantities: a point $\mathbf{Q}$ lying anywhere on the plane, and a unit vector $\mathbf{n}$ that is perpendicular (i.e., normal)

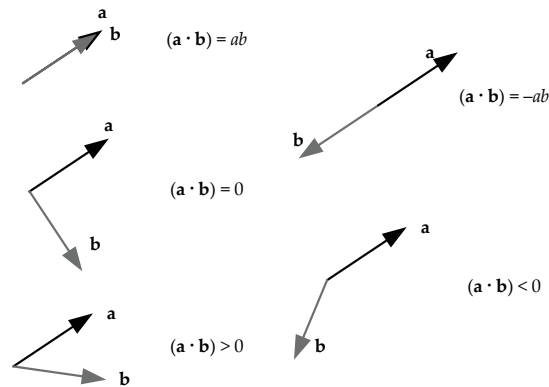


Figure 5.11. Some common dot product tests.

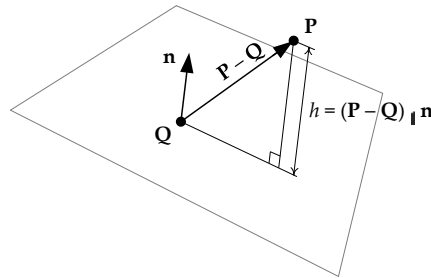


Figure 5.12. The dot product can be used to find the height of a point above or below a plane.

to the plane. To find the height h of a point P above the plane, we first calculate a vector from any point on the plane (Q will do nicely) to the point in question P . So we have $\mathbf{v} = \mathbf{P} - \mathbf{Q}$. The dot product of vector $\mathbf{v}$ with the unit-length normal vector $\mathbf{n}$ is just the projection of $\mathbf{v}$ onto the line defined by $\mathbf{n}$. But that is exactly the height we're looking for. Therefore,

$$h = \mathbf{v} \cdot \mathbf{n} = (\mathbf{P} - \mathbf{Q}) \cdot \mathbf{n}. \quad (5.2)$$

This is illustrated in Figure 5.12.

5.2.4.8 Cross Product

The *cross product* (also known as the *outer product* or *vector product*) of two vectors yields another *vector* that is *perpendicular* to the two vectors being multiplied, as shown in Figure 5.13. The cross product operation is only defined in

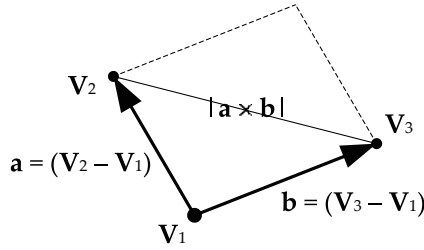


Figure 5.14. Area of a parallelogram expressed as the magnitude of a cross product.

three dimensions:

$$\begin{aligned}\mathbf{a} \times \mathbf{b} &= [(a_y b_z - a_z b_y), (a_z b_x - a_x b_z), (a_x b_y - a_y b_x)] \\ &= (a_y b_z - a_z b_y)\mathbf{i} + (a_z b_x - a_x b_z)\mathbf{j} + (a_x b_y - a_y b_x)\mathbf{k}.\end{aligned}$$

Magnitude of the Cross Product

The magnitude of the cross product vector is the product of the magnitudes of the two vectors and the sine of the angle between them. (This is similar to the definition of the dot product, but it replaces the cosine with the sine.)

$$|\mathbf{a} \times \mathbf{b}| = |\mathbf{a}| |\mathbf{b}| \sin \theta.$$

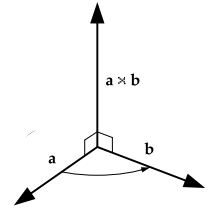
The magnitude of the cross product $|\mathbf{a} \times \mathbf{b}|$ is equal to the area of the parallelogram whose sides are $\mathbf{a}$ and $\mathbf{b}$, as shown in Figure 5.14. Since a triangle is one half of a parallelogram, the area of a triangle whose vertices are specified by the position vectors $\mathbf{V}_1$, $\mathbf{V}_2$ and $\mathbf{V}_3$ can be calculated as one half of the magnitude of the cross product of any two of its sides:

$$A_{\text{triangle}} = \frac{1}{2} |(\mathbf{V}_2 - \mathbf{V}_1) \times (\mathbf{V}_3 - \mathbf{V}_1)|.$$

Direction of the Cross Product

When using a right-handed coordinate system, you can use the *right-hand rule* to determine the direction of the cross product. Simply cup your fingers such that they point in the direction you'd rotate vector $\mathbf{a}$ to move it on top of vector $\mathbf{b}$, and the cross product $(\mathbf{a} \times \mathbf{b})$ will be in the direction of your thumb.

Note that the cross product is defined by the *left-hand rule* when using a left-handed coordinate system. This means that the direction of the cross product changes depending on the choice of coordinate system. This might seem odd

Figure 5.13. The cross product of vectors $\mathbf{a}$ and $\mathbf{b}$ (right-handed).

at first, but remember that the handedness of a coordinate system does *not* affect the mathematical calculations we carry out—it only changes our *visualization* of what the numbers look like in 3D space. When converting from a right-handed system to a left-handed system or vice versa, the numerical representations of all the points and vectors stay the same, but one axis flips. Our visualization of everything is therefore mirrored along that flipped axis. So if a cross product just happens to align with the axis we're flipping (e.g., the z -axis), it needs to flip when the axis flips. If it didn't, the mathematical definition of the cross product itself would have to be changed so that the z -coordinate of the cross product comes out negative in the new coordinate system. I wouldn't lose too much sleep over all of this. Just remember: when *visualizing* a cross product, use the right-hand rule in a right-handed coordinate system and the left-hand rule in a left-handed coordinate system.

Properties of the Cross Product

The cross product is *not commutative* (i.e., order matters):

$$\mathbf{a} \times \mathbf{b} \neq \mathbf{b} \times \mathbf{a}.$$

However, it is *anti-commutative*:

$$\mathbf{a} \times \mathbf{b} = -(\mathbf{b} \times \mathbf{a}).$$

The cross product is distributive over addition:

$$\mathbf{a} \times (\mathbf{b} + \mathbf{c}) = (\mathbf{a} \times \mathbf{b}) + (\mathbf{a} \times \mathbf{c}).$$

And it combines with scalar multiplication as follows:

$$(s\mathbf{a}) \times \mathbf{b} = \mathbf{a} \times (s\mathbf{b}) = s(\mathbf{a} \times \mathbf{b}).$$

The Cartesian basis vectors are related by cross products as follows:

$$\begin{aligned}\mathbf{i} \times \mathbf{j} &= -(\mathbf{j} \times \mathbf{i}) = \mathbf{k} \\ \mathbf{j} \times \mathbf{k} &= -(\mathbf{k} \times \mathbf{j}) = \mathbf{i} \\ \mathbf{k} \times \mathbf{i} &= -(\mathbf{i} \times \mathbf{k}) = \mathbf{j}\end{aligned}$$

These three cross products define the direction of *positive rotations* about the Cartesian axes. The positive rotations go from x to y (about z), from y to z (about x) and from z to x (about y). Notice how the rotation about the y -axis “reversed” alphabetically, in that it goes from z to x (not from x to z). As we'll

see below, this gives us a hint as to why the matrix for rotation about the y -axis looks *inverted* when compared to the matrices for rotation about the x - and z -axes.

The Cross Product in Action

The cross product has a number of applications in games. One of its most common uses is for finding a vector that is perpendicular to two other vectors. As we'll see in Section 5.3.10.2, if we know an object's local unit basis vectors, ($\mathbf{i}_{\text{local}}$, $\mathbf{j}_{\text{local}}$ and $\mathbf{k}_{\text{local}}$), we can easily find a matrix representing the object's orientation. Let's assume that all we know is the object's $\mathbf{k}_{\text{local}}$ vector—i.e., the direction in which the object is facing. If we assume that the object has no roll about $\mathbf{k}_{\text{local}}$, then we can find $\mathbf{i}_{\text{local}}$ by taking the cross product between $\mathbf{k}_{\text{local}}$ (which we already know) and the world-space up vector $\mathbf{j}_{\text{world}}$ (which equals $[0\ 1\ 0]$). We do so as follows: $\mathbf{i}_{\text{local}} = \text{normalize}(\mathbf{j}_{\text{world}} \times \mathbf{k}_{\text{local}})$. We can then find $\mathbf{j}_{\text{local}}$ by simply crossing $\mathbf{i}_{\text{local}}$ and $\mathbf{k}_{\text{local}}$ as follows: $\mathbf{j}_{\text{local}} = \mathbf{k}_{\text{local}} \times \mathbf{i}_{\text{local}}$.

A very similar technique can be used to find a unit vector normal to the surface of a triangle or some other plane. Given three points on the plane, $\mathbf{P}_1$, $\mathbf{P}_2$ and $\mathbf{P}_3$, the normal vector is just $\mathbf{n} = \text{normalize}((\mathbf{P}_2 - \mathbf{P}_1) \times (\mathbf{P}_3 - \mathbf{P}_1))$.

Cross products are also used in physics simulations. When a force is applied to an object, it will give rise to rotational motion if and only if it is applied off-center. This rotational force is known as a *torque*, and it is calculated as follows. Given a force $\mathbf{F}$, and a vector $\mathbf{r}$ from the center of mass to the point at which the force is applied, the torque $\mathbf{N} = \mathbf{r} \times \mathbf{F}$.

5.2.4.9 Pseudovectors and Exterior Algebra

We mentioned in Section 5.2.2 that the cross product doesn't actually produce a vector—it produces a special kind of mathematical object known as a *pseudovector*. The difference between a vector and a pseudovector is pretty subtle. In fact, you can't tell the difference between them at all when performing the kinds of transformations we normally encounter in game programming—translation, rotation and scaling. It's only when you *reflect* the coordinate system (as happens when you move from a left-handed coordinate system to a right-handed system) that the special nature of pseudovectors becomes apparent. Under reflection, a vector transforms into its mirror image, as you'd probably expect. But when a pseudovector is reflected, it transforms into its mirror image and also *changes direction*.

Positions and all of the derivatives thereof (linear velocity, acceleration, jerk) are represented by true vectors (also known as *polar vectors* or *contravariant vectors*). Angular velocities and magnetic fields are represented by pseu-

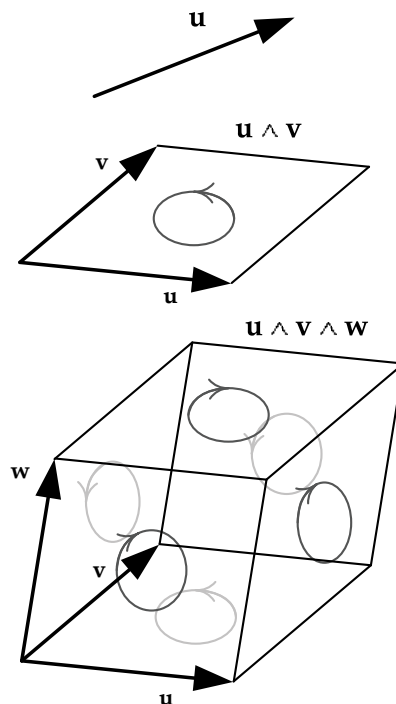


Figure 5.15. In the exterior algebra (Grassman algebra), a single wedge product yields a pseudovector or bivector, and two wedge products yield a pseudoscalar or trivector.

dovectors (also known as *axial vectors*, *covariant vectors*, *bivectors* or *2-blades*). The surface normal of a triangle (which is calculated using a cross product) is also a pseudovector.

It's pretty interesting to note that the cross product ($\mathbf{A} \times \mathbf{B}$), the scalar triple product ($\mathbf{A} \cdot (\mathbf{B} \times \mathbf{C})$) and the determinant of a matrix are all inter-related, and pseudovectors lie at the heart of it all. Mathematicians have come up with a set of algebraic rules, called an *exterior algebra* or *Grassman algebra*, which describe how vectors and pseudovectors work and allow us to calculate areas of parallelograms (in 2D), volumes of parallelepipeds (in 3D), and so on in higher dimensions.

We won't get into all the details here, but the basic idea of Grassman algebra is to introduce a special kind of vector product known as the *wedge product*, denoted $\mathbf{A} \wedge \mathbf{B}$. A pairwise wedge product yields a pseudovector and is equivalent to a cross product, which also represents the *signed area* of the parallelogram formed by the two vectors (where the sign tells us whether we're rotating from $\mathbf{A}$ to $\mathbf{B}$ or vice versa). Doing two wedge products in a row, $\mathbf{A} \wedge \mathbf{B} \wedge \mathbf{C}$,

is equivalent to the scalar triple product $\mathbf{A} \cdot (\mathbf{B} \times \mathbf{C})$ and produces another strange mathematical object known as a *pseudoscalar* (also known as a *trivector* or a *3-blade*), which can be interpreted as the *signed volume* of the parallelepiped formed by the three vectors (see Figure 5.15). This extends into higher dimensions as well.

What does all this mean for us as game programmers? Not too much. All we really need to keep in mind is that some vectors in our code are actually pseudovectors, so that we can transform them properly when changing handedness, for example. Of course if you really want to geek out, you can impress your friends by talking about *exterior algebras* and *wedge products* and explaining how cross products aren't really vectors. Which might make you look cool at your next social engagement ...or not.

For more information, see <http://en.wikipedia.org/wiki/Pseudovector>, http://en.wikipedia.org/wiki/Exterior_algebra, and http://www.terathon.com/gdc12_lengyel.pdf.

5.2.5 Linear Interpolation of Points and Vectors

In games, we often need to find a vector that is midway between two known vectors. For example, if we want to smoothly animate an object from point $\mathbf{A}$ to point $\mathbf{B}$ over the course of two seconds at 30 frames per second, we would need to find 60 intermediate positions between $\mathbf{A}$ and $\mathbf{B}$.

A *linear interpolation* is a simple mathematical operation that finds an intermediate point between two known points. The name of this operation is often shortened to LERP. The operation is defined as follows, where β ranges from 0 to 1 inclusive:

$$\begin{aligned}\mathbf{L} &= \text{LERP}(\mathbf{A}, \mathbf{B}, \beta) = (1 - \beta)\mathbf{A} + \beta\mathbf{B} \\ &= [(1 - \beta)A_x + \beta B_x, (1 - \beta)A_y + \beta B_y, (1 - \beta)A_z + \beta B_z]\end{aligned}$$

Geometrically, $\mathbf{L} = \text{LERP}(\mathbf{A}, \mathbf{B}, \beta)$ is the position vector of a point that lies β percent of the way along the line segment from point $\mathbf{A}$ to point $\mathbf{B}$, as shown in Figure 5.16. Mathematically, the LERP function is just a *weighted average* of the two input vectors, with weights $(1 - \beta)$ and β , respectively. Notice that the weights always add to 1, which is a general requirement for any weighted average.

5.3 Matrices

A *matrix* is a rectangular array of $m \times n$ scalars. Matrices are a convenient way of representing linear transformations such as translation, rotation and scale.

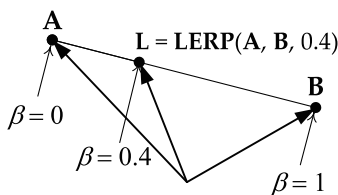


Figure 5.16. Linear interpolation (LERP) between points **A** and **B**, with $\beta = 0.4$.

A matrix **M** is usually written as a grid of scalars M_{rc} enclosed in square brackets, where the subscripts r and c represent the row and column indices of the entry, respectively. For example, if **M** is a 3×3 matrix, it could be written as follows:

$$\mathbf{M} = \begin{bmatrix} M_{11} & M_{12} & M_{13} \\ M_{21} & M_{22} & M_{23} \\ M_{31} & M_{32} & M_{33} \end{bmatrix}.$$

We can think of the rows and/or columns of a 3×3 matrix as 3D vectors. When all of the row and column vectors of a 3×3 matrix are of unit magnitude, we call it a *special orthogonal* matrix. This is also known as an *isotropic* matrix, or an *orthonormal* matrix. Such matrices represent pure rotations.

Under certain constraints, a 4×4 matrix can represent arbitrary 3D *transformations*, including *translations*, *rotations*, and changes in *scale*. These are called *transformation matrices*, and they are the kinds of matrices that will be most useful to us as game engineers. The transformations represented by a matrix are applied to a point or vector via matrix multiplication. We'll investigate how this works below.

An *affine* matrix is a 4×4 transformation matrix that preserves parallelism of lines and relative distance ratios, but not necessarily absolute lengths and angles. An affine matrix is any combination of the following operations: rotation, translation, scale and/or shear.

5.3.1 Matrix Multiplication

The product **P** of two matrices **A** and **B** is written $\mathbf{P} = \mathbf{AB}$. If **A** and **B** are transformation matrices, then the product **P** is another transformation matrix that performs *both* of the original transformations. For example, if **A** is a scale matrix and **B** is a rotation, the matrix **P** would both scale *and* rotate the points or vectors to which it is applied. This is particularly useful in game programming, because we can precalculate a single matrix that performs a whole sequence of transformations and then apply all of those transformations to a large number of vectors efficiently.

To calculate a matrix product, we simply take dot products between the rows of the $n_A \times m_A$ matrix $\mathbf{A}$ and the columns of the $n_B \times m_B$ matrix $\mathbf{B}$. Each dot product becomes one component of the resulting matrix $\mathbf{P}$. The two matrices can be multiplied as long as the *inner dimensions* are equal (i.e., $m_A = n_B$). For example, if $\mathbf{A}$ and $\mathbf{B}$ are 3×3 matrices, then $\mathbf{P} = \mathbf{AB}$ may be expressed as follows:

$$\begin{aligned} \mathbf{P} &= \begin{bmatrix} P_{11} & P_{12} & P_{13} \\ P_{21} & P_{22} & P_{23} \\ P_{31} & P_{32} & P_{33} \end{bmatrix} \\ &= \begin{bmatrix} \mathbf{A}_{\text{row1}} \cdot \mathbf{B}_{\text{col1}} & \mathbf{A}_{\text{row1}} \cdot \mathbf{B}_{\text{col2}} & \mathbf{A}_{\text{row1}} \cdot \mathbf{B}_{\text{col3}} \\ \mathbf{A}_{\text{row2}} \cdot \mathbf{B}_{\text{col1}} & \mathbf{A}_{\text{row2}} \cdot \mathbf{B}_{\text{col2}} & \mathbf{A}_{\text{row2}} \cdot \mathbf{B}_{\text{col3}} \\ \mathbf{A}_{\text{row3}} \cdot \mathbf{B}_{\text{col1}} & \mathbf{A}_{\text{row3}} \cdot \mathbf{B}_{\text{col2}} & \mathbf{A}_{\text{row3}} \cdot \mathbf{B}_{\text{col3}} \end{bmatrix}. \end{aligned}$$

Matrix multiplication is not commutative. In other words, the order in which matrix multiplication is done matters:

$$\mathbf{AB} \neq \mathbf{BA}$$

We'll see exactly why this matters in Section 5.3.2.

Matrix multiplication is often called *concatenation*, because the product of n transformation matrices is a matrix that concatenates, or chains together, the original sequence of transformations in the order the matrices were multiplied.

5.3.2 Representing Points and Vectors as Matrices

Points and vectors can be represented as *row matrices* ($1 \times n$) or *column matrices* ($n \times 1$), where n is the dimension of the space we're working with (usually 2 or 3). For example, the vector $\mathbf{v} = (3, 4, -1)$ can be written either as

$$\mathbf{v}_1 = [3 \quad 4 \quad -1]$$

or as

$$\mathbf{v}_2 = \begin{bmatrix} 3 \\ 4 \\ -1 \end{bmatrix} = \mathbf{v}_1^T.$$

Here, the superscripted T represents matrix *transposition* (see Section 5.3.5).

The choice between column and row vectors is a completely arbitrary one, but it does affect the order in which matrix multiplications are written. This happens because when multiplying matrices, the inner dimensions of the two matrices must be equal, so

- to multiply a $1 \times n$ row vector by an $n \times n$ matrix, the vector must appear to the *left* of the matrix ($\mathbf{v}'_{1 \times n} = \mathbf{v}_{1 \times n} \mathbf{M}_{n \times n}$), whereas
- to multiply an $n \times n$ matrix by an $n \times 1$ column vector, the vector must appear to the *right* of the matrix ($\mathbf{v}'_{n \times 1} = \mathbf{M}_{n \times n} \mathbf{v}_{n \times 1}$).

If multiple transformation matrices **A**, **B** and **C** are applied in order to a vector **v**, the transformations “read” from *left to right* when using *row* vectors, but from *right to left* when using *column* vectors. The easiest way to remember this is to realize that the matrix *closest* to the vector is applied first. This is illustrated by the parentheses below:

$$\begin{aligned}\mathbf{v}' &= (((\mathbf{v}\mathbf{A})\mathbf{B})\mathbf{C}) && \text{Row vectors: read left-to-right;} \\ \mathbf{v}'^T &= (\mathbf{C}^T(\mathbf{B}^T(\mathbf{A}^T\mathbf{v}^T))) && \text{Column vectors: read right-to-left.}\end{aligned}$$

In this book we’ll adopt the *row vector convention*, because the left-to-right order of transformations is most intuitive to read for English-speaking people. That said, be very careful to check which convention is used by your game engine, and by other books, papers or web pages you may read. You can usually tell by seeing whether vector-matrix multiplications are written with the vector on the left (for row vectors) or the right (for column vectors) of the matrix. When using column vectors, you’ll need to *transpose* all the matrices shown in this book.

5.3.3 The Identity Matrix

The *identity matrix* is a matrix that, when multiplied by any other matrix, yields the very same matrix. It is usually represented by the symbol **I**. The identity matrix is always a square matrix with 1’s along the diagonal and 0’s everywhere else:

$$\mathbf{I}_{3 \times 3} = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix};$$

$$\mathbf{A}\mathbf{I} = \mathbf{I}\mathbf{A} \equiv \mathbf{A}.$$

5.3.4 Matrix Inversion

The *inverse* of a matrix **A** is another matrix (denoted $\mathbf{A}^{-1}$) that *undoes* the effects of matrix **A**. So, for example, if **A** rotates objects by 37 degrees about the z-axis, then $\mathbf{A}^{-1}$ will rotate by -37 degrees about the z-axis. Likewise, if **A** scales objects to be twice their original size, then $\mathbf{A}^{-1}$ scales objects to be half-sized. When a matrix is multiplied by its own inverse, the result is *always* the identity matrix, so $\mathbf{A}(\mathbf{A}^{-1}) \equiv (\mathbf{A}^{-1})\mathbf{A} \equiv \mathbf{I}$. Not all matrices have

inverses. However, all *affine* matrices (combinations of pure rotations, translations, scales and shears) do have inverses. Gaussian elimination or lower-upper (LU) decomposition can be used to find the inverse, if one exists.

Since we'll be dealing with matrix multiplication a lot, it's important to note here that the inverse of a sequence of concatenated matrices can be written as the *reverse concatenation* of the individual matrices' inverses. For example,

$$(\mathbf{ABC})^{-1} = \mathbf{C}^{-1}\mathbf{B}^{-1}\mathbf{A}^{-1}.$$

5.3.5 Transposition

The *transpose* of a matrix $\mathbf{M}$ is denoted $\mathbf{M}^T$. It is obtained by reflecting the entries of the original matrix across its diagonal. In other words, the rows of the original matrix become the columns of the transposed matrix, and vice versa:

$$\begin{bmatrix} a & b & c \\ d & e & f \\ g & h & i \end{bmatrix}^T = \begin{bmatrix} a & d & g \\ b & e & h \\ c & f & i \end{bmatrix}.$$

The transpose is useful for a number of reasons. For one thing, the inverse of an orthonormal (pure rotation) matrix is exactly equal to its transpose—which is good news, because it's much cheaper to transpose a matrix than it is to find its inverse in general. Transposition can also be important when moving data from one math library to another, because some libraries use column vectors while others expect row vectors. The matrices used by a row-vector-based library will be *transposed* relative to those used by a library that employs the column vector convention.

As with the inverse, the transpose of a sequence of concatenated matrices can be rewritten as the reverse concatenation of the individual matrices' transposes. For example,

$$(\mathbf{ABC})^T = \mathbf{C}^T\mathbf{B}^T\mathbf{A}^T.$$

This will prove useful when we consider how to apply transformation matrices to points and vectors.

5.3.6 Homogeneous Coordinates

You may recall from high-school algebra that a 2×2 matrix can represent a rotation in two dimensions. To rotate a vector $\mathbf{r}$ through an angle of ϕ degrees (where positive rotations are counterclockwise), we can write

$$\begin{bmatrix} r'_x & r'_y \end{bmatrix} = \begin{bmatrix} r_x & r_y \end{bmatrix} \begin{bmatrix} \cos \phi & \sin \phi \\ -\sin \phi & \cos \phi \end{bmatrix}.$$

It's probably no surprise that rotations in three dimensions can be represented by a 3×3 matrix. The two-dimensional example above is really just a three-dimensional rotation about the z -axis, so we can write

$$\begin{bmatrix} r'_x & r'_y & r'_z \end{bmatrix} = \begin{bmatrix} r_x & r_y & r_z \end{bmatrix} \begin{bmatrix} \cos \phi & \sin \phi & 0 \\ -\sin \phi & \cos \phi & 0 \\ 0 & 0 & 1 \end{bmatrix}.$$

The question naturally arises: Can a 3×3 matrix be used to represent *translations*? Sadly, the answer is no. The result of translating a point $\mathbf{r}$ by a translation $\mathbf{t}$ requires adding the components of $\mathbf{t}$ to the components of $\mathbf{r}$ individually:

$$\mathbf{r} + \mathbf{t} = \begin{bmatrix} (r_x + t_x) & (r_y + t_y) & (r_z + t_z) \end{bmatrix}.$$

Matrix multiplication involves multiplication and addition of matrix elements, so the idea of using multiplication for translation seems promising. But, unfortunately, there is no way to arrange the components of $\mathbf{t}$ within a 3×3 matrix such that the result of multiplying it with the column vector $\mathbf{r}$ yields sums like $(r_x + t_x)$.

The good news is that we *can* obtain sums like this if we use a 4×4 matrix. What would such a matrix look like? Well, we know that we don't want any rotational effects, so the upper 3×3 should contain an identity matrix. If we arrange the components of $\mathbf{t}$ across the bottom-most row of the matrix and set the fourth element of the $\mathbf{r}$ vector (usually called w) equal to 1, then taking the dot product of the vector $\mathbf{r}$ with column 1 of the matrix will yield $(1 \cdot r_x) + (0 \cdot r_y) + (0 \cdot r_z) + (t_x \cdot 1)$, which is exactly what we want. If the bottom right-hand corner of the matrix contains a 1 and the rest of the fourth column contains zeros, then the resulting vector will also have a 1 in its w component. Here's what the final 4×4 translation matrix looks like:

$$\begin{aligned} \mathbf{r} + \mathbf{t} &= \begin{bmatrix} r_x & r_y & r_z & 1 \end{bmatrix} \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix} \\ &= \begin{bmatrix} (r_x + t_x) & (r_y + t_y) & (r_z + t_z) & 1 \end{bmatrix}. \end{aligned}$$

When a point or vector is extended from three dimensions to four in this manner, we say that it has been written in *homogeneous coordinates*. A point in homogeneous coordinates always has $w = 1$. Most of the 3D matrix math done by game engines is performed using 4×4 matrices with four-element points and vectors written in homogeneous coordinates.

5.3.6.1 Transforming Direction Vectors

Mathematically, points (position vectors) and direction vectors are treated in subtly different ways. When transforming a point by a matrix, the translation, rotation and scale of the matrix are all applied to the point. But when transforming a direction by a matrix, the *translational* effects of the matrix are ignored. This is because direction vectors have no translation per se—applying a translation to a direction would alter its magnitude, which is usually not what we want.

In homogeneous coordinates, we achieve this by defining points to have their w components equal to one, while direction vectors have their w components equal to *zero*. In the example below, notice how the $w = 0$ component of the vector $\mathbf{v}$ multiplies with the $\mathbf{t}$ vector in the matrix, thereby eliminating translation in the final result:

$$\begin{bmatrix} \mathbf{v} & 0 \end{bmatrix} \begin{bmatrix} \mathbf{U} & 0 \\ \mathbf{t} & 1 \end{bmatrix} = [(\mathbf{v}\mathbf{U} + 0\mathbf{t}) \quad 0] = [\mathbf{v}\mathbf{U} \quad 0].$$

Technically, a point in homogeneous (four-dimensional) coordinates can be converted into non-homogeneous (three-dimensional) coordinates by dividing the x , y and z components by the w component:

$$\begin{bmatrix} x & y & z & w \end{bmatrix} \equiv \begin{bmatrix} \frac{x}{w} & \frac{y}{w} & \frac{z}{w} \end{bmatrix}.$$

This sheds some light on why we set a point's w component to one and a vector's w component to zero. Dividing by $w = 1$ has no effect on the coordinates of a point, but dividing a pure direction vector's components by $w = 0$ would yield infinity. A point at infinity in 4D can be rotated but not translated, because no matter what translation we try to apply, the point will remain at infinity. So in effect, a pure direction vector in three-dimensional space acts like a point at infinity in four-dimensional homogeneous space.

5.3.7 Atomic Transformation Matrices

Any affine transformation matrix can be created by simply concatenating a sequence of 4×4 matrices representing pure translations, pure rotations, pure scale operations and/or pure shears. These atomic transformation building blocks are presented below. (We'll omit shear from these discussions, as it tends to be used only rarely in games.)

Notice that all affine 4×4 transformation matrices can be partitioned into four components:

$$\mathbf{M}_{\text{affine}} = \begin{bmatrix} \mathbf{U}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{t}_{1 \times 3} & 1 \end{bmatrix}.$$

- the upper 3×3 matrix $\mathbf{U}$, which represents the rotation and/or scale,
- a 1×3 translation vector $\mathbf{t}$,
- a 3×1 vector of zeros $\mathbf{0} = [0 \ 0 \ 0]^T$, and
- a scalar 1 in the bottom-right corner of the matrix.

When a point is multiplied by a matrix that has been partitioned like this, the result is as follows:

$$[\mathbf{r}'_{1 \times 3} \ 1] = [\mathbf{r}_{1 \times 3} \ 1] \begin{bmatrix} \mathbf{U}_{3 \times 3} & \mathbf{0}_{3 \times 1} \\ \mathbf{t}_{1 \times 3} & 1 \end{bmatrix} = [(\mathbf{r}\mathbf{U} + \mathbf{t}) \ 1].$$

5.3.7.1 Translation

The following matrix translates a point by the vector $\mathbf{t}$:

$$\begin{aligned} \mathbf{r} + \mathbf{t} &= [r_x \ r_y \ r_z \ 1] \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & 1 & 0 & 0 \\ 0 & 0 & 1 & 0 \\ t_x & t_y & t_z & 1 \end{bmatrix} \\ &= [(r_x + t_x) \ (r_y + t_y) \ (r_z + t_z) \ 1], \end{aligned} \quad (5.3)$$

or in partitioned shorthand:

$$[\mathbf{r} \ 1] \begin{bmatrix} \mathbf{I} & \mathbf{0} \\ \mathbf{t} & 1 \end{bmatrix} = [(\mathbf{r} + \mathbf{t}) \ 1].$$

To invert a pure translation matrix, simply negate the vector $\mathbf{t}$ (i.e., negate t_x , t_y and t_z).

5.3.7.2 Rotation

All 4×4 pure rotation matrices have the form

$$[\mathbf{r} \ 1] \begin{bmatrix} \mathbf{R} & \mathbf{0} \\ \mathbf{0} & 1 \end{bmatrix} = [\mathbf{r}\mathbf{R} \ 1].$$

The $\mathbf{t}$ vector is zero, and the upper 3×3 matrix $\mathbf{R}$ contains cosines and sines of the rotation angle, measured in radians.

The following matrix represents rotation about the x -axis by an angle ϕ .

$$\text{rotate}_x(\mathbf{r}, \phi) = [r_x \ r_y \ r_z \ 1] \begin{bmatrix} 1 & 0 & 0 & 0 \\ 0 & \cos \phi & \sin \phi & 0 \\ 0 & -\sin \phi & \cos \phi & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (5.4)$$

The matrix below represents rotation about the y -axis by an angle θ . (Notice that this one is *transposed* relative to the other two—the positive and negative sine terms have been reflected across the diagonal.)

$$\text{rotate}_y(\mathbf{r}, \theta) = \begin{bmatrix} r_x & r_y & r_z & 1 \end{bmatrix} \begin{bmatrix} \cos \theta & 0 & -\sin \theta & 0 \\ 0 & 1 & 0 & 0 \\ \sin \theta & 0 & \cos \theta & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (5.5)$$

The following matrix represents rotation about the z -axis by an angle γ :

$$\text{rotate}_z(\mathbf{r}, \gamma) = \begin{bmatrix} r_x & r_y & r_z & 1 \end{bmatrix} \begin{bmatrix} \cos \gamma & \sin \gamma & 0 & 0 \\ -\sin \gamma & \cos \gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}. \quad (5.6)$$

Here are a few observations about these matrices:

- The 1 within the upper 3×3 always appears on the axis we're rotating about, while the sine and cosine terms are off-axis.
- Positive rotations go from x to y (about z), from y to z (about x) and from z to x (about y). The z to x rotation “wraps around,” which is why the rotation matrix about the y -axis is transposed relative to the other two. (Use the right-hand or left-hand rule to remember this.)
- The inverse of a pure rotation is just its transpose. This works because inverting a rotation is equivalent to rotating by the negative angle. You may recall that $\cos(-\theta) = \cos(\theta)$ while $\sin(-\theta) = -\sin(\theta)$, so negating the angle causes the two sine terms to effectively switch places, while the cosine terms stay put.

5.3.7.3 Scale

The following matrix scales the point $\mathbf{r}$ by a factor of s_x along the x -axis, s_y along the y -axis and s_z along the z -axis:

$$\begin{aligned} \mathbf{rS} &= \begin{bmatrix} r_x & r_y & r_z & 1 \end{bmatrix} \begin{bmatrix} s_x & 0 & 0 & 0 \\ 0 & s_y & 0 & 0 \\ 0 & 0 & s_z & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix} \\ &= \begin{bmatrix} s_x r_x & s_y r_y & s_z r_z & 1 \end{bmatrix}, \end{aligned} \quad (5.7)$$

or in partitioned shorthand:

$$\begin{bmatrix} \mathbf{r} & 1 \end{bmatrix} \begin{bmatrix} \mathbf{S}_{3 \times 3} & 0 \\ 0 & 1 \end{bmatrix} = \begin{bmatrix} \mathbf{rS}_{3 \times 3} & 1 \end{bmatrix}.$$

Here are some observations about this kind of matrix:

- To invert a scaling matrix, simply substitute s_x , s_y and s_z with their reciprocals (i.e., $1/s_x$, $1/s_y$ and $1/s_z$).
- When the scale factor along all three axes is the same ($s_x = s_y = s_z$), we call this *uniform scale*. Spheres remain spheres under uniform scale, whereas under nonuniform scale they become ellipsoids. To keep the mathematics of bounding sphere checks simple and fast, many game engines impose the restriction that only uniform scale may be applied to renderable geometry or collision primitives.
- When a uniform scale matrix $\mathbf{S}_u$ and a rotation matrix $\mathbf{R}$ are concatenated, the order of multiplication is unimportant (i.e., $\mathbf{S}_u\mathbf{R} = \mathbf{R}\mathbf{S}_u$). This only works for *uniform scale*!

5.3.8 4×3 Matrices

The rightmost column of an affine 4×4 matrix always contains the vector $[0 \ 0 \ 0 \ 1]^T$. As such, game programmers often omit the fourth column to save memory. You'll encounter 4×3 affine matrices frequently in game math libraries.

5.3.9 Coordinate Spaces

We've seen how to apply transformations to points and direction vectors using 4×4 matrices. We can extend this idea to rigid objects by realizing that such an object can be thought of as an infinite collection of points. Applying a transformation to a rigid object is like applying that same transformation to every point within the object. For example, in computer graphics an object is usually represented by a mesh of triangles, each of which has three vertices represented by points. In this case, the object can be transformed by applying a transformation matrix to all of its vertices in turn.

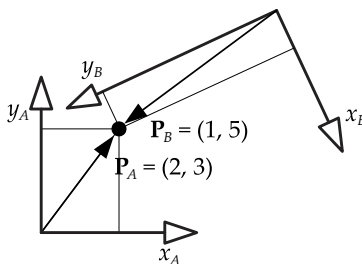


Figure 5.17. Position vectors for the point P relative to different coordinate axes.

We said above that a point is a vector whose tail is fixed to the origin of some coordinate system. This is another way of saying that a point (position vector) is always expressed *relative* to a set of coordinate axes. The triplet of numbers representing a point changes numerically whenever we select a new set of coordinate axes. In Figure 5.17, we see a point P represented by two different position vectors—the vector $\mathbf{P}_A$ gives the position of P relative to the “A” axes, while the vector $\mathbf{P}_B$ gives the position of that same point relative to a different set of axes “B.”

In physics, a set of coordinate axes represents a frame of reference, so we sometimes refer to a set of axes as a *coordinate frame* (or just a *frame*). People in the game industry also use the term *coordinate space* (or simply *space*) to refer to a set of coordinate axes. In the following sections, we’ll look at a few of the most common coordinate spaces used in games and computer graphics.

5.3.9.1 Model Space

When a triangle mesh is created in a tool such as Maya or 3DStudioMAX, the positions of the triangles’ vertices are specified relative to a Cartesian coordinate system, which we call *model space* (also known as *object space* or *local space*). The model-space origin is usually placed at a central location within the object, such as at its center of mass, or on the ground between the feet of a humanoid or animal character.

Most game objects have an inherent directionality. For example, an airplane has a nose, a tail fin and wings that correspond to the front, up and left/right directions. The model-space axes are usually aligned to these natural directions on the model, and they’re given intuitive names to indicate their directionality as illustrated in Figure 5.18.

- *Front*. This name is given to the axis that points in the direction that the object naturally travels or faces. In this book, we’ll use the symbol $\mathbf{F}$ to

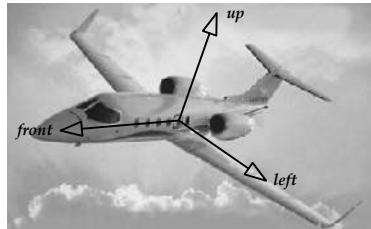


Figure 5.18. One possible choice of the model-space front, left and up axis basis vectors for an airplane.

refer to a unit basis vector along the front axis.

- *Up*. This name is given to the axis that points towards the top of the object. The unit basis vector along this axis will be denoted $\mathbf{U}$.
- *Left or right*. The name “left” or “right” is given to the axis that points toward the left or right side of the object. Which name is chosen depends on whether your game engine uses left-handed or right-handed coordinates. The unit basis vector along this axis will be denoted $\mathbf{L}$ or $\mathbf{R}$, as appropriate.

The mapping between the (*front*, *up*, *left*) labels and the (x , y , z) axes is completely arbitrary. A common choice when working with right-handed axes is to assign the label *front* to the positive z -axis, the label *left* to the positive x -axis and the label *up* to the positive y -axis (or in terms of unit basis vectors, $\mathbf{F} = \mathbf{k}$, $\mathbf{L} = \mathbf{i}$ and $\mathbf{U} = \mathbf{j}$). However, it’s equally common for $+x$ to be *front* and $+z$ to be *right* ($\mathbf{F} = \mathbf{i}$, $\mathbf{R} = \mathbf{k}$, $\mathbf{U} = \mathbf{j}$). I’ve also worked with engines in which the z -axis is oriented vertically. The only real requirement is that you stick to one convention consistently throughout your engine.

As an example of how intuitive axis names can reduce confusion, consider *Euler angles* (*pitch*, *yaw*, *roll*), which are often used to describe an aircraft’s orientation. It’s not possible to define pitch, yaw, and roll angles in terms of the ($\mathbf{i}$, $\mathbf{j}$, $\mathbf{k}$) basis vectors because their orientation is arbitrary. However, we *can* define pitch, yaw and roll in terms of the ($\mathbf{L}$, $\mathbf{U}$, $\mathbf{F}$) basis vectors, because their orientations are clearly defined. Specifically,

- *pitch* is rotation about $\mathbf{L}$ or $\mathbf{R}$,
- *yaw* is rotation about $\mathbf{U}$, and
- *roll* is rotation about $\mathbf{F}$.

5.3.9.2 World Space

World space is a fixed coordinate space, in which the positions, orientations and scales of all objects in the game world are expressed. This coordinate space ties all the individual objects together into a cohesive virtual world.

The location of the world-space origin is arbitrary, but it is often placed near the center of the playable game space to minimize the reduction in floating-point precision that can occur when (x , y , z) coordinates grow very large. Likewise, the orientation of the x -, y - and z -axes is arbitrary, although most of the engines I’ve encountered use either a y -up or a z -up convention. The y -up convention was probably an extension of the two-dimensional convention found in most mathematics textbooks, where the y -axis is shown going up and the x -axis going to the right. The z -up convention is also common,

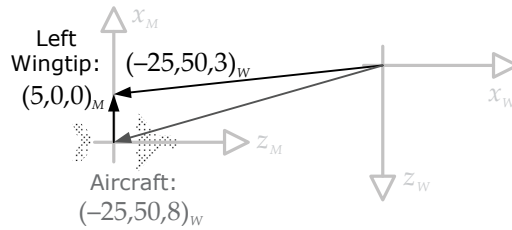


Figure 5.19. A Lear jet whose left wingtip is at $(5, 0, 0)$ in model space. If the jet is rotated by 90 degrees about the world-space y -axis, and its model-space origin translated to $(-25, 50, 8)$ in world space, then its left wingtip would end up at $(-25, 50, 3)$ when expressed in world-space coordinates.

because it allows a top-down orthographic view of the game world to look like a traditional two-dimensional xy -plot.

As an example, let's say that our aircraft's left wingtip is at $(5, 0, 0)$ in model space. (In our game, front vectors correspond to the positive z -axis in model space with y up, as shown in Figure 5.18.) Now imagine that the jet is facing down the positive x -axis in world space, with its model-space origin at some arbitrary location, such as $(-25, 50, 8)$. Because the F vector of the airplane, which corresponds to $+z$ in model space, is facing down the $+x$ -axis in world space, we know that the jet has been rotated by 90 degrees about the world y -axis. So, if the aircraft were sitting at the world-space origin, its left wingtip would be at $(0, 0, -5)$ in world space. But because the aircraft's origin has been translated to $(-25, 50, 8)$, the final position of the jet's left wingtip in world space is $(-25, 50, [8 - 5]) = (-25, 50, 3)$. This is illustrated in Figure 5.19.

We could of course populate our friendly skies with more than one Lear jet. In that case, all of their left wingtips would have coordinates of $(5, 0, 0)$ in model space. But in world space, the left wingtips would have all sorts of interesting coordinates, depending on the orientation and translation of each aircraft.

5.3.9.3 View Space

View space (also known as *camera space*) is a coordinate frame fixed to the camera. The view space origin is placed at the focal point of the camera. Again, any axis orientation scheme is possible. However, a y -up convention with z increasing in the direction the camera is facing (left-handed) is typical because it allows z coordinates to represent depths into the screen. Other engines and APIs, such as OpenGL, define view space to be right-handed, in which case the camera faces towards *negative* z , and z coordinates represent negative depths.

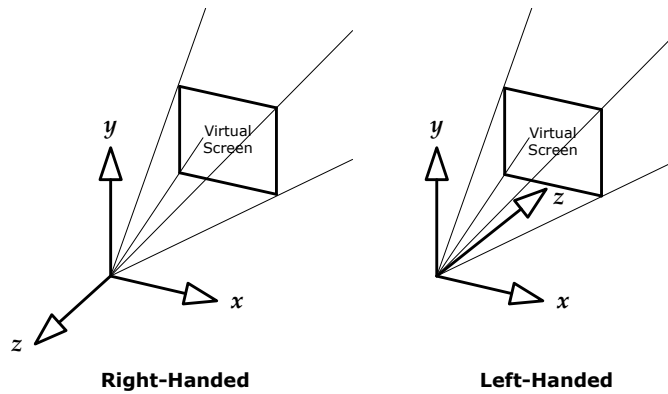


Figure 5.20. Left- and right-handed examples of view space, also known as camera space.

Two possible definitions of view space are illustrated in Figure 5.20.

5.3.10 Change of Basis

In games and computer graphics, it is often quite useful to convert an object's position, orientation and scale from one coordinate system into another. We call this operation a *change of basis*.

5.3.10.1 Coordinate Space Hierarchies

Coordinate frames are relative. That is, if you want to quantify the position, orientation and scale of a set of axes in three-dimensional space, you must specify these quantities *relative to* some other set of axes (otherwise the numbers would have no meaning). This implies that coordinate spaces form a *hierarchy*—every coordinate space is a *child* of some other coordinate space, and the other space acts as its *parent*. World space has no parent; it is at the root of the coordinate-space tree, and all other coordinate systems are ultimately specified relative to it, either as direct children or more-distant relatives.

5.3.10.2 Building a Change of Basis Matrix

The matrix that transforms points and directions from any child coordinate system C to its parent coordinate system P can be written $\mathbf{M}_{C \rightarrow P}$ (pronounced “C to P”). The subscript indicates that this matrix transforms points and directions from child space to parent space. Any child-space position vector $\mathbf{P}_C$ can

be transformed into a parent-space position vector $\mathbf{P}_P$ as follows:

$$\begin{aligned}\mathbf{P}_P &= \mathbf{P}_C \mathbf{M}_{C \rightarrow P}; \\ \mathbf{M}_{C \rightarrow P} &= \begin{bmatrix} \mathbf{i}_C & 0 \\ \mathbf{j}_C & 0 \\ \mathbf{k}_C & 0 \\ \mathbf{t}_C & 1 \end{bmatrix} \\ &= \begin{bmatrix} i_{Cx} & i_{Cy} & i_{Cz} & 0 \\ j_{Cx} & j_{Cy} & j_{Cz} & 0 \\ k_{Cx} & k_{Cy} & k_{Cz} & 0 \\ t_{Cx} & t_{Cy} & t_{Cz} & 1 \end{bmatrix}.\end{aligned}$$

In this equation,

- $\mathbf{i}_C$ is the unit basis vector along the child space x -axis, expressed in parent-space coordinates;
- $\mathbf{j}_C$ is the unit basis vector along the child space y -axis, in parent space;
- $\mathbf{k}_C$ is the unit basis vector along the child space z -axis, in parent space; and
- $\mathbf{t}_C$ is the translation of the child coordinate system relative to parent space.

This result should not be too surprising. The $\mathbf{t}_C$ vector is just the translation of the child-space axes relative to parent space, so if the rest of the matrix were identity, the point $(0, 0, 0)$ in child space would become $\mathbf{t}_C$ in parent space, just as we'd expect. The $\mathbf{i}_C$, $\mathbf{j}_C$ and $\mathbf{k}_C$ unit vectors form the upper 3×3 of the matrix, which is a pure rotation matrix because these vectors are of unit length. We can see this more clearly by considering a simple example, such as a situation in which child space is rotated by an angle γ about the z -axis, with no translation. Recall from Equation (5.6) that the matrix for such a rotation is given by

$$\text{rotate}_z(\mathbf{r}, \gamma) = \begin{bmatrix} r_x & r_y & r_z & 1 \end{bmatrix} \begin{bmatrix} \cos \gamma & \sin \gamma & 0 & 0 \\ -\sin \gamma & \cos \gamma & 0 & 0 \\ 0 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 \end{bmatrix}.$$

But in Figure 5.21, we can see that the coordinates of the $\mathbf{i}_C$ and $\mathbf{j}_C$ vectors, expressed in parent space, are $\mathbf{i}_C = [\cos \gamma \ \sin \gamma \ 0]$ and $\mathbf{j}_C = [-\sin \gamma \ \cos \gamma \ 0]$. When we plug these vectors into our formula for $\mathbf{M}_{C \rightarrow P}$, with $\mathbf{k}_C = [0 \ 0 \ 1]$, it exactly matches the matrix $\text{rotate}_z(\mathbf{r}, \gamma)$ from Equation (5.6).

Scaling the Child Axes

Scaling of the child coordinate system is accomplished by simply scaling the unit basis vectors appropriately. For example, if child space is scaled up by a factor of two, then the basis vectors $\mathbf{i}_C$, $\mathbf{j}_C$ and $\mathbf{k}_C$ will be of length 2 instead of unit length.

5.3.10.3 Extracting Unit Basis Vectors from a Matrix

The fact that we can build a change of basis matrix out of a translation and three Cartesian basis vectors gives us another powerful tool: Given *any* affine 4×4 transformation matrix, we can go in the other direction and extract the child-space basis vectors $\mathbf{i}_C$, $\mathbf{j}_C$ and $\mathbf{k}_C$ from it by simply isolating the appropriate rows of the matrix (or columns if your math library uses column vectors).

This can be incredibly useful. Let's say we are given a vehicle's model-to-world transform as an affine 4×4 matrix (a very common representation). This is really just a change of basis matrix, transforming points in model space into their equivalents in world space. Let's further assume that in our game, the positive z -axis always points in the direction that an object is facing. So, to find a unit vector representing the vehicle's facing direction, we can simply extract $\mathbf{k}_C$ directly from the model-to-world matrix (by grabbing its third row). This vector will already be normalized and ready to go.

5.3.10.4 Transforming Coordinate Systems versus Vectors

We've said that the matrix $\mathbf{M}_{C \rightarrow P}$ transforms points and directions from child space into parent space. Recall that the fourth row of $\mathbf{M}_{C \rightarrow P}$ contains $\mathbf{t}_C$, the translation of the child coordinate axes relative to the world-space axes. Therefore, another way to visualize the matrix $\mathbf{M}_{C \rightarrow P}$ is to imagine it taking the parent *coordinate axes* and transforming them *into* the child axes. This is the

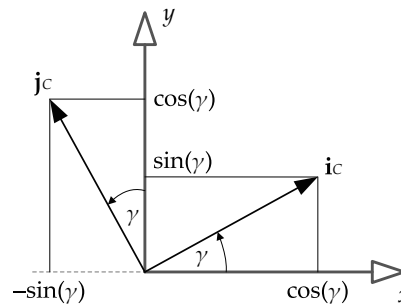


Figure 5.21. Change of basis when child axes are rotated by an angle γ relative to parent.

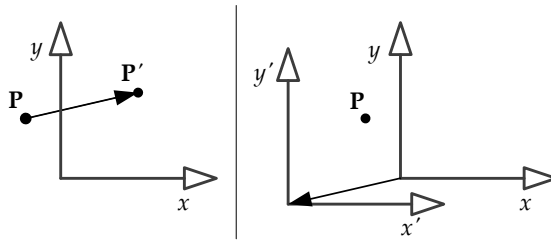


Figure 5.22. Two ways to interpret a transformation matrix. On the left, the point moves against a fixed set of axes. On the right, the axes move in the opposite direction while the point remains fixed.

reverse of what happens to points and direction vectors. In other words, if a matrix transforms *vectors* from child space to parent space, then it also transforms *coordinate axes* from parent space to child space. This makes sense when you think about it—moving a point 20 units to the right with the coordinate axes fixed is the same as moving the coordinate axes 20 units to the left with the point fixed. This concept is illustrated in Figure 5.22.

Of course, this is just another point of potential confusion. If you’re thinking in terms of coordinate axes, then transformations go in one direction, but if you’re thinking in terms of points and vectors, they go in the other direction! As with many confusing things in life, your best bet is probably to choose a single “canonical” way of thinking about things and stick with it. For example, in this book we’ve chosen the following conventions:

- Transformations apply to vectors (not coordinate axes).
- Vectors are written as rows (not columns).

Taken together, these two conventions allow us to read sequences of matrix multiplications from left to right and have them make sense (e.g., in the expression $\mathbf{r}_D = \mathbf{r}_A \mathbf{M}_{A \rightarrow B} \mathbf{M}_{B \rightarrow C} \mathbf{M}_{C \rightarrow D}$, the B’s and C’s in effect “cancel out,” leaving only $\mathbf{r}_D = \mathbf{r}_A \mathbf{M}_{A \rightarrow D}$). Obviously if you start thinking about the coordinate axes moving around rather than the points and vectors, you either have to read the transforms from right to left, or flip one of these two conventions around. It doesn’t really matter what conventions you choose as long as *you* find them easy to remember and work with.

That said, it’s important to note that certain problems are easier to think about in terms of vectors being transformed, while others are easier to work with when you imagine the coordinate axes moving around. Once you get good at thinking about 3D vector and matrix math, you’ll find it pretty easy to flip back and forth between conventions as needed to suit the problem at hand.

5.3.11 Transforming Normal Vectors

A *normal vector* is a special kind of vector, because in addition to (usually!) being of unit length, it carries with it the additional requirement that it should always remain *perpendicular* to whatever surface or plane it is associated with. Special care must be taken when transforming a normal vector to ensure that both its length and perpendicularity properties are maintained.

In general, if a point or (non-normal) vector can be rotated from space A to space B via the 3×3 matrix $\mathbf{M}_{A \rightarrow B}$, then a normal vector $\mathbf{n}$ will be transformed from space A to space B via the *inverse transpose* of that matrix, $(\mathbf{M}_{A \rightarrow B}^{-1})^T$. We will not prove or derive this result here (see [32, Section 3.5] for an excellent derivation). However, we will observe that if the matrix $\mathbf{M}_{A \rightarrow B}$ contains only uniform scale and no shear, then the angles between all surfaces and vectors in space B will be the same as they were in space A. In this case, the matrix $\mathbf{M}_{A \rightarrow B}$ will actually work just fine for any vector, normal or non-normal. However, if $\mathbf{M}_{A \rightarrow B}$ contains nonuniform scale or shear (i.e., is *non-orthogonal*), then the angles between surfaces and vectors are *not* preserved when moving from space A to space B. A vector that was normal to a surface in space A will not necessarily be perpendicular to that surface in space B. The inverse transpose operation accounts for this distortion, bringing normal vectors back into perpendicularity with their surfaces even when the transformation involves nonuniform scale or shear. Another way of looking at this is that the inverse transpose is required because a surface normal is really a *pseudovector* rather than a regular vector (see Section 5.2.4.9).

5.3.12 Storing Matrices in Memory

In the C and C++ languages, a two-dimensional array is often used to store a matrix. Recall that in C/C++ two-dimensional array syntax, the first subscript is the row and the second is the column, and the column index varies fastest as you move through memory sequentially.

```
float m[4][4]; // [row][col], col varies fastest

// "flatten" the array to demonstrate ordering
float* pm = &m[0][0];
ASSERT( &pm[0] == &m[0][0] );
ASSERT( &pm[1] == &m[0][1] );
ASSERT( &pm[2] == &m[0][2] );
// etc.
```

We have two choices when storing a matrix in a two-dimensional C/C++ array. We can either

1. store the vectors (i_C, j_C, k_C, t_C) *contiguously* in memory (i.e., each row contains a single vector), or
2. store the vectors *strided* in memory (i.e., each column contains one vector).

The benefit of approach (1) is that we can address any one of the four vectors by simply indexing into the matrix and interpreting the four contiguous values we find there as a 4-element vector. This layout also has the benefit of matching up exactly with *row vector* matrix equations (which is another reason why I've selected row vector notation for this book). Approach (2) is sometimes necessary when doing fast matrix-vector multiplies using a vector-enabled (SIMD) microprocessor, as we'll see later in this chapter. In most game engines I've personally encountered, matrices are stored using approach (1), with the *vectors* in the *rows* of the two-dimensional C/C++ array. This is shown below:

```
float M[4][4];

M[0][0]=ix;  M[0][1]=iy;  M[0][2]=iz;  M[0][3]=0.0f;
M[1][0]=jx;  M[1][1]=jy;  M[1][2]=jz;  M[1][3]=0.0f;
M[2][0]=kx;  M[2][1]=ky;  M[2][2]=kz;  M[2][3]=0.0f;
M[3][0]=tx;  M[3][1]=ty;  M[3][2]=tz;  M[3][3]=1.0f;
```

The matrix looks like this when viewed in a debugger:

```
M[] []
  [0]
    [0] ix
    [1] iy
    [2] iz
    [3] 0.0000
  [1]
    [0] jx
    [1] jy
    [2] jz
    [3] 0.0000
  [2]
    [0] kx
    [1] ky
    [2] kz
    [3] 0.0000
  [3]
    [0] tx
    [1] ty
    [2] tz
    [3] 1.0000
```

One easy way to determine which layout your engine uses is to find a function that builds a 4×4 translation matrix. (Every good 3D math library provides such a function.) You can then inspect the source code to see where the elements of the $\mathbf{t}$ vector are being stored. If you don't have access to the source code of your math library (which is pretty rare in the game industry), you can always call the function with an easy-to-recognize translation like $(4, 3, 2)$, and then inspect the resulting matrix. If row 3 contains the values $4.0f, 3.0f, 2.0f, 1.0f$, then the vectors are in the rows, otherwise the vectors are in the columns.

5.4 Quaternions

We've seen that a 3×3 matrix can be used to represent an arbitrary rotation in three dimensions. However, a matrix is not always an ideal representation of a rotation, for a number of reasons:

1. We need nine floating-point values to represent a rotation, which seems excessive considering that we only have three degrees of freedom—pitch, yaw and roll.
2. Rotating a vector requires a vector-matrix multiplication, which involves three dot products, or a total of nine multiplications and six additions. We would like to find a rotational representation that is less expensive to calculate, if possible.
3. In games and computer graphics, it's often important to be able to find rotations that are some percentage of the way between two known rotations. For example, if we are to smoothly animate a camera from some starting orientation A to some final orientation B over the course of a few seconds, we need to be able to find lots of intermediate rotations between A and B over the course of the animation. It turns out to be difficult to do this when the A and B orientations are expressed as matrices.

Thankfully, there is a rotational representation that overcomes these three problems. It is a mathematical object known as a *quaternion*. A quaternion looks a lot like a four-dimensional vector, but it *behaves* quite differently. We usually write quaternions using non-italic, non-boldface type, like this: $q = [q_x \ q_y \ q_z \ q_w]$.

Quaternions were developed by Sir William Rowan Hamilton in 1843 as an extension to the complex numbers. (Specifically, a quaternion may be interpreted as a four-dimensional complex number, with a single real axis

and three imaginary axes represented by the imaginary numbers i , j and k . As such, a quaternion can be written in “complex form” as follows: $q = iq_x + jq_y + kq_z + q_w$.) Quaternions were first used to solve problems in the area of mechanics. Technically speaking, a quaternion obeys a set of rules known as a four-dimensional *normed division algebra* over the real numbers. Thankfully, we won’t need to understand the details of these rather esoteric algebraic rules. For our purposes, it will suffice to know that the *unit-length* quaternions (i.e., all quaternions obeying the constraint $q_x^2 + q_y^2 + q_z^2 + q_w^2 = 1$) represent three-dimensional rotations.

There are a lot of great papers, web pages and presentations on quaternions available on the web for further reading. Here’s one of my favorites: http://graphics.ucsd.edu/courses/cse169_w05/CSE169_04.ppt.

5.4.1 Unit Quaternions as 3D Rotations

A unit quaternion can be visualized as a three-dimensional vector plus a fourth scalar coordinate. The vector part $\mathbf{q}_V$ is the unit axis of rotation, scaled by the sine of the half-angle of the rotation. The scalar part q_S is the cosine of the half-angle. So the unit quaternion q can be written as follows:

$$\begin{aligned} q &= [\mathbf{q}_V \quad q_S] \\ &= [\mathbf{a} \sin \frac{\theta}{2} \quad \cos \frac{\theta}{2}], \end{aligned}$$

where $\mathbf{a}$ is a unit vector along the axis of rotation, and θ is the angle of rotation. The direction of the rotation follows the *right-hand rule*, so if your thumb points in the direction of $\mathbf{a}$, positive rotations will be in the direction of your curved fingers.

Of course, we can also write q as a simple four-element vector:

$$\begin{aligned} q &= [q_x \quad q_y \quad q_z \quad q_w], \text{ where} \\ q_x &= q_{V_x} = a_x \sin \frac{\theta}{2}, \\ q_y &= q_{V_y} = a_y \sin \frac{\theta}{2}, \\ q_z &= q_{V_z} = a_z \sin \frac{\theta}{2}, \\ q_w &= q_S = \cos \frac{\theta}{2}. \end{aligned}$$

A unit quaternion is very much like an axis+angle representation of a rotation (i.e., a four-element vector of the form $[\mathbf{a} \quad \theta]$). However, quaternions are more convenient mathematically than their axis+angle counterparts, as we shall see below.

5.4.2 Quaternion Operations

Quaternions support some of the familiar operations from vector algebra, such as magnitude and vector addition. However, we must remember that the sum of two unit quaternions does not represent a 3D rotation, because such a quaternion would not be of unit length. As a result, you won't see any quaternion sums in a game engine, unless they are scaled in some way to preserve the unit length requirement.

5.4.2.1 Quaternion Multiplication

One of the most important operations we will perform on quaternions is that of multiplication. Given two quaternions p and q representing two rotations $\mathbf{P}$ and $\mathbf{Q}$, respectively, the product pq represents the composite rotation (i.e., rotation $\mathbf{Q}$ followed by rotation $\mathbf{P}$). There are actually quite a few different kinds of quaternion multiplication, but we'll restrict this discussion to the variety used in conjunction with 3D rotations, namely the Grassman product. Using this definition, the product pq is defined as follows:

$$pq = [(p_s q_v + q_s p_v + p_v \times q_v) \quad (p_s q_s - p_v \cdot q_v)] .$$

Notice how the Grassman product is defined in terms of a vector part, which ends up in the x , y and z components of the resultant quaternion, and a scalar part, which ends up in the w component.

5.4.2.2 Conjugate and Inverse

The *inverse* of a quaternion q is denoted q^{-1} and is defined as a quaternion that, when multiplied by the original, yields the scalar 1 (i.e., $qq^{-1} = 0\mathbf{i} + 0\mathbf{j} + 0\mathbf{k} + 1$). The quaternion $[0 \quad 0 \quad 0 \quad 1]$ represents a zero rotation (which makes sense since $\sin(0) = 0$ for the first three components, and $\cos(0) = 1$ for the last component).

In order to calculate the inverse of a quaternion, we must first define a quantity known as the *conjugate*. This is usually denoted q^* and it is defined as follows:

$$q^* = [-q_v \quad q_s] .$$

In other words, we negate the vector part but leave the scalar part unchanged.

Given this definition of the quaternion conjugate, the inverse quaternion q^{-1} is defined as follows:

$$q^{-1} = \frac{q^*}{|q|^2} .$$

Our quaternions are always of unit length (i.e., $|q| = 1$), because they represent 3D rotations. So, for our purposes, the inverse and the conjugate are identical:

$$q^{-1} = q^* = [-q_V \quad q_S] \quad \text{when} \quad |q| = 1.$$

This fact is incredibly useful, because it means we can always avoid doing the (relatively expensive) division by the squared magnitude when inverting a quaternion, as long as we know a priori that the quaternion is normalized. This also means that inverting a quaternion is generally much faster than inverting a 3×3 matrix—a fact that you may be able to leverage in some situations when optimizing your engine.

Conjugate and Inverse of a Product

The conjugate of a quaternion product (pq) is equal to the reverse product of the conjugates of the individual quaternions:

$$(pq)^* = q^*p^*.$$

Likewise, the inverse of a quaternion product is equal to the reverse product of the inverses of the individual quaternions:

$$(pq)^{-1} = q^{-1}p^{-1}. \quad (5.8)$$

This is analogous to the reversal that occurs when transposing or inverting matrix products.

5.4.3 Rotating Vectors with Quaternions

How can we apply a quaternion rotation to a vector? The first step is to rewrite the vector in *quaternion form*. A vector is a sum involving the unit basis vectors $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$. A quaternion is a sum involving $\mathbf{i}$, $\mathbf{j}$ and $\mathbf{k}$, but with a fourth scalar term as well. So it makes sense that a vector can be written as a quaternion with its scalar term q_S equal to zero. Given the vector $\mathbf{v}$, we can write a corresponding quaternion $\mathbf{v} = [\mathbf{v} \ 0] = [v_x \ v_y \ v_z \ 0]$.

In order to rotate a vector $\mathbf{v}$ by a quaternion q , we premultiply the vector (written in its quaternion form $\mathbf{v}$) by q and then post-multiply it by the *inverse* quaternion q^{-1} . Therefore, the rotated vector $\mathbf{v}'$ can be found as follows:

$$\mathbf{v}' = \text{rotate}(q, \mathbf{v}) = q\mathbf{v}q^{-1}.$$

This is equivalent to using the quaternion conjugate, because our quaternions are always unit length:

$$\mathbf{v}' = \text{rotate}(q, \mathbf{v}) = q\mathbf{v}q^*. \quad (5.9)$$

The rotated vector $\mathbf{v}'$ is obtained by simply extracting it from its quaternion form $\mathbf{v}'$.

Quaternion multiplication can be useful in all sorts of situations in real games. For example, let's say that we want to find a unit vector describing the direction in which an aircraft is flying. We'll further assume that in our game, the positive z-axis always points toward the front of an object by convention. So the forward unit vector of any object *in model space* is always $\mathbf{F}_M \equiv [0 \ 0 \ 1]$ by definition. To transform this vector into world space, we can simply take our aircraft's orientation quaternion q and use it with Equation (5.9) to rotate our model-space vector $\mathbf{F}_M$ into its world-space equivalent $\mathbf{F}_W$ (after converting these vectors into quaternion form, of course):

$$\mathbf{F}_W = q\mathbf{F}_Mq^{-1} = q [0 \ 0 \ 1 \ 0] q^{-1}.$$

5.4.3.1 Quaternion Concatenation

Rotations can be *concatenated* in exactly the same way that matrix-based transformations can, by multiplying the quaternions together. For example, consider three distinct rotations, represented by the quaternions q_1 , q_2 and q_3 , with matrix equivalents $\mathbf{R}_1$, $\mathbf{R}_2$ and $\mathbf{R}_3$. We want to apply rotation 1 first, followed by rotation 2 and finally rotation 3. The composite rotation matrix $\mathbf{R}_{\text{net}}$ can be found and applied to a vector $\mathbf{v}$ as follows:

$$\begin{aligned}\mathbf{R}_{\text{net}} &= \mathbf{R}_1\mathbf{R}_2\mathbf{R}_3; \\ \mathbf{v}' &= \mathbf{v}\mathbf{R}_1\mathbf{R}_2\mathbf{R}_3 \\ &= \mathbf{v}\mathbf{R}_{\text{net}}.\end{aligned}$$

Likewise, the composite rotation quaternion q_{net} can be found and applied to vector $\mathbf{v}$ (in its quaternion form, v) as follows:

$$\begin{aligned}q_{\text{net}} &= q_3q_2q_1; \\ v' &= q_3q_2q_1 v q_1^{-1}q_2^{-1}q_3^{-1} \\ &= q_{\text{net}} v q_{\text{net}}^{-1}.\end{aligned}$$

Notice how the quaternion product must be performed in an order *opposite* to that in which the rotations are applied ($q_3q_2q_1$). This is because quaternion rotations always multiply on *both* sides of the vector, with the uninverted quaternions on the left and the inverted quaternions on the right. As we saw in Equation (5.8), the inverse of a quaternion product is the reverse product of the individual inverses, so the uninverted quaternions read right-to-left while the inverted quaternions read left-to-right.

5.4.4 Quaternion-Matrix Equivalence

We can convert any 3D rotation freely between a 3×3 matrix representation $\mathbf{R}$ and a quaternion representation $\mathbf{q}$. If we let $\mathbf{q} = [\mathbf{q}_V \ q_S] = [q_{Vx} \ q_{Vy} \ q_{Vz} \ q_S]$ = $[x \ y \ z \ w]$, then we can find $\mathbf{R}$ as follows:

$$\mathbf{R} = \begin{bmatrix} 1 - 2y^2 - 2z^2 & 2xy + 2zw & 2xz - 2yw \\ 2xy - 2zw & 1 - 2x^2 - 2z^2 & 2yz + 2xw \\ 2xz + 2yw & 2yz - 2xw & 1 - 2x^2 - 2y^2 \end{bmatrix}.$$

Likewise, given $\mathbf{R}$, we can find $\mathbf{q}$ as follows (where $\mathbf{q}[0] = q_{Vx}$, $\mathbf{q}[1] = q_{Vy}$, $\mathbf{q}[2] = q_{Vz}$ and $\mathbf{q}[3] = q_S$). This code assumes that we are using row vectors in C/C++ (i.e., that the rows of the matrix correspond to the rows of the matrix $\mathbf{R}$ shown above). The code was adapted from a *Gamasutra* article by Nick Bobic, published on July 5, 1998, which is available here: http://www.gamasutra.com/view/feature/3278/rotating_objects_using_quaternions.php. For a discussion of some even faster methods for converting a matrix to a quaternion, leveraging various assumptions about the nature of the matrix, see <http://www.euclideanspace.com/maths/geometry/rotations/conversions/matrixToQuaternion/index.htm>.

```
void matrixToQuaternion(
    const float  R[3][3],
    float        q[/*4*/])
{
    float trace = R[0][0] + R[1][1] + R[2][2];

    // check the diagonal
    if (trace > 0.0f)
    {
        float s = sqrt(trace + 1.0f);
        q[3] = s * 0.5f;

        float t = 0.5f / s;
        q[0] = (R[2][1] - R[1][2]) * t;
        q[1] = (R[0][2] - R[2][0]) * t;
        q[2] = (R[1][0] - R[0][1]) * t;
    }
    else
    {
        // diagonal is negative
        int i = 0;
        if (R[1][1] > R[0][0]) i = 1;
        if (R[2][2] > R[i][i]) i = 2;

        static const int NEXT[3] = {1, 2, 0};
```

```

int j = NEXT[i];
int k = NEXT[j];

float s = sqrt((R[i][j]
               - (R[j][j] + R[k][k]))
               + 1.0f);

q[i] = s * 0.5f;

float t;
if (s != 0.0) t = 0.5f / s;
else        t = s;

q[3] = (R[k][j] - R[j][k]) * t;
q[j] = (R[j][i] + R[i][j]) * t;
q[k] = (R[k][i] + R[i][k]) * t;
    }
}

```

Let's pause for a moment to consider notational conventions. In this book, we write our quaternions like this: $[x \ y \ z \ w]$. This differs from the $[w \ x \ y \ z]$ convention found in many academic papers on quaternions as an extension of the complex numbers. Our convention arises from an effort to be consistent with the common practice of writing homogeneous vectors as $[x \ y \ z \ 1]$ (with the $w = 1$ at the end). The academic convention arises from the parallels between quaternions and complex numbers. Regular two-dimensional complex numbers are typically written in the form $c = a + jb$, and the corresponding quaternion notation is $q = w + ix + jy + kz$. So be careful out there—make sure you know which convention is being used before you dive into a paper head first!

5.4.5 Rotational Linear Interpolation

Rotational interpolation has many applications in the animation, dynamics and camera systems of a game engine. With the help of quaternions, rotations can be easily interpolated just as vectors and points can.

The easiest and least computationally intensive approach is to perform a four-dimensional vector LERP on the quaternions you wish to interpolate. Given two quaternions q_A and q_B representing rotations A and B, we can find an intermediate rotation q_{LERP} that is β percent of the way from A to B as fol-

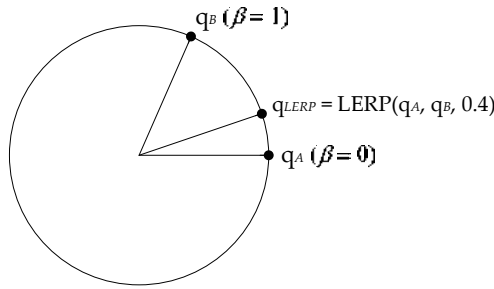


Figure 5.23. Linear interpolation (LERP) between two quaternions q_A and q_B .

lows:

$$q_{\text{LERP}} = \text{LERP}(q_A, q_B, \beta) = \frac{(1 - \beta)q_A + \beta q_B}{|(1 - \beta)q_A + \beta q_B|}$$

$$= \text{normalize} \left(\begin{bmatrix} (1 - \beta)q_{Ax} + \beta q_{Bx} \\ (1 - \beta)q_{Ay} + \beta q_{By} \\ (1 - \beta)q_{Az} + \beta q_{Bz} \\ (1 - \beta)q_{Aw} + \beta q_{Bw} \end{bmatrix}^T \right).$$

Notice that the resultant interpolated quaternion had to be renormalized. This is necessary because the LERP operation does not preserve a vector's length in general.

Geometrically, $q_{\text{LERP}} = \text{LERP}(q_A, q_B, \beta)$ is the quaternion whose orientation lies β percent of the way from orientation A to orientation B, as shown (in two dimensions for clarity) in Figure 5.23. Mathematically, the LERP operation results in a *weighted average* of the two quaternions, with weights $(1 - \beta)$ and β (notice that these two weights sum to 1).

5.4.5.1 Spherical Linear Interpolation

The problem with the LERP operation is that it does not take account of the fact that quaternions are really points on a four-dimensional *hypersphere*. A LERP effectively interpolates along a *chord* of the hypersphere, rather than along the surface of the hypersphere itself. This leads to rotation animations that do not have a constant angular speed when the parameter β is changing at a constant rate. The rotation will appear slower at the end points and faster in the middle of the animation.

To solve this problem, we can use a variant of the LERP operation known as *spherical linear interpolation*, or SLERP for short. The SLERP operation uses sines and cosines to interpolate along a *great circle* of the 4D hypersphere,

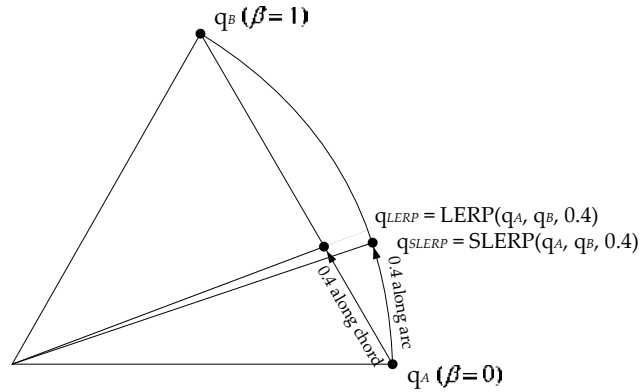


Figure 5.24. Spherical linear interpolation along a great circle arc of a 4D hypersphere.

rather than along a chord, as shown in Figure 5.24. This results in a constant angular speed when β varies at a constant rate.

The formula for SLERP is similar to the LERP formula, but the weights $(1 - \beta)$ and β are replaced with weights w_p and w_q involving sines of the angle between the two quaternions.

$$\text{SLERP}(p, q, \beta) = w_p p + w_q q,$$

where

$$w_p = \frac{\sin(1 - \beta)\theta}{\sin \theta},$$

$$w_q = \frac{\sin \beta\theta}{\sin \theta}.$$

The cosine of the angle between any two unit-length quaternions can be found by taking their four-dimensional dot product. Once we know $\cos \theta$, we can calculate the angle θ and the various sines we need quite easily:

$$\cos \theta = p \cdot q = p_x q_x + p_y q_y + p_z q_z + p_w q_w;$$

$$\theta = \cos^{-1}(p \cdot q).$$

5.4.5.2 To SLERP or Not to SLERP (That's Still the Question)

The jury is still out on whether or not to use SLERP in a game engine. Jonathan Blow wrote a great article positing that SLERP is too expensive, and LERP's quality is not really that bad—therefore, he suggests, we should understand SLERP but avoid it in our game engines (see <http://number-none.com/product/Understanding%20Slerp,%20Then%20Not%20Using%20It/index.html>).

On the other hand, some of my colleagues at Naughty Dog have found that a good SLERP implementation performs nearly as well as LERP. (For example, on the PS3's SPUs, Naughty Dog's Ice team's implementation of SLERP takes 20 cycles per joint, while its LERP implementation takes 16.25 cycles per joint.) Therefore, I'd personally recommend that you profile your SLERP and LERP implementations before making any decisions. If the performance hit for SLERP isn't unacceptable, I say go for it, because it may result in slightly better-looking animations. But if your SLERP is slow (and you cannot speed it up, or you just don't have the time to do so), then LERP is usually good enough for most purposes.

5.5 Comparison of Rotational Representations

We've seen that rotations can be represented in quite a few different ways. This section summarizes the most common rotational representations and outlines their pros and cons. No one representation is ideal in all situations. Using the information in this section, you should be able to select the best representation for a particular application.

5.5.1 Euler Angles

We briefly explored *Euler angles* in Section 5.3.9.1. A rotation represented via Euler angles consists of three scalar values: yaw, pitch and roll. These quantities are sometimes represented by a 3D vector $[\theta_Y \ \theta_P \ \theta_R]$.

The benefits of this representation are its simplicity, its small size (three floating-point numbers) and its intuitive nature—yaw, pitch and roll are easy to visualize. You can also easily interpolate simple rotations about a single axis. For example, it's trivial to find intermediate rotations between two distinct yaw angles by linearly interpolating the scalar θ_Y . However, Euler angles cannot be interpolated easily when the rotation is about an arbitrarily oriented axis.

In addition, Euler angles are prone to a condition known as *gimbal lock*. This occurs when a 90-degree rotation causes one of the three principal axes to “collapse” onto another principal axis. For example, if you rotate by 90 degrees about the x -axis, the y -axis collapses onto the z -axis. This prevents any further rotations about the original y -axis, because rotations about y and z have effectively become equivalent.

Another problem with Euler angles is that the order in which the rotations are performed around each axis matters. The order could be PYR, YPR, RYP and so on, and each ordering may produce a different composite rotation. No

one standard rotation order exists for Euler angles across all disciplines (although certain disciplines do follow specific conventions). So the rotation angles $[\theta_Y \ \theta_P \ \theta_R]$ do not uniquely define a particular rotation—you need to know the rotation order to interpret these numbers properly.

A final problem with Euler angles is that they depend upon the mapping from the x -, y - and z -axes onto the natural *front*, *left/right* and *up* directions for the object being rotated. For example, yaw is always defined as rotation about the *up* axis, but without additional information we cannot tell whether this corresponds to a rotation about x , y or z .

5.5.2 3×3 Matrices

A 3×3 matrix is a convenient and effective rotational representation for a number of reasons. It does not suffer from gimbal lock, and it can represent arbitrary rotations uniquely. Rotations can be applied to points and vectors in a straightforward manner via matrix multiplication (i.e., a series of dot products). Most CPUs and all GPUs now have built-in support for hardware-accelerated dot products and matrix multiplication. Rotations can also be reversed by finding an inverse matrix, which for a pure rotation matrix is the same thing as finding the transpose—a trivial operation. And 4×4 matrices offer a way to represent arbitrary affine transformations—rotations, translations and scaling—in a totally consistent way.

However, rotation matrices are not particularly intuitive. Looking at a big table of numbers doesn't help one picture the corresponding transformation in three-dimensional space. Also, rotation matrices are not easily interpolated. Finally, a rotation matrix takes up a lot of storage (nine floating-point numbers) relative to Euler angles (three floats).

5.5.3 Axis + Angle

We can represent rotations as a unit vector, defining the axis of rotation plus a scalar for the angle of rotation. This is known as an *axis+angle* representation, and it is sometimes denoted by the four-dimensional vector $[\mathbf{a} \ \theta] = [a_x \ a_y \ a_z \ \theta]$, where $\mathbf{a}$ is the axis of rotation and θ the angle in radians. In a right-handed coordinate system, the direction of a positive rotation is defined by the right-hand rule, while in a left-handed system, we use the left-hand rule instead.

The benefits of the axis+angle representation are that it is reasonably intuitive and also compact. (It only requires four floating-point numbers, as opposed to the nine required for a 3×3 matrix.)

One important limitation of the axis+angle representation is that rotations cannot be easily interpolated. Also, rotations in this format cannot be applied to points and vectors in a straightforward way—one needs to convert the axis+angle representation into a matrix or quaternion first.

5.5.4 Quaternions

As we've seen, a unit-length quaternion can represent 3D rotations in a manner analogous to the axis+angle representation. The primary difference between the two representations is that a quaternion's axis of rotation is scaled by the sine of the half-angle of rotation, and instead of storing the angle in the fourth component of the vector, we store the cosine of the half-angle.

The quaternion formulation provides two immense benefits over the axis+angle representation. First, it permits rotations to be concatenated and applied directly to points and vectors via quaternion multiplication. Second, it permits rotations to be easily interpolated via simple LERP or SLERP operations. Its small size (four floating-point numbers) is also a benefit over the matrix formulation.

5.5.5 SRT Transformations

By itself, a quaternion can only represent a rotation, whereas a 4×4 matrix can represent an arbitrary affine transformation (rotation, translation and scale). When a quaternion is combined with a *translation vector* and a *scale factor* (either a scalar for uniform scaling or a vector for nonuniform scaling), then we have a viable alternative to the 4×4 matrix representation of affine transformations. We sometimes call this an *SRT transform*, because it contains a scale factor, a rotation quaternion and a translation vector. (It's also sometimes called an *SQT*, because the rotation is a quaternion.)

$$\text{SRT} = [s \quad \mathbf{q} \quad \mathbf{t}] \quad (\text{uniform scale } s),$$

or

$$\text{SRT} = [s \quad \mathbf{q} \quad \mathbf{t}] \quad (\text{nonuniform scale vector } \mathbf{s}).$$

SRT transforms are widely used in computer animation because of their smaller size (eight floats for uniform scale, or ten floats for nonuniform scale, as opposed to the 12 floating-point numbers needed for a 4×3 matrix) and their ability to be easily interpolated. The translation vector and scale factor are interpolated via LERP, and the quaternion can be interpolated with either LERP or SLERP.

5.5.6 Dual Quaternions

A *rigid transformation* is a transformation involving a rotation and a translation—a “corkscrew” motion. Such transformations are prevalent in animation and robotics. A rigid transformation can be represented using a mathematical object known as a *dual quaternion*. The dual quaternion representation offers a number of benefits over the typical vector-quaternion representation. The key benefit is that linear interpolation blending can be performed in a constant-speed, shortest-path, coordinate-invariant manner, similar to using LERP for translation vectors and SLERP for rotational quaternions (see Section 5.4.5.1), but in a way that is easily generalizable to blends involving three or more transforms.

A dual quaternion is like an ordinary quaternion, except that its four components are *dual numbers* instead of regular real-valued numbers. A dual number can be written as the sum of a non-dual part and a dual part as follows: $\hat{a} = a + \varepsilon b$. Here ε is a magical number called the *dual unit*, defined in such a way that $\varepsilon^2 = 0$ (yet without ε itself being zero). This is analogous to the imaginary number $j = \sqrt{-1}$ used when writing a complex number as the sum of a real and an imaginary part: $c = a + jb$.

Because each dual number can be represented by two real numbers (the non-dual and dual parts, a and b), a dual quaternion can be represented by an eight-element vector. It can also be represented as the sum of two ordinary quaternions, where the second one is multiplied by the dual unit, as follows: $\hat{q} = q_a + \varepsilon q_b$.

A full discussion of dual numbers and dual quaternions is beyond our scope here. However, the excellent paper entitled, “Dual Quaternions for Rigid Transformation Blending” by Kavan et al. outlines the theory and practice of using dual quaternions to represent rigid transformations—it is available online at <https://bit.ly/2vjD5sz>. Note that in this paper, a dual number is written in the form $\hat{a} = a_0 + \varepsilon a_\varepsilon$, whereas I have used $a + \varepsilon b$ above to underscore the similarity between dual numbers and complex numbers.¹

5.5.7 Rotations and Degrees of Freedom

The term “*degrees of freedom*” (or *DOF* for short) refers to the number of mutually independent ways in which an object’s physical state (position and orientation) can change. You may have encountered the phrase “six degrees of

¹Personally I would have preferred the symbol a_1 over a_0 , so that a dual number would be written $\hat{a} = (1)a_1 + (\varepsilon)a_\varepsilon$. Just as when we plot a complex number in the complex plane, we can think of the real unit as a “basis vector” along the real axis, and the dual unit ε as a “basis vector” along the dual axis.

freedom” in fields such as mechanics, robotics and aeronautics. This refers to the fact that a three-dimensional object (whose motion is not artificially constrained) has three degrees of freedom in its translation (along the x -, y - and z -axes) and three degrees of freedom in its rotation (about the x -, y - and z -axes), for a total of six degrees of freedom.

The DOF concept will help us to understand how different rotational representations can employ different numbers of floating-point parameters, yet all specify rotations with only three degrees of freedom. For example, Euler angles require three floats, but axis+angle and quaternion representations use four floats, and a 3×3 matrix takes up nine floats. How can these representations all describe 3-DOF rotations?

The answer lies in *constraints*. All 3D rotational representations employ three or more floating-point parameters, but some representations also have one or more constraints on those parameters. The constraints indicate that the parameters are not *independent*—a change to one parameter induces changes to the other parameters in order to maintain the validity of the constraint(s). If we subtract the number of constraints from the number of floating-point parameters, we arrive at the number of degrees of freedom—and this number should always be three for a 3D rotation:

$$N_{\text{DOF}} = N_{\text{parameters}} - N_{\text{constraints}}. \quad (5.10)$$

The following list shows Equation (5.10) in action for each of the rotational representations we’ve encountered in this book.

- *Euler Angles*. 3 parameters – 0 constraints = 3 DOF.
- *Axis+Angle*. 4 parameters – 1 constraint = 3 DOF.
Constraint: Axis is constrained to be unit length.
- *Quaternion*. 4 parameters – 1 constraint = 3 DOF.
Constraint: Quaternion is constrained to be unit length.
- *3×3 Matrix*. 9 parameters – 6 constraints = 3 DOF.
Constraints: All three rows and all three columns must be of unit length (when treated as three-element vectors).

5.6 Other Useful Mathematical Objects

As game engineers, we will encounter a host of other mathematical objects in addition to points, vectors, matrices and quaternions. This section briefly outlines the most common of these.

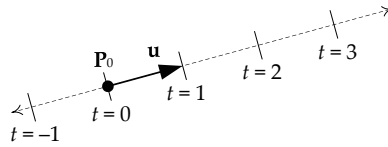


Figure 5.25. Parametric equation of a line.

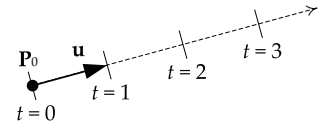


Figure 5.26. Parametric equation of a ray.

5.6.1 Lines, Rays and Line Segments

An infinite line can be represented by a point $\mathbf{P}_0$ plus a unit vector $\mathbf{u}$ in the direction of the line. A *parametric equation* of a line traces out every possible point $\mathbf{P}$ along the line by starting at the initial point $\mathbf{P}_0$ and moving an arbitrary distance t along the direction of the unit vector $\mathbf{v}$. The infinitely large set of points $\mathbf{P}$ becomes a *vector function* of the scalar parameter t :

$$\mathbf{P}(t) = \mathbf{P}_0 + t\mathbf{u}, \quad \text{where } -\infty < t < \infty. \quad (5.11)$$

This is depicted in Figure 5.25.

A *ray* is a line that extends to infinity in only one direction. This is easily expressed as $\mathbf{P}(t)$ with the constraint $t \geq 0$, as shown in Figure 5.26.

A *line segment* is bounded at both ends by $\mathbf{P}_0$ and $\mathbf{P}_1$. It too can be represented by $\mathbf{P}(t)$, in either one of the following two ways (where $\mathbf{L} = \mathbf{P}_1 - \mathbf{P}_0$, $L = |\mathbf{L}|$ is the length of the line segment, and $\mathbf{u} = (1/L)\mathbf{L}$ is a unit vector in the direction of $\mathbf{L}$):

1. $\mathbf{P}(t) = \mathbf{P}_0 + t\mathbf{u}$, where $0 \leq t \leq L$, or
2. $\mathbf{P}(t) = \mathbf{P}_0 + t\mathbf{L}$, where $0 \leq t \leq 1$.

The latter format, depicted in Figure 5.27, is particularly convenient because the parameter t is normalized; in other words, t always goes from zero to one, no matter which particular line segment we are dealing with. This means we do not have to store the constraint L in a separate floating-point parameter; it is already encoded in the vector $\mathbf{L} = L\mathbf{u}$ (which we have to store anyway).

5.6.2 Spheres

Spheres are ubiquitous in game engine programming. A sphere is typically defined as a center point $\mathbf{C}$ plus a radius r , as shown in Figure 5.28. This packs nicely into a four-element vector, $[C_x \ C_y \ C_z \ r]$. As we saw when we discussed SIMD vector processing, there are distinct benefits to being able to pack data into a vector containing four 32-bit floats (i.e., a 128-bit package).

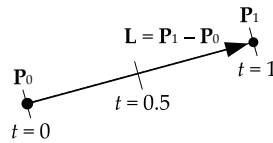
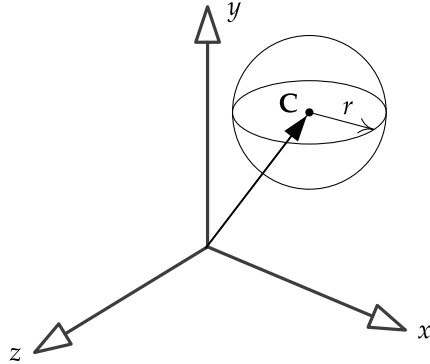
Figure 5.27. Parametric equation of a line segment, with normalized parameter t .

Figure 5.28. Point-radius representation of a sphere.

5.6.3 Planes

A *plane* is a 2D surface in 3D space. As you may recall from high-school algebra, the equation of a plane is often written as follows:

$$Ax + By + Cz + D = 0.$$

This equation is satisfied only for the locus of points $\mathbf{P} = [x \ y \ z]$ that lie on the plane.

Planes can be represented by a point $\mathbf{P}_0$ and a unit vector $\mathbf{n}$ that is normal to the plane. This is sometimes called *point-normal form*, as depicted in Figure 5.29.

It's interesting to note that when the parameters A , B and C from the traditional plane equation are interpreted as a 3D vector, that vector lies in the direction of the plane normal. If the vector $[A \ B \ C]$ is normalized to unit length, then the normalized vector $[a \ b \ c] = \mathbf{n}$, and the normalized parameter $d = D / \sqrt{A^2 + B^2 + C^2}$ is just the distance from the plane to the origin. The sign of d is positive if the plane's normal vector $\mathbf{n}$ is pointing toward the origin (i.e., the origin is on the "front" side of the plane) and negative if the normal is pointing away from the origin (i.e., the origin is "behind" the plane).

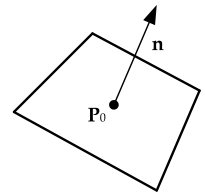


Figure 5.29. A plane in point-normal form.

Another way of looking at this is that the plane equation and the point-normal form are really just two ways of writing the same equation. Imagine testing whether or not an arbitrary point $\mathbf{P} = [x \ y \ z]$ lies on the plane. To do this, we find the signed distance from point $\mathbf{P}$ to the origin along the normal $\mathbf{n} = [a \ b \ c]$, and if this signed distance is equal to the signed distance $d = -\mathbf{n} \cdot \mathbf{P}_0$ from the plane from the origin, then $\mathbf{P}$ must lie on the plane. So let's set them equal and expand some terms:

$$\begin{aligned}
 (\text{signed distance } \mathbf{P} \text{ to origin}) &= (\text{signed distance plane to origin}) \\
 \mathbf{n} \cdot \mathbf{P} &= \mathbf{n} \cdot \mathbf{P}_0 \\
 \mathbf{n} \cdot \mathbf{P} - \mathbf{n} \cdot \mathbf{P}_0 &= 0 \\
 ax + by + cz - \mathbf{n} \cdot \mathbf{P}_0 &= 0 \\
 ax + by + cz + d &= 0.
 \end{aligned} \tag{5.12}$$

Equation (5.12) only holds when the point $\mathbf{P}$ lies on the plane. But what happens when the point $\mathbf{P}$ does *not* lie on the plane? In this case, the left-hand side of the plane equation ($ax + by + cz$, which is equal to $\mathbf{n} \cdot \mathbf{P}$) tells how far “off” the point is from being on the plane. This expression calculates the difference between the distance from $\mathbf{P}$ to the origin and the distance from the plane to the origin. In other words, the left-hand side of Equation (5.12) gives us the perpendicular distance h between the point and the plane! This is just another way to write Equation (5.2) from Section 5.2.4.7.

$$\begin{aligned}
 h &= (\mathbf{P} - \mathbf{P}_0) \cdot \mathbf{n}; \\
 h &= ax + by + cz + d.
 \end{aligned} \tag{5.13}$$

A plane can actually be packed into a four-element vector, much like a sphere can. To do so, we observe that to describe a plane uniquely, we need only the normal vector $\mathbf{n} = [a \ b \ c]$ and the distance from the origin d . The four-element vector $\mathbf{L} = [\mathbf{n} \ d] = [a \ b \ c \ d]$ is a compact and convenient way to represent and store a plane in memory. Note that when $\mathbf{P}$ is written in homogeneous coordinates with $w = 1$, the equation $(\mathbf{L} \cdot \mathbf{P}) = 0$ is yet another way of writing $(\mathbf{n} \cdot \mathbf{P}) = -d$. These equations are satisfied for all points $\mathbf{P}$ that lie on the plane $\mathbf{L}$.

Planes defined in four-element vector form can be easily transformed from one coordinate space to another. Given a matrix $\mathbf{M}_{A \rightarrow B}$ that transforms points and (non-normal) vectors from space A to space B, we already know that to transform a *normal vector* such as the plane's $\mathbf{n}$ vector, we need to use the inverse transpose of that matrix, $(\mathbf{M}_{A \rightarrow B}^{-1})^T$. So it shouldn't be a big surprise to learn that applying the inverse transpose of a matrix to a four-element plane vector $\mathbf{L}$ will, in fact, correctly transform that plane from space A to space B. We won't derive or prove this result any further here, but a thorough explanation of why this little “trick” works is provided in Section 4.2.3 of [32].

5.6.4 Axis-Aligned Bounding Boxes (AABB)

An axis-aligned bounding box (AABB) is a 3D *cuboid* whose six rectangular faces are aligned with a particular coordinate frame's mutually orthogonal axes. As such, an AABB can be represented by a six-element vector containing the minimum and maximum coordinates along each of the 3 principal axes, $[x_{\min}, y_{\min}, z_{\min}, x_{\max}, y_{\max}, z_{\max}]$, or two points $\mathbf{P}_{\min}$ and $\mathbf{P}_{\max}$.

This simple representation allows for a particularly convenient and inexpensive method of testing whether a point $\mathbf{P}$ is inside or outside any given AABB. We simply test if all of the following conditions are true:

$$\begin{aligned} P_x &\geq x_{\min} \text{ and } P_x \leq x_{\max} \text{ and} \\ P_y &\geq y_{\min} \text{ and } P_y \leq y_{\max} \text{ and} \\ P_z &\geq z_{\min} \text{ and } P_z \leq z_{\max}. \end{aligned}$$

Because intersection tests are so speedy, AABBs are often used as an “early out” collision check; if the AABBs of two objects do not intersect, then there is no need to do a more detailed (and more expensive) collision test.

5.6.5 Oriented Bounding Boxes (OBB)

An oriented bounding box (OBB) is a *cuboid* that has been oriented so as to align in some logical way with the object it bounds. Usually an OBB aligns with the local-space axes of the object. Hence, it acts like an AABB in local space, although it may not necessarily align with the world-space axes.

Various techniques exist for testing whether or not a point lies within an OBB, but one common approach is to transform the point into the OBB's “aligned” coordinate system and then use an AABB intersection test as presented above.

5.6.6 Frusta

As shown in Figure 5.30, a *frustum* is a group of six planes that define a *truncated pyramid* shape. Frusta are commonplace in 3D rendering because they conveniently define the viewable region of the 3D world when rendered via a perspective projection from the point of view of a virtual camera. Four of the planes bound the edges of the screen space, while the other two planes represent the the near and far clipping planes (i.e., they define the minimum and maximum z coordinates possible for any visible point).

One convenient representation of a frustum is as an array of six planes, each of which is represented in point-normal form (i.e., one point and one normal vector per plane).

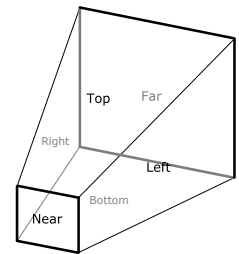


Figure 5.30. A frustum.

Testing whether a point lies inside a frustum is a bit involved, but the basic idea is to use dot products to determine whether the point lies on the front or back side of each plane. If it lies inside all six planes, it is inside the frustum.

A helpful trick is to transform the world-space point being tested by applying the camera's perspective projection to it. This takes the point from world space into a space known as *homogeneous clip space*. In this space, the frustum is just an axis-aligned cuboid (AABB). This permits much simpler in/out tests to be performed.

5.6.7 Convex Polyhedral Regions

A *convex polyhedral region* is defined by an arbitrary set of planes, all with normals pointing inward (or outward). The test for whether a point lies inside or outside the volume defined by the planes is relatively straightforward; it is similar to a frustum test, but with possibly more planes. Convex regions are very useful for implementing arbitrarily shaped trigger regions in games. Many engines employ this technique; for example, the Quake engine's ubiquitous *brushes* are just volumes bounded by planes in exactly this way.

5.7 Random Number Generation

Random numbers are ubiquitous in game engines, so it behooves us to have a brief look at the two most common random number generators (RNG), the linear congruential generator and the Mersenne Twister. It's important to realize that random number generators don't actually generate random numbers—they merely produce a complex, but totally deterministic, predefined sequence of values. For this reason, we call the sequences they produce *pseudorandom*, and technically speaking we should really call them "pseudorandom number generators" (PRNG). What differentiates a good generator from a bad one is how long the sequence of numbers is before it repeats (its *period*), and how well the sequences hold up under various well-known randomness tests.

5.7.1 Linear Congruential Generators

Linear congruential generators are a very fast and simple way to generate a sequence of pseudorandom numbers. Depending on the platform, this algorithm is sometimes used in the C standard library's `rand()` function. However, your mileage may vary, so don't count on `rand()` being based on any particular algorithm. If you want to be sure, you'll be better off implementing your own random number generator.

The linear congruential algorithm is explained in detail in the book *Numerical Recipes in C*, so I won't go into the details of it here.

What I will say is that this random number generator does not produce particularly high-quality pseudorandom sequences. Given the same initial seed value, the sequence is always exactly the same. The numbers produced do not meet many of the criteria widely accepted as desirable, such as a long period, low- and high-order bits that have similarly long periods, and absence of sequential or spatial correlation between the generated values.

5.7.2 Mersenne Twister

The Mersenne Twister pseudorandom number generator algorithm was designed specifically to improve upon the various problems of the linear congruential algorithm. Wikipedia provides the following description of the benefits of the algorithm:

1. It was designed to have a colossal period of $2^{19937} - 1$ (the creators of the algorithm proved this property). In practice, there is little reason to use larger ones, as most applications do not require 2^{19937} unique combinations ($2^{19937} \approx 4.3 \times 10^{6001}$).
2. It has a very high order of dimensional equidistribution. Note that this means, by default, that there is negligible serial correlation between successive values in the output sequence.
3. It passes numerous tests for statistical randomness, including the stringent Diehard tests.
4. It is fast.

Various implementations of the Twister are available on the web, including a particularly cool one that uses SIMD vector instructions for an extra speed boost, called *SFMT* (SIMD-oriented fast Mersenne Twister). SFMT can be downloaded from <http://www.math.sci.hiroshima-u.ac.jp/~m-mat/MT/SFMT/index.html>.

5.7.3 Mother-of-All, Xorshift and KISS99

In 1994, George Marsaglia, a computer scientist and mathematician best known for developing the Diehard battery of tests of randomness (<http://www.stat.fsu.edu/pub/diehard>), published a pseudorandom number generation algorithm that is much simpler to implement and runs faster than the Mersenne Twister algorithm. He claimed that it could produce a sequence

of 32-bit pseudorandom numbers with a period of non-repetition of 2^{250} . It passed all of the Diehard tests and still stands today as one of the best pseudorandom number generators for high-speed applications. He called his algorithm the *Mother of All Pseudorandom Number Generators*, because it seemed to him to be the only random number generator one would ever need.

Later, Marsaglia published another generator called Xorshift, which is between Mersenne and Mother-of-All in terms of randomness, but runs slightly faster than Mother.

Marsaglia also developed a series of random number generators that are collectively called KISS (Keep It Simple Stupid). The KISS99 algorithm is a popular choice, because it has a large period (2^{123}) and passes all tests in the TestU01 test suite (<https://bit.ly/2r5FmSP>).

You can read about George Marsaglia at http://en.wikipedia.org/wiki/George_Marsaglia, and about the Mother-of-All generator at <ftp://ftp.forth.org/pub/C/mother.c> and at <http://www.agner.org/random>. You can download a PDF of George's paper on Xorshift at <http://www.jstatsoft.org/v08/i14/paper>.

5.7.4 PCG

Another very popular and high-quality family of pseudorandom number generators is called PCG. It works by combining a congruential generator for its state transitions (the "CG" in PCG) with permutation functions to generate its output (the "P" in PCG). You can read more about this family of PRNGs at <http://www.pcg-random.org/>.